Spatial Data Structures

Acknowledgement: Marco Tarini

Logistics: Final format

- Open questions
- E.g.,
  - Describe the steps of an algorithm
  - What are the differences between two things
  - Light derivation with given formulas

University of Victoria | Computer Science

Logistics: Assignment 3

- **Due: November 2 before class (10 am Pacific)**
- Many things are from Assignment 2 but now we are rendering a scene
- Recaps and walk-through

University of Victoria | Computer Science

Assignment 3: Basic Ray Tracing Effects

- See “05 - Ray Tracing” and “06 - Procedural Synthesis” slides on the course repo
- Goal: shadows, reflections, procedural texture, and a perspective camera
- 5 exercises + 2 bonus exercises

Build and Run

- `mkdir build; cd build; cmake ..; make; ./assignment3`
- Writes **raytrace.png** into the build folder
- Out of the box you get a blank image, because the intersections return −1

University of Victoria | Computer Science

Starter code

- Scene and Globals
  - Camera
  - Objects
  - Materials
  - Perlin noise random vector field
  - Lights
- The Pipeline
  - main → setup_scene → raytrace_scene → shoot_ray → write_matrix_to_png

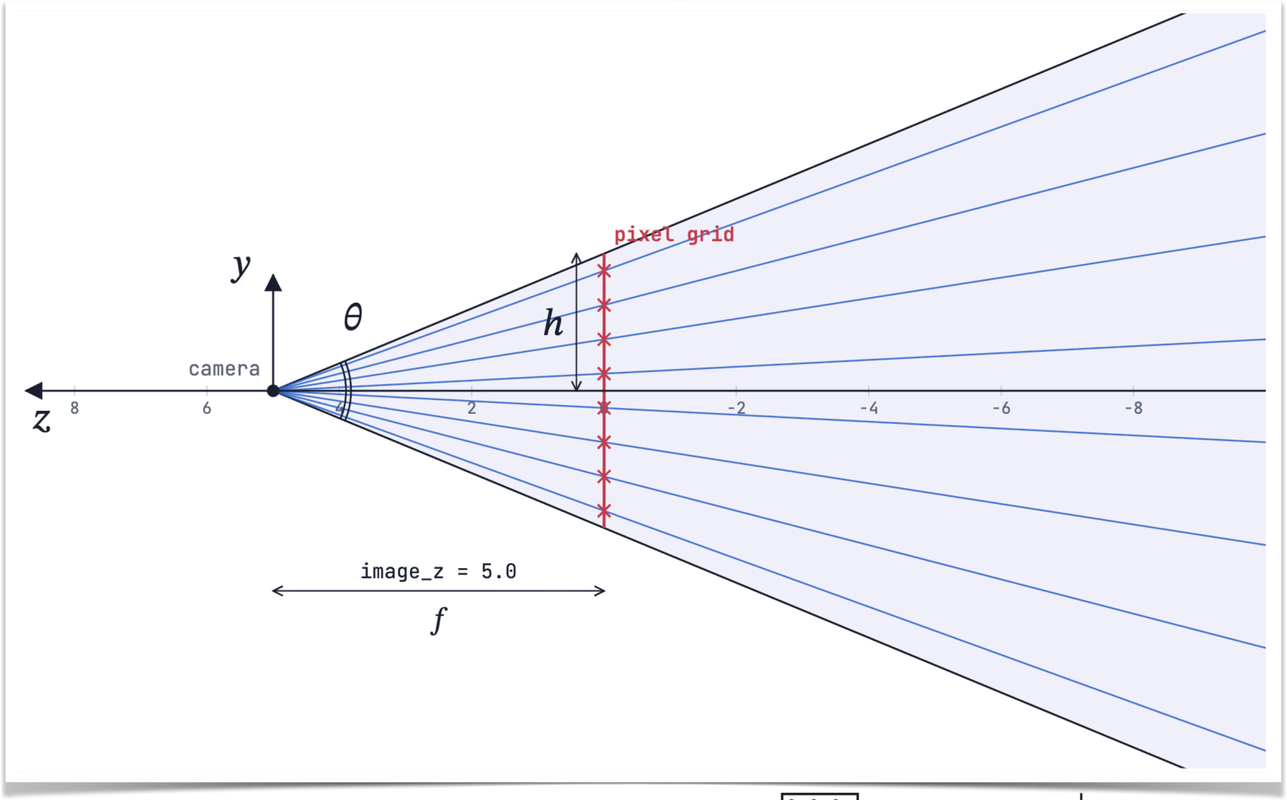
Ex 0: Intersections (5 pt)

- Fill in ray_sphere_intersection and ray_parallelogram_intersection
- Return t (or −1), and write the hit point p and the normal N
- Same math as Assignment 2
  - I'll cover the solution next Thursday (October 15)

Ex 1: Field of View (5 pt)

- Fill the starter code to compute the correct value of h (image_y in the code). You can use the Law of Sines or the trigonometric circle.

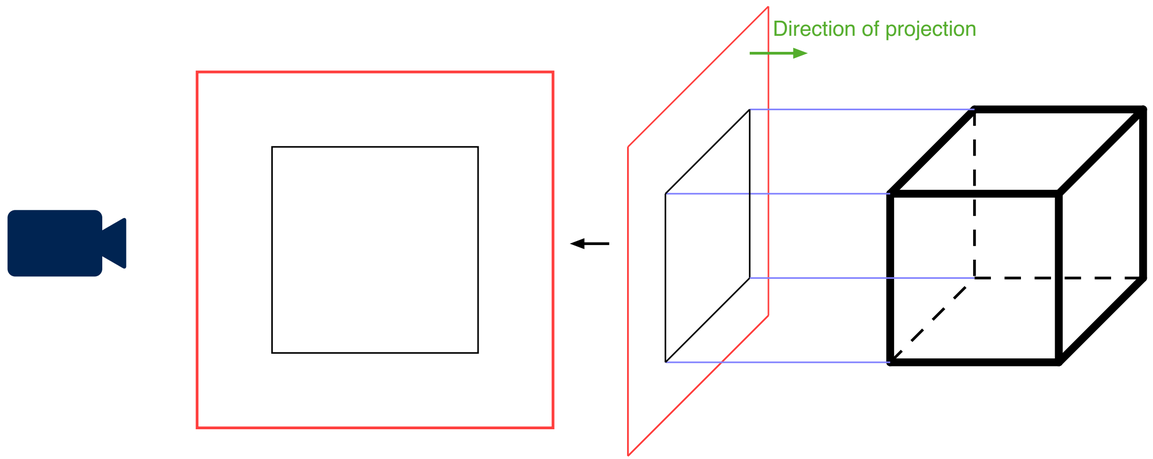
```
//Camera settings
const double focal_length = 10;
const double field_of_view = 0.7854; //45 degrees
const double image_z = 5;
const bool is_perspective = false;
const Vector3d camera_position(0, 0, 5);
const double camera_aperture = 0.05;
```



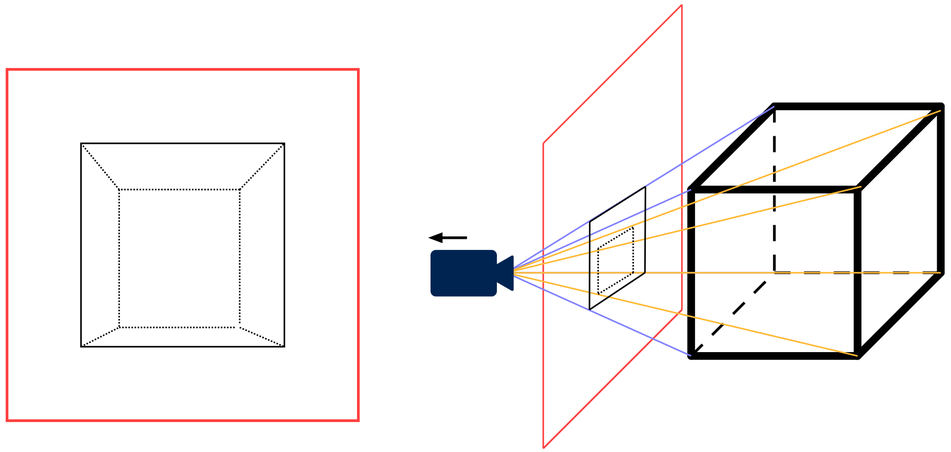
University of Victoria | Computer Science

Ex 1: Field of View (5 pt)

- Implement the perspective camera similarly to Assignment 2



Orthographic

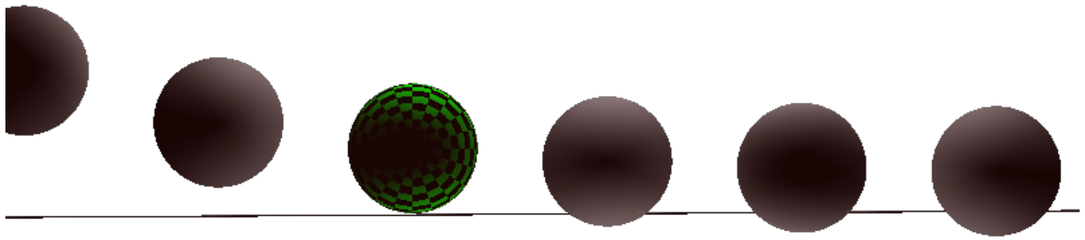


Perspective

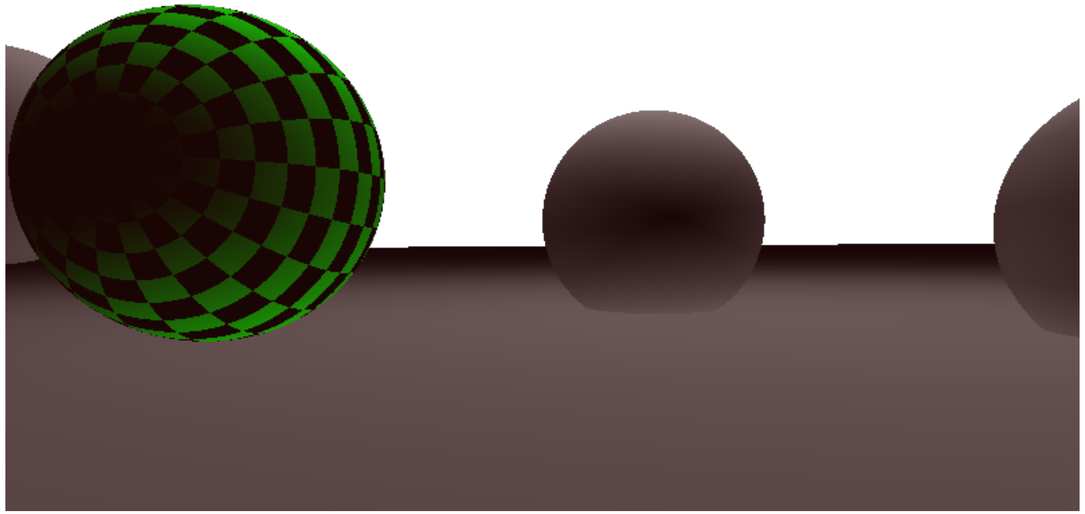
University of Victoria | Computer Science

Ex 1: Field of View (5 pt)

- Implement the perspective camera similarly to Assignment 2



Orthographic



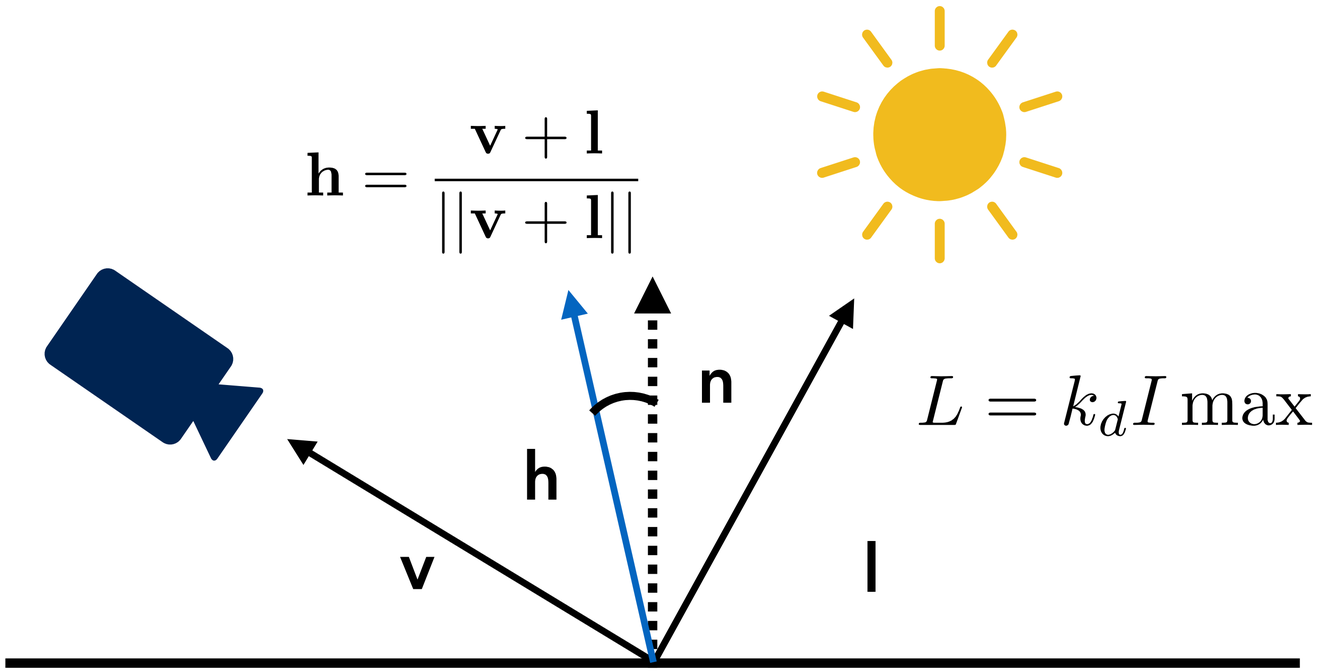
Perspective

University of Victoria | Computer Science

Ex 2: Phong Shading (5 of 10 pt)

- The diffuse term is already there
- Add the specular term in **shoot_ray**

$$\mathbf{h} = \frac{\mathbf{v} + \mathbf{l}}{||\mathbf{v} + \mathbf{l}||}$$



$$L = k_d I \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s I \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

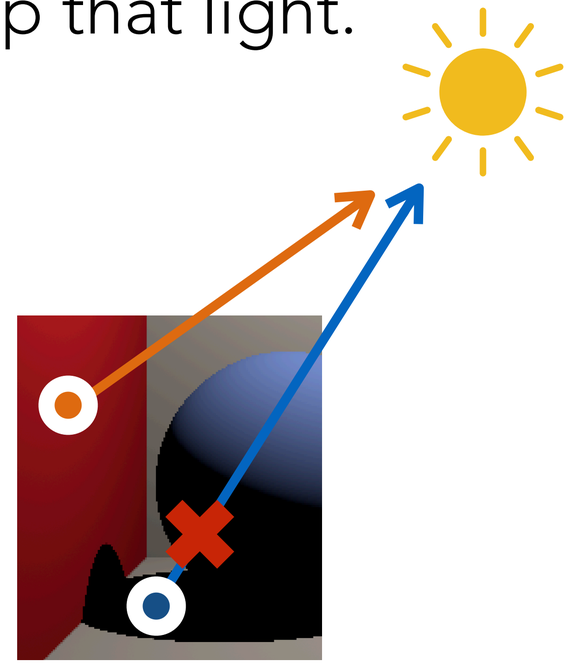
Phong exponent

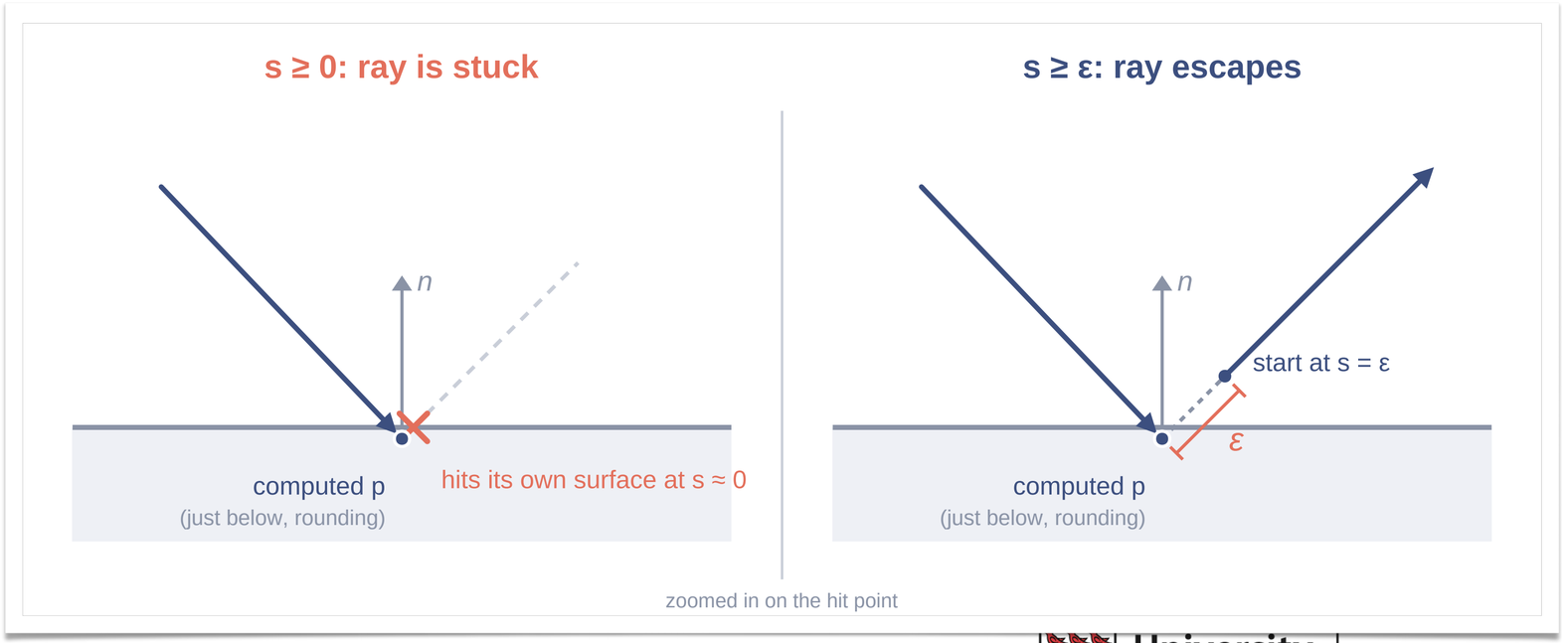
specular coefficient

University of Victoria | Computer Science

Ex 2: Shadow Rays (5 of 10 pt)

- Fill in **is_light_visible**
- Cast a ray from the hit point toward *each* light. If something blocks it, skip that light.

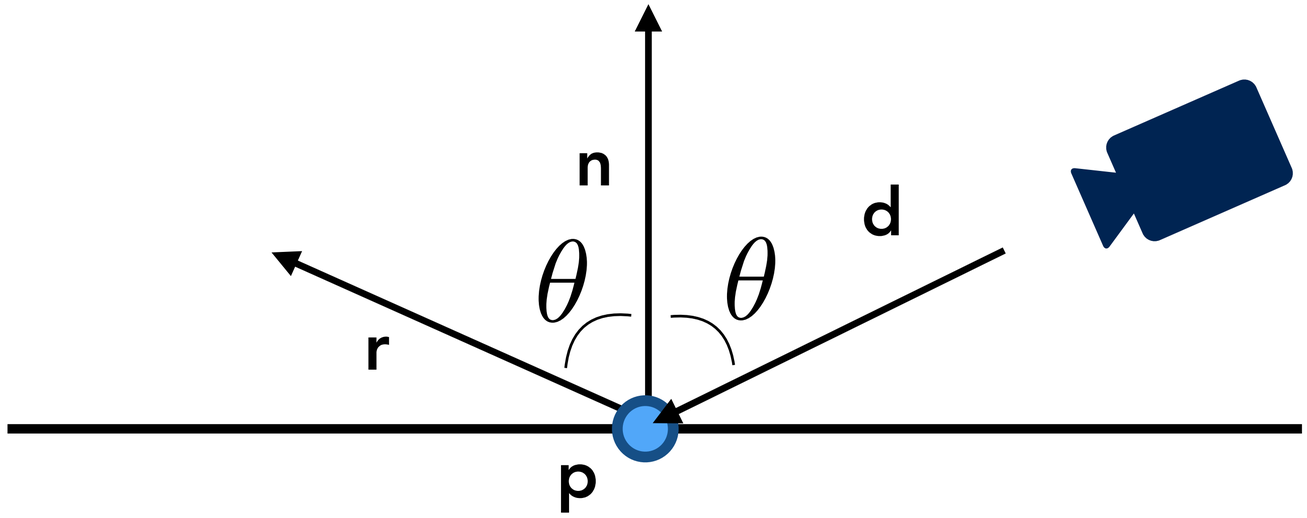


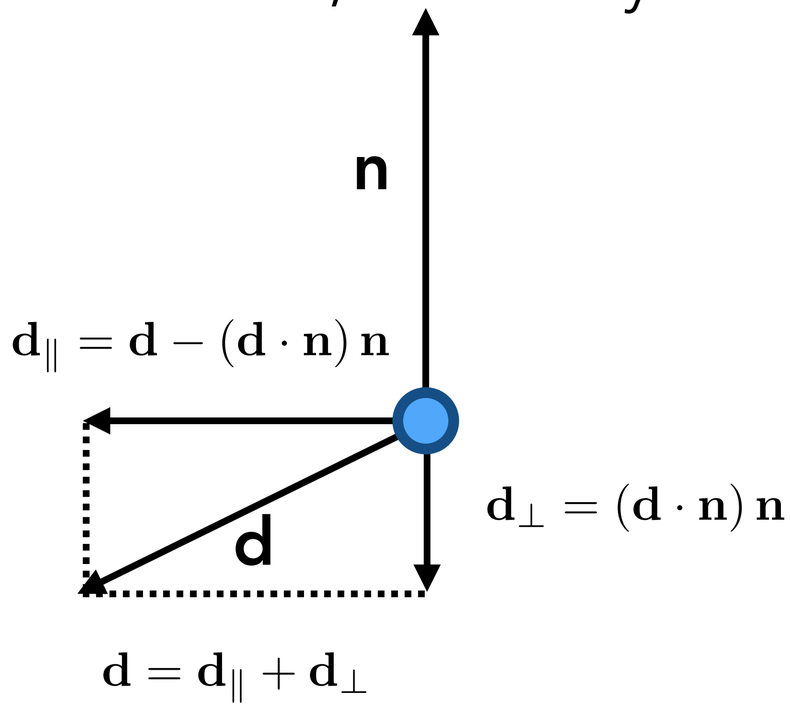


University of Victoria | Computer Science

Ex 3: (Mirror) Reflection (10 pt)

- Recursively call **shoot_ray** along the reflected direction, scaled by **refl_color**



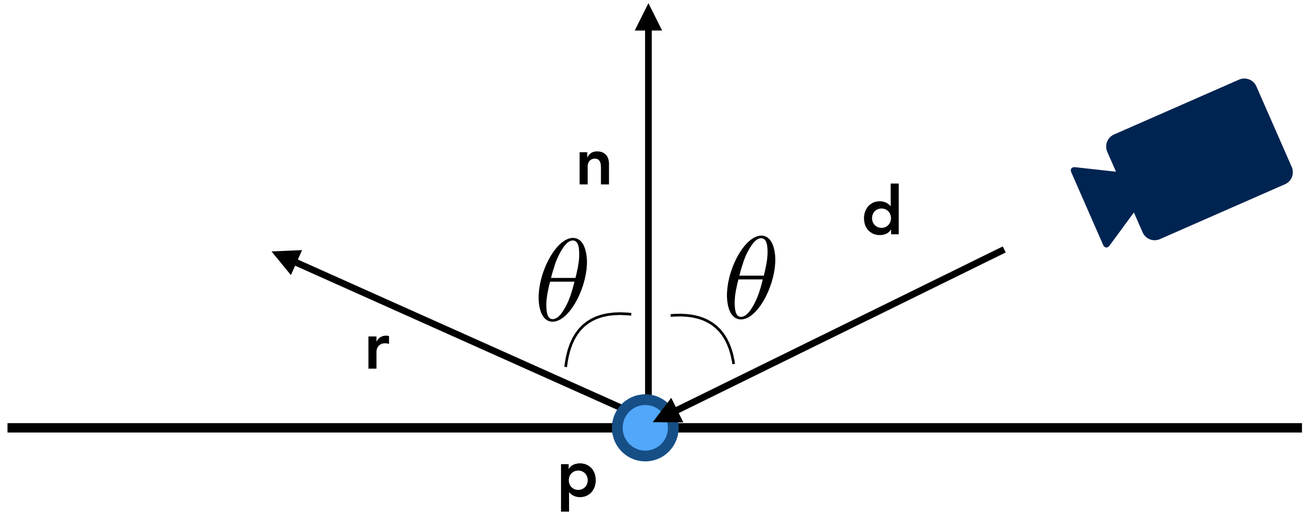


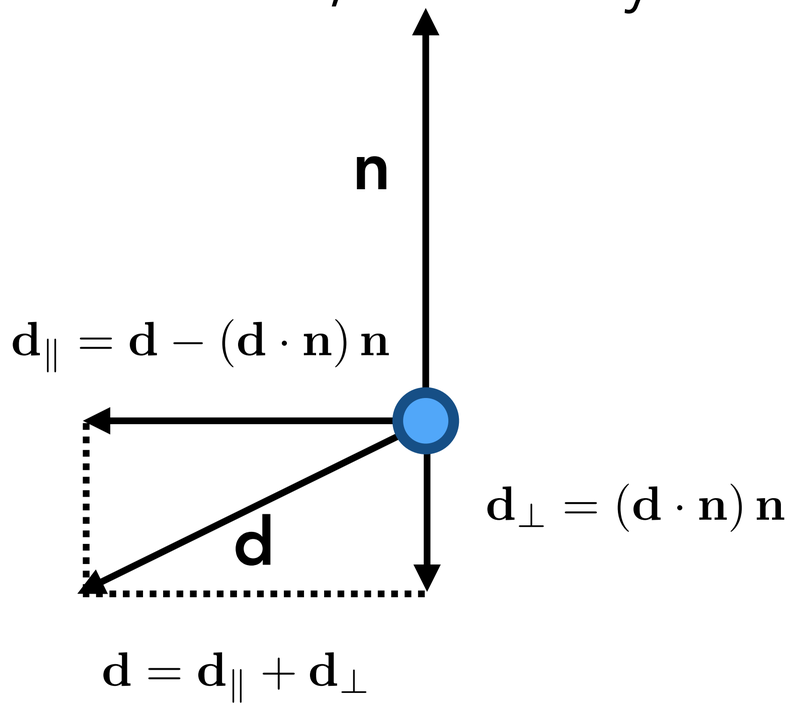
$$L = L_{\text{local}} + k_m \,\text{raycolor}\big(\mathbf{p} + s\mathbf{r}\big), \quad s \in [\epsilon, \infty)$$

University of Victoria | Computer Science

Ex 3: (Mirror) Reflection (10 pt)

- Recursively call **shoot_ray** along the reflected direction, scaled by **refl_color**





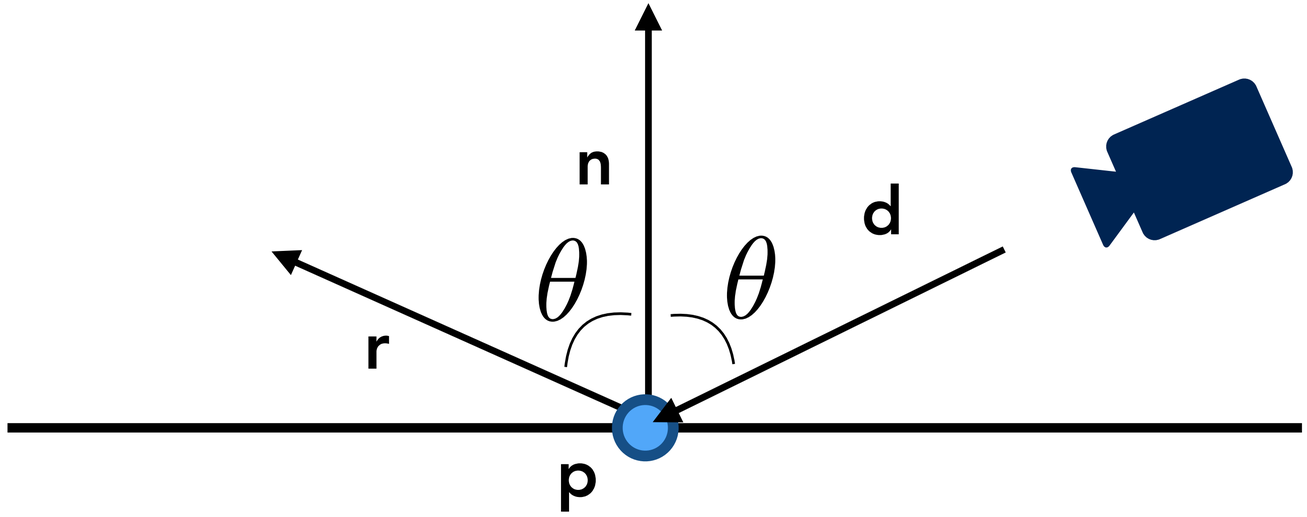
$$\mathbf{r} = \mathbf{d}_{\parallel} - \mathbf{d}_{\perp}$$

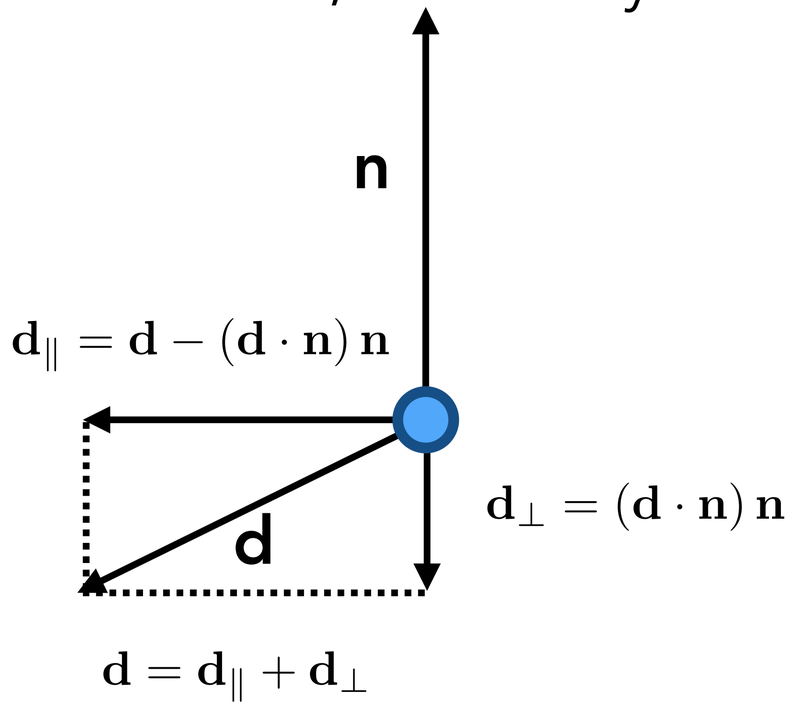
$$L = L_{\text{local}} + k_m \,\text{raycolor}\big(\mathbf{p} + s\mathbf{r}\big), \quad s \in [\epsilon, \infty)$$

University of Victoria | Computer Science

Ex 3: (Mirror) Reflection (10 pt)

- Recursively call **shoot_ray** along the reflected direction, scaled by **refl_color**





$$\mathbf{r} = \mathbf{d}_{\parallel} - \mathbf{d}_{\perp}$$

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\mathbf{n}$$

$$L = L_{\text{local}} + k_m \,\text{raycolor}\big(\mathbf{p} + s\mathbf{r}\big), \quad s \in [\epsilon, \infty)$$

University of Victoria | Computer Science

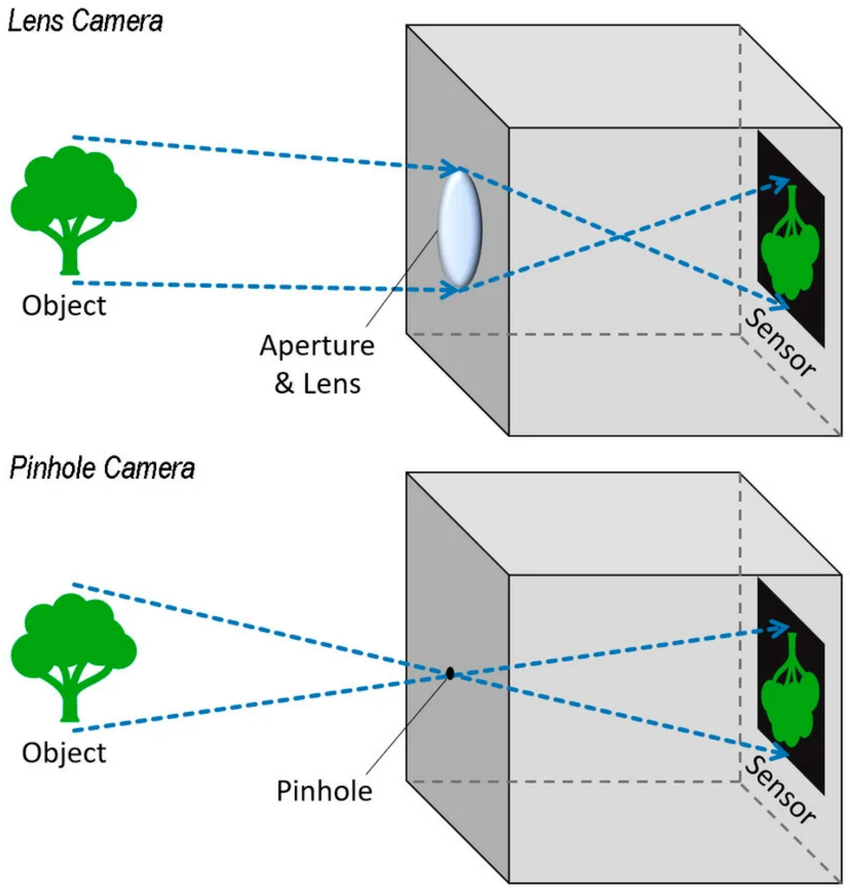
Ex 4: Perlin Noise (10 pt)

- Fill in **lerp**, **dotGridGradient**, and the grid cell lookup in **perlin**
- Uncomment the two lines in procedural_texture (they replace the checkerboard)

University of Victoria | Computer Science

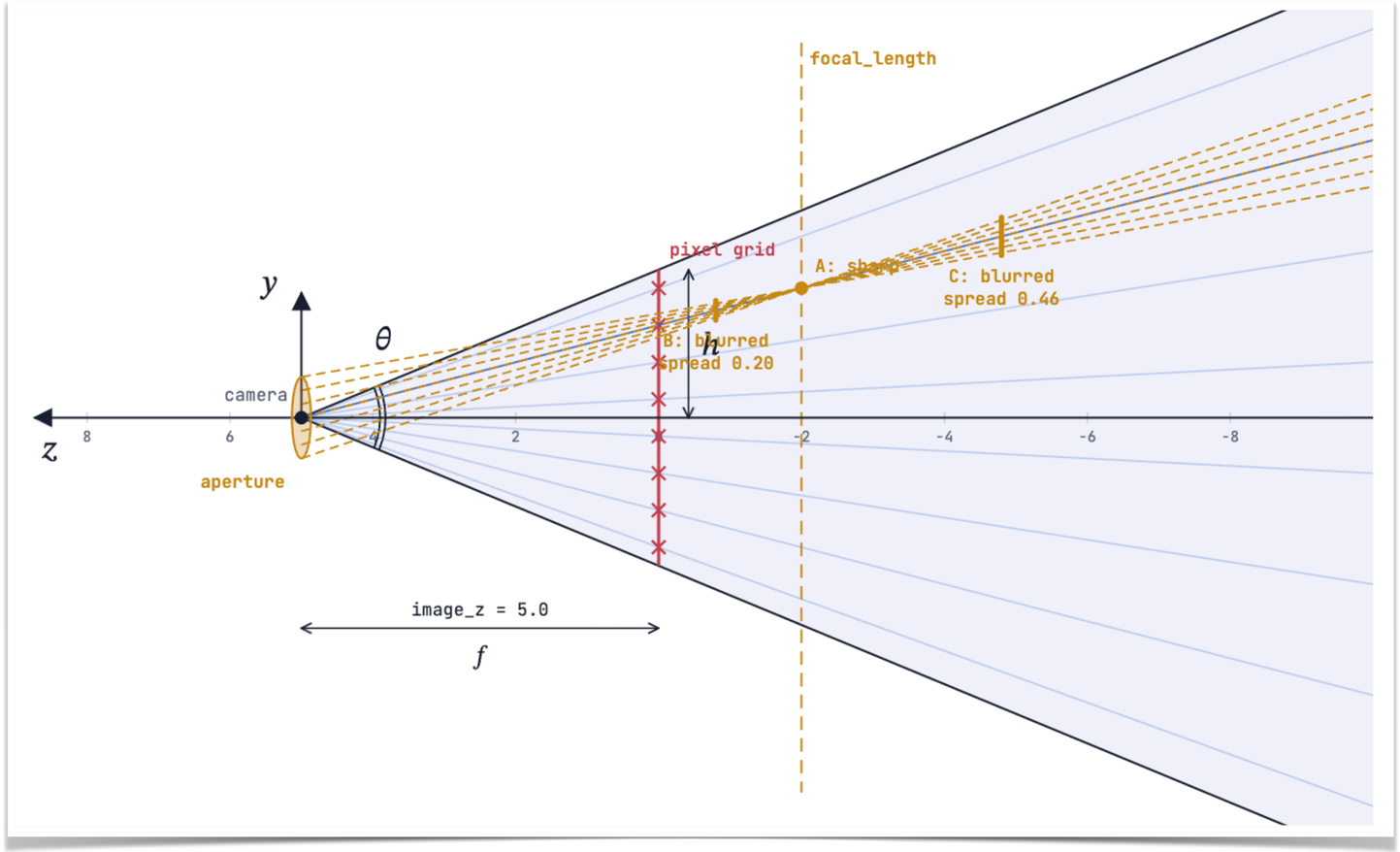
Bonus 1: Depth of Field (10 pt)

- A pinhole camera has an infinitely small aperture, so everything is in focus
- A real lens has a finite aperture, so rays from one scene point spread over a disk
- Points on the focal plane stay sharp, and blur grows with distance from it



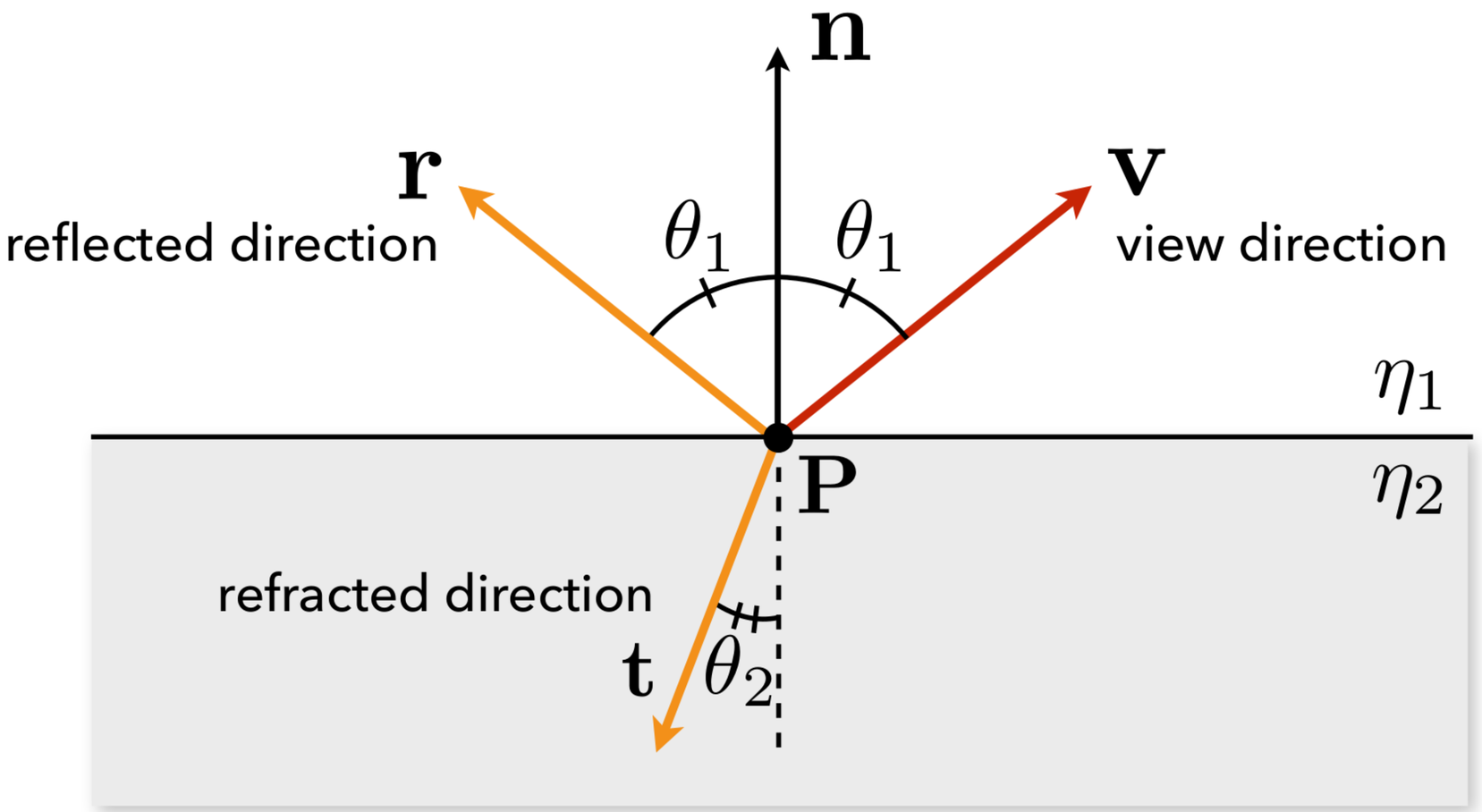
Kahl, Benjamin. "Hardware acceleration of progressive refinement radiosity using Nvidia RTX." arXiv preprint arXiv:2303.14831 (2023).

Bonus 1: Depth of Field (10 pt)



University of Victoria | Computer Science

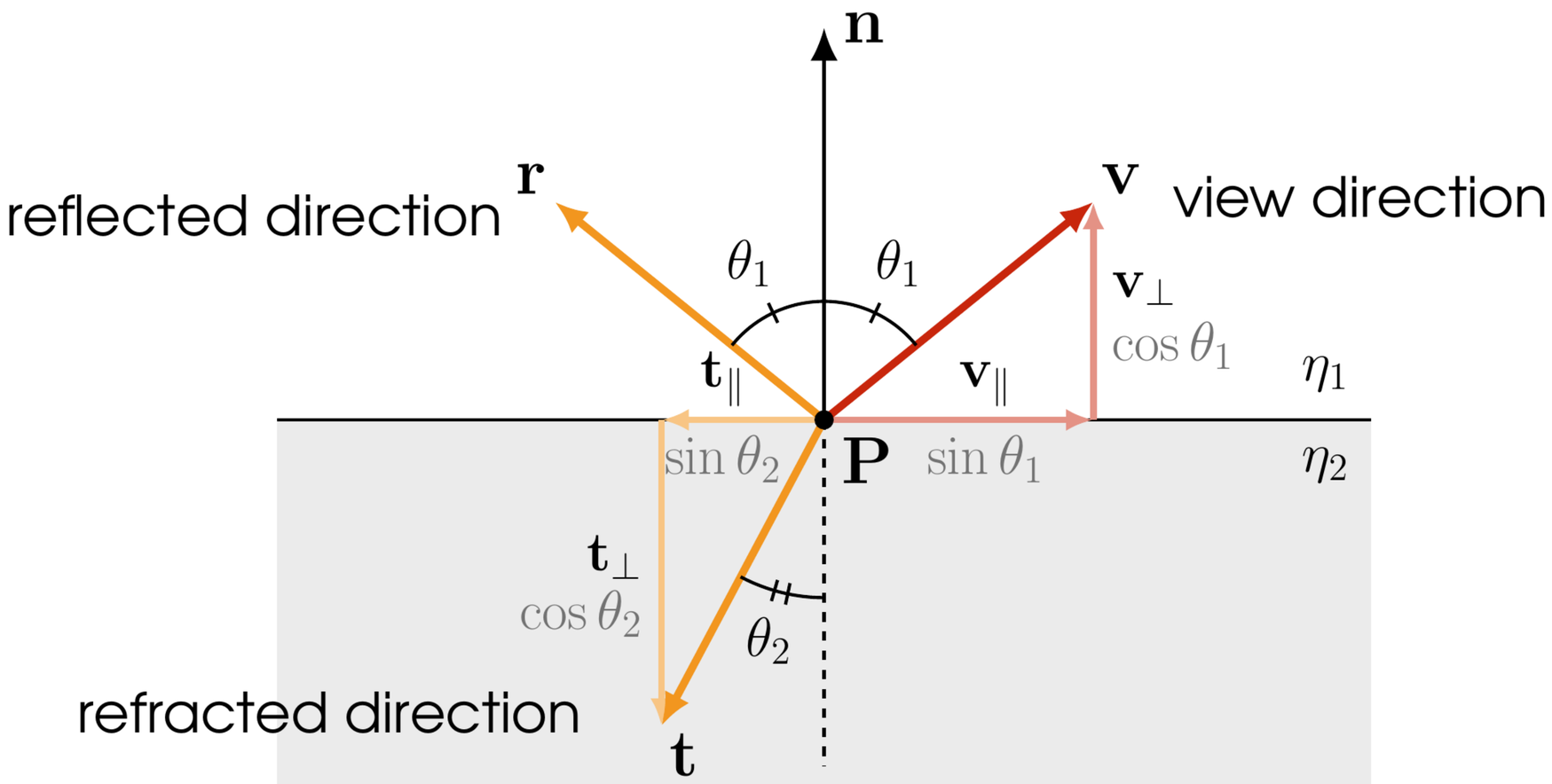
Bonus 2: Refraction (15 pt)



$$\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$$

University of Victoria | Computer Science

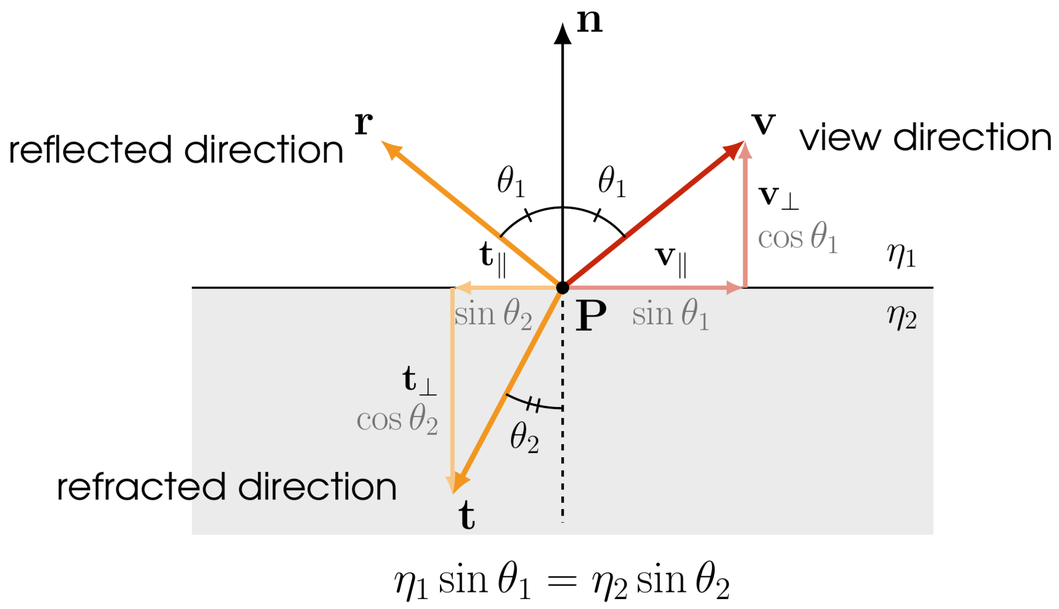
Bonus 2: Refraction (15 pt)



$$\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$$

University of Victoria | Computer Science

Bonus 2: Refraction (15 pt)



$$\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$$

θ_2 is unknown

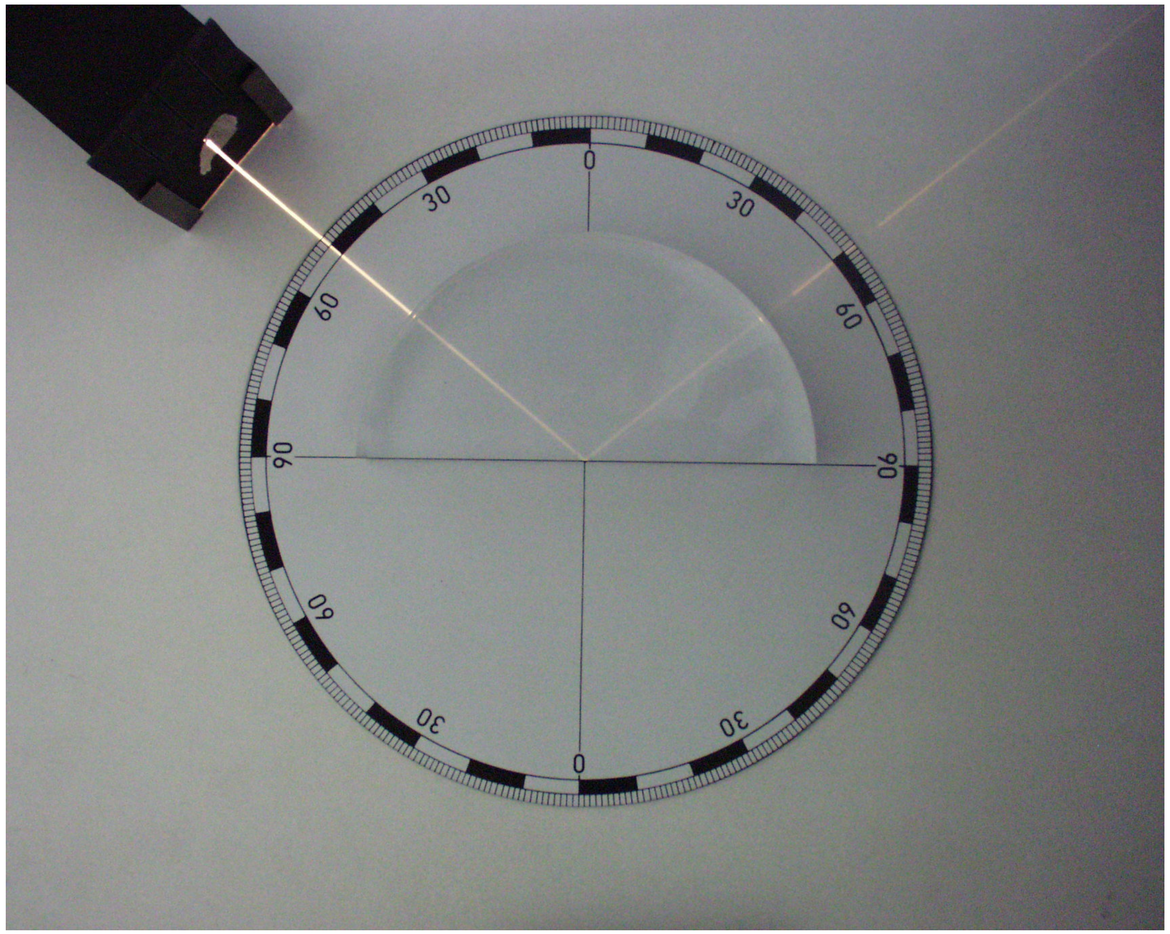
$$\cos\theta_1 = \mathbf{v}\cdot\mathbf{n}, \qquad \mathbf{v}_\parallel = \mathbf{v} - \cos\theta_1\,\mathbf{n}, \qquad \|\mathbf{v}_\parallel\| = \sin\theta_1$$

$$\mathbf{t}_\parallel = -\frac{\sin\theta_2}{\sin\theta_1}\,\mathbf{v}_\parallel = -\eta\,(\mathbf{v} - \cos\theta_1\,\mathbf{n})$$

$$\cos\theta_2 = \sqrt{1-\sin^2\theta_2} = ?$$

University of Victoria | Computer Science

Bonus 2: Refraction (15 pt)





https://en.wikipedia.org/wiki/Total_internal_reflection

Spatial Data Structures

Acknowledgement: Marco Tarini

Types of Queries

- Graphic applications often require spatial queries
  - Is object X intersection with object Y? (Intersection)
  - Find the k points closer to a specific point p (k-Nearest Neighbours, knn)
  - What is the volume of the intersection between two objects?



https://commons.wikimedia.org/wiki/File:Monmouth_castle_point_cloud,_created_with_Photosynth_01.jpg

Types of Queries

- Brute force search is expensive. Instead, you can solve these queries with an initial preprocessing that creates a data structure which supports efficient queries $\mathcal{O}(n^2)$
- The data structure to use is application-specific

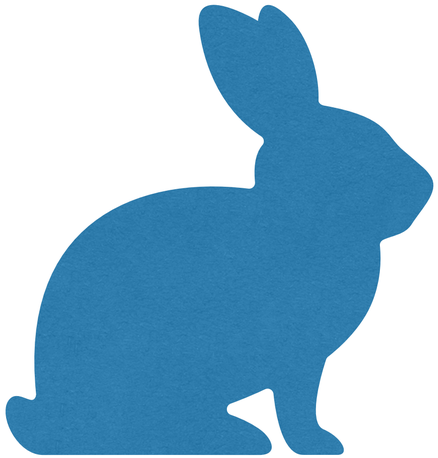
University of Victoria | Computer Science

Two Main Ideas

1. You can explicitly index the space itself (Spatial Index)

2. You can "sort" the primitives in the scene, which implicitly induces a partition of the space (Bounding Volume Hierarchies)

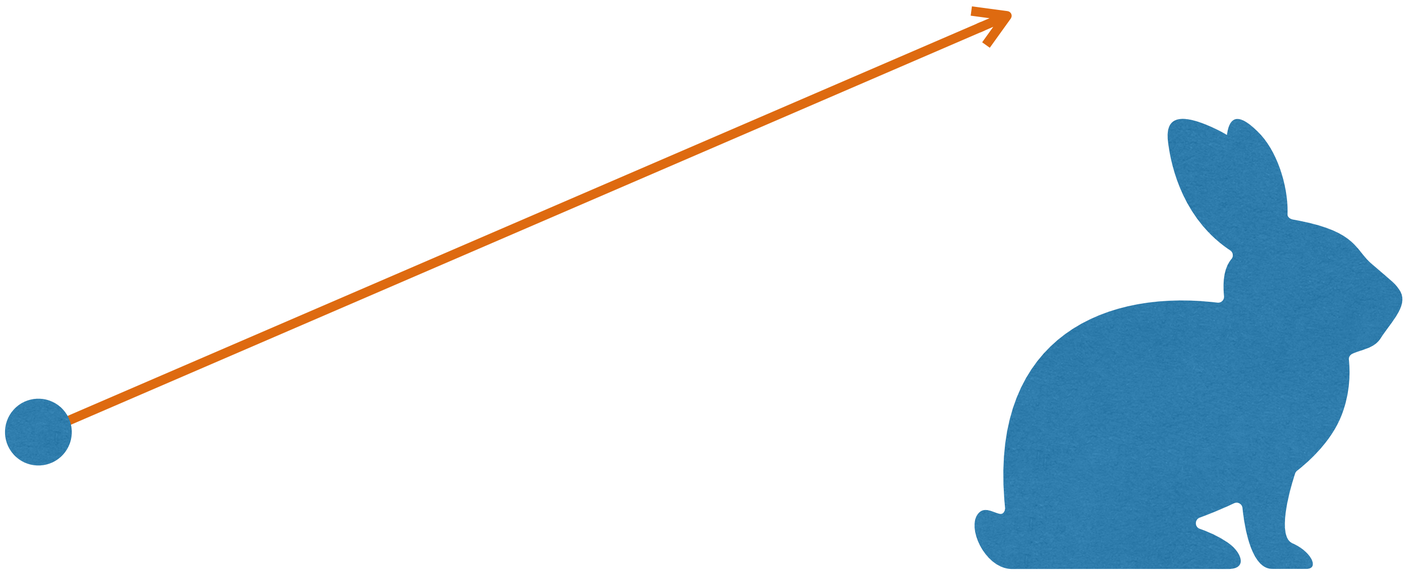




Two Main Ideas

1. You can explicitly index the space itself (Spatial Index)

2. You can “sort” the primitives in the scene, which implicitly induces a partition of the space (Bounding Volume Hierarchies)

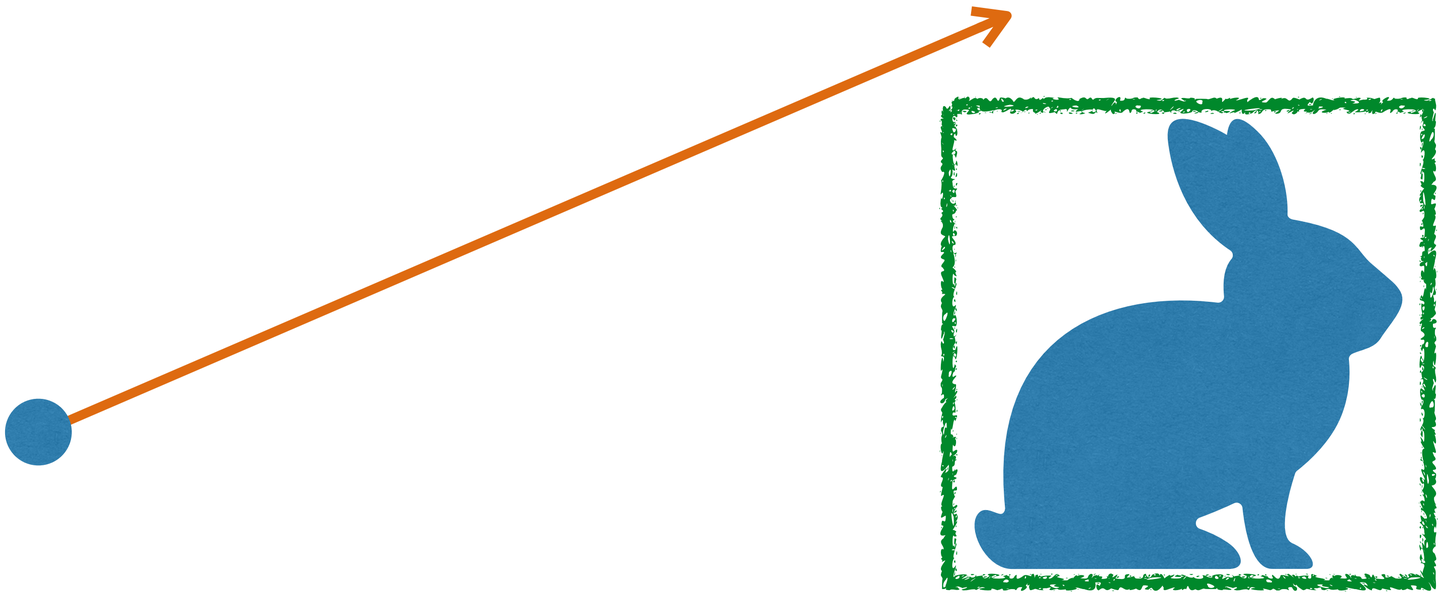


University of Victoria | Computer Science

Two Main Ideas

1. You can explicitly index the space itself (Spatial Index)

2. You can “sort” the primitives in the scene, which implicitly induces a partition of the space (Bounding Volume Hierarchies)

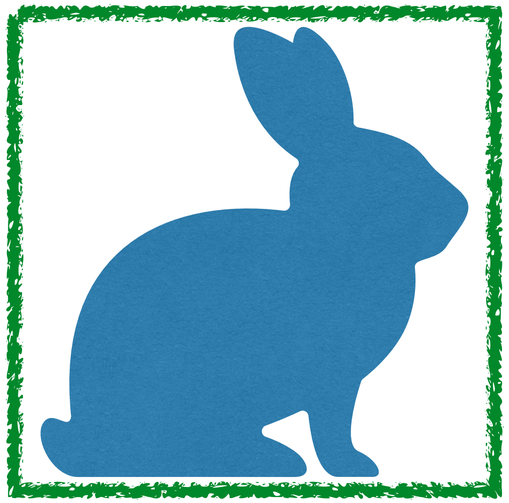


University of Victoria | Computer Science

Two Main Ideas

1. You can explicitly index the space itself (Spatial Index)

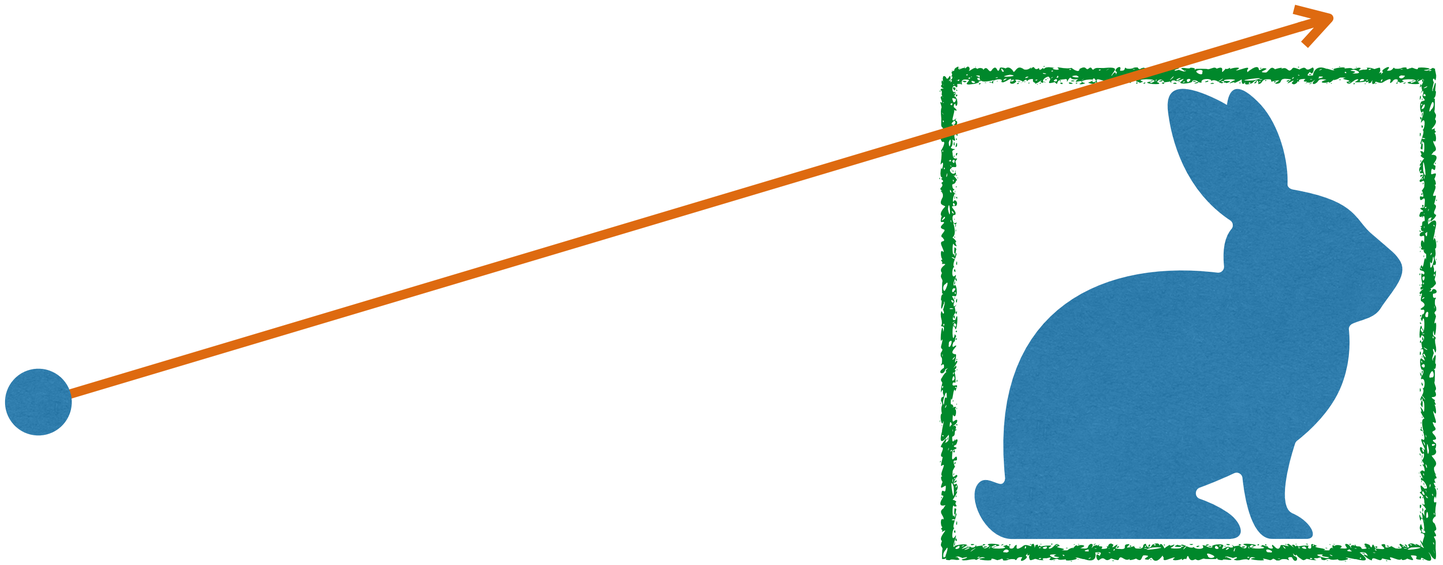
2. You can “sort” the primitives in the scene, which implicitly induces a partition of the space (Bounding Volume Hierarchies)



Two Main Ideas

1. You can explicitly index the space itself (Spatial Index)

2. You can "sort" the primitives in the scene, which implicitly induces a partition of the space (Bounding Volume Hierarchies)



Spatial Indexing Structures

University of Victoria | Computer Science

Spatial Indexing Structures

- Data structures to accelerate queries of the kind:
  "I'm here. Which object is around me?"
- Tasks:
  - (1) construction / update
    - for **static** parts of the scene, a preprocessing.
    - for **moving** parts of the scene, an update.
  - (2) access / usage
    - as fast as possible

University of Victoria | Computer Science

Spatial Indexing Structures

- The most common structures are:
  - Regular Grid
  - kD-Tree
  - Oct-Tree/Quad-Tree
  - BSP Tree

University of Victoria | Computer Science

Regular Grid (aka lattice)

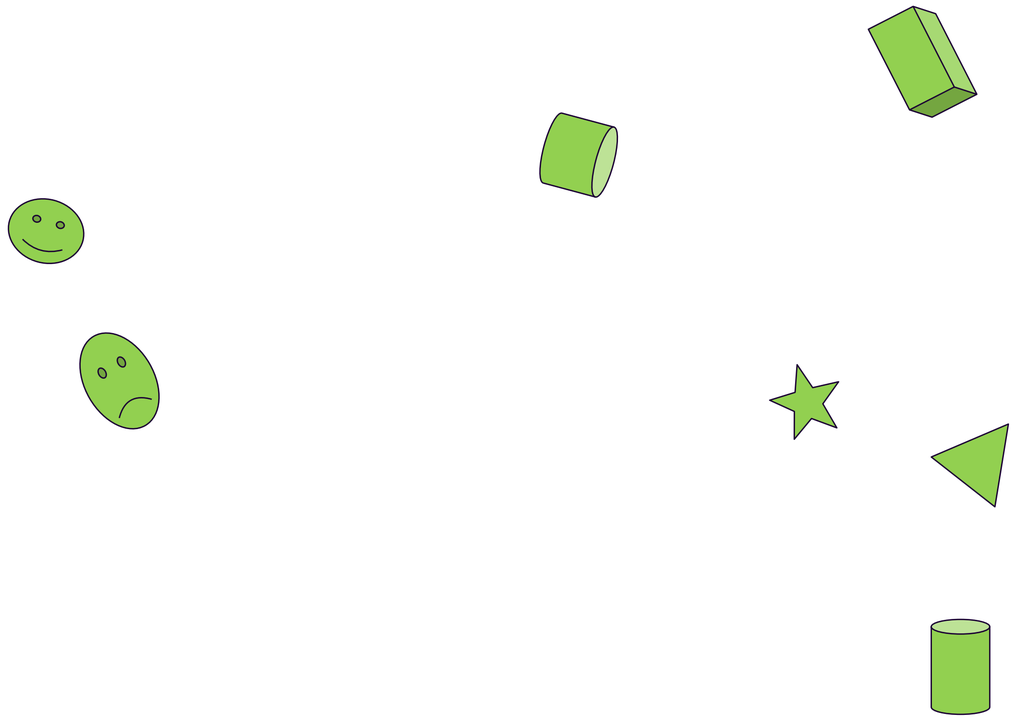


Image Copyright: Marco Tarini

Regular Grid (aka lattice)

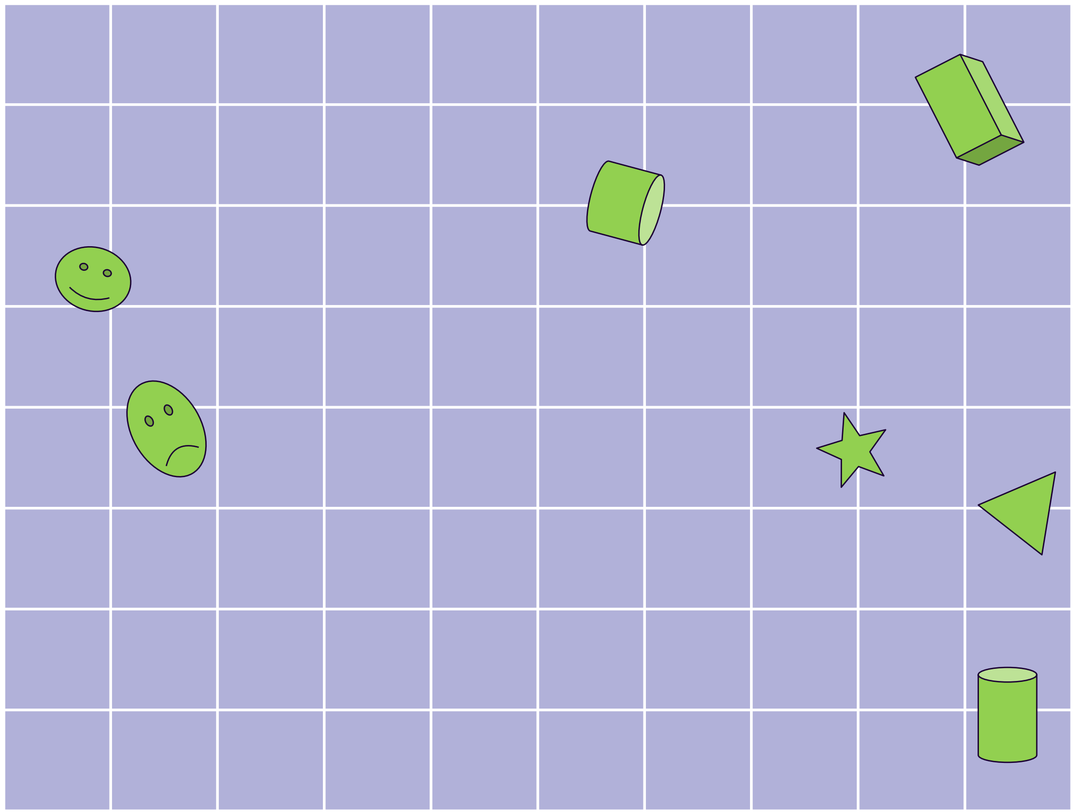


Image Copyright: Marco Tarini

Regular Grid (aka lattice)

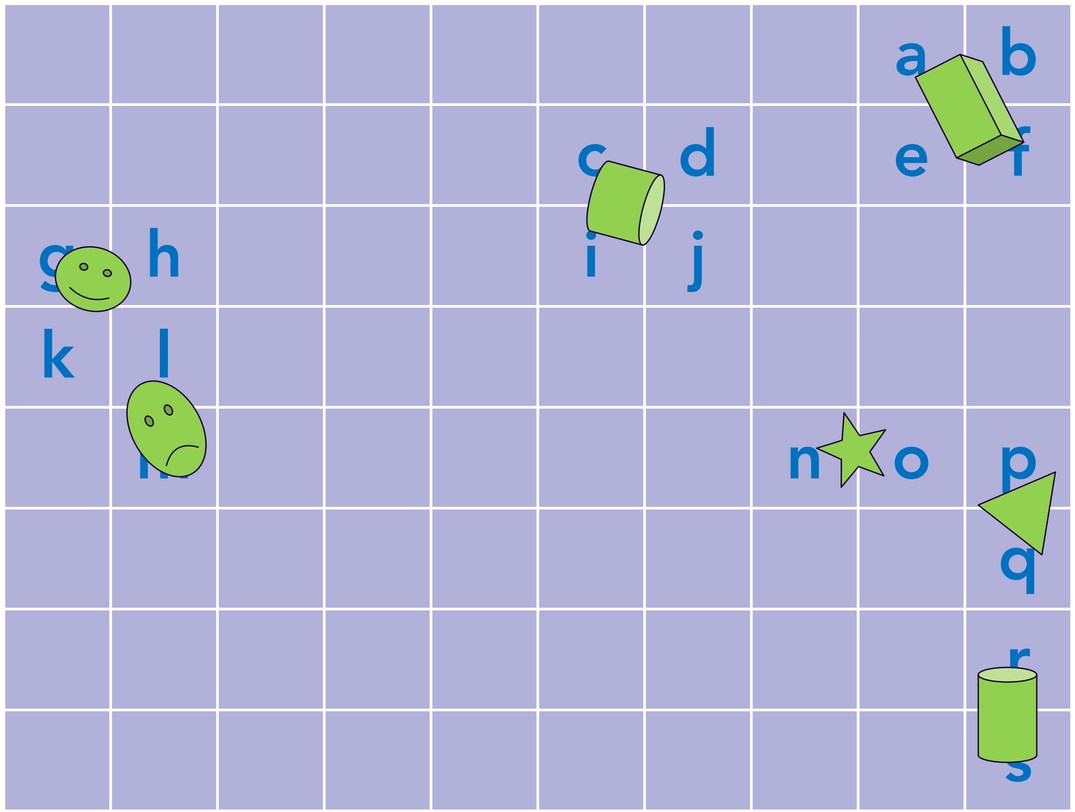


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Regular Grid (aka lattice)

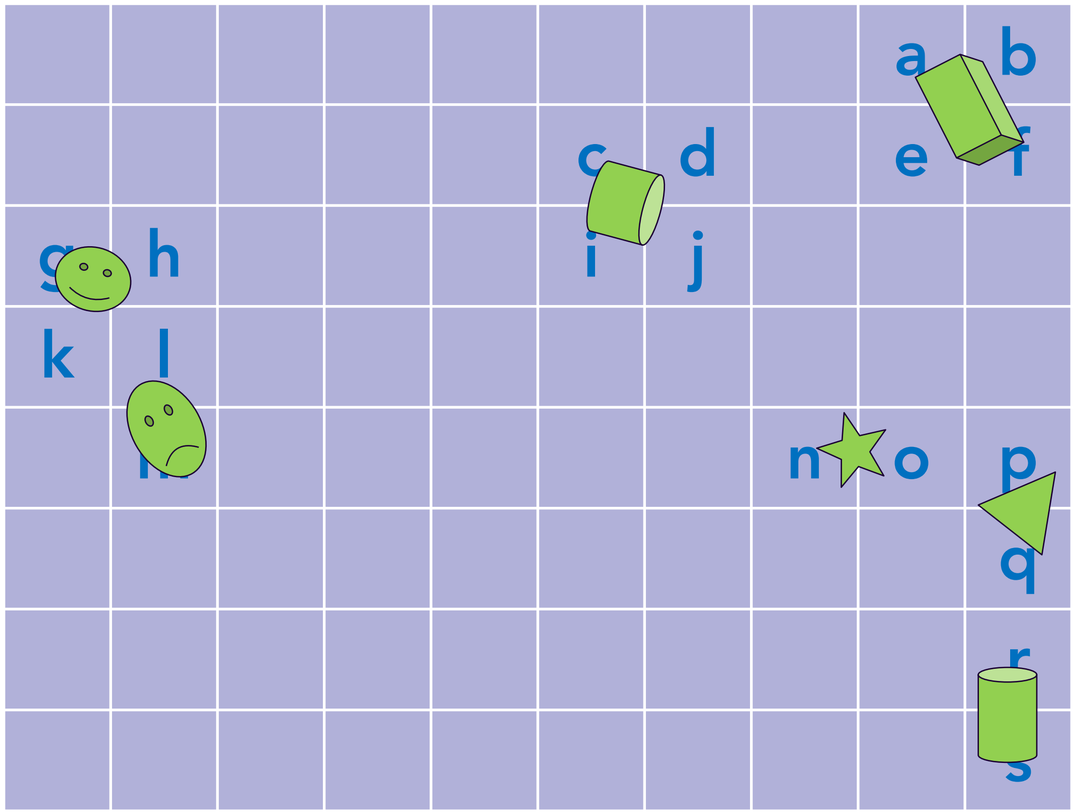




Image Copyright: Marco Tarini

University of Victoria | Computer Science

Regular Grid (aka lattice)

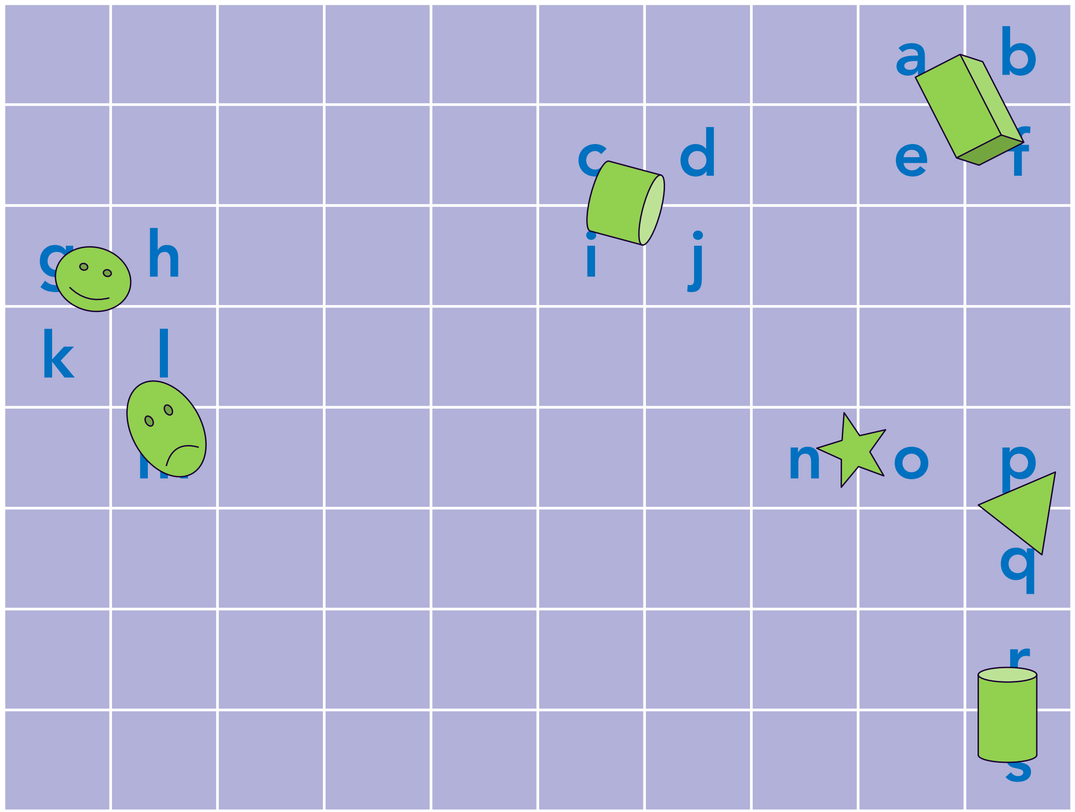
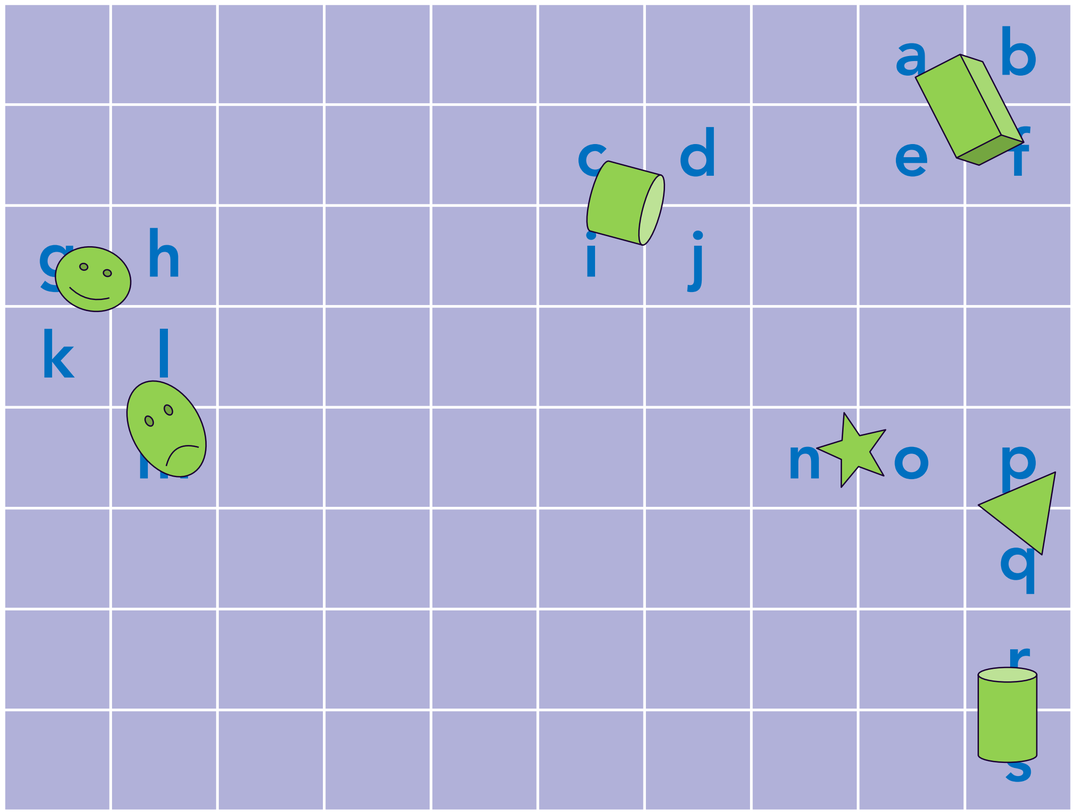




Image Copyright: Marco Tarini

Regular Grid (aka lattice)



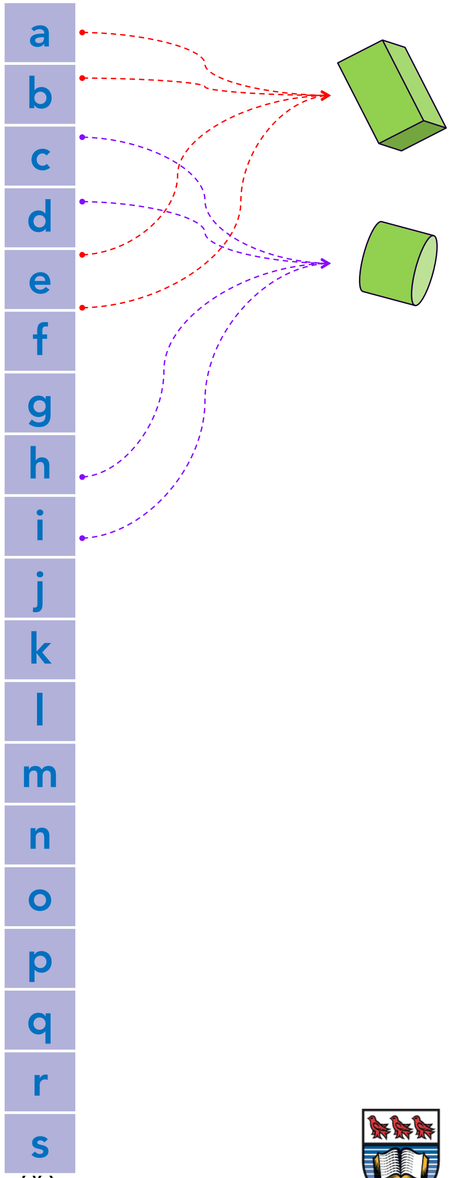
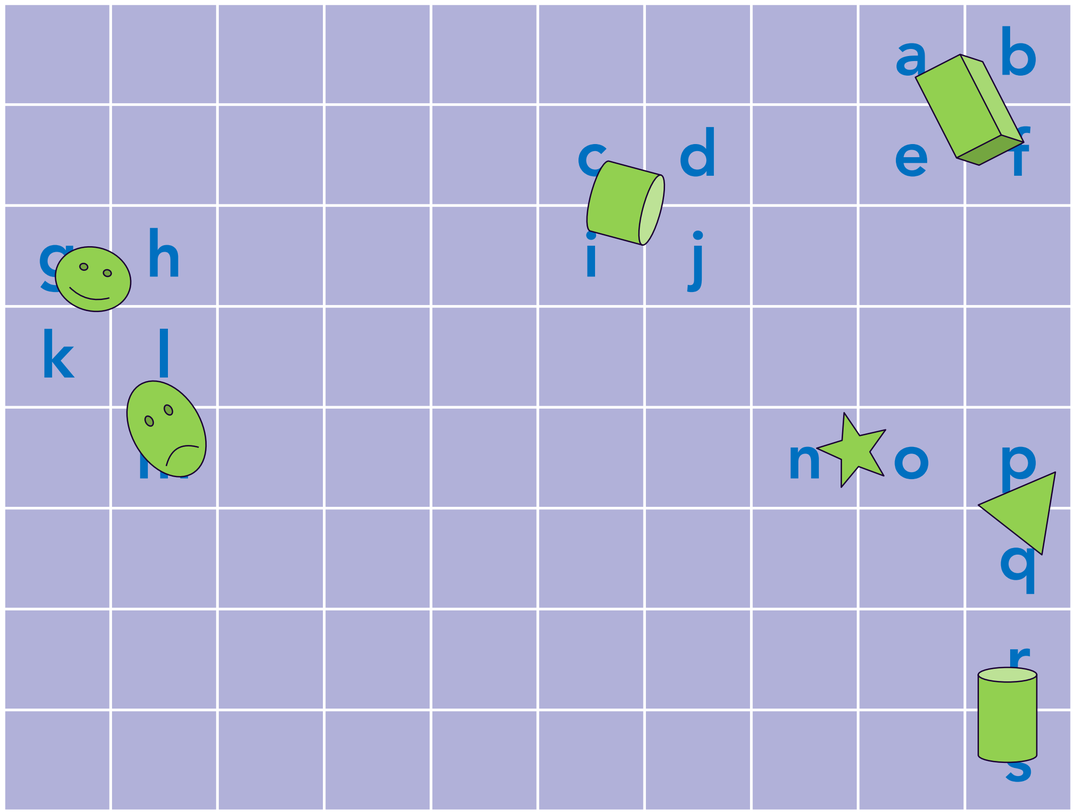


Image Copyright: Marco Tarini

Regular Grid (aka lattice)



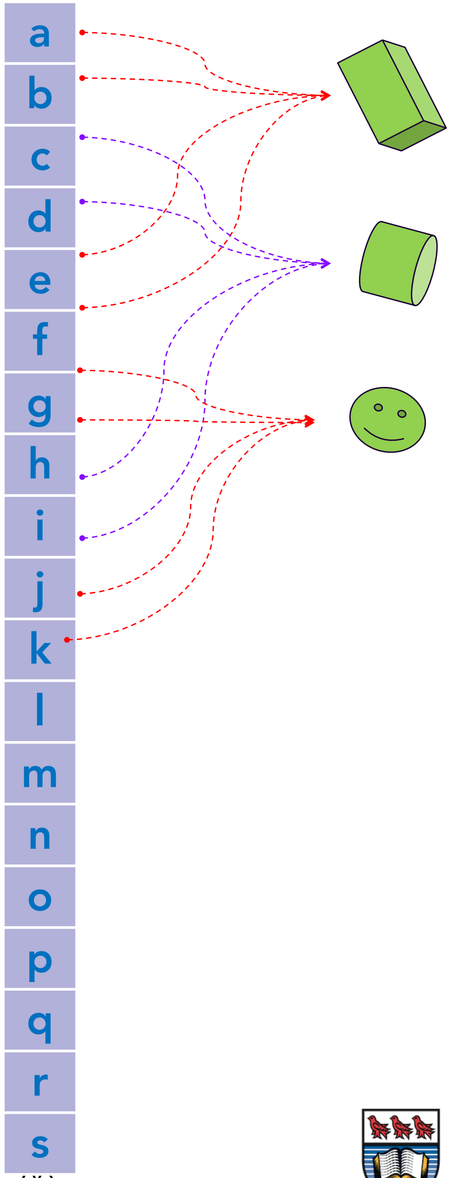
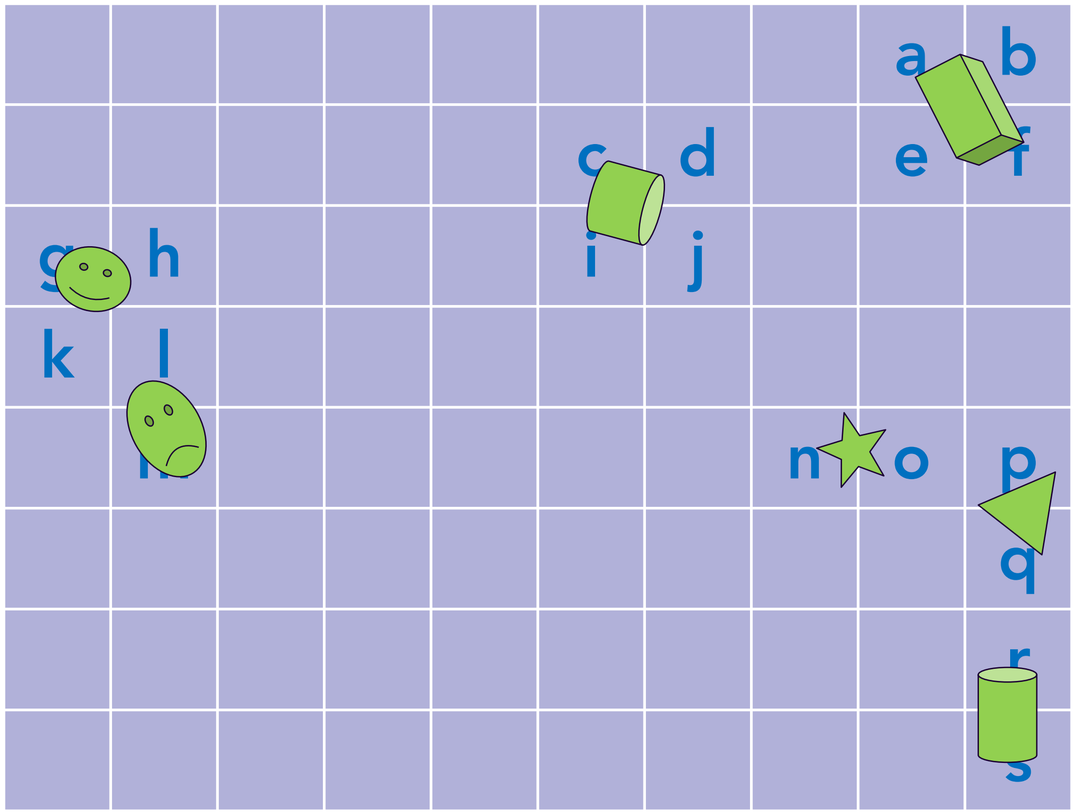


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Regular Grid (aka lattice)



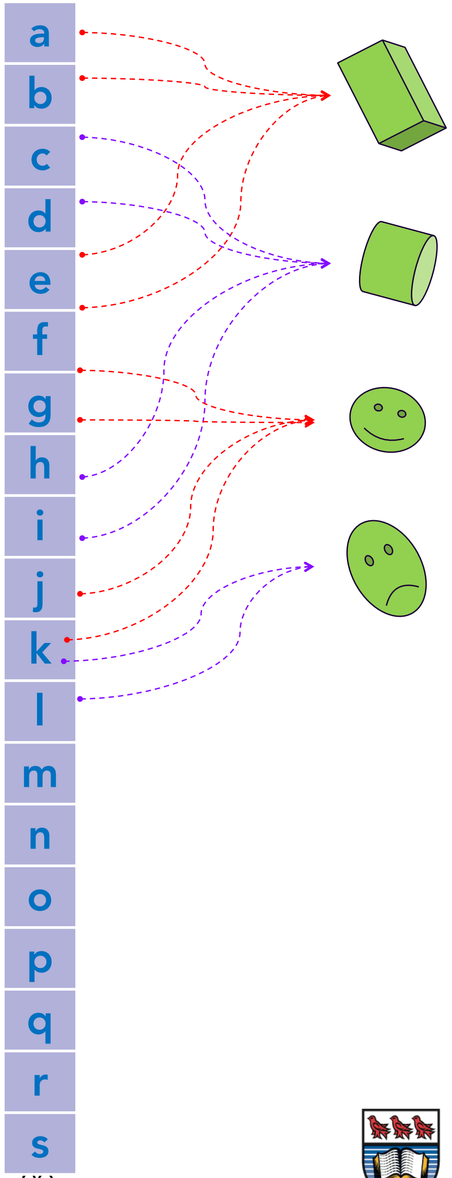
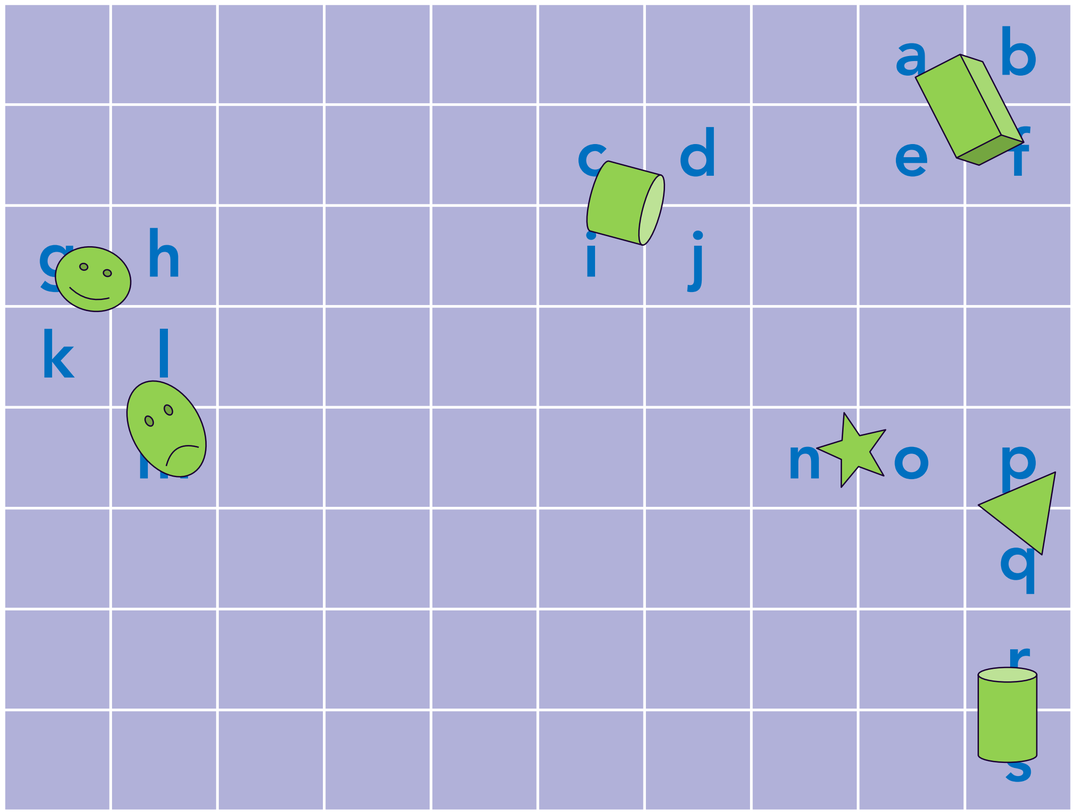


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Regular Grid (aka lattice)



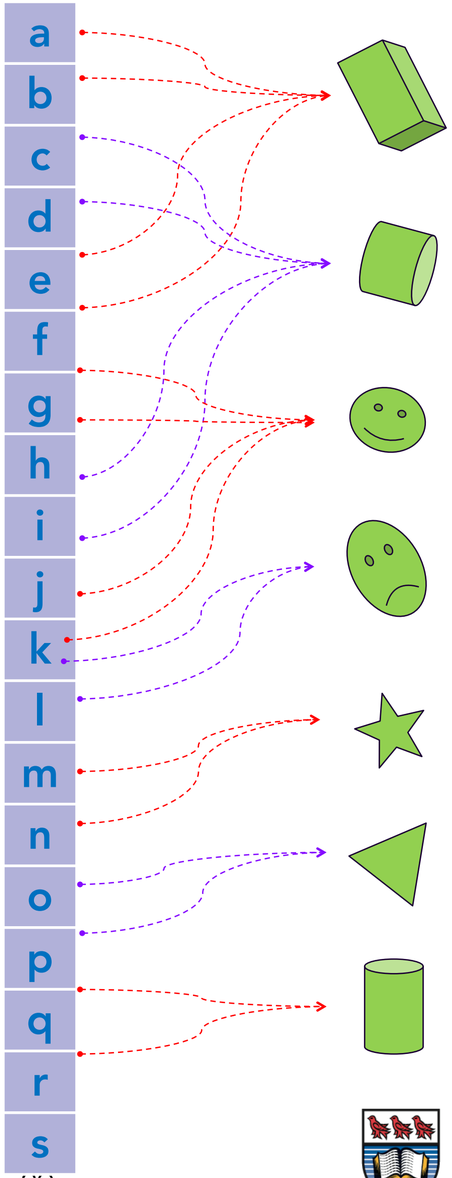
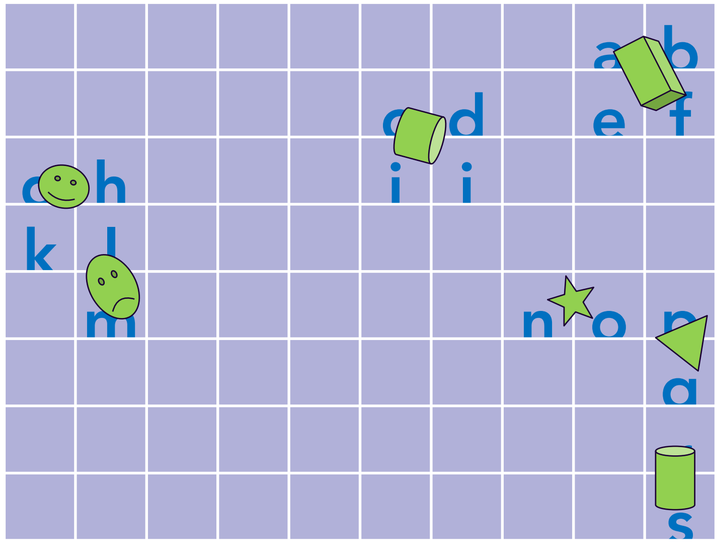


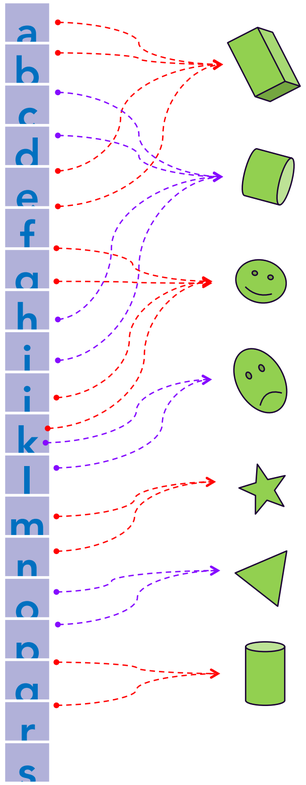
Image Copyright: Marco Tarini

University of Victoria | Computer Science

Regular Grid (or: lattice)

- Array 3D of cells (same size)
  - each cell: a list of pointers to colliding objects
- Indexing function:
  - Point3D \ cell index, (constant time!)
- Construction: (“scatter” approach)
  - for each object B[i]
    - find the cells C[j] which it touches
    - add a pointer in C[j] to B[i]
- Queries: (“gather” approach)
  - given a point to test *p*,
    find cell C[j], test all objects linked to it

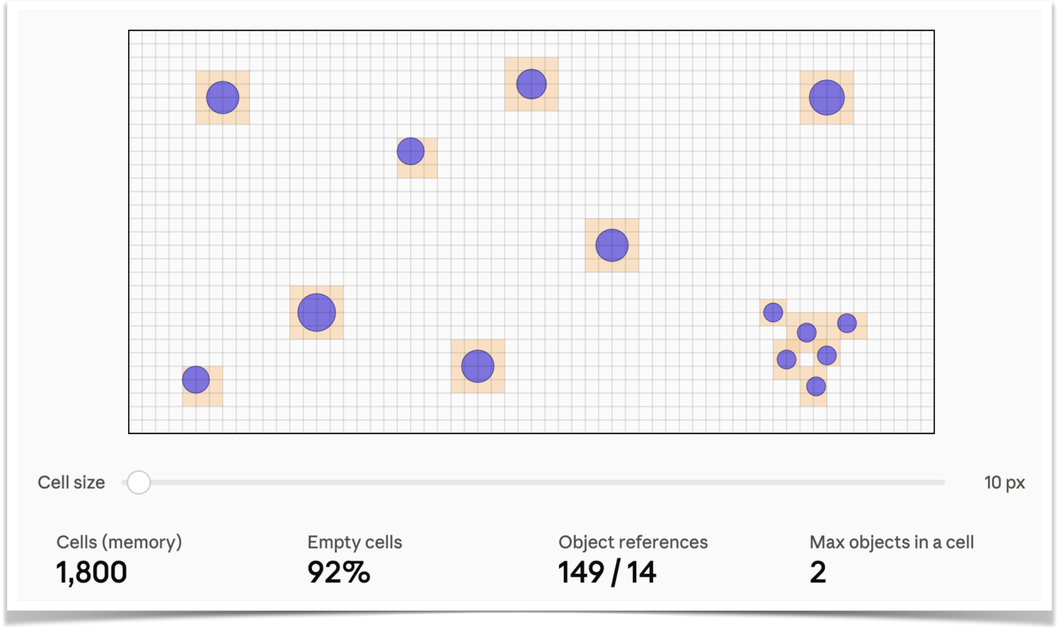


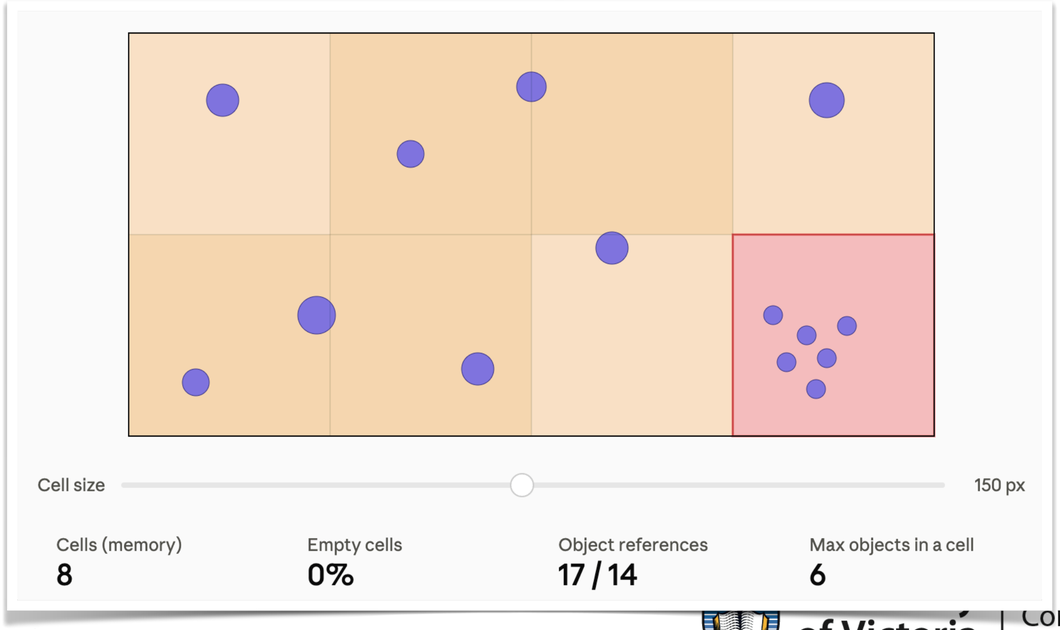


University of Victoria | Computer Science

Regular Grid (or: lattice)

- Problem: cell size
  - too small: memory occupancy too large quadratic with inverse of cell size!
  - too big: too many objects in one cell
  - sometimes, no cell size is good enough





Spatial trees

- Hierarchical structure: a tree
  - each node: a subpart of the 3D space
  - root: all the world
  - child nodes: partitions of the father
  - objects linked to leaves

University of Victoria | Computer Science

kD-tree

k = 2

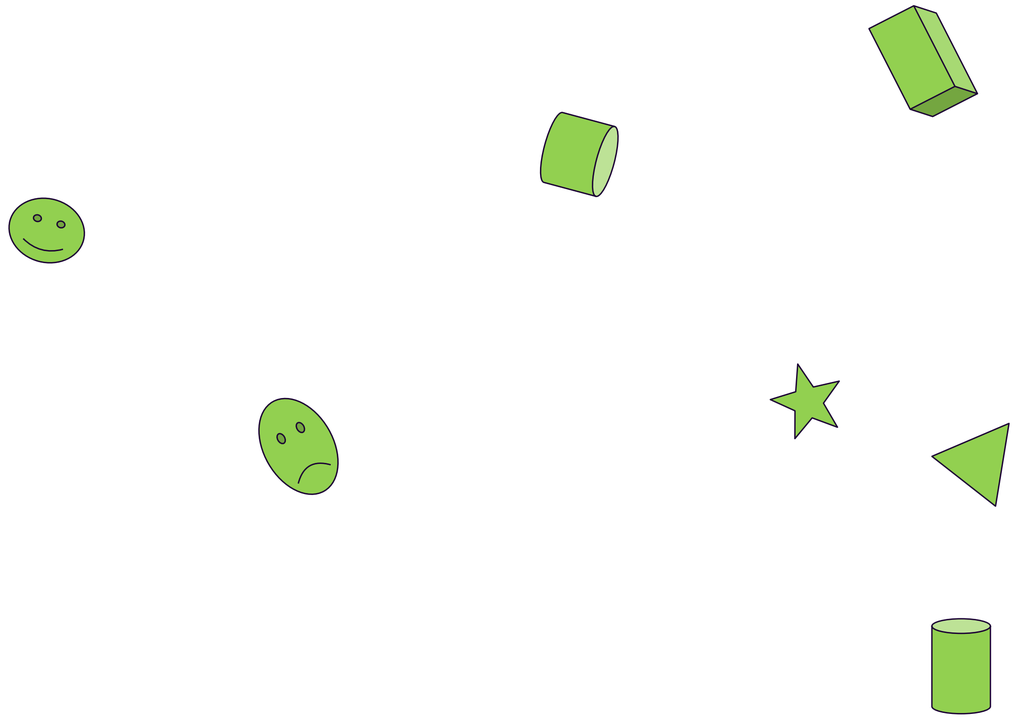
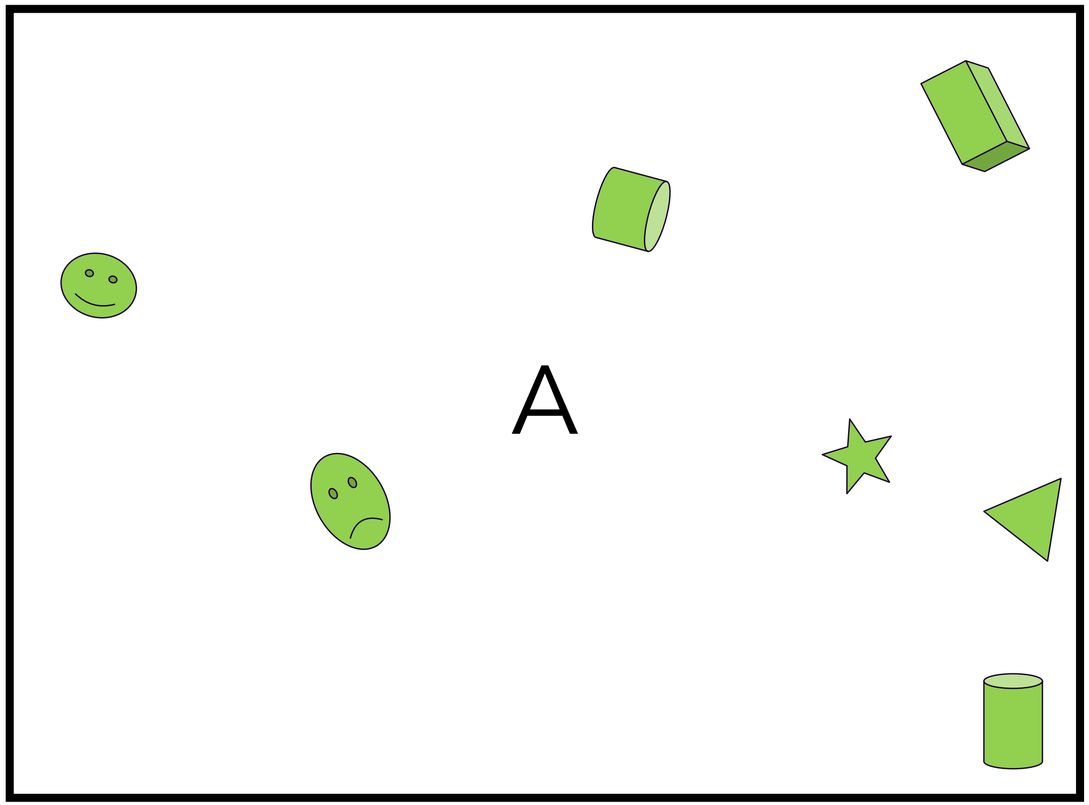


Image Copyright: Marco Tarini

kD-tree

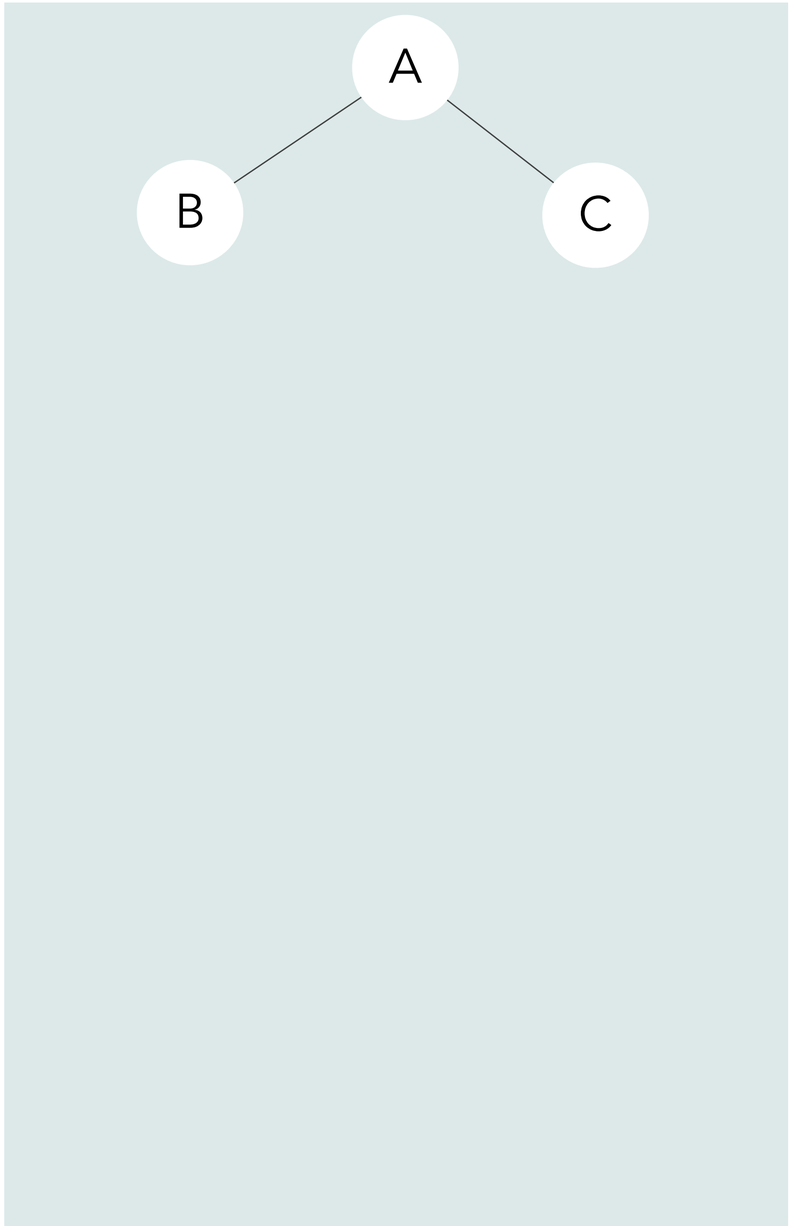
k = 2



A

Image Copyright: Marco Tarini

kD-tree



k = 2

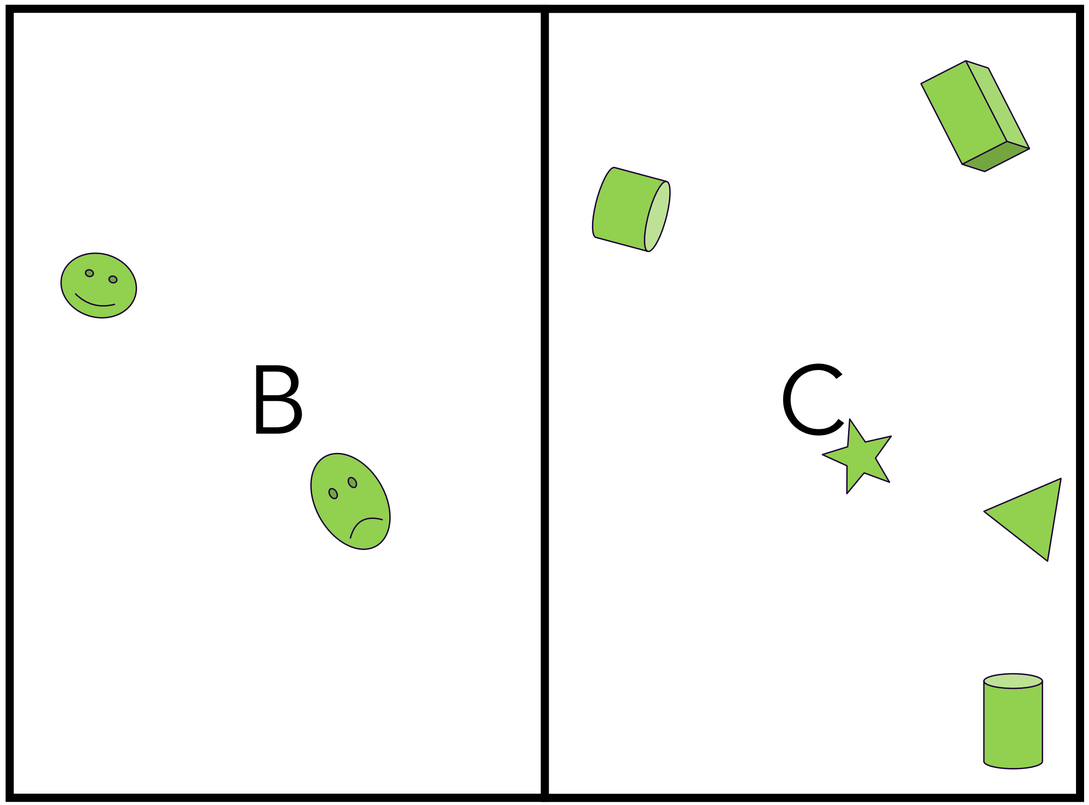
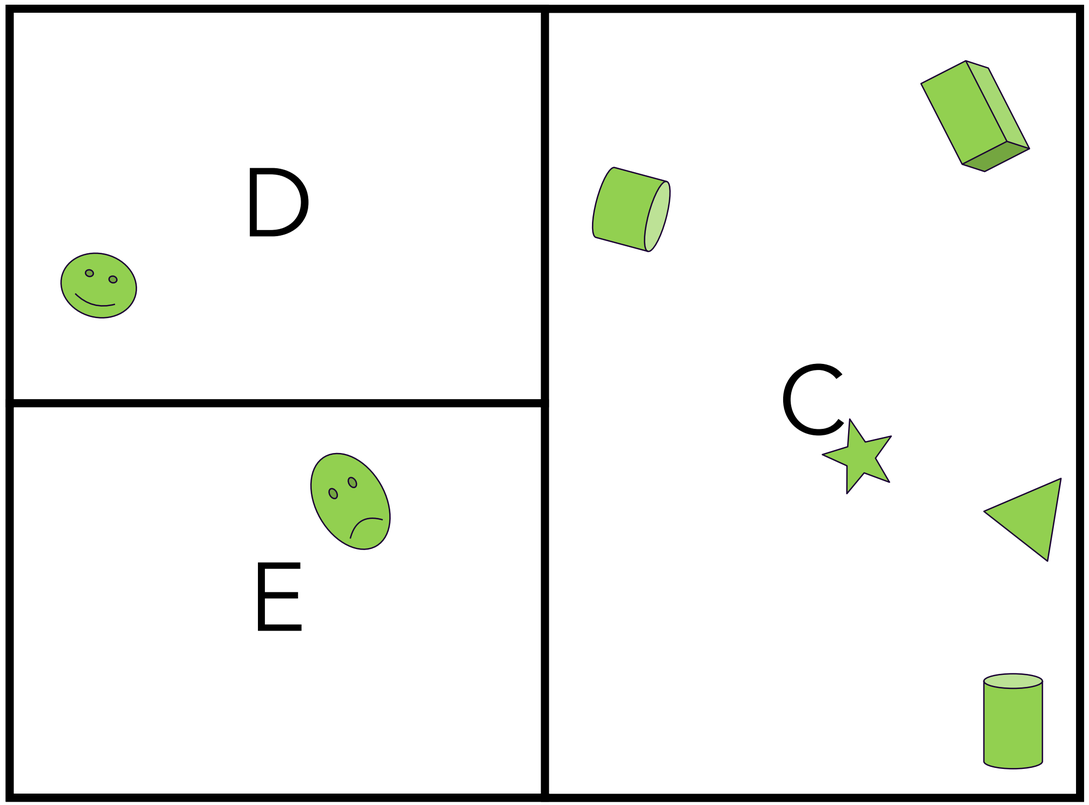


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree

k = 2



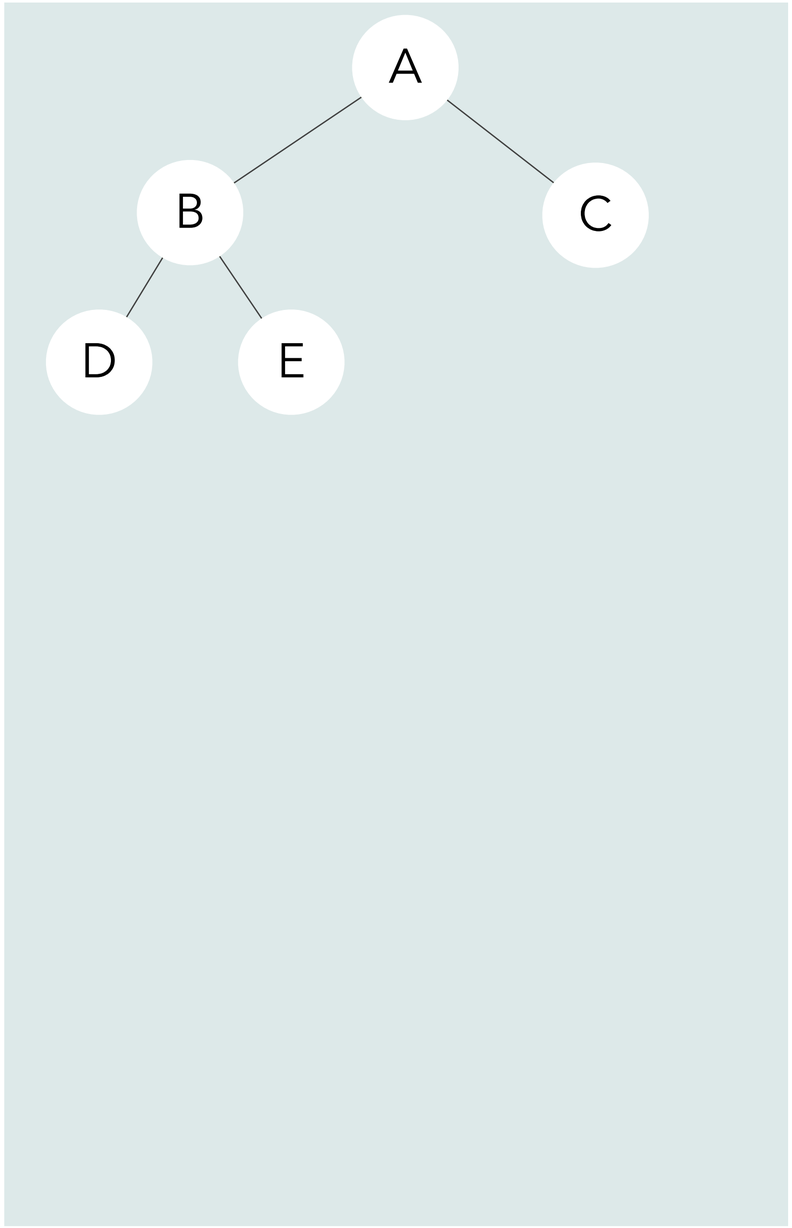
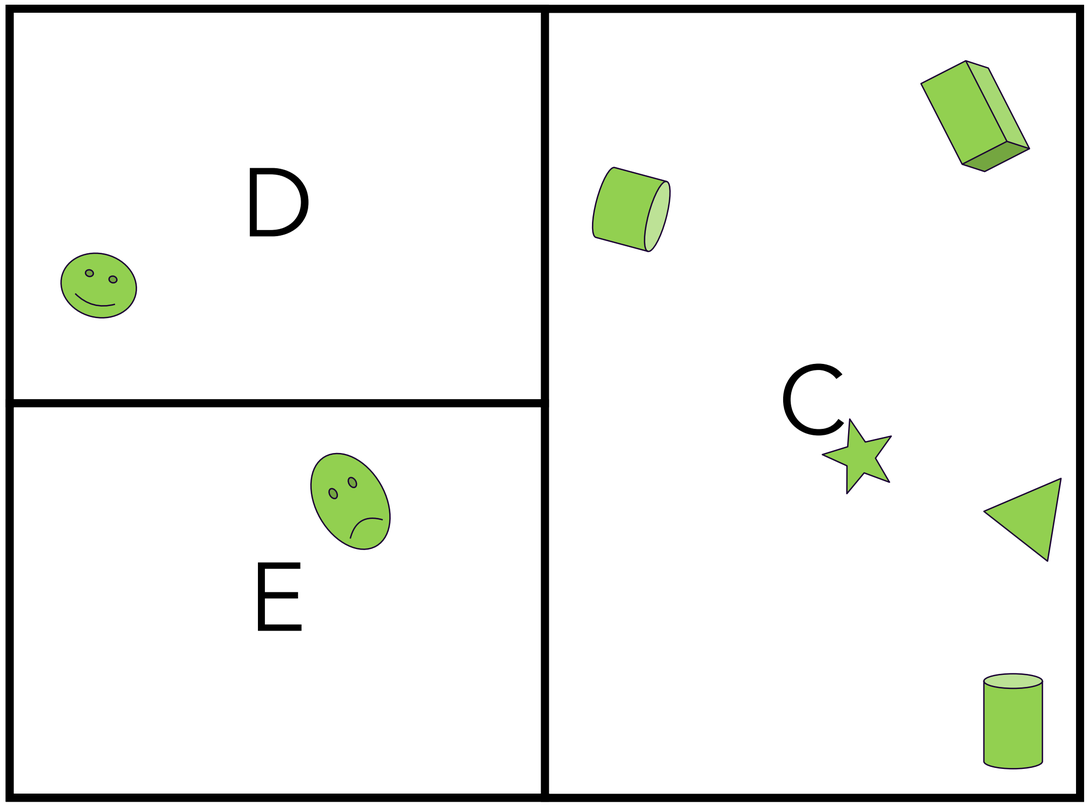


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree

k = 2



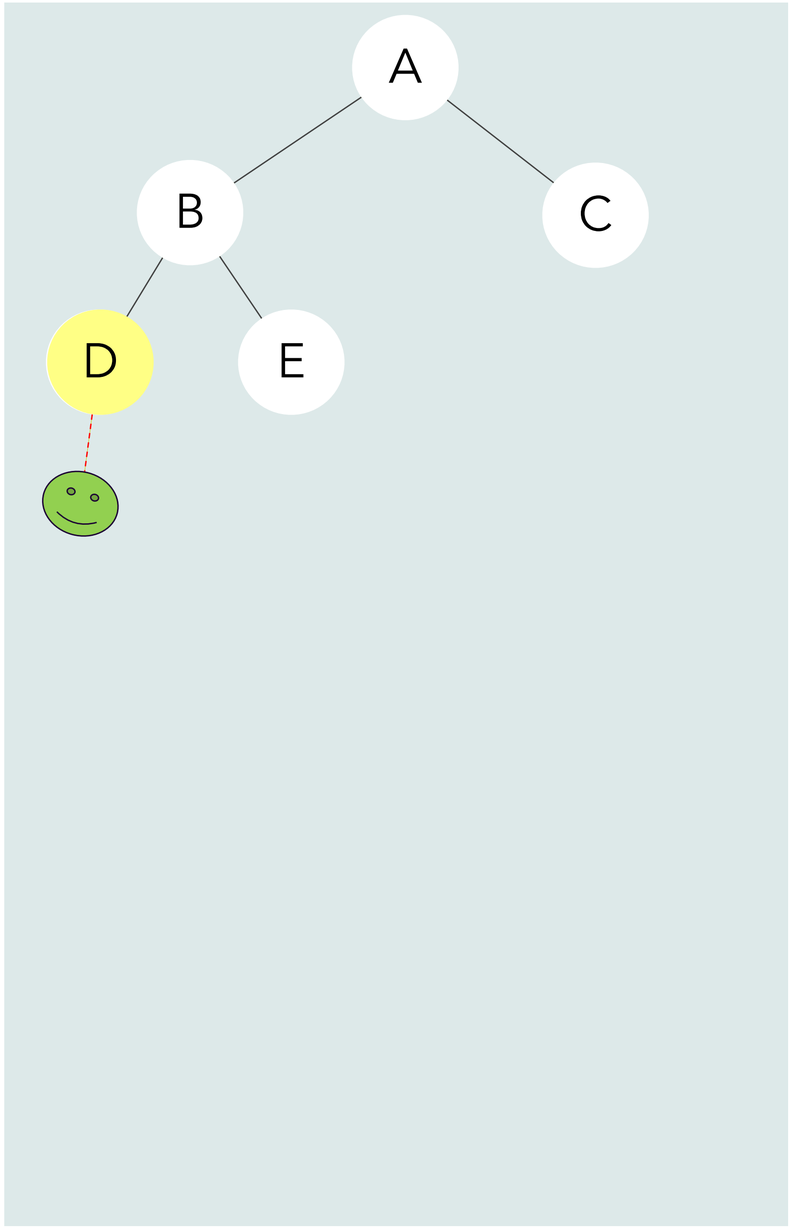
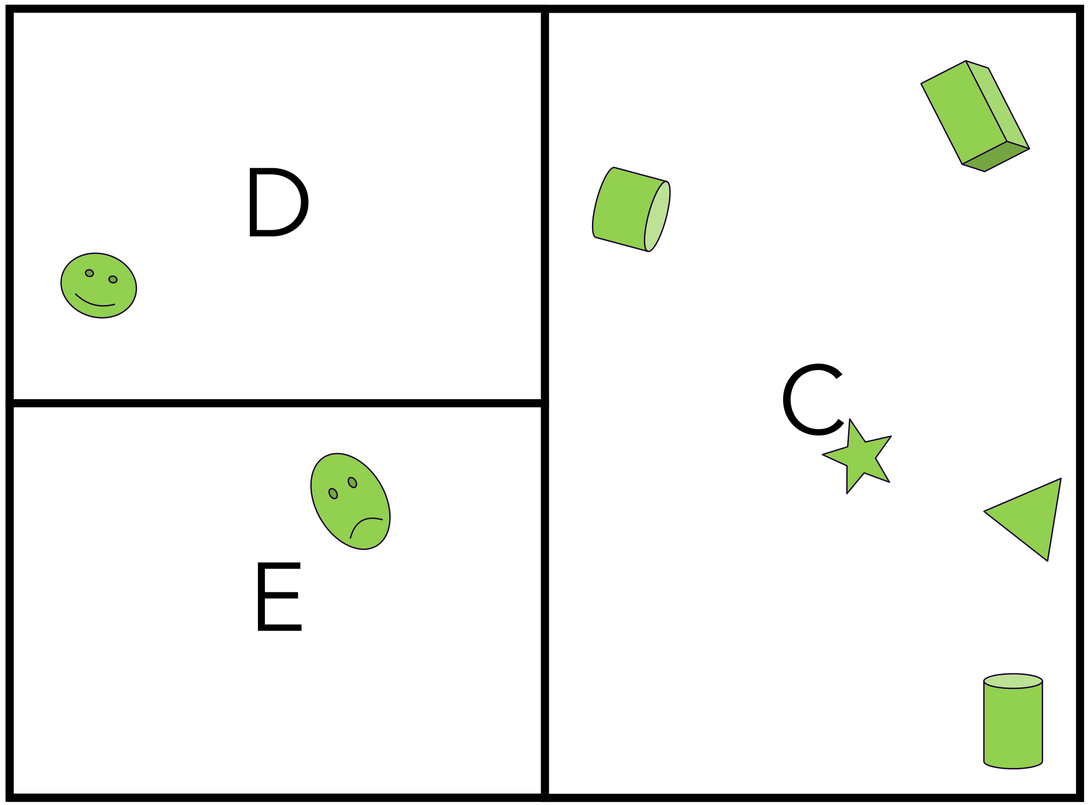


Image Copyright: Marco Tarini

kD-tree

k = 2



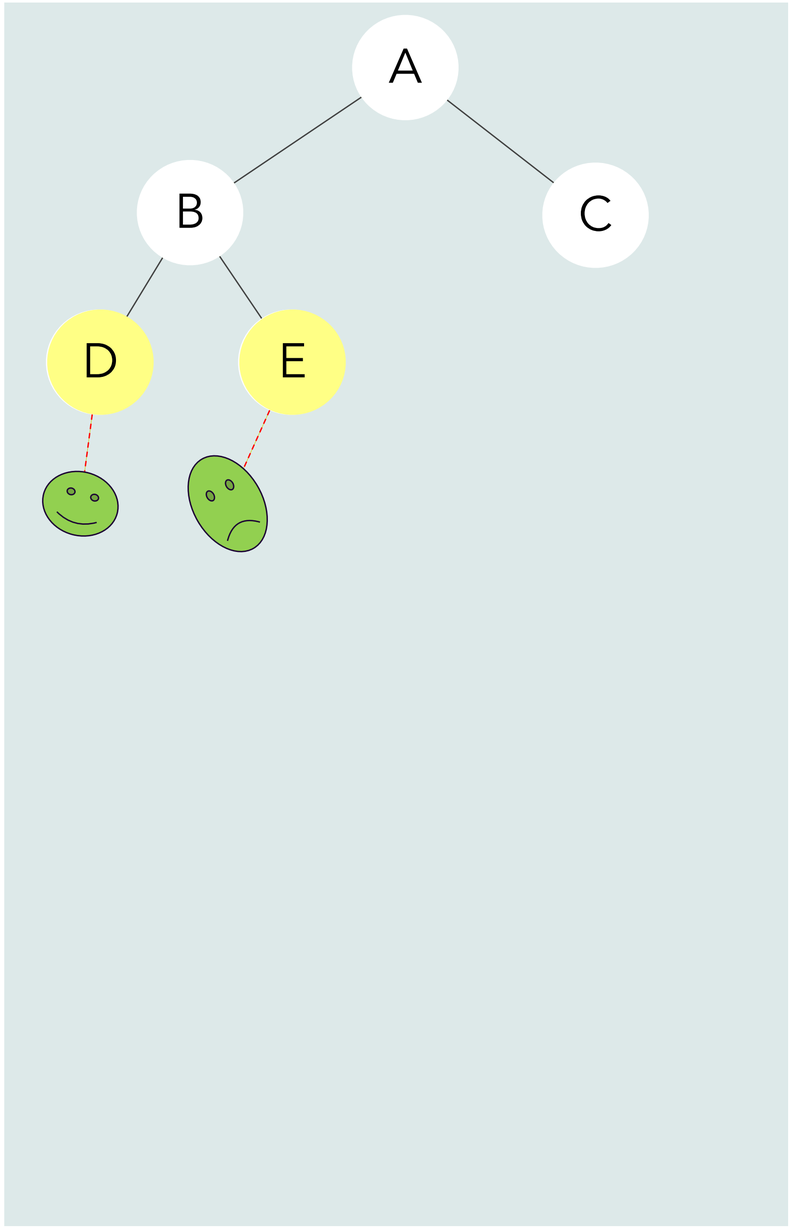
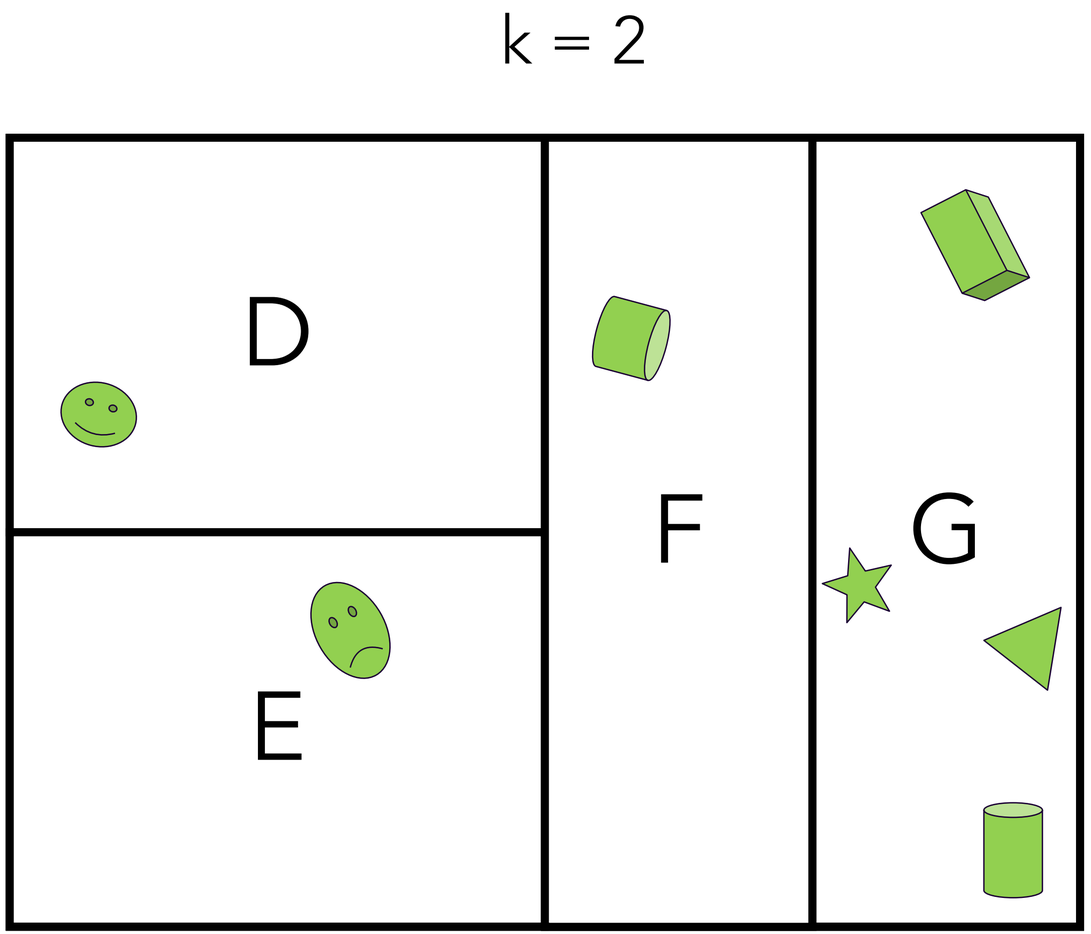


Image Copyright: Marco Tarini

kD-tree



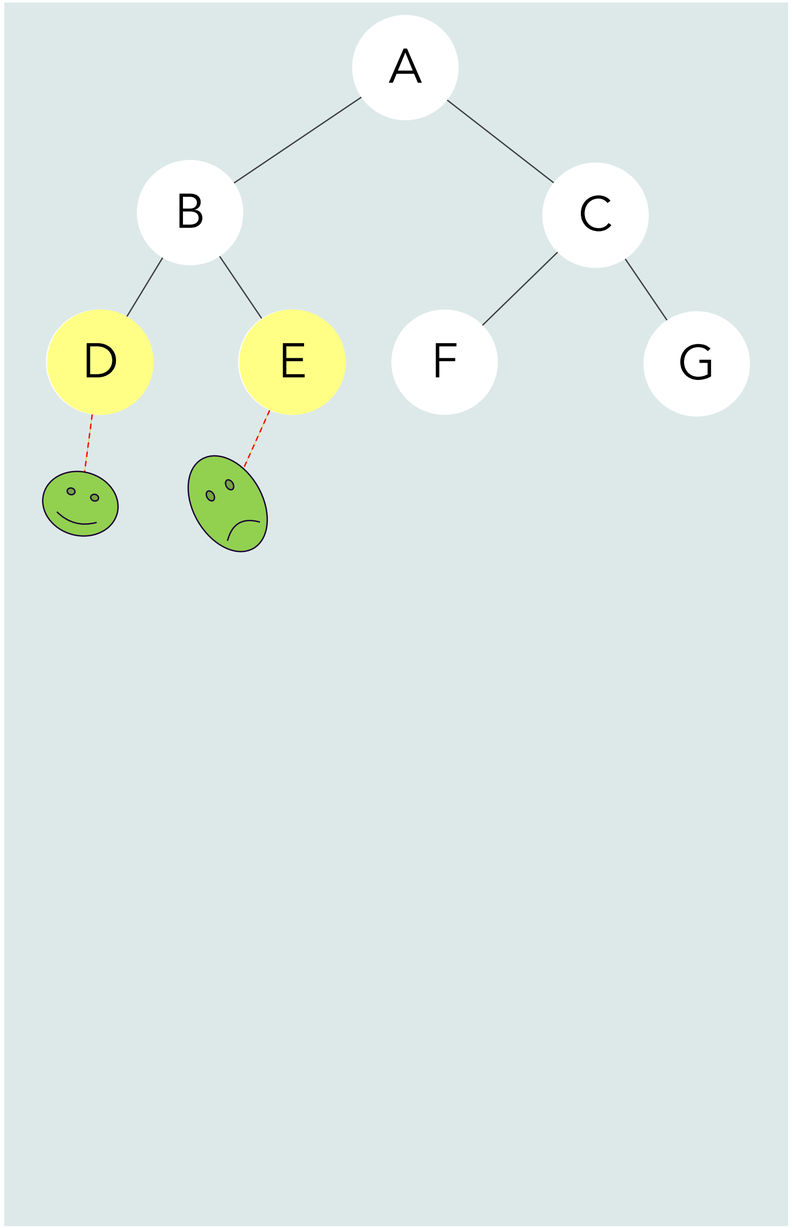
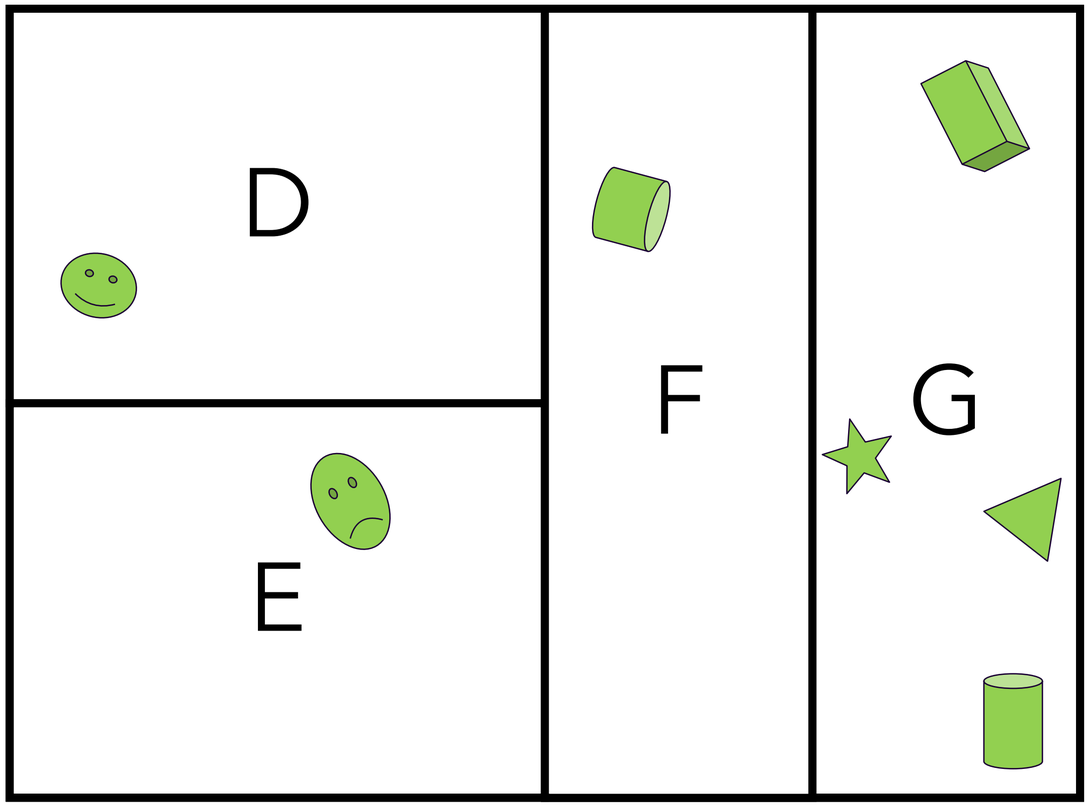


Image Copyright: Marco Tarini

kD-tree

k = 2



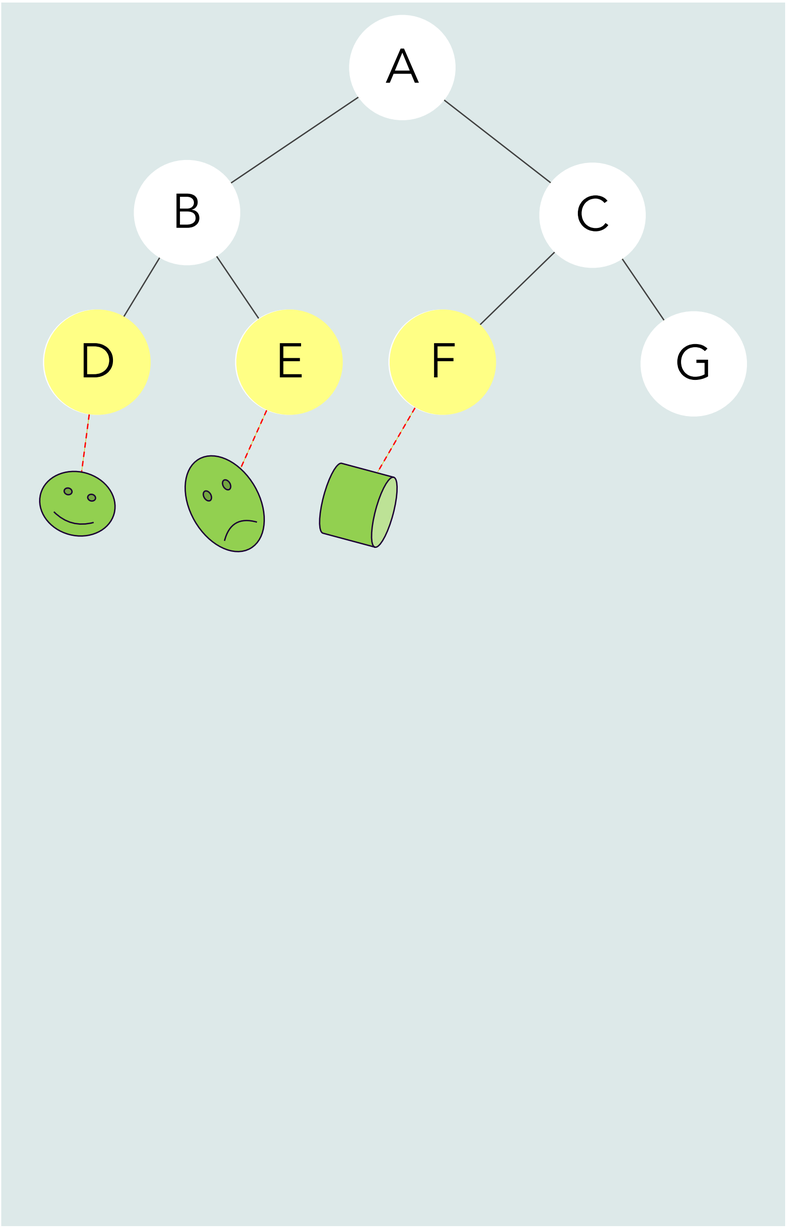
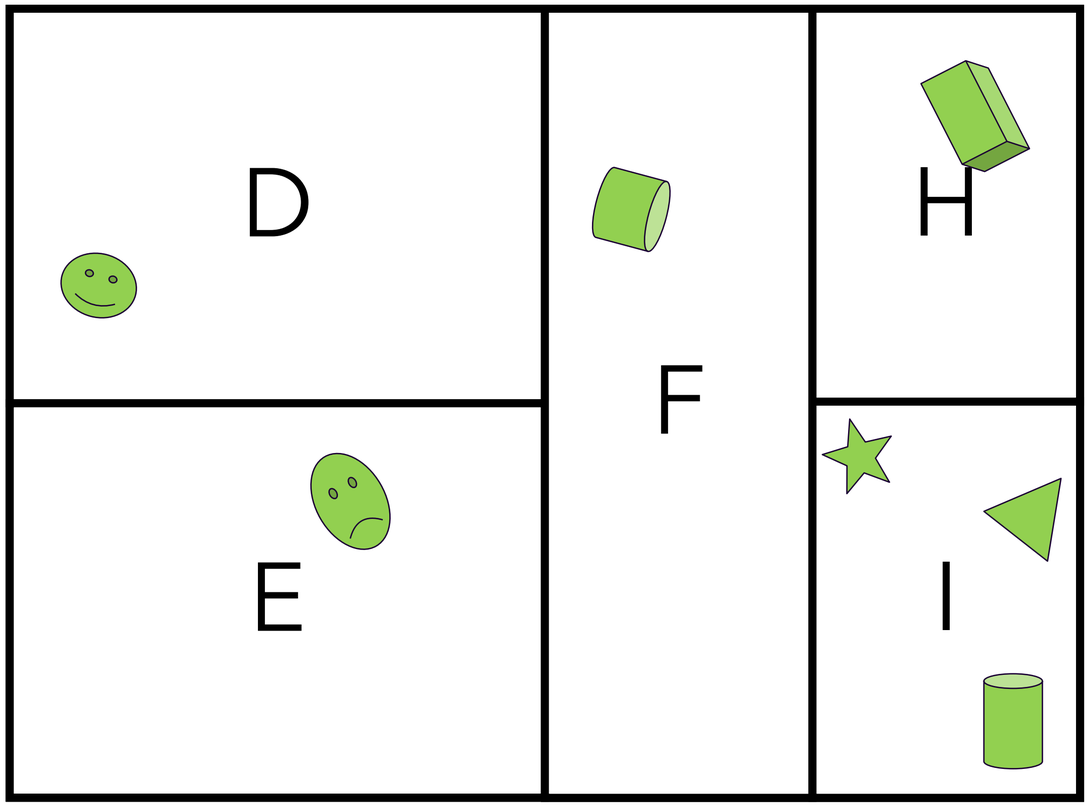


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree

k = 2



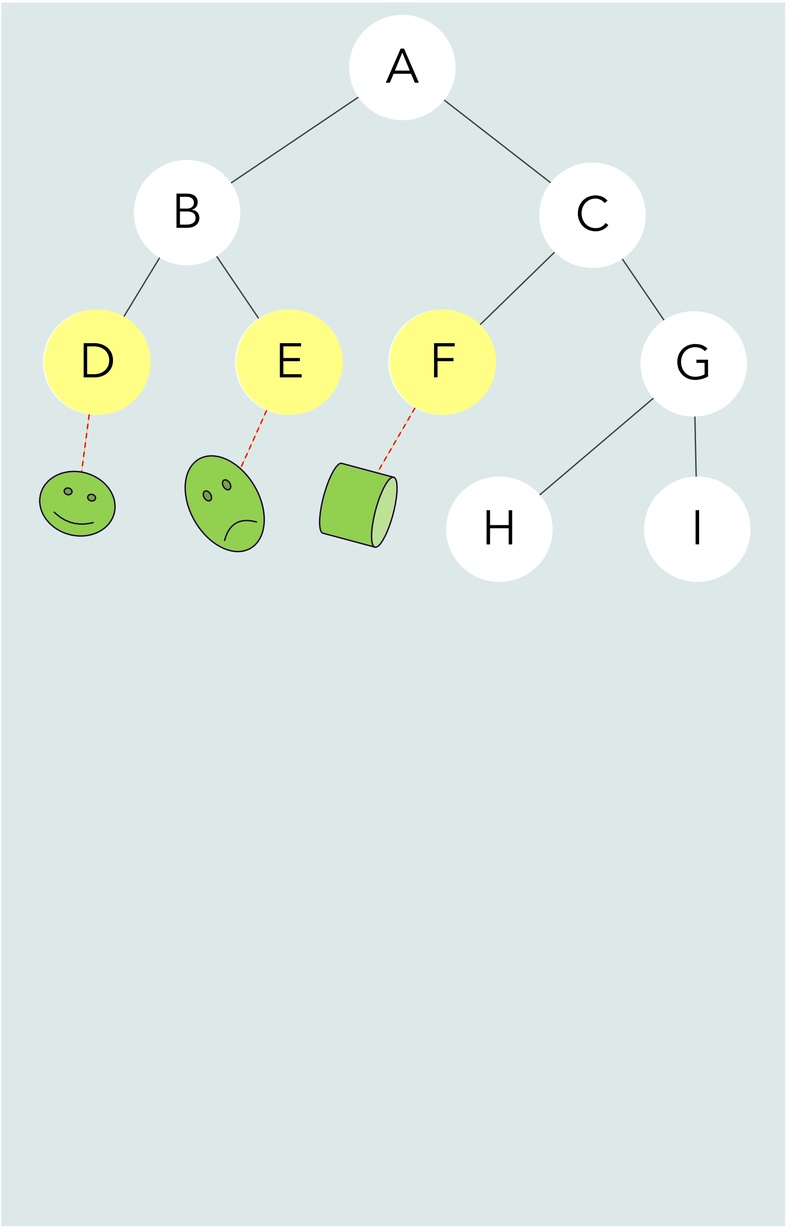
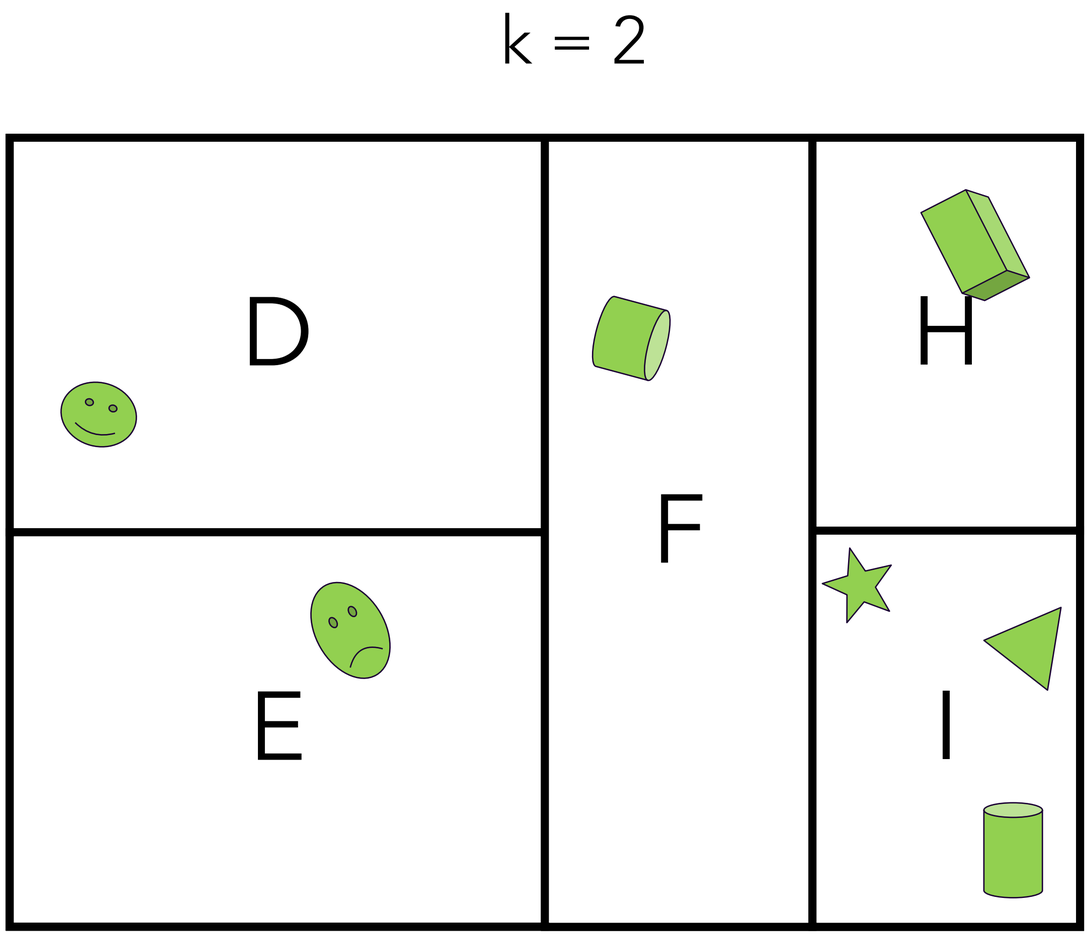


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



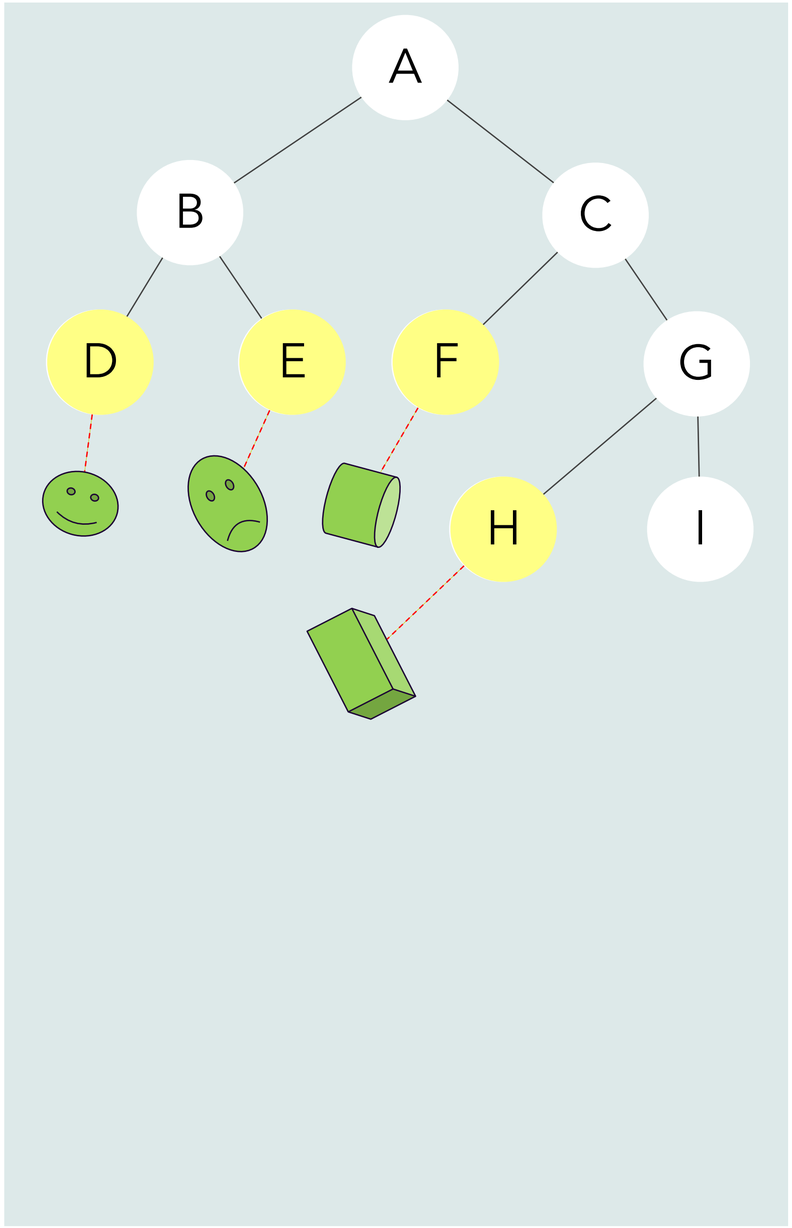
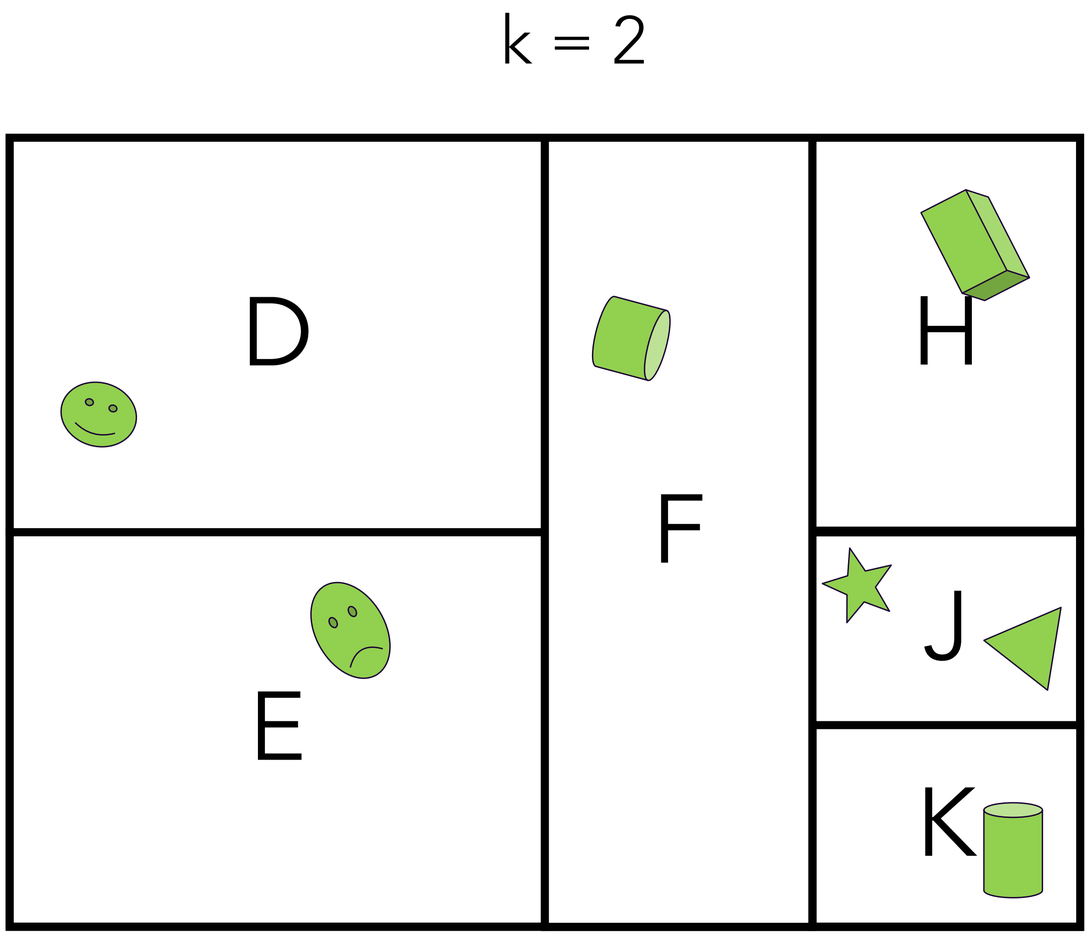


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



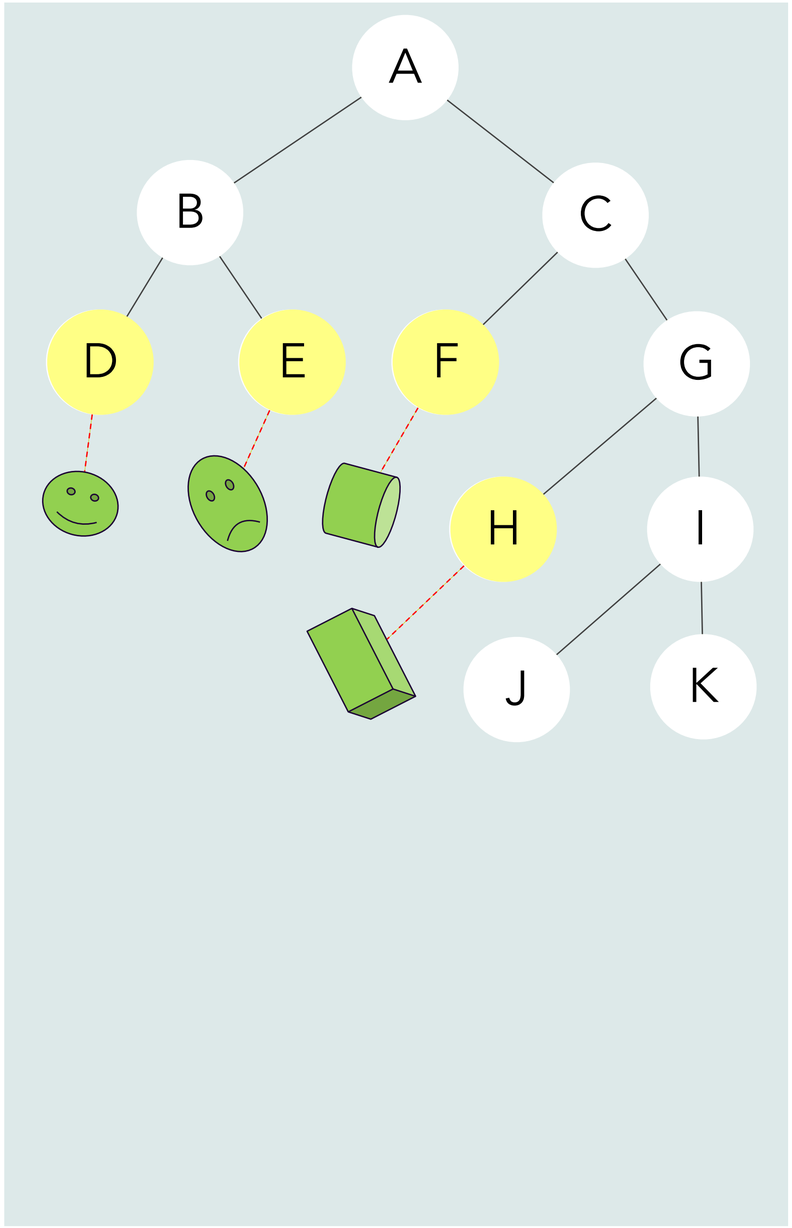
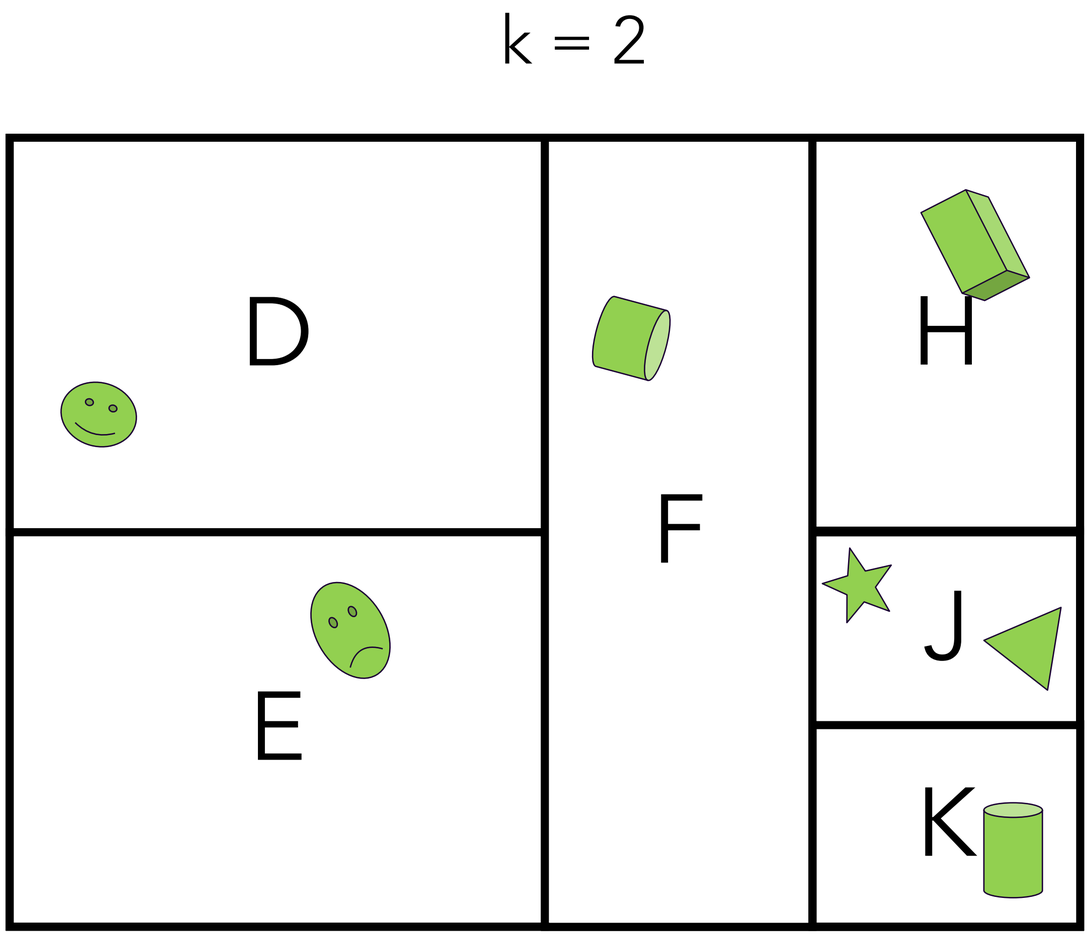


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



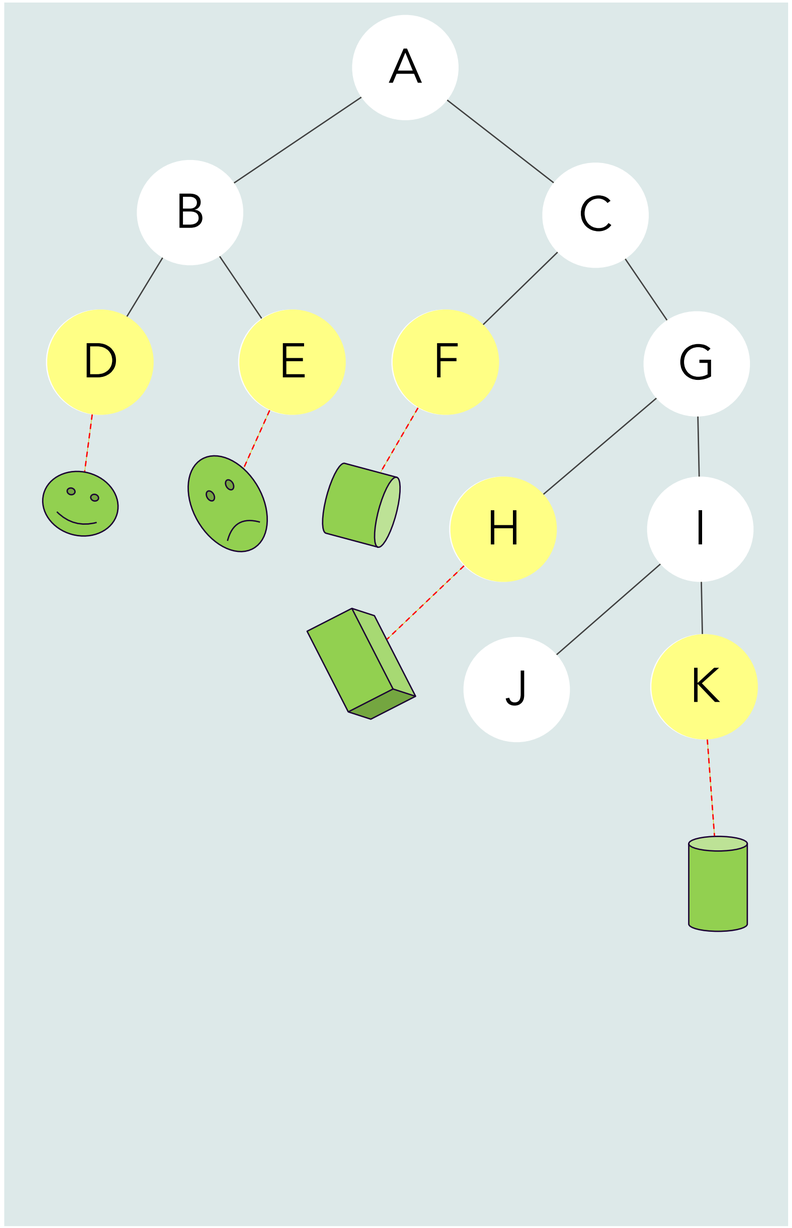
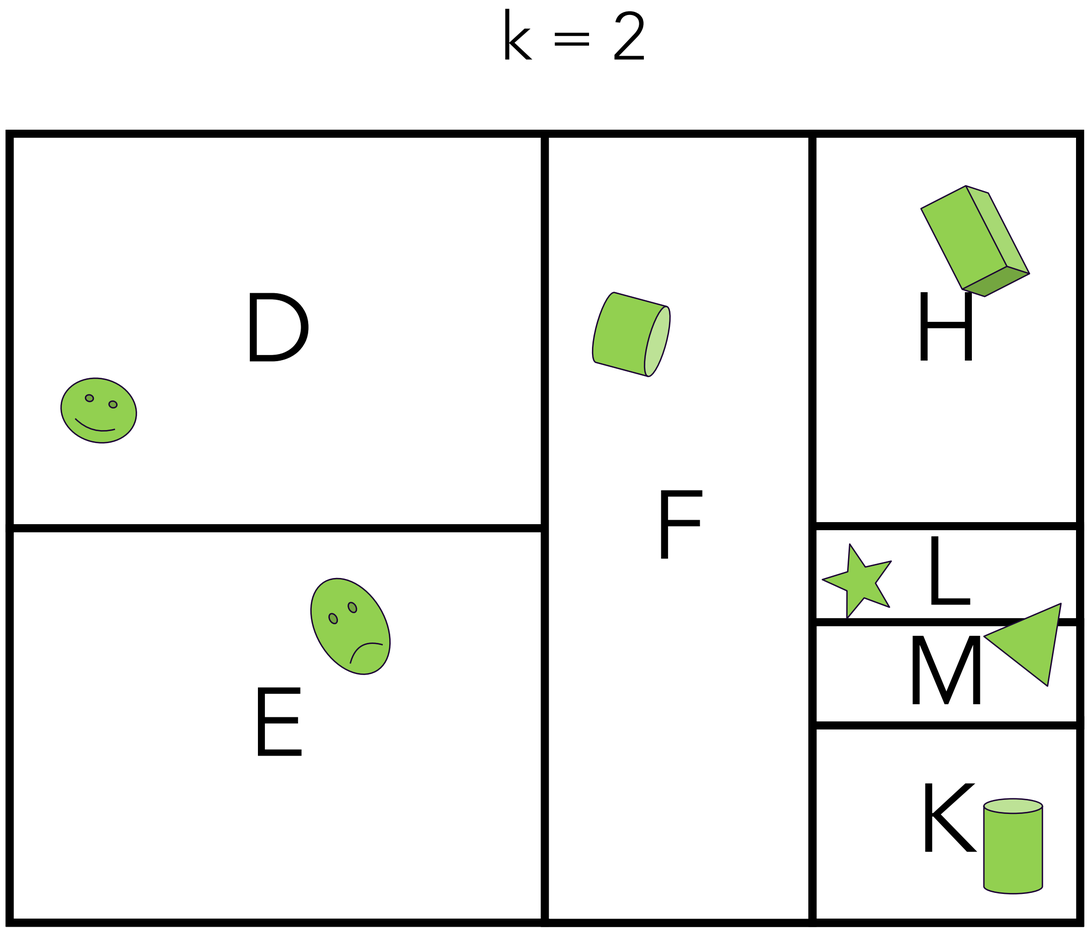


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



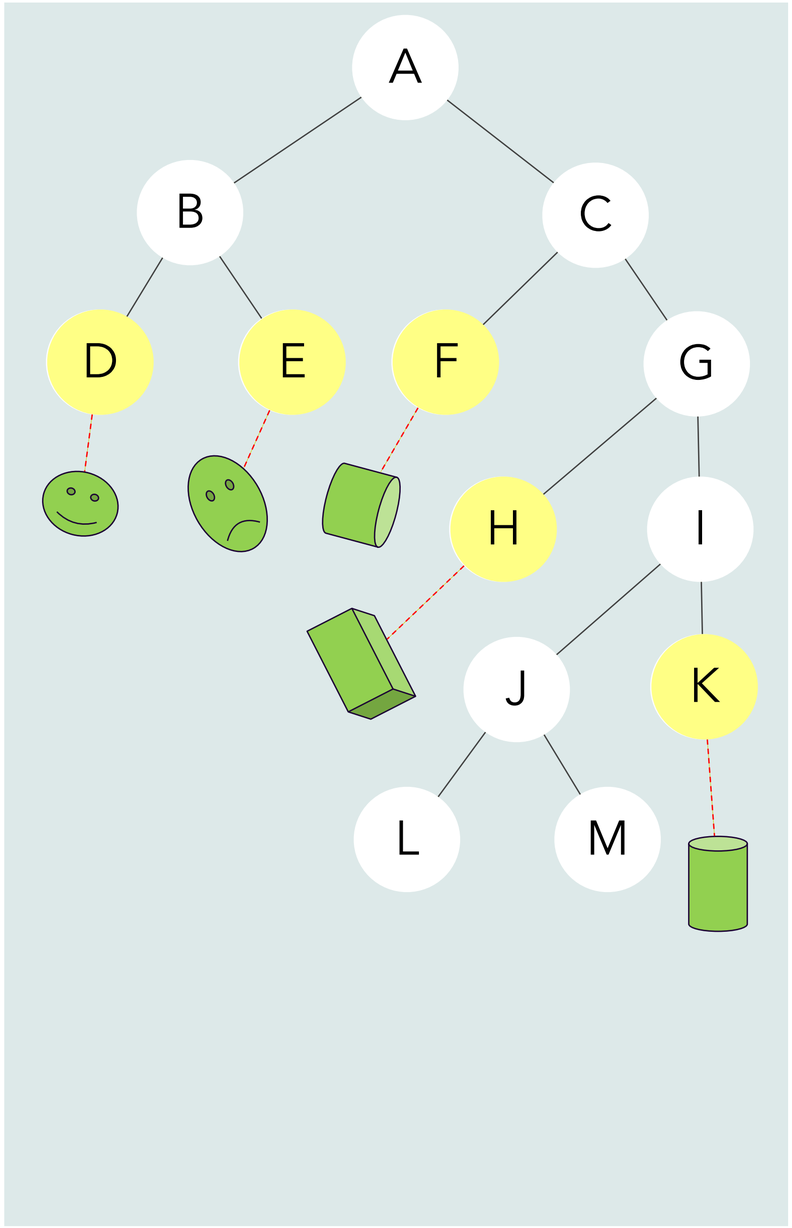
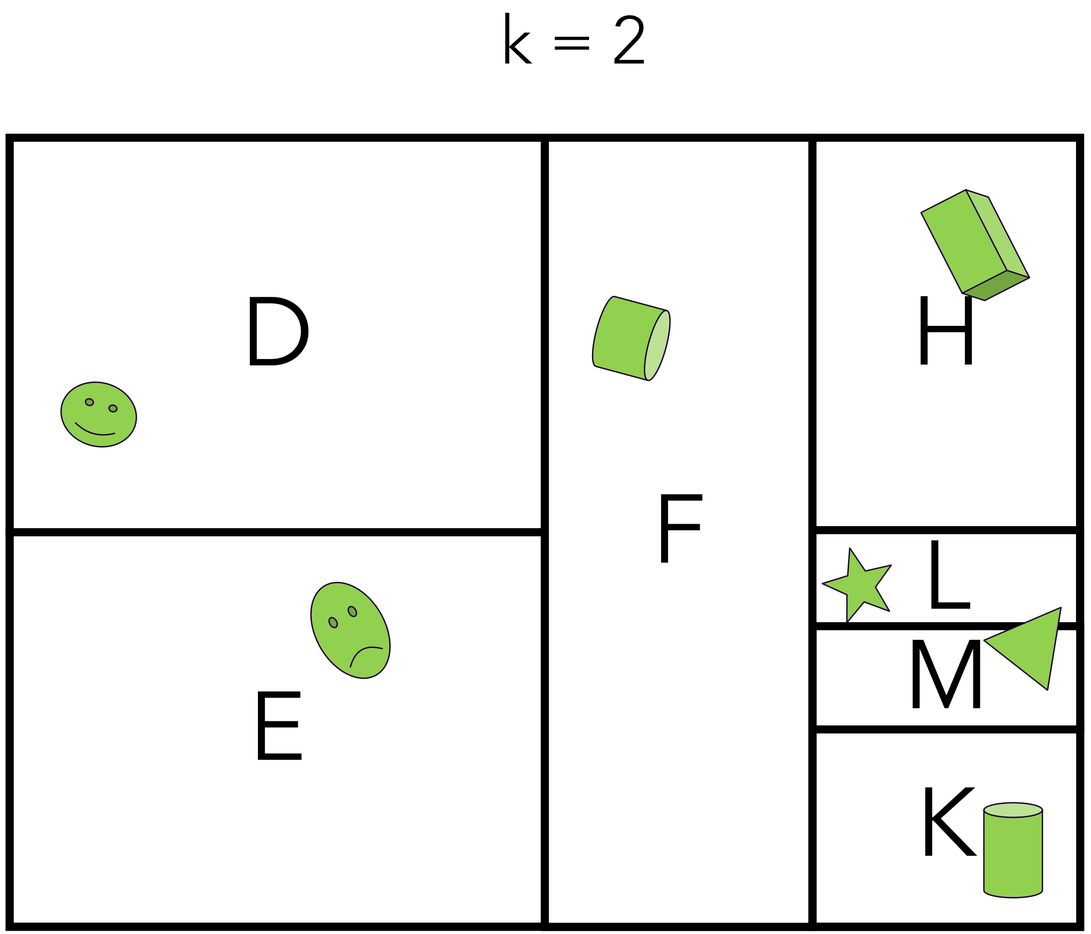


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



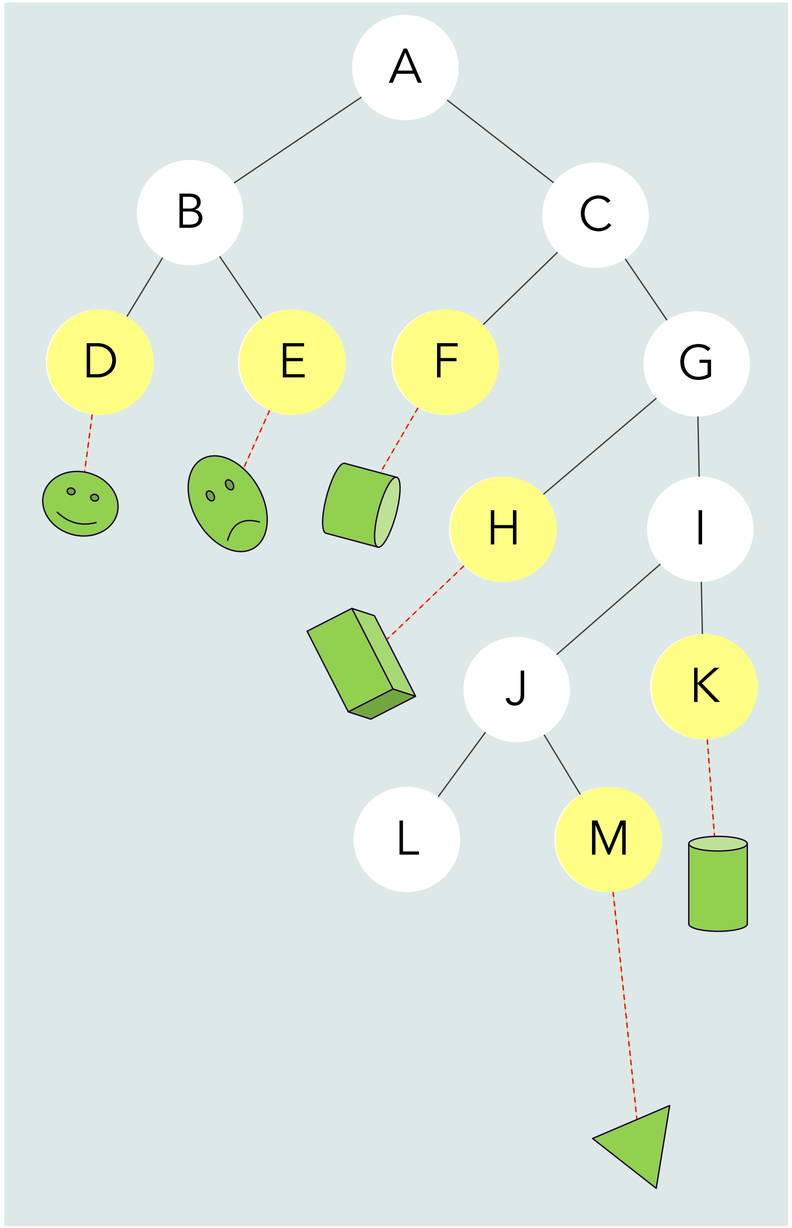
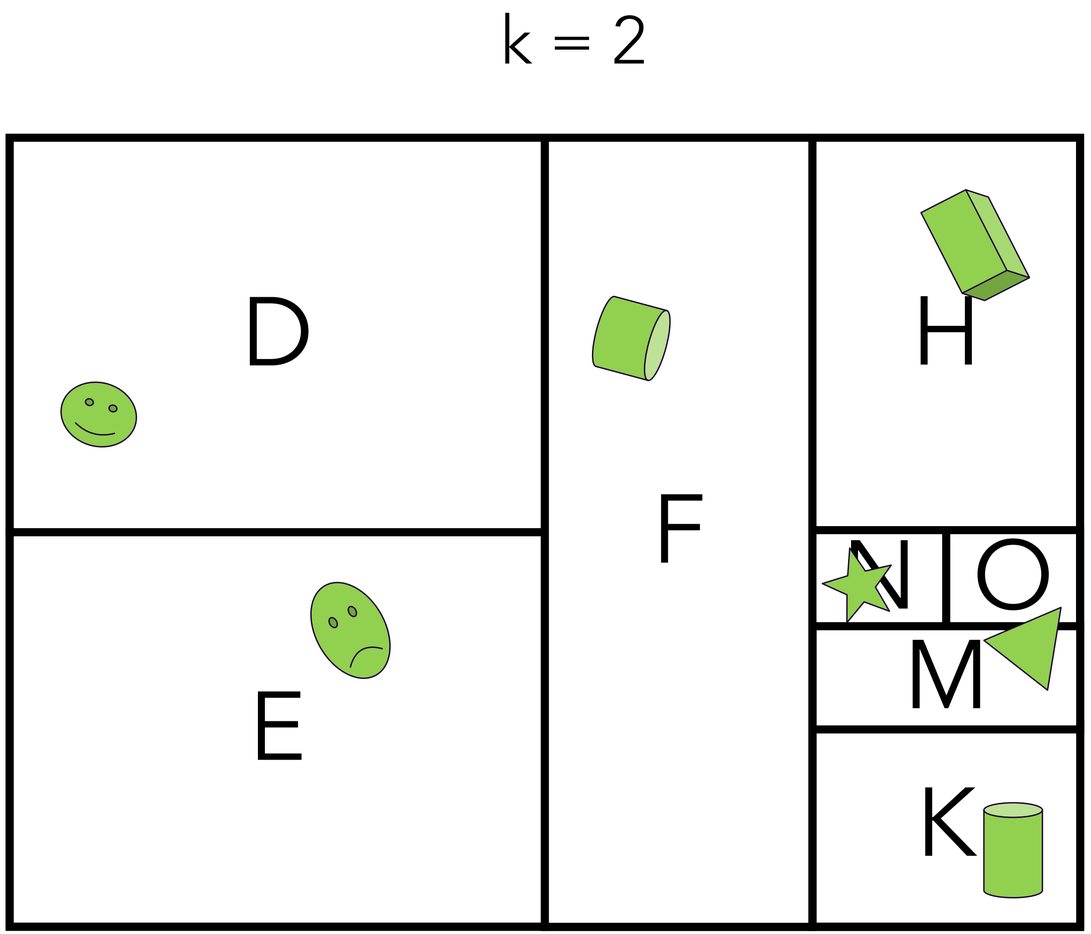


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



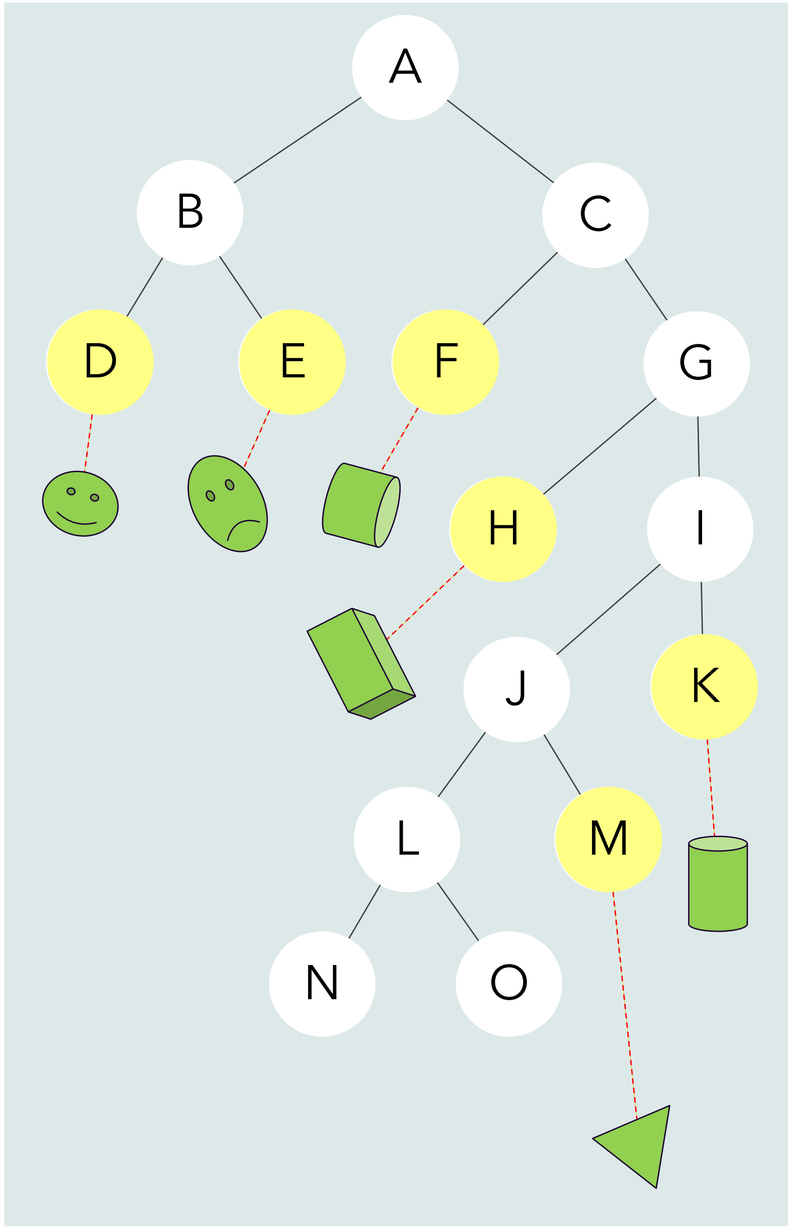
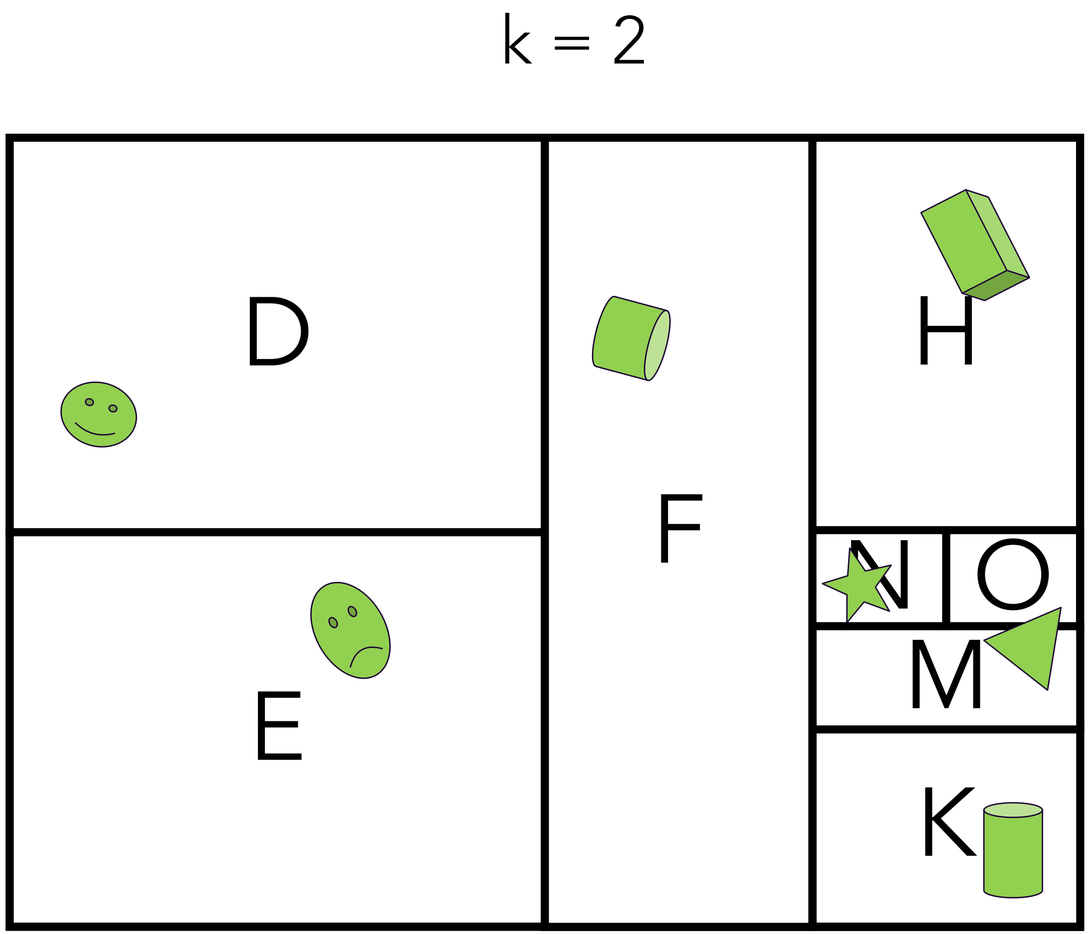


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



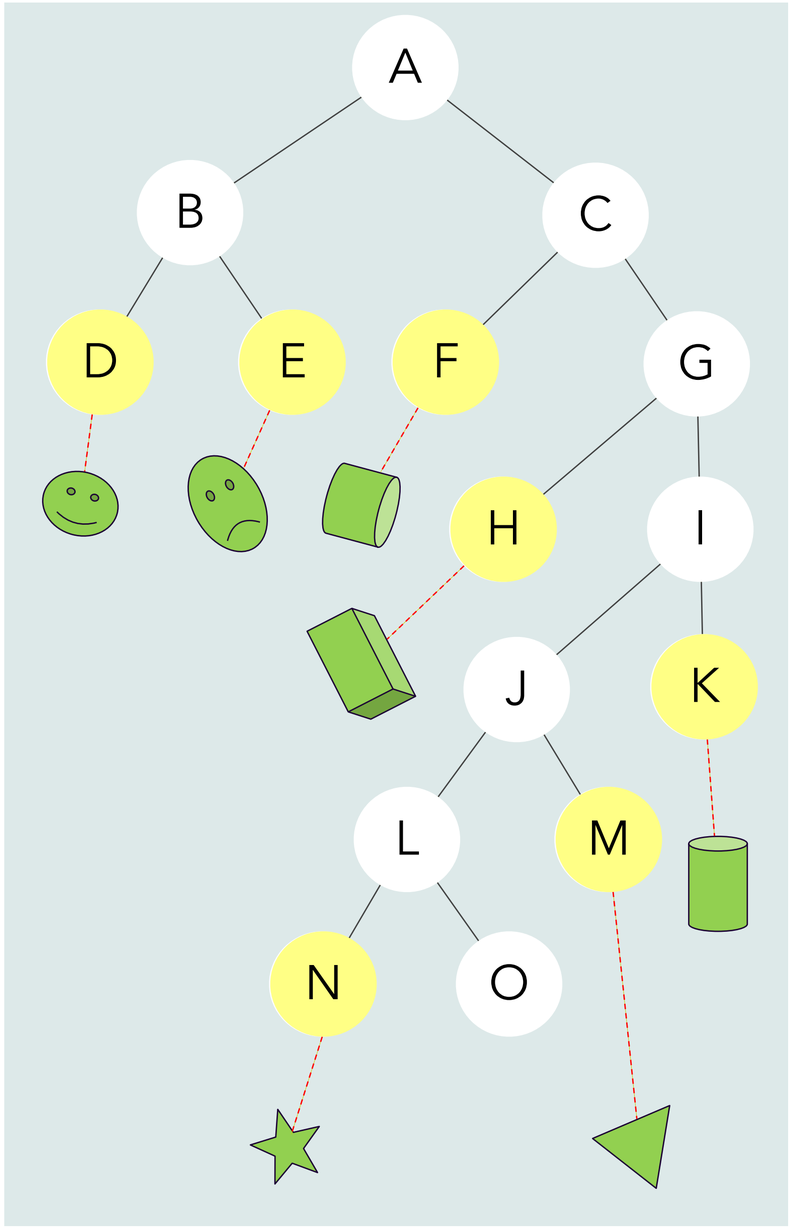
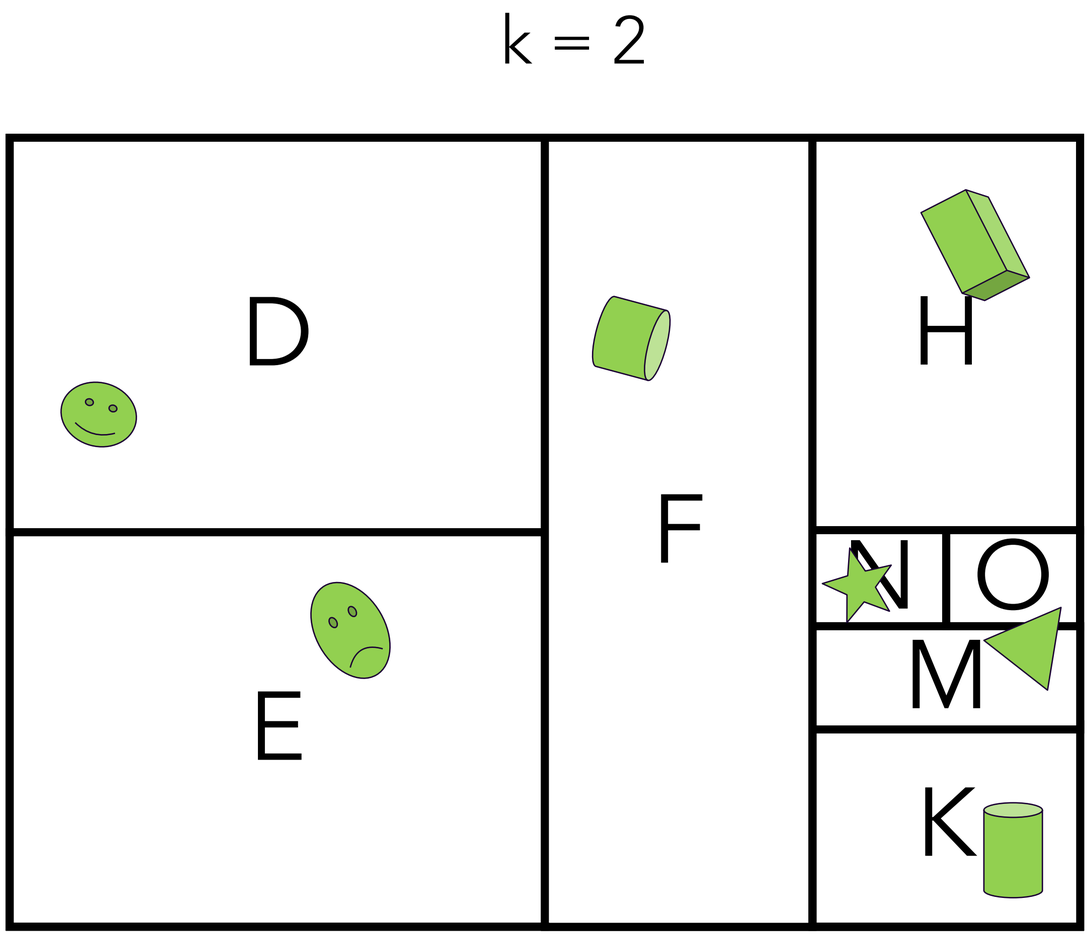


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



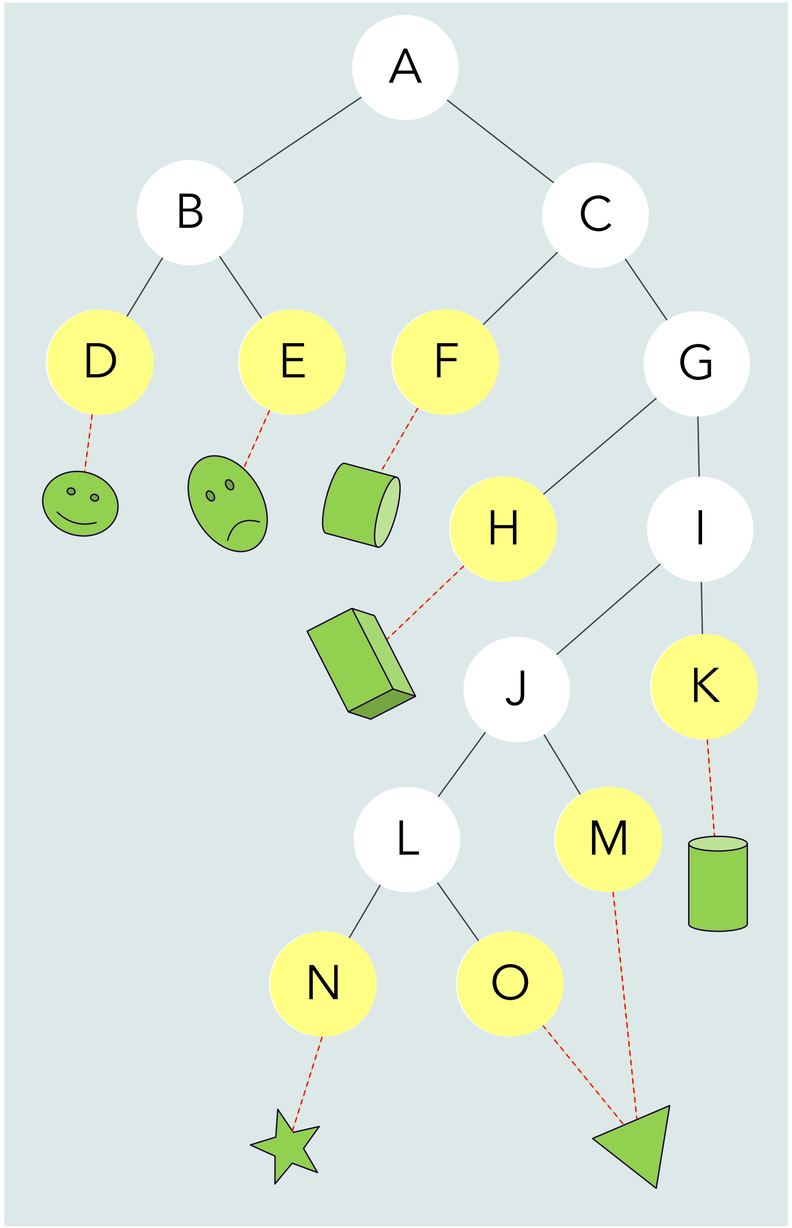
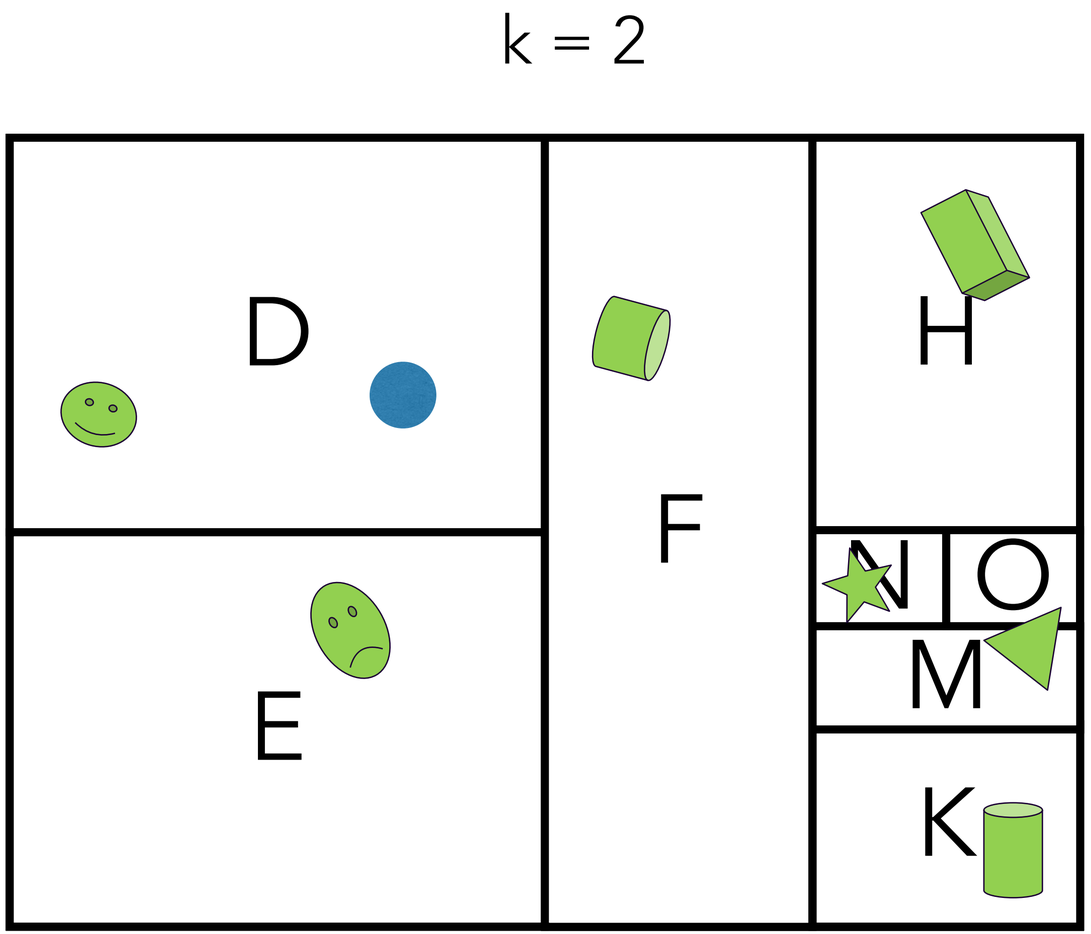


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



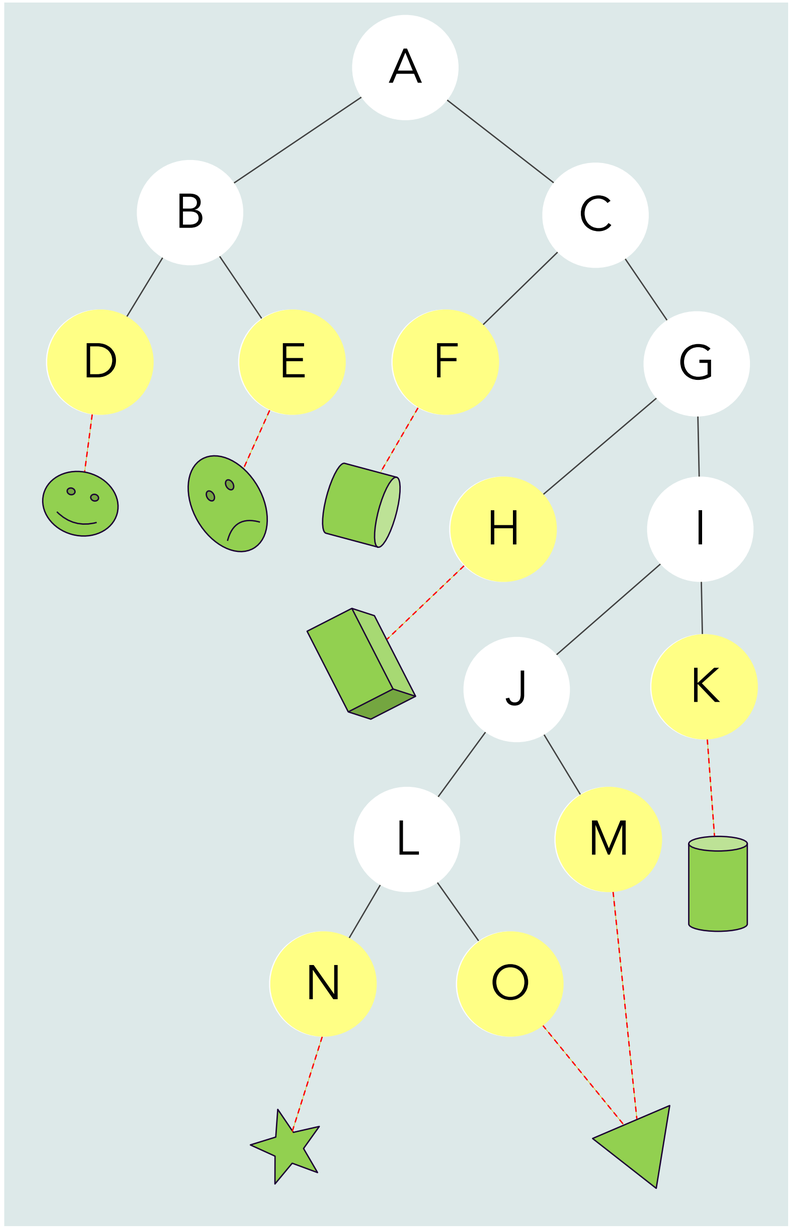
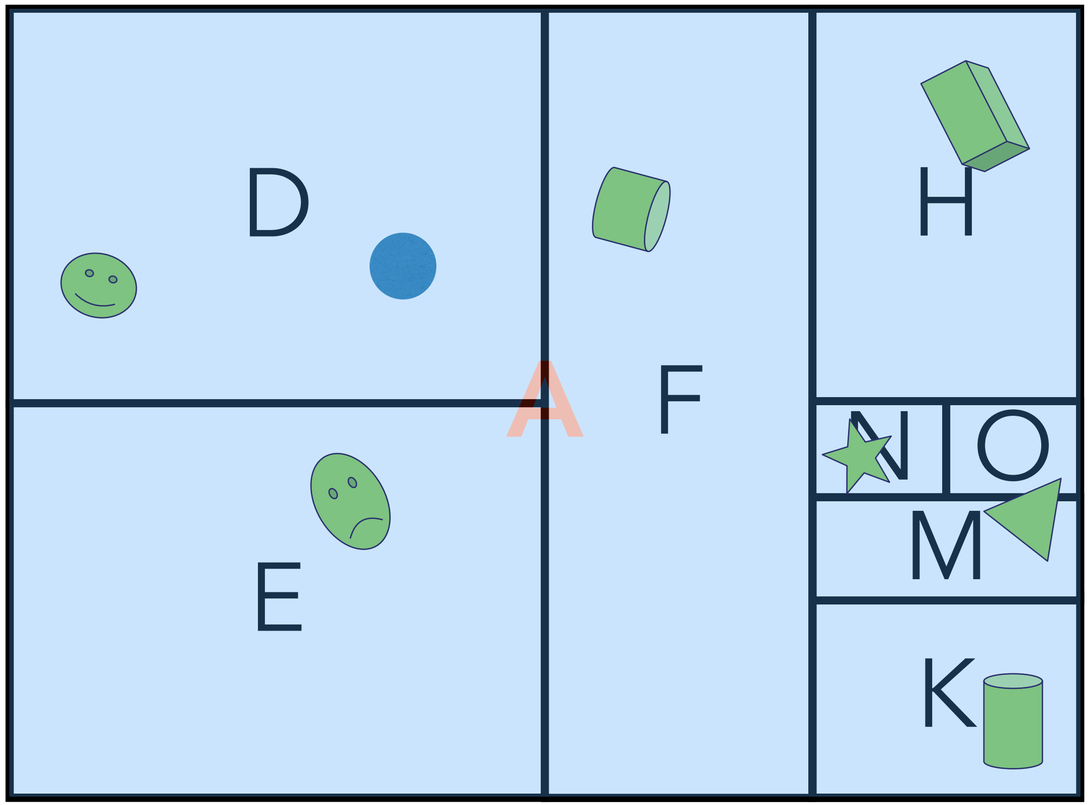


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree

k = 2



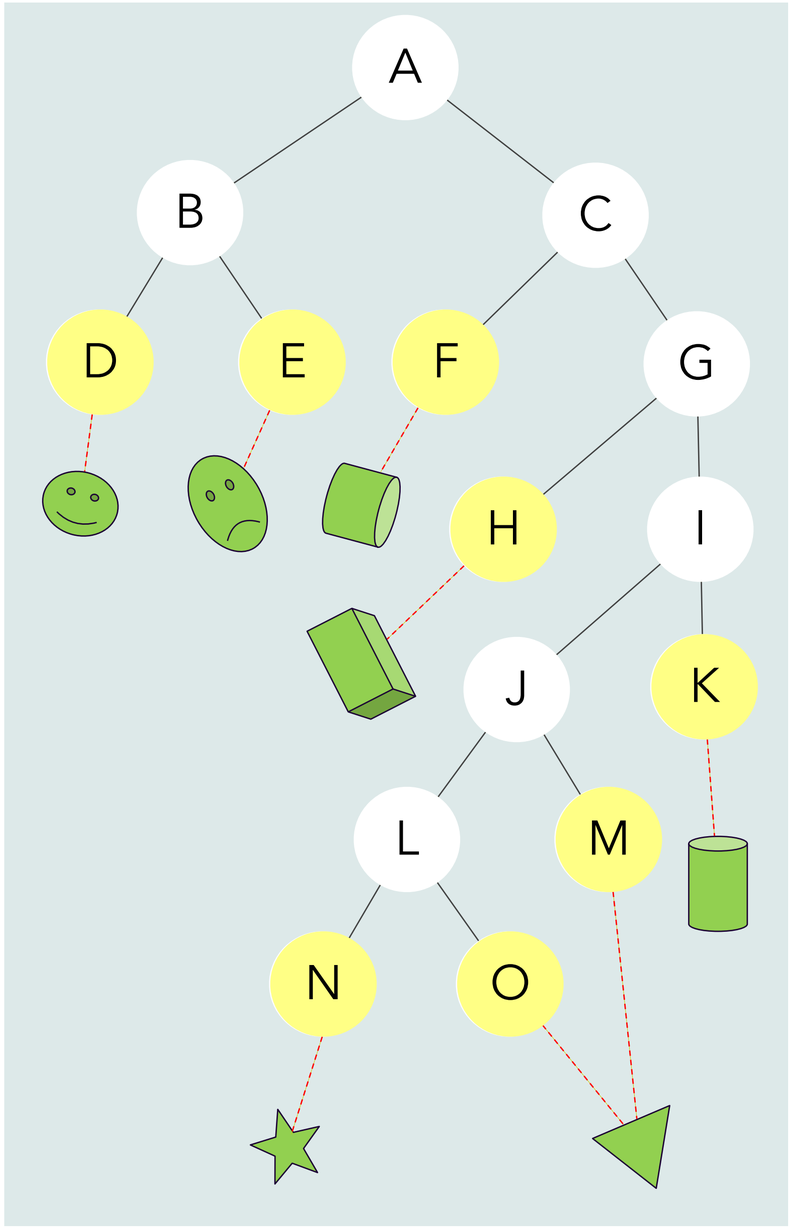
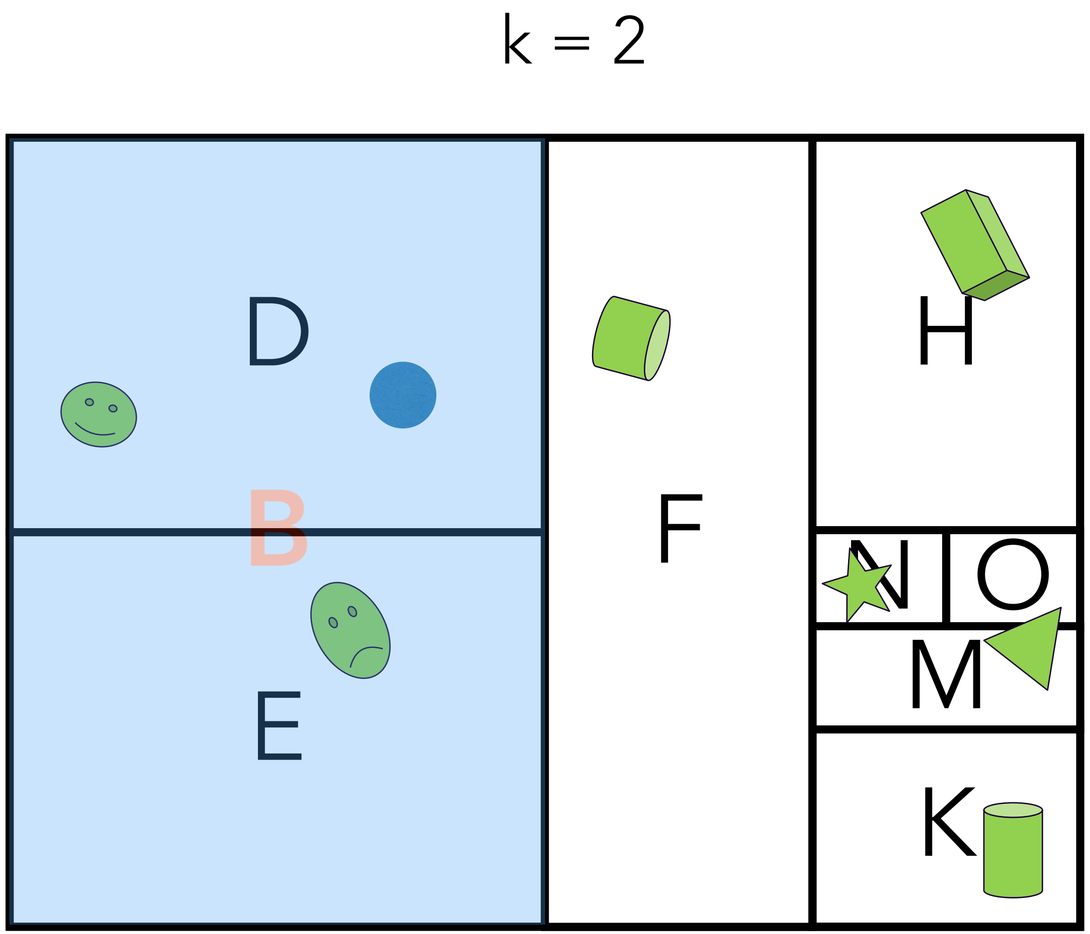


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



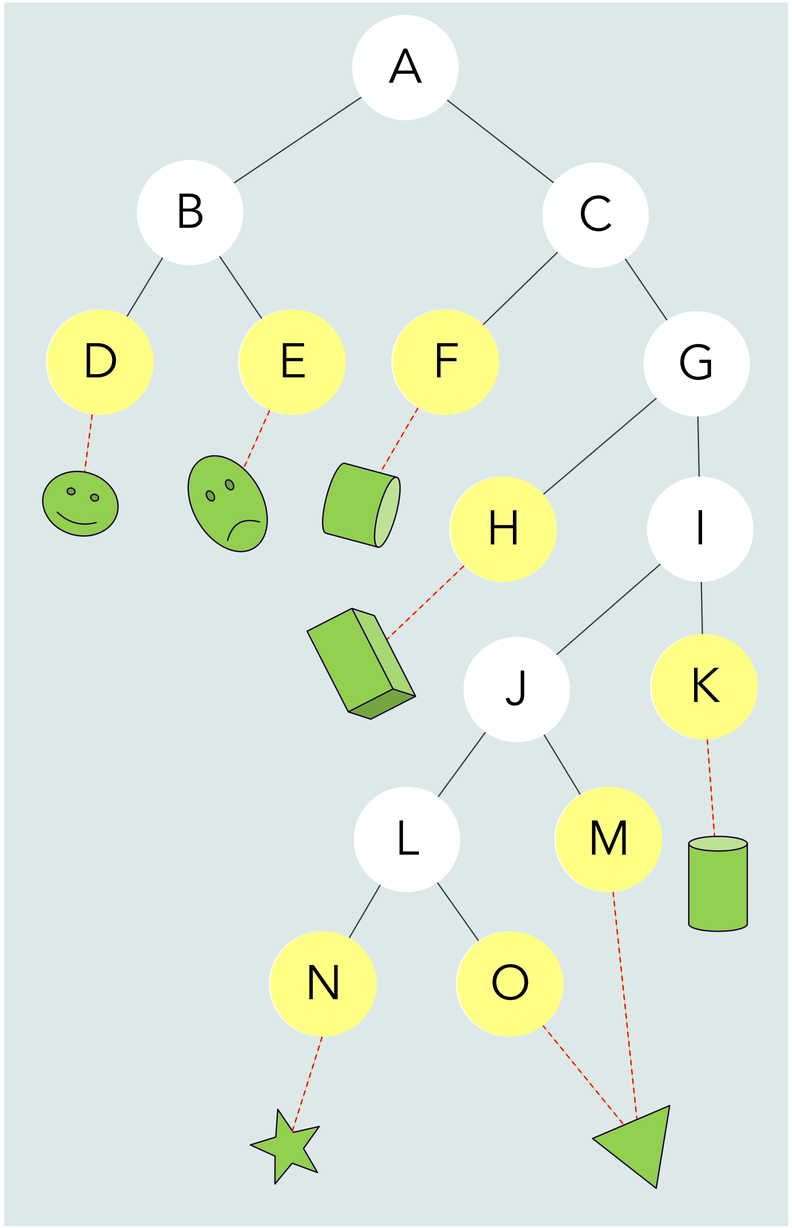
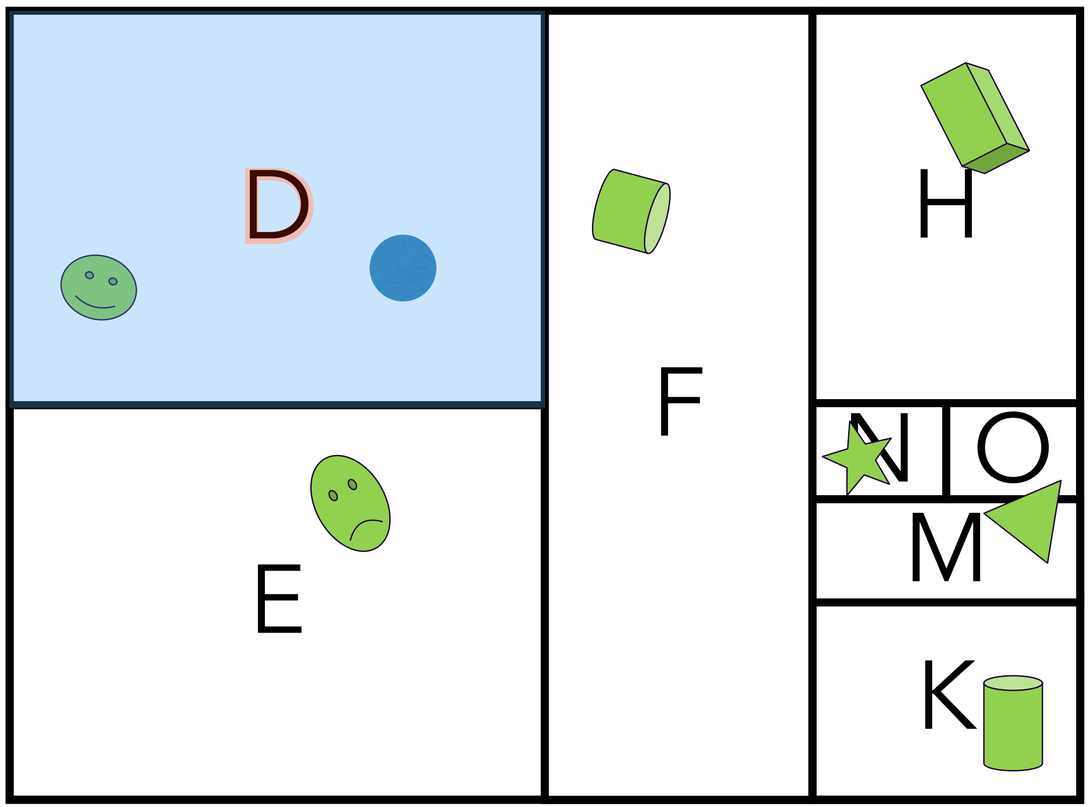


Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-tree



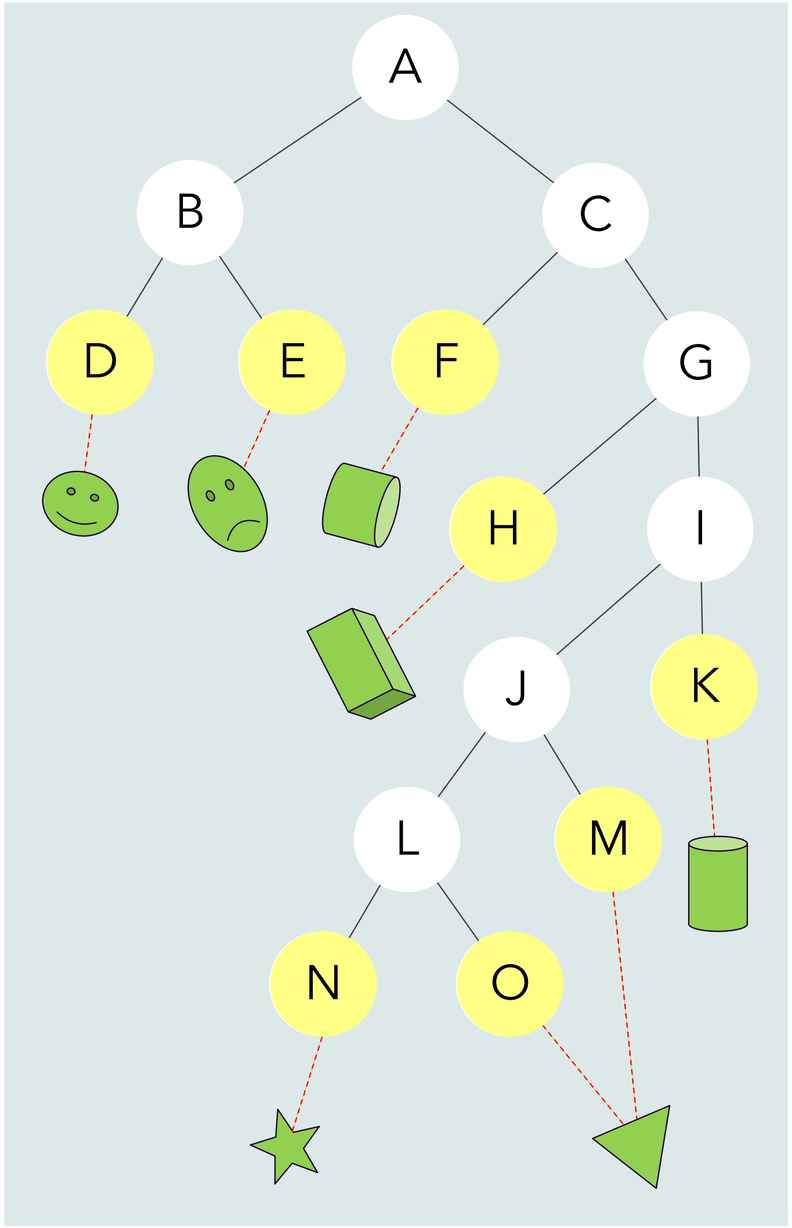


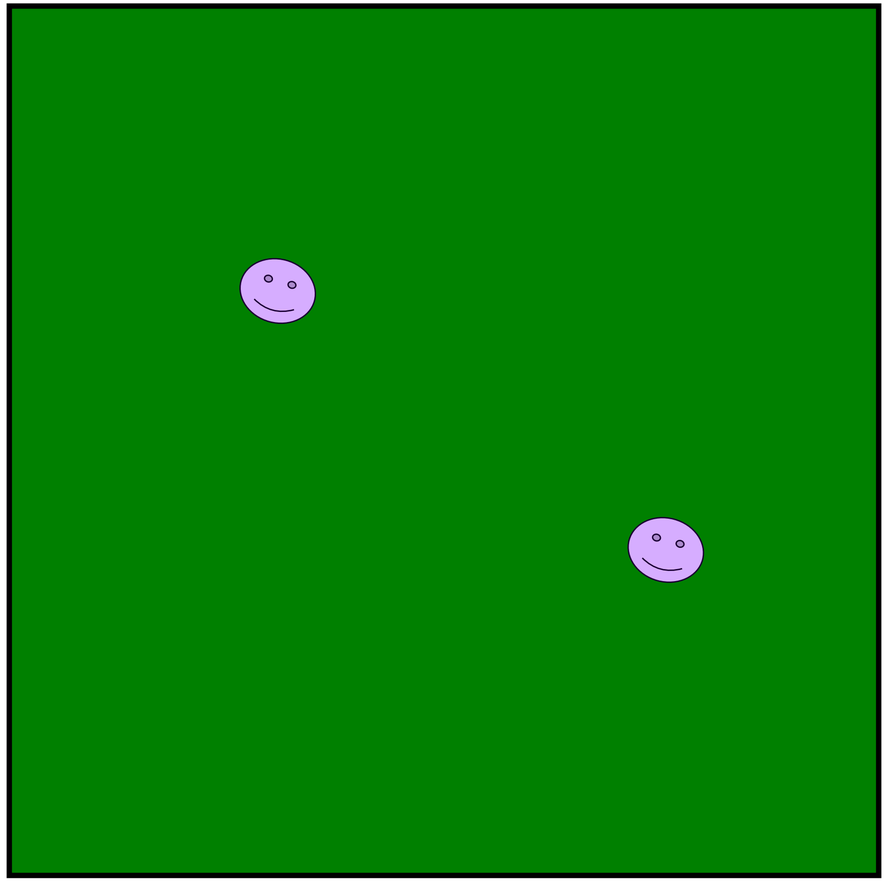
Image Copyright: Marco Tarini

University of Victoria | Computer Science

kD-trees

- kD-tree:
  - binary tree
  - each node: split over one dimension (in 3D: X,Y,Z)
  - variant:
    - each node optimizes (and stores) which dimension, or
    - always same order: e.g. X then Y then Z
  - variant:
    - each node optimizes the split point, or
    - always in the middle

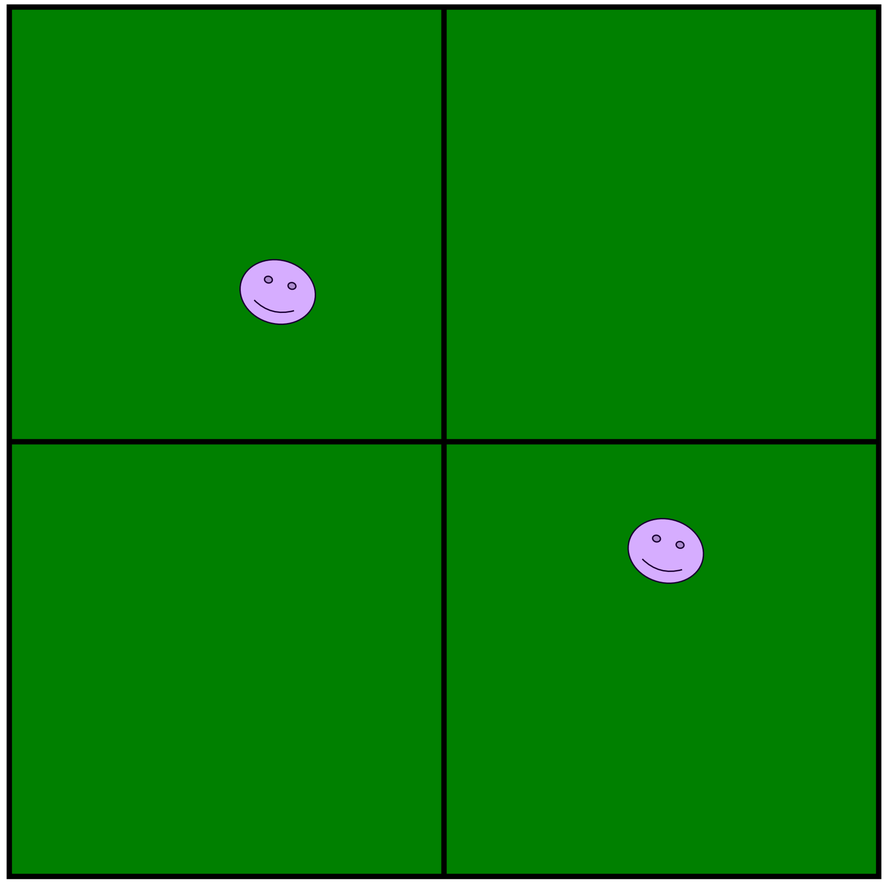
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

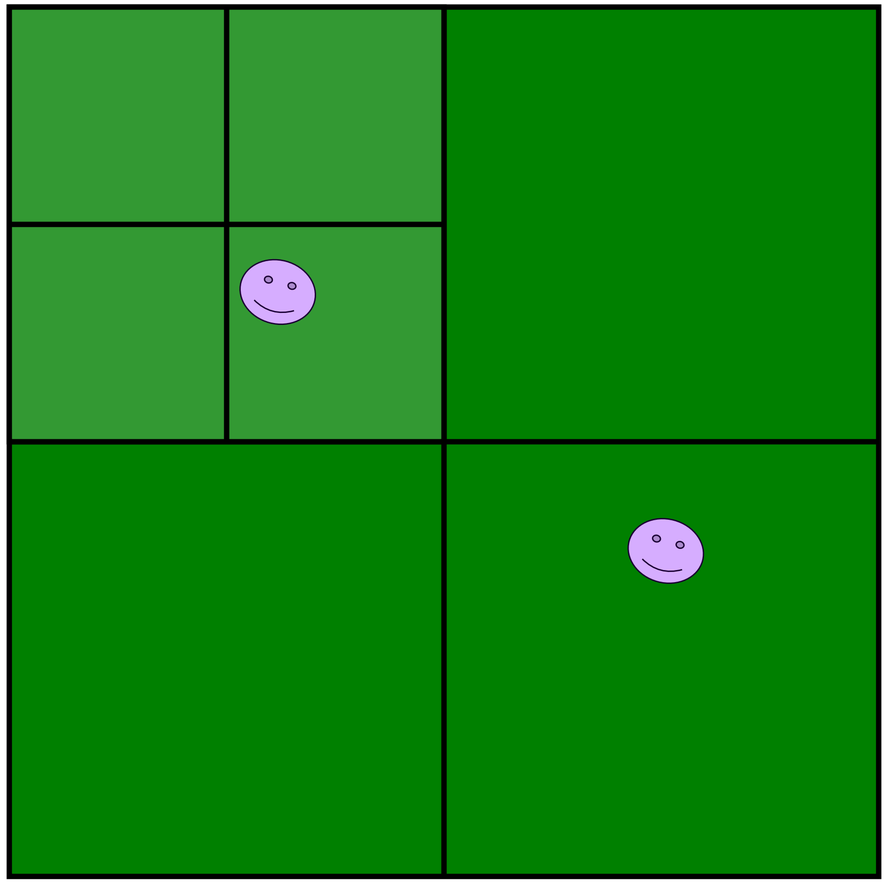
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

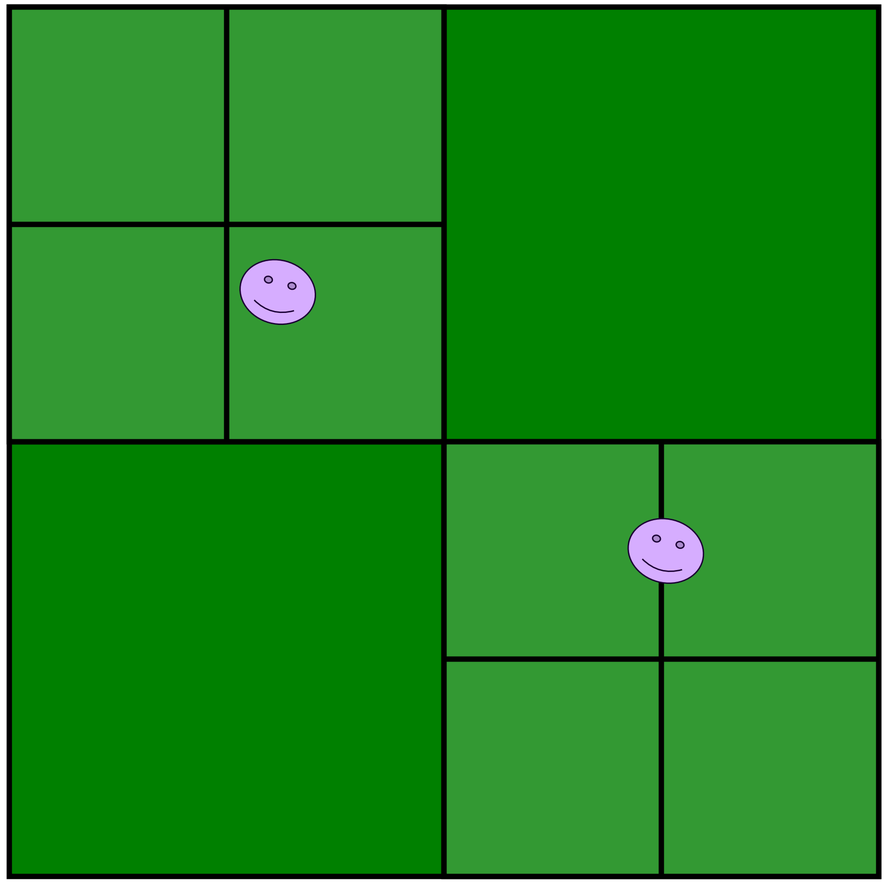
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

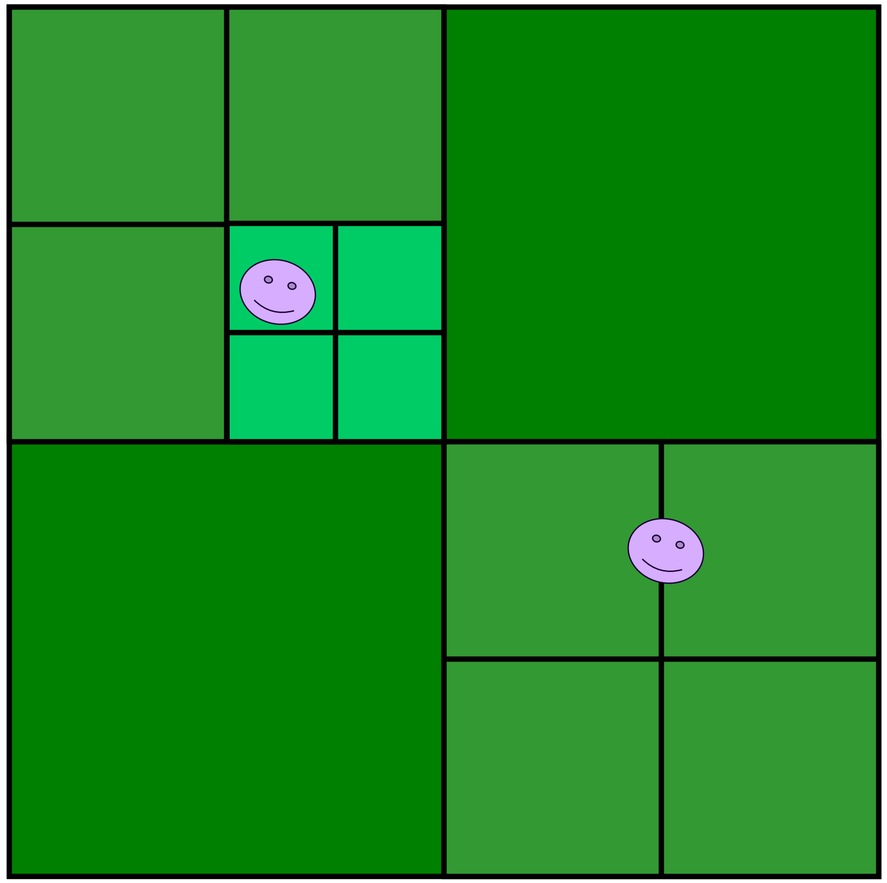
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

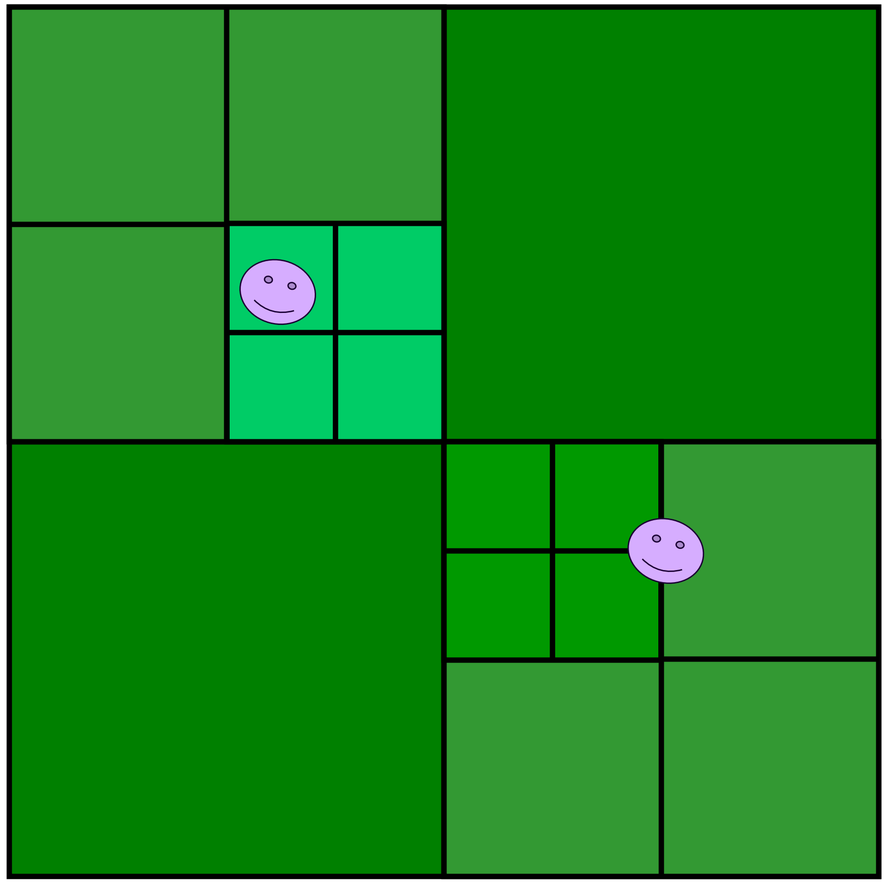
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

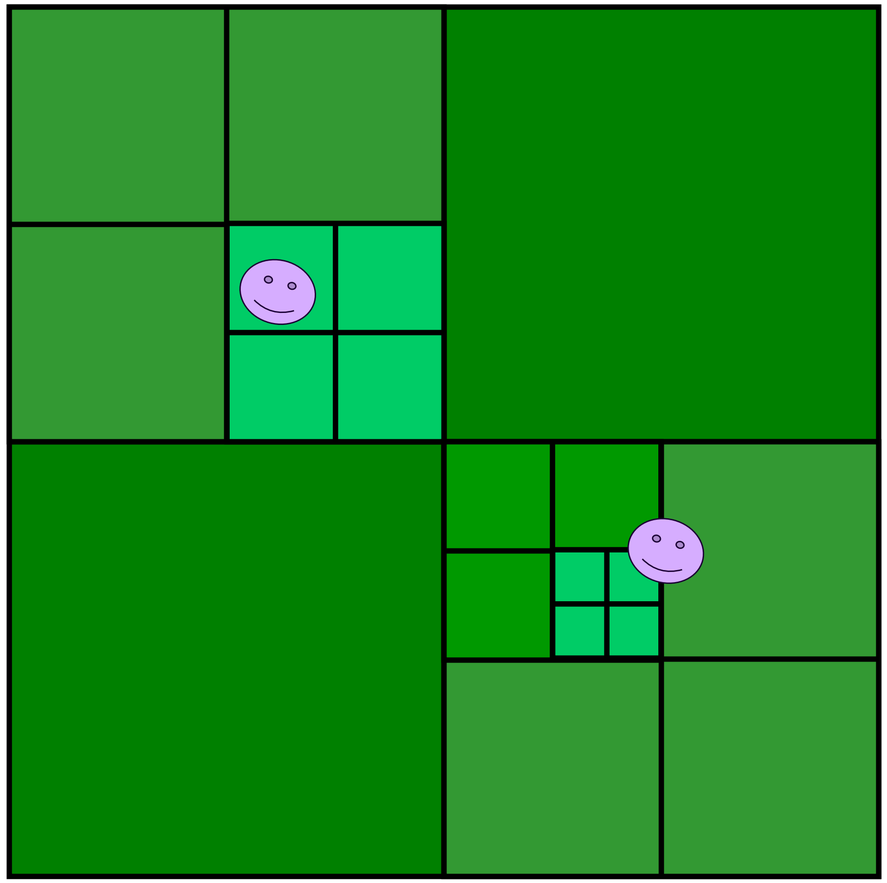
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

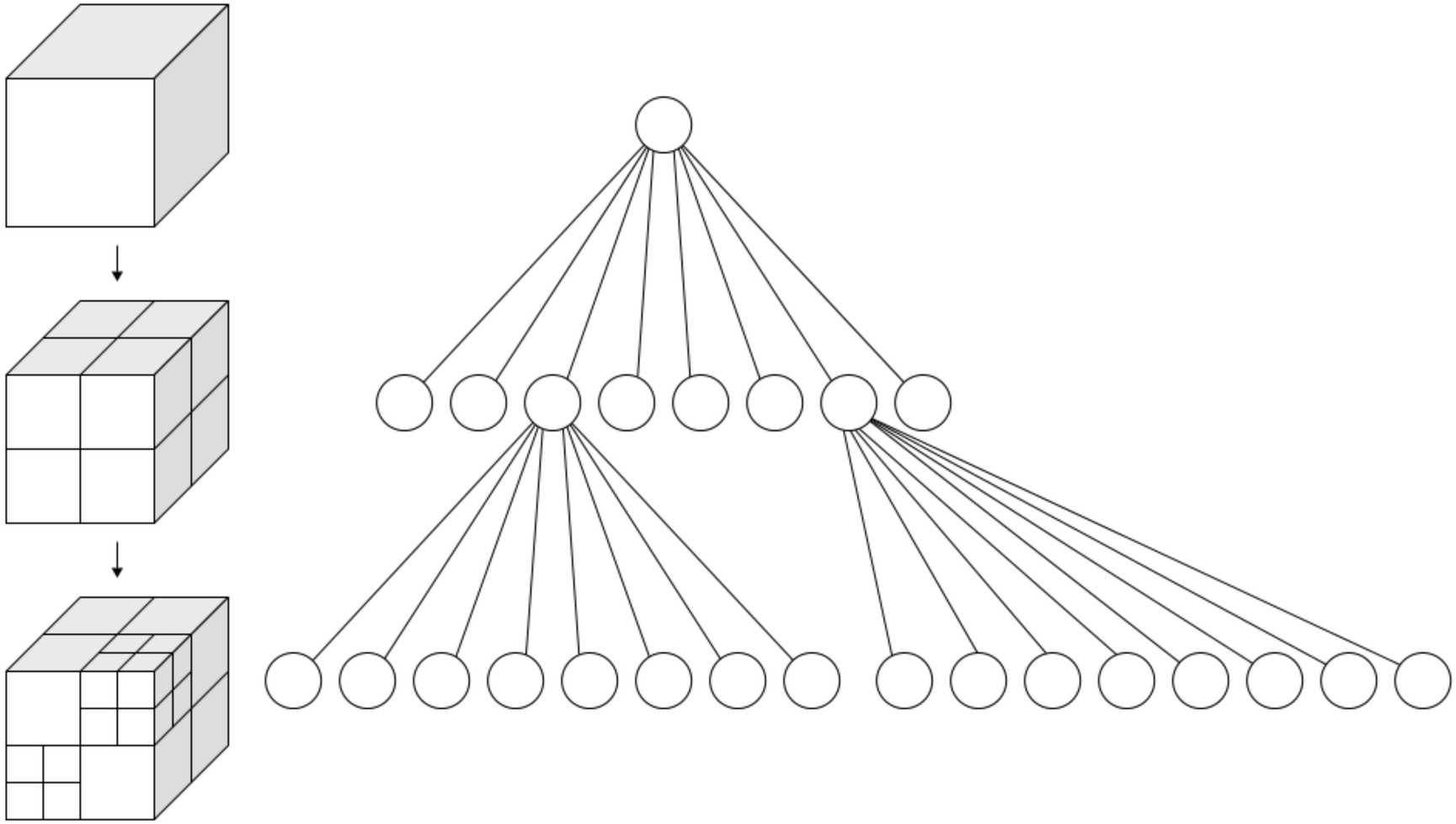
Quad-Tree (2D)



Max depth = 4

Image Copyright: Marco Tarini

Oc-Tree (3D)



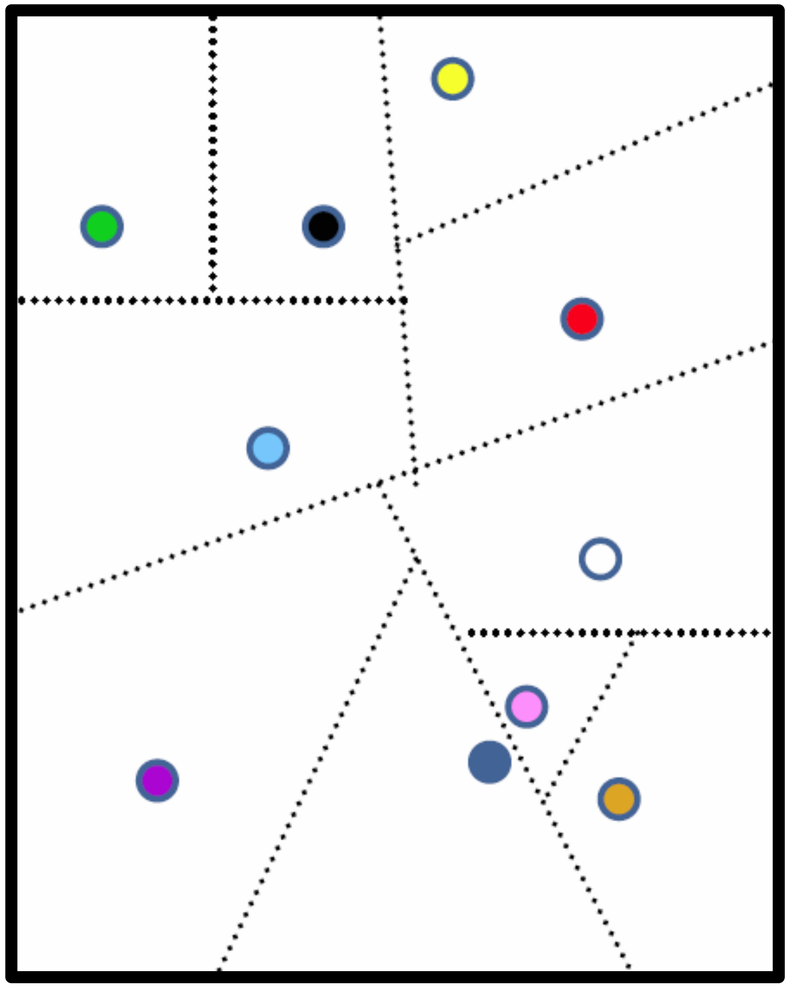
Quad trees (in 2D)
Oct trees (in 3D)

- Similar to kD-trees, but:
  - tree: branching factor: 4 (2D) or 8 (3D)
  - each node: splits into all dimensions at once,
    (in the middle)

- Construction (just as kD-trees):
  - continue splitting until a end nodes has few enough objects
    (or limit level reached)

University of Victoria | Computer Science

BSP-tree
Binary Spatial Partitioning tree



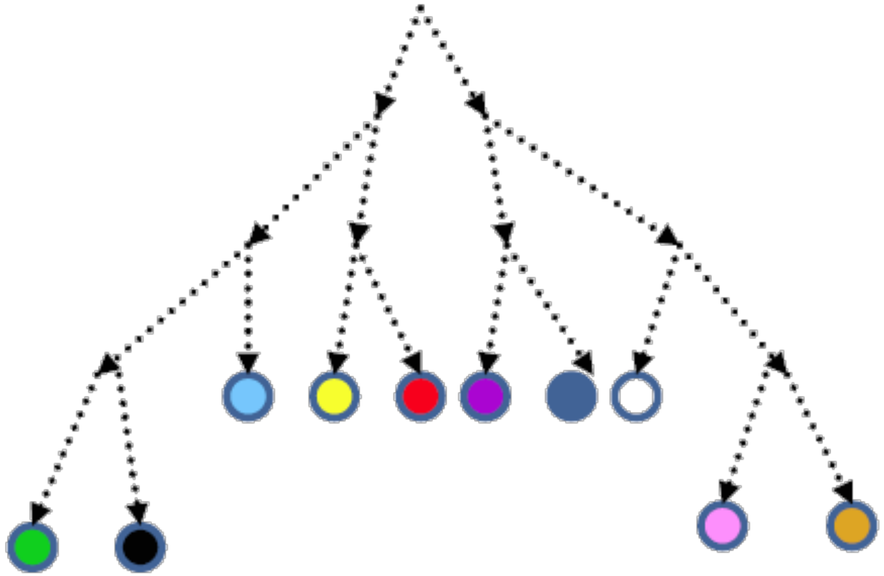


Image Copyright: Marco Tarini

BSP-trees for the Concave Polyhedron proxy

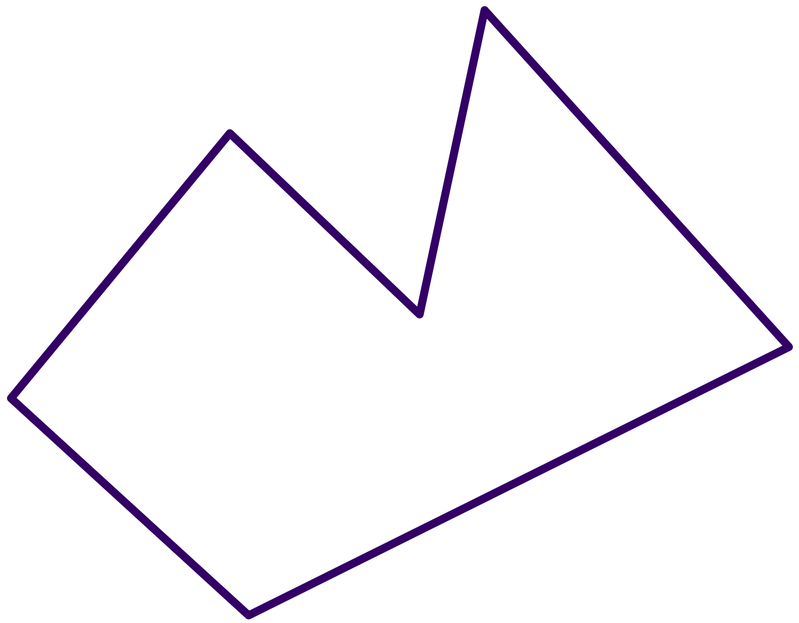


Image Copyright: Marco Tarini

BSP-trees for Inside-Outside Test

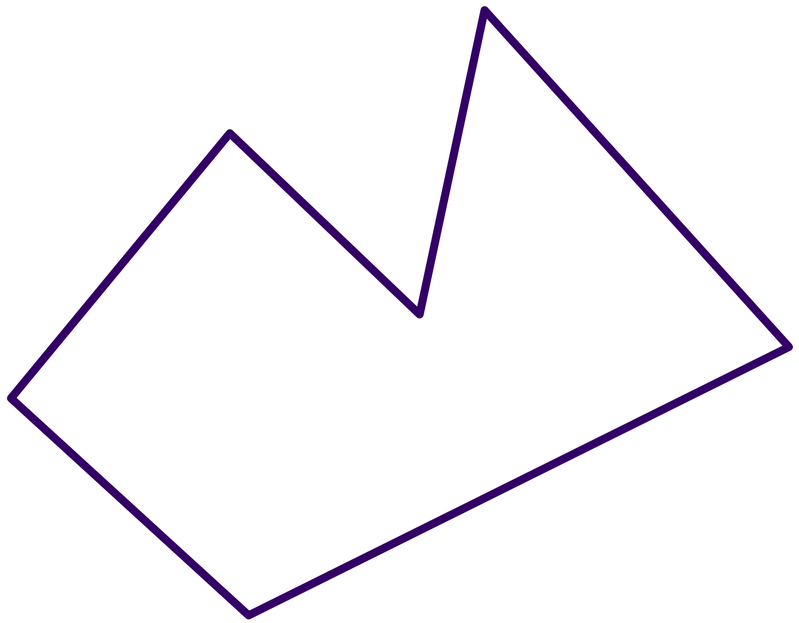


Image Copyright: Marco Tarini

BSP-trees for Inside-Outside Test

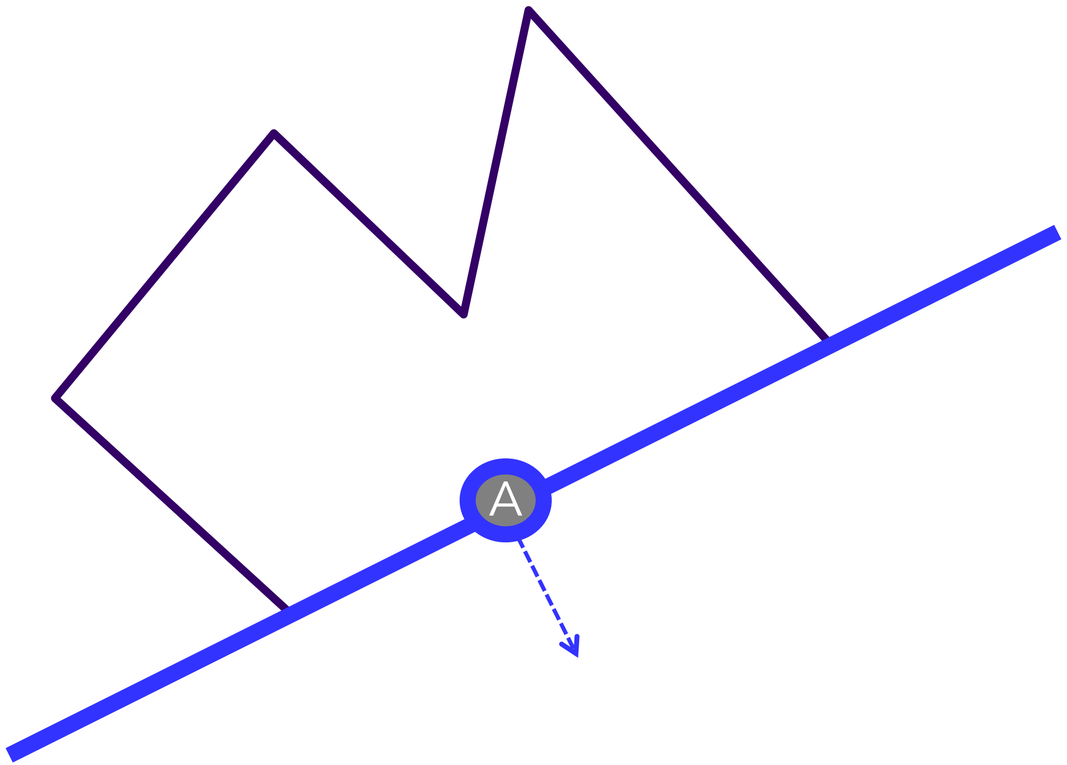
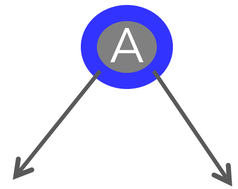


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



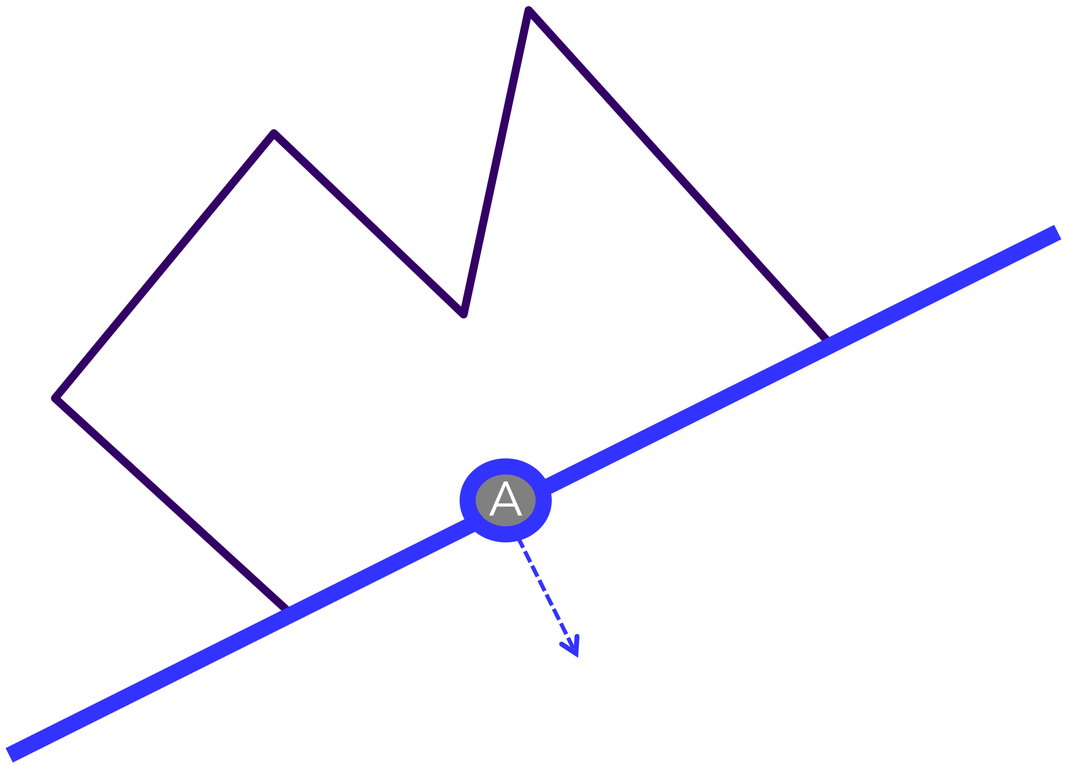
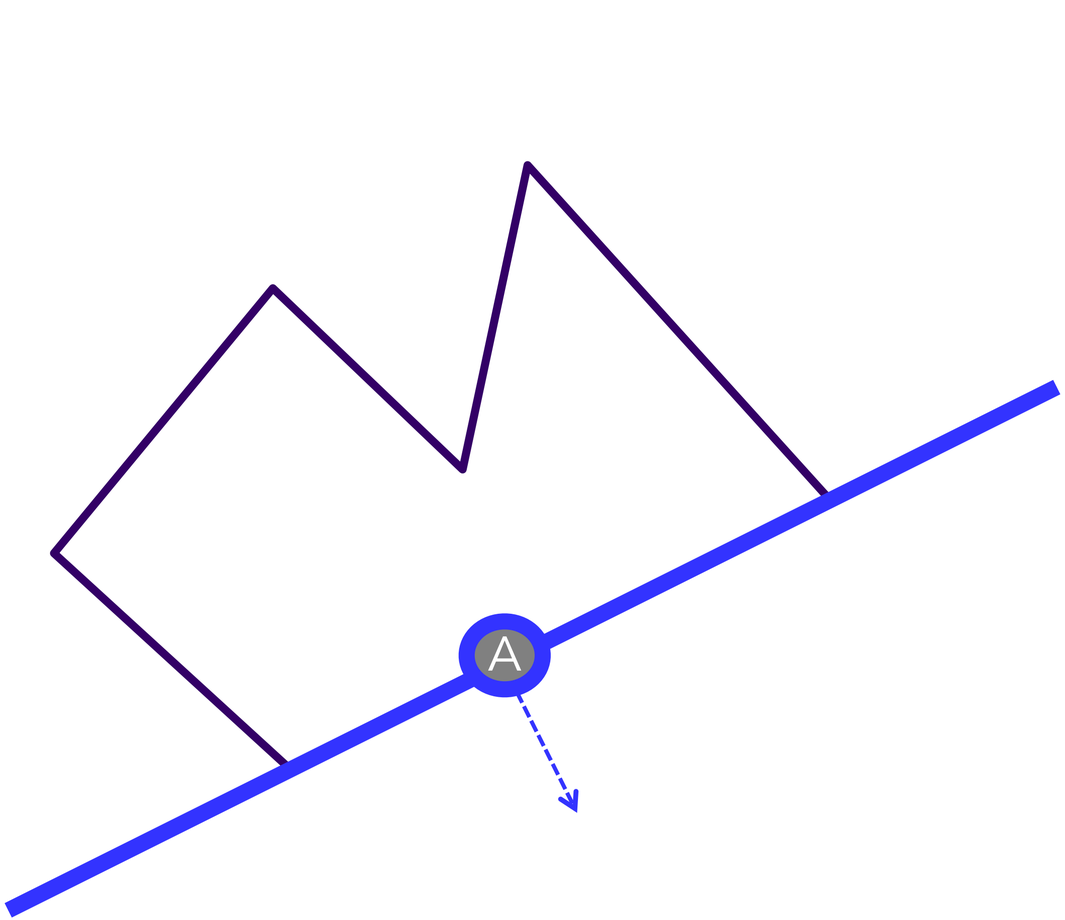


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



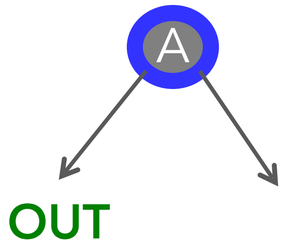
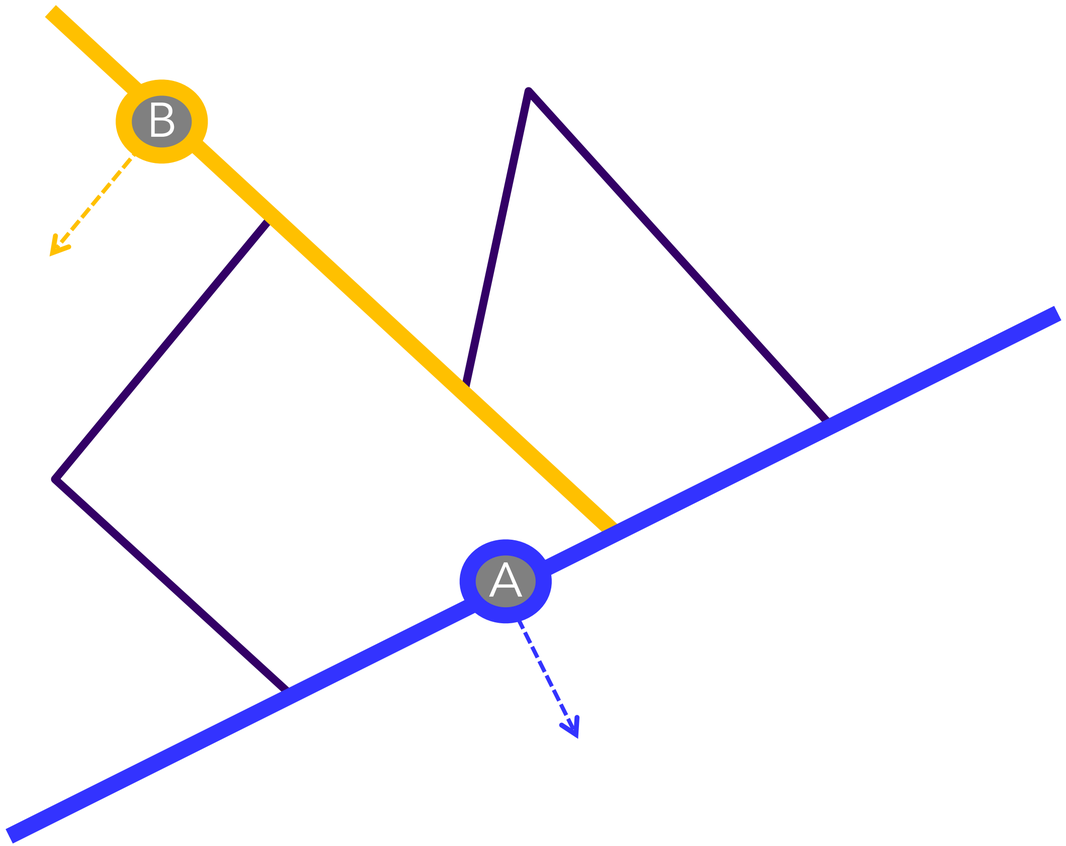


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



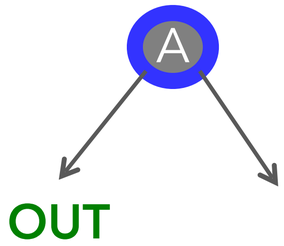
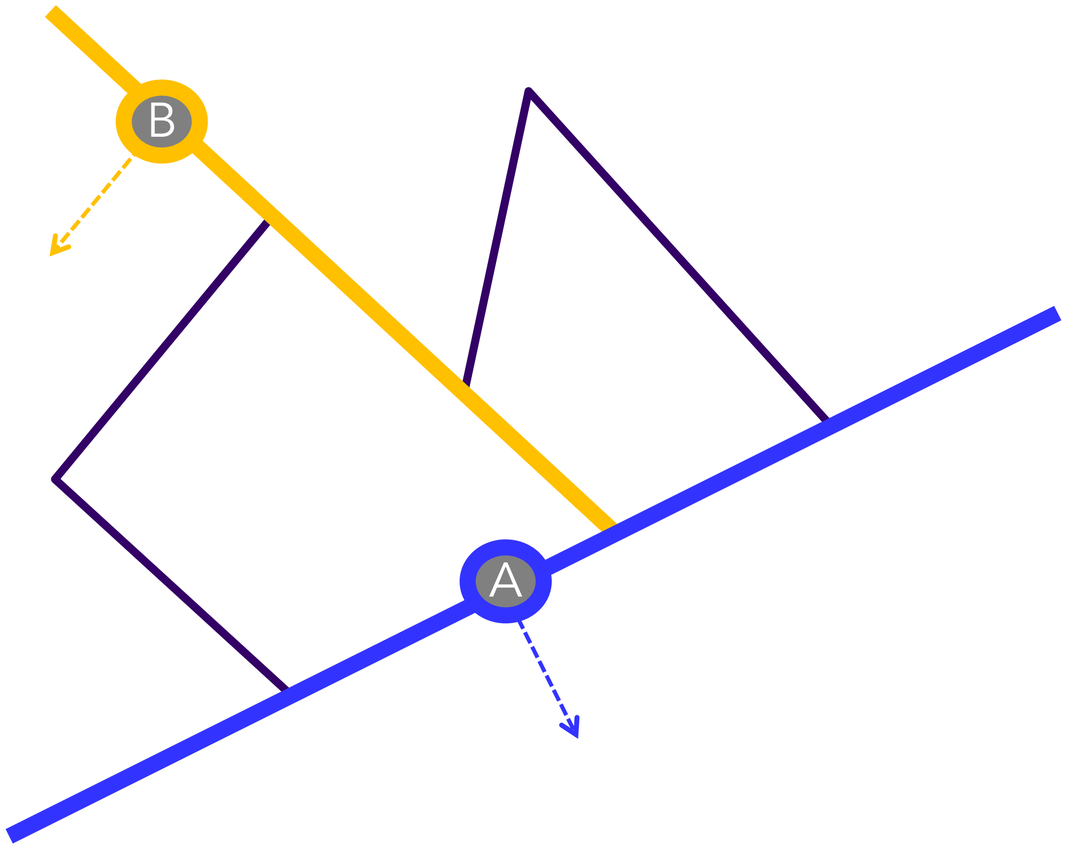


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



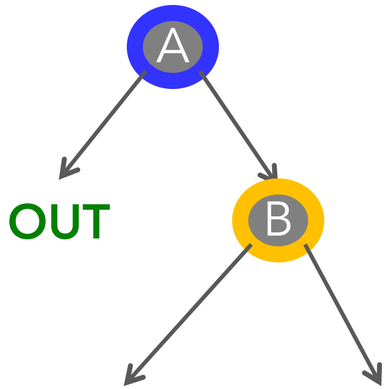
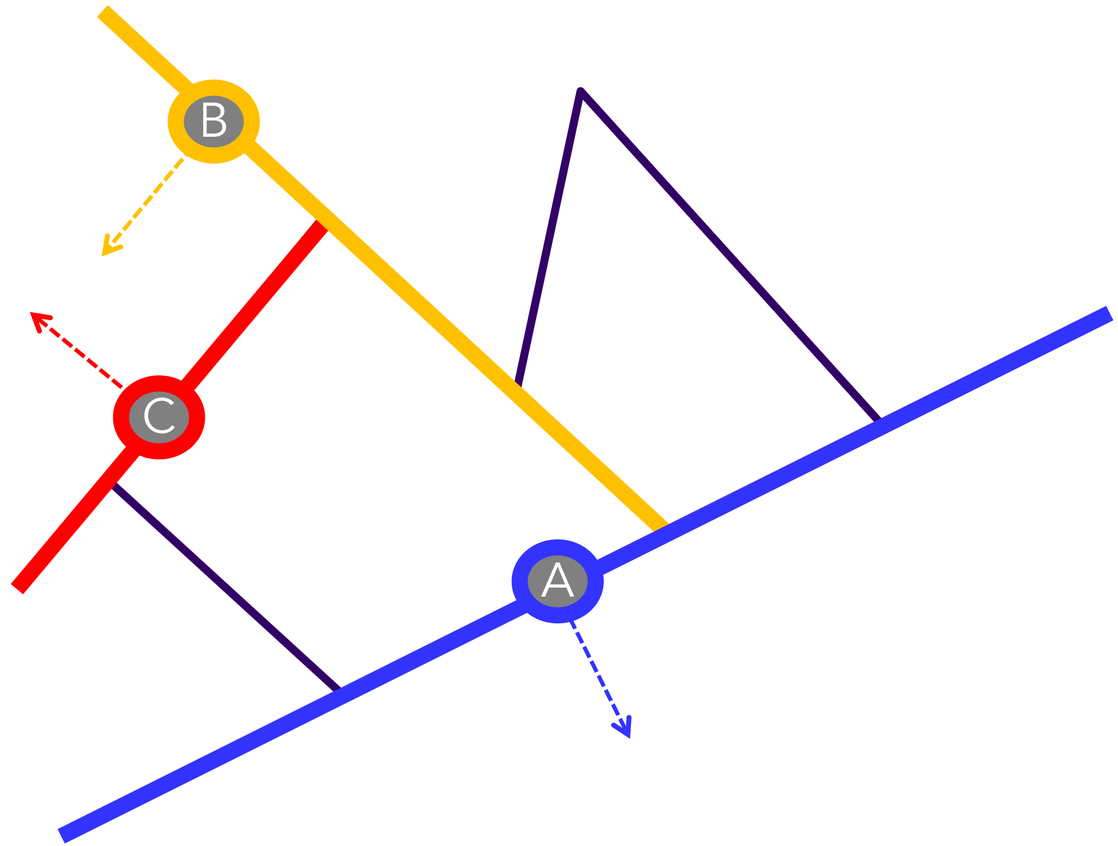


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



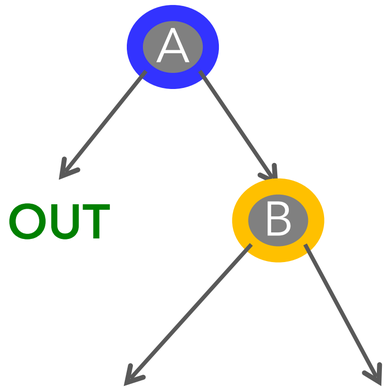
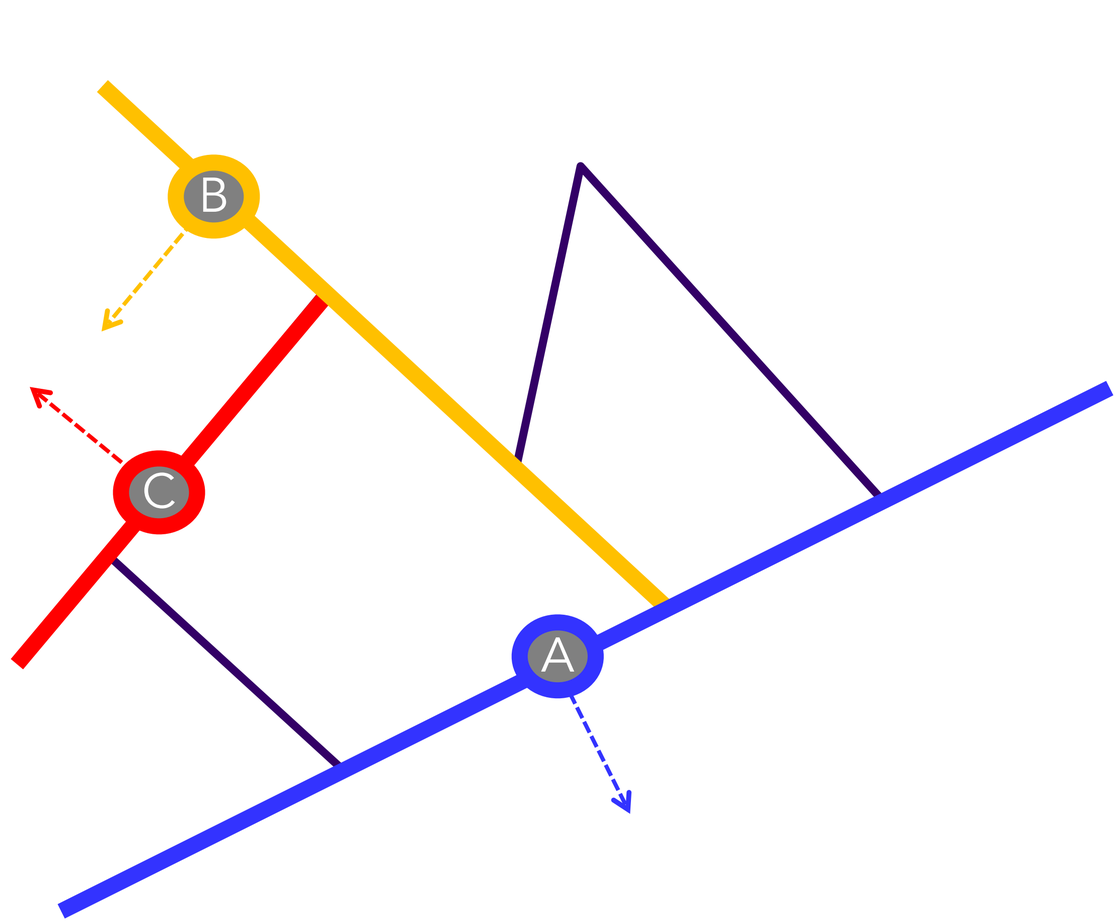


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



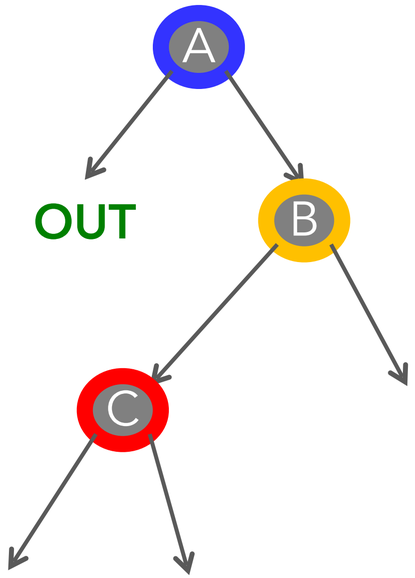
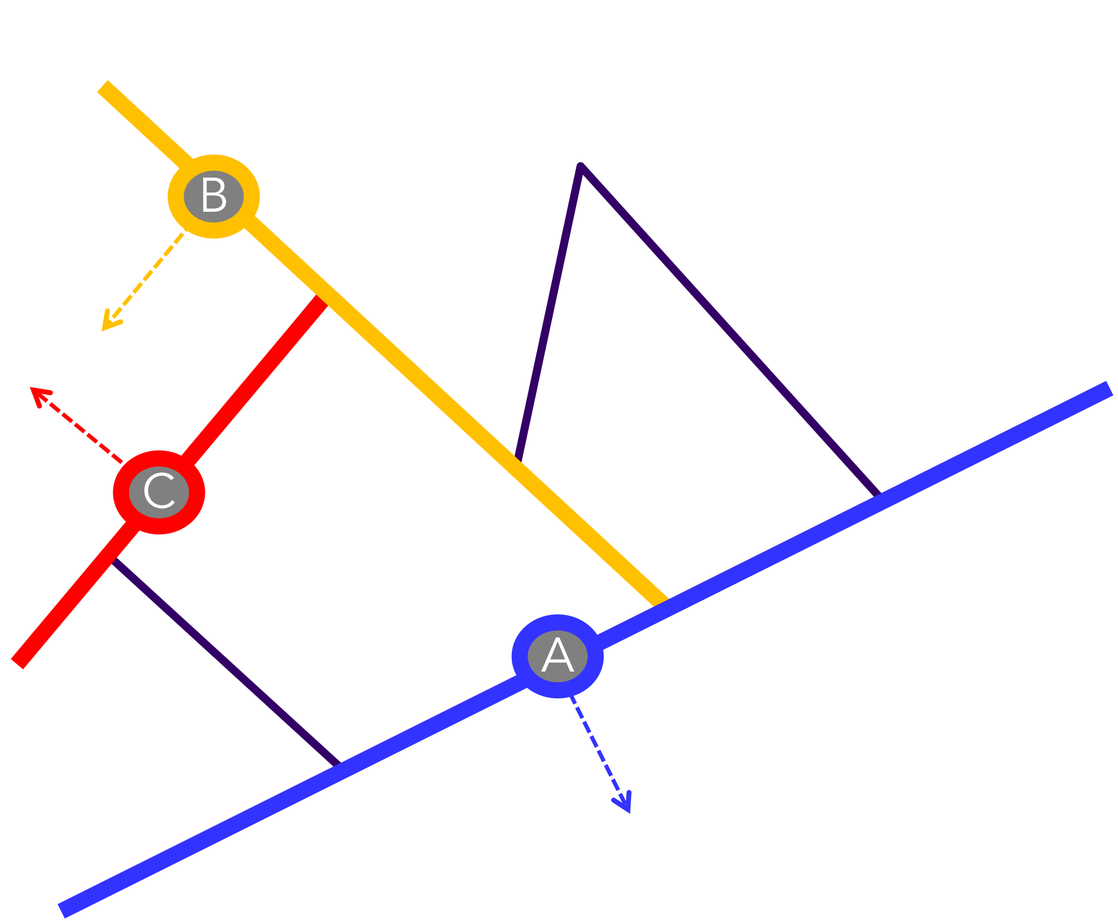


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



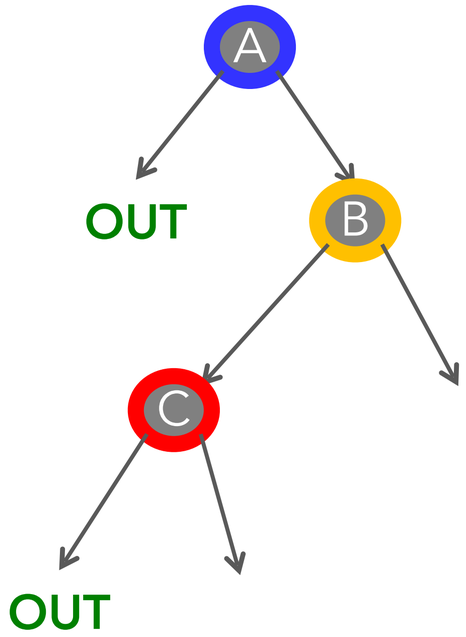
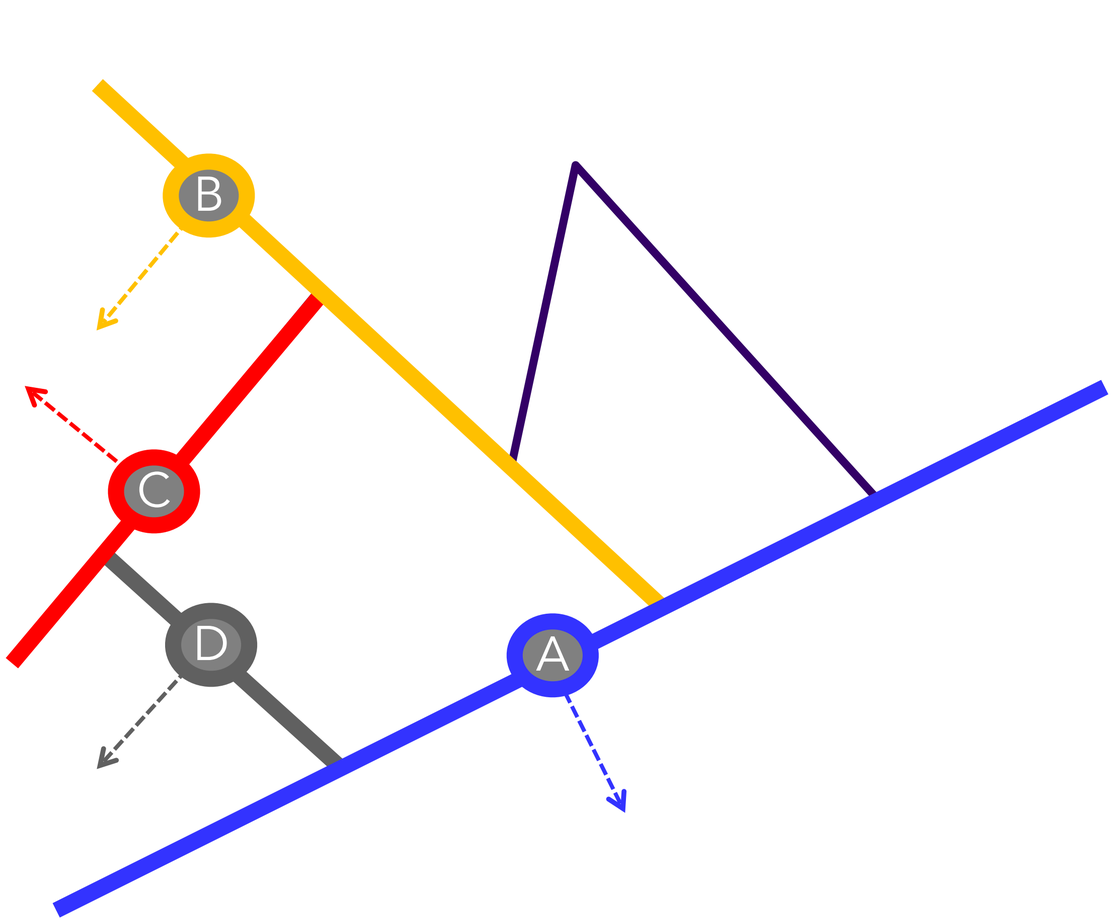


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



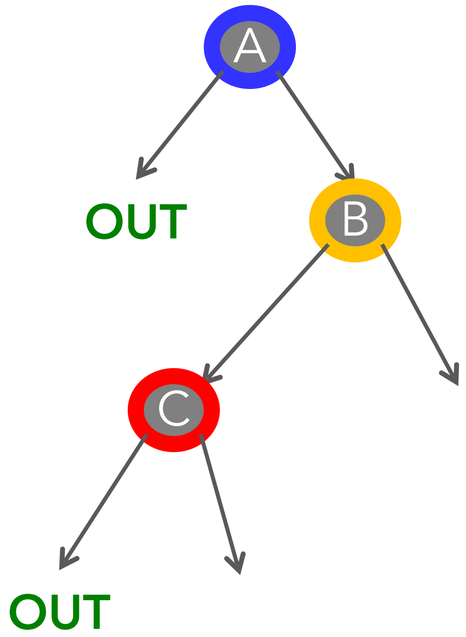
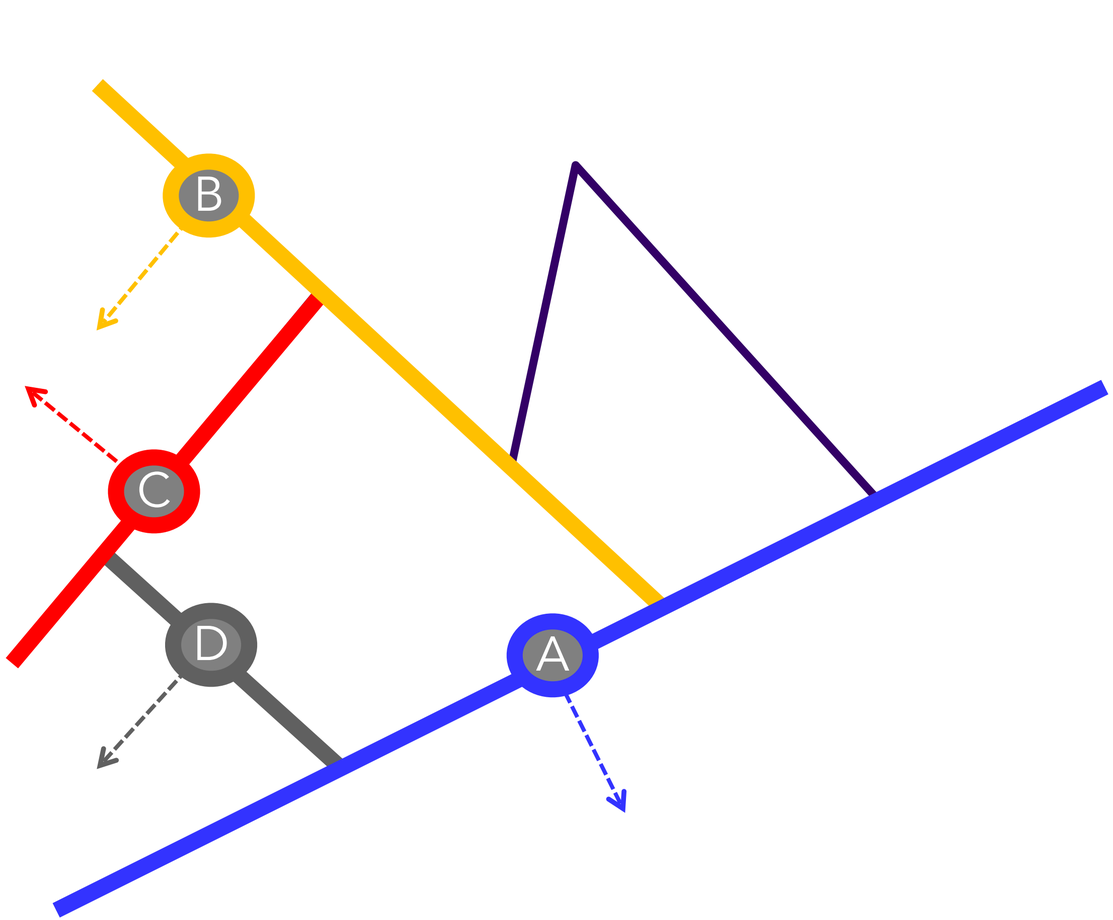


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



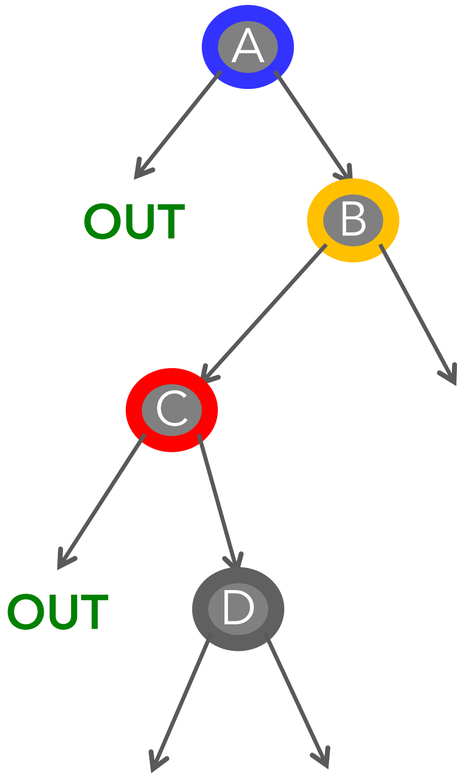
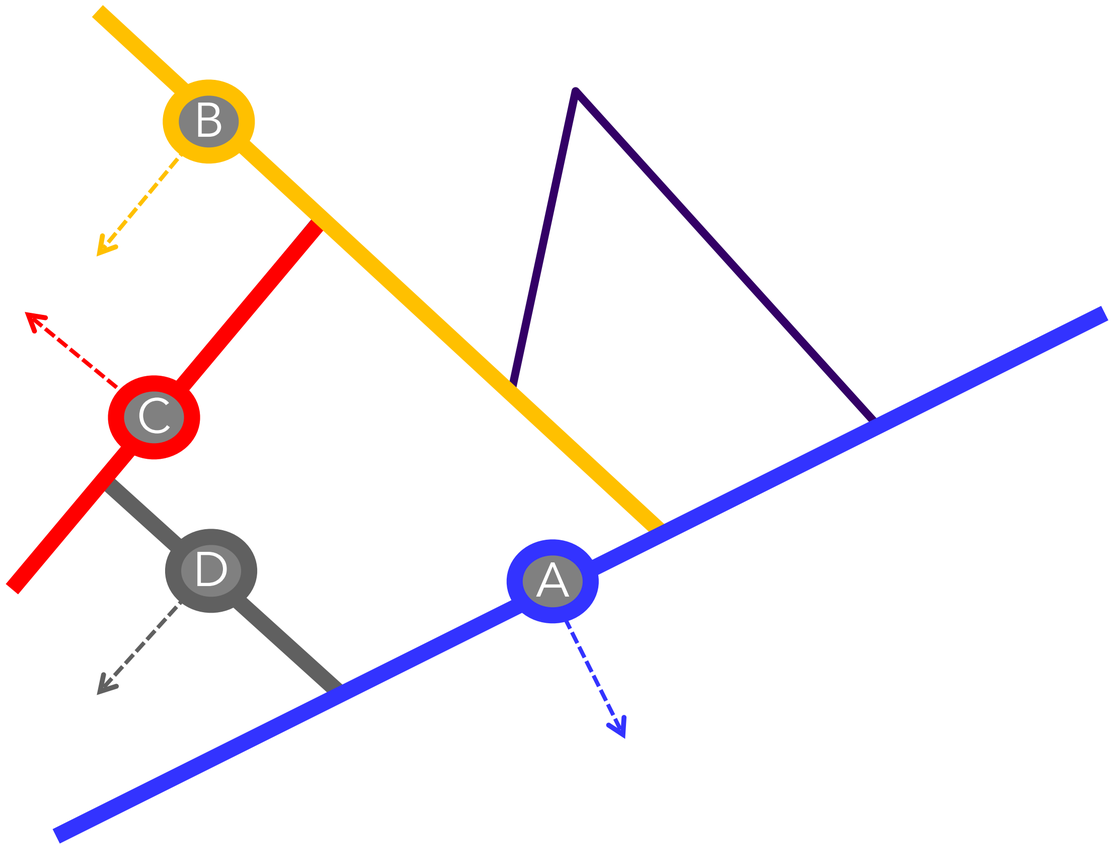


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



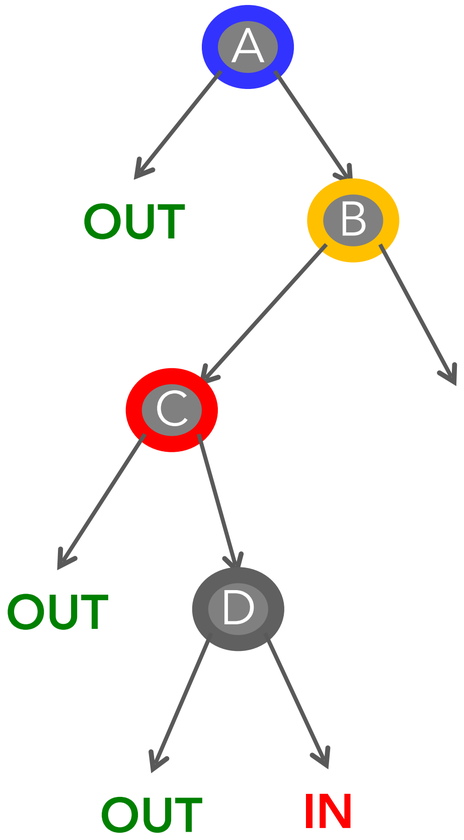
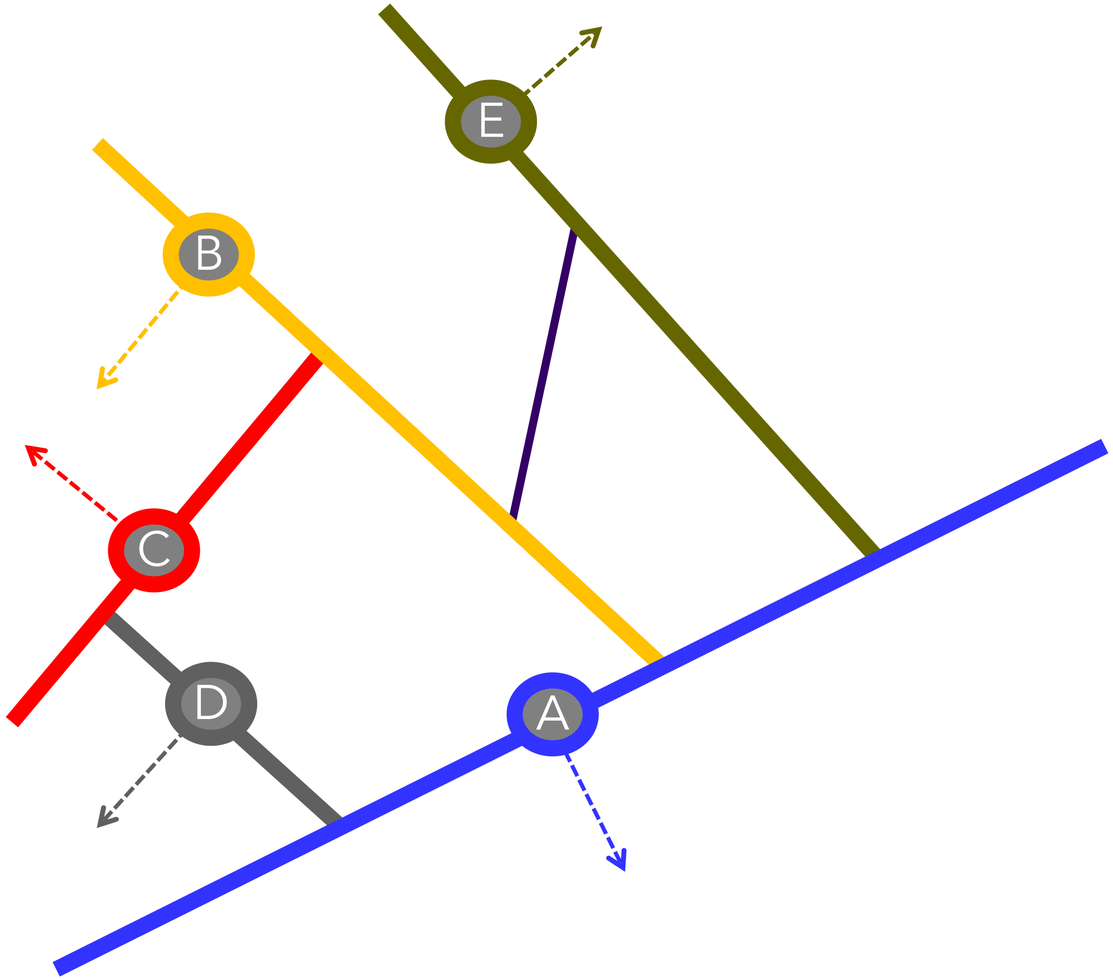


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



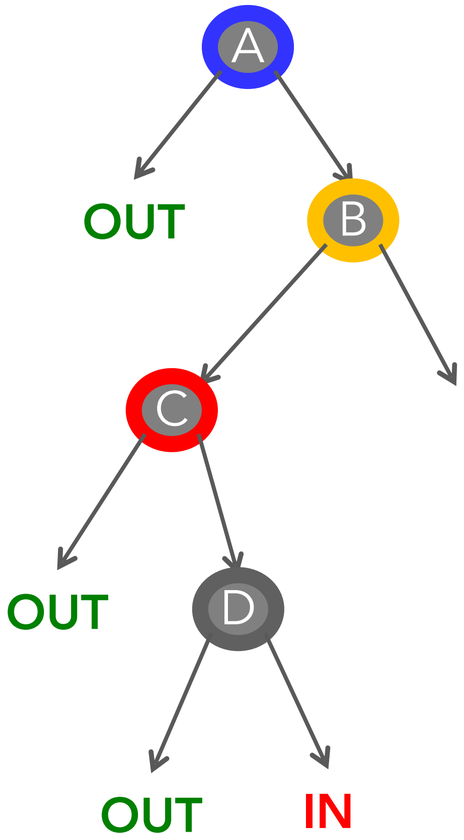
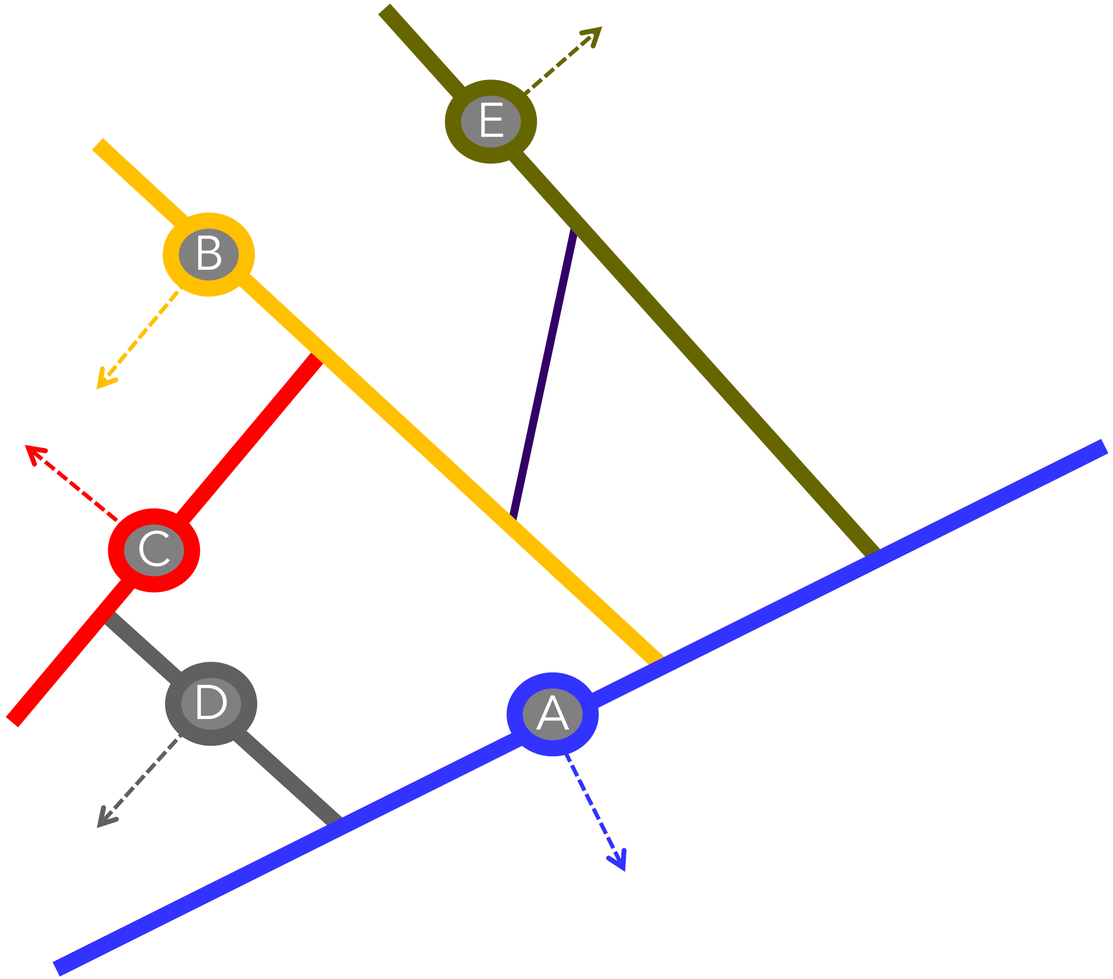


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



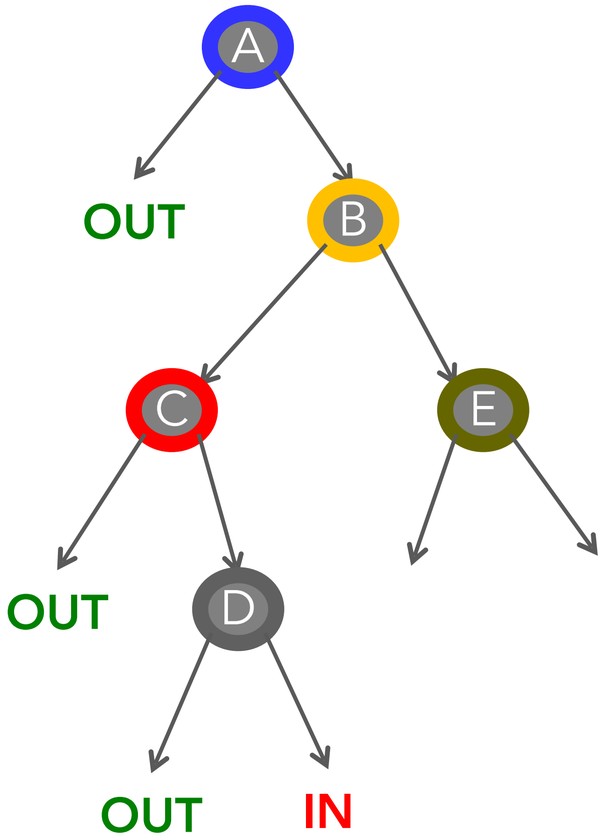
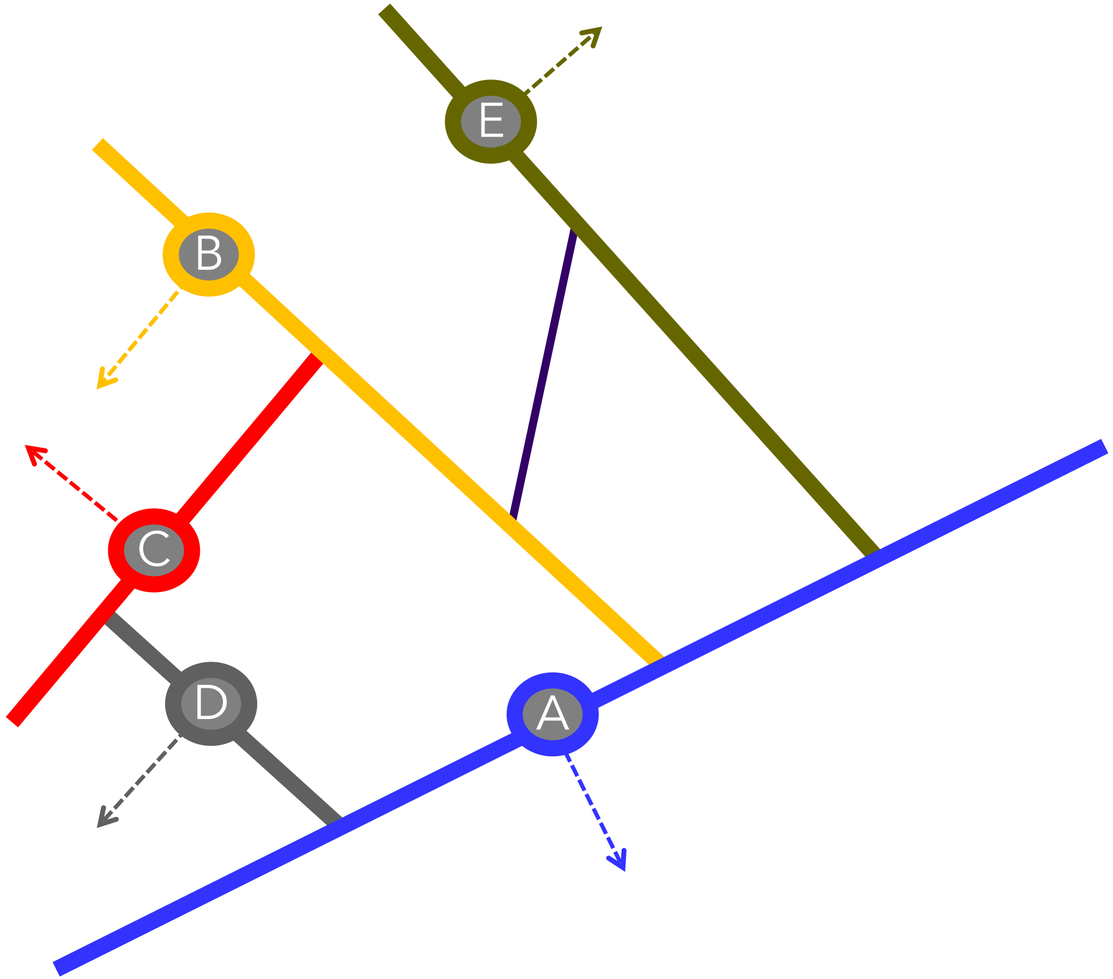


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



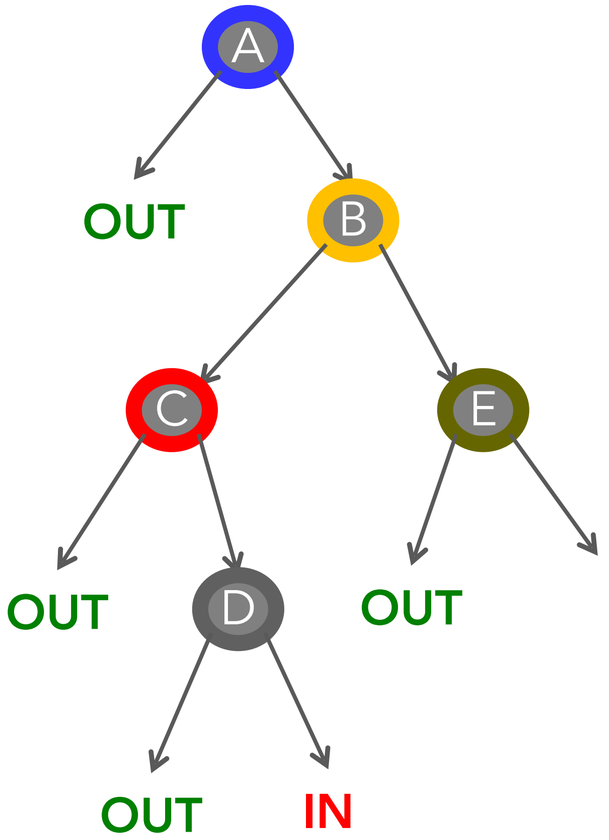
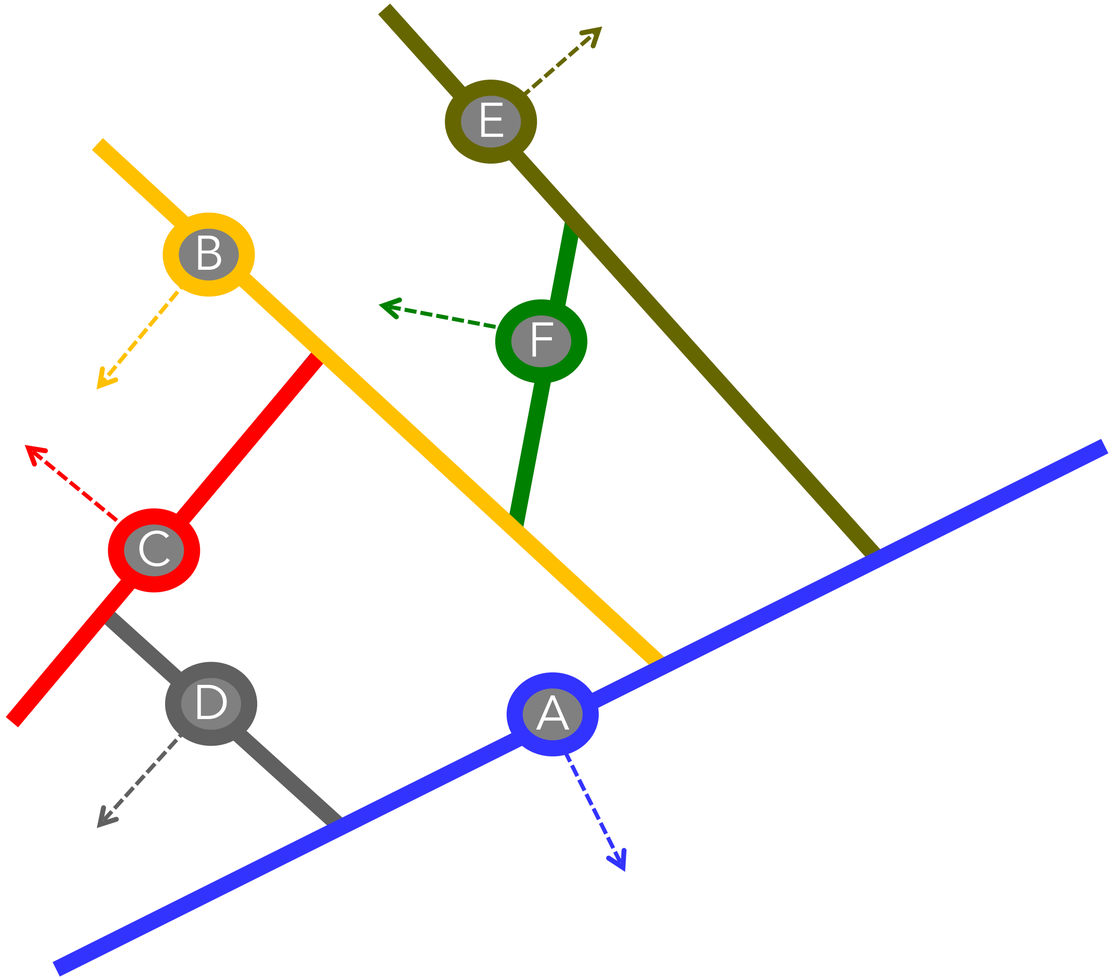


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



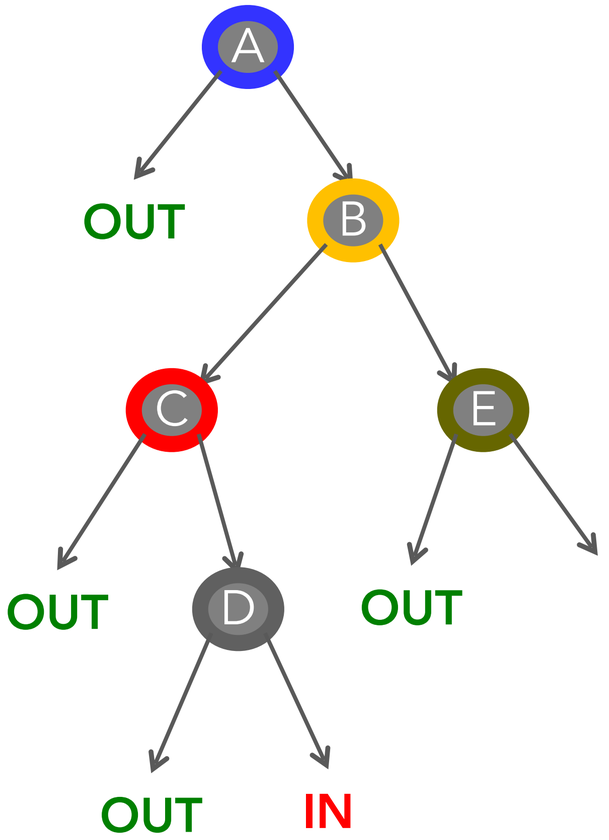
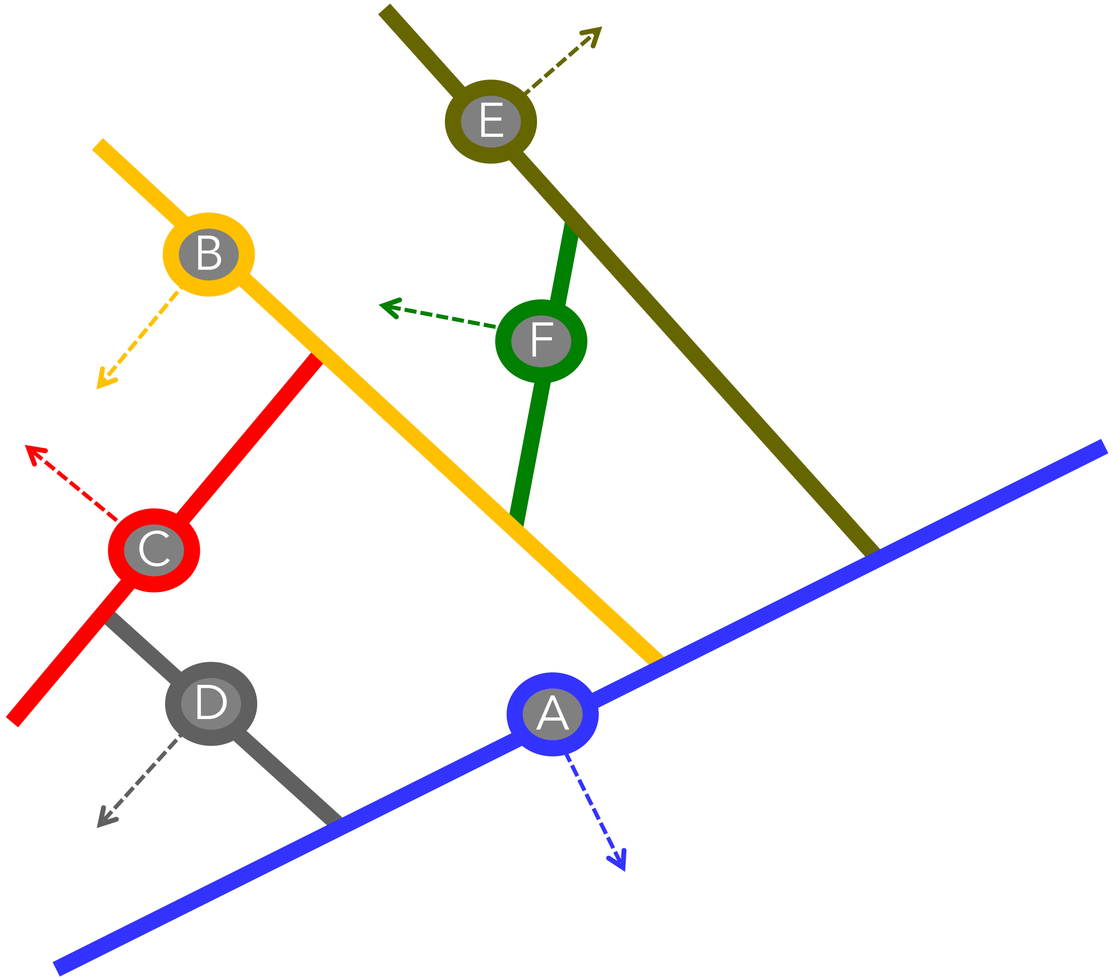


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



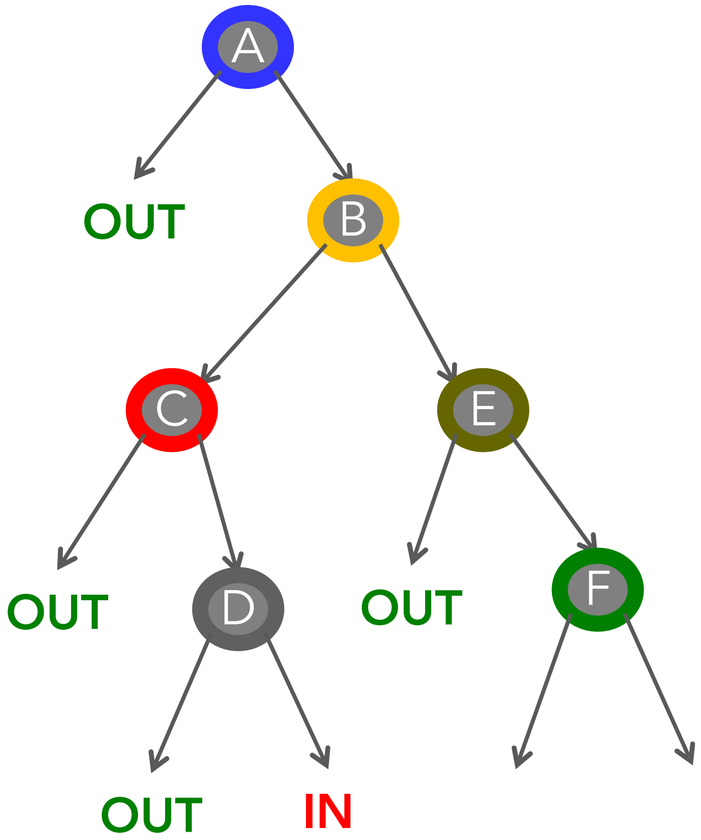
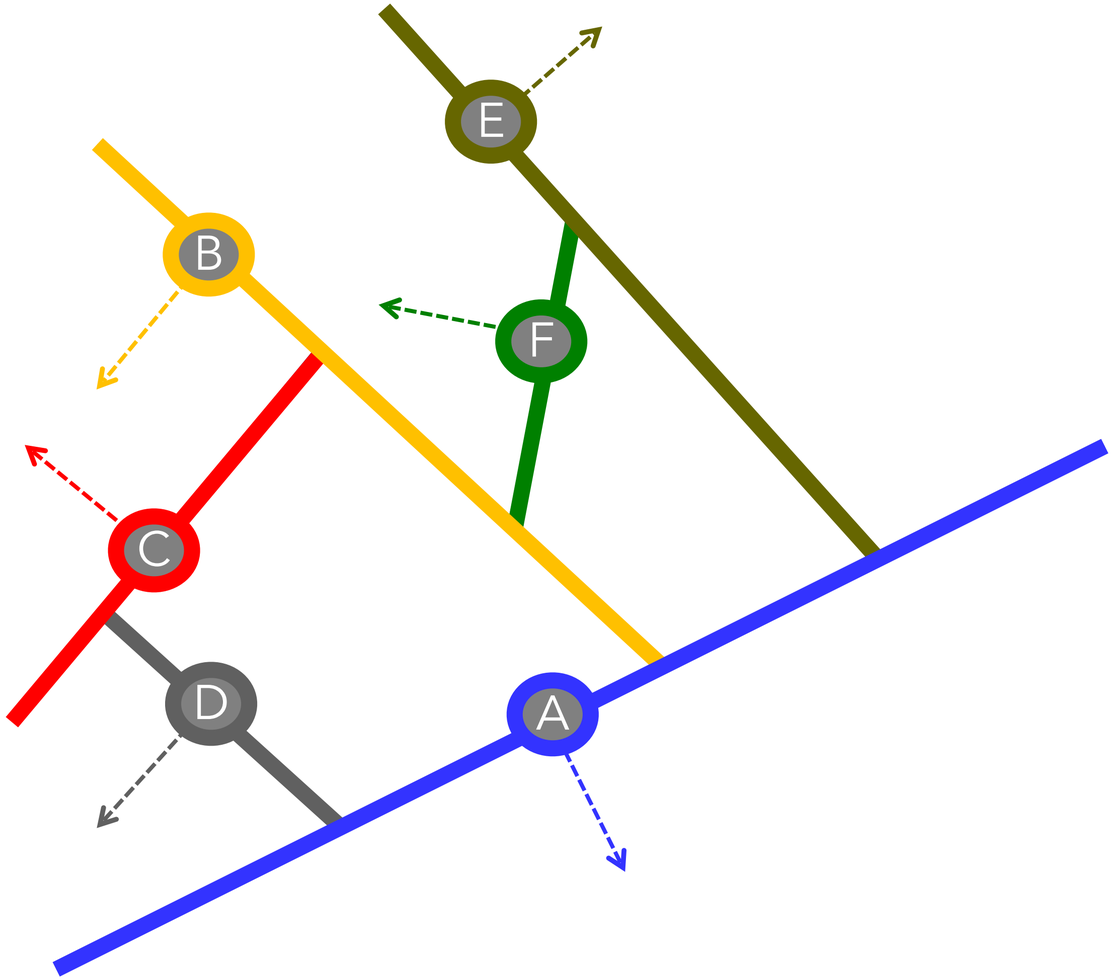


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



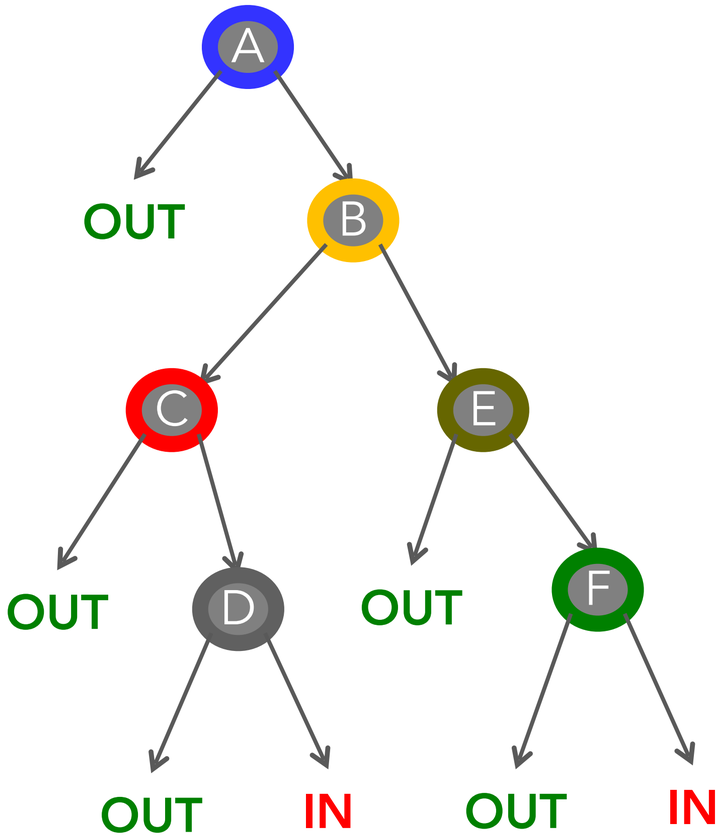
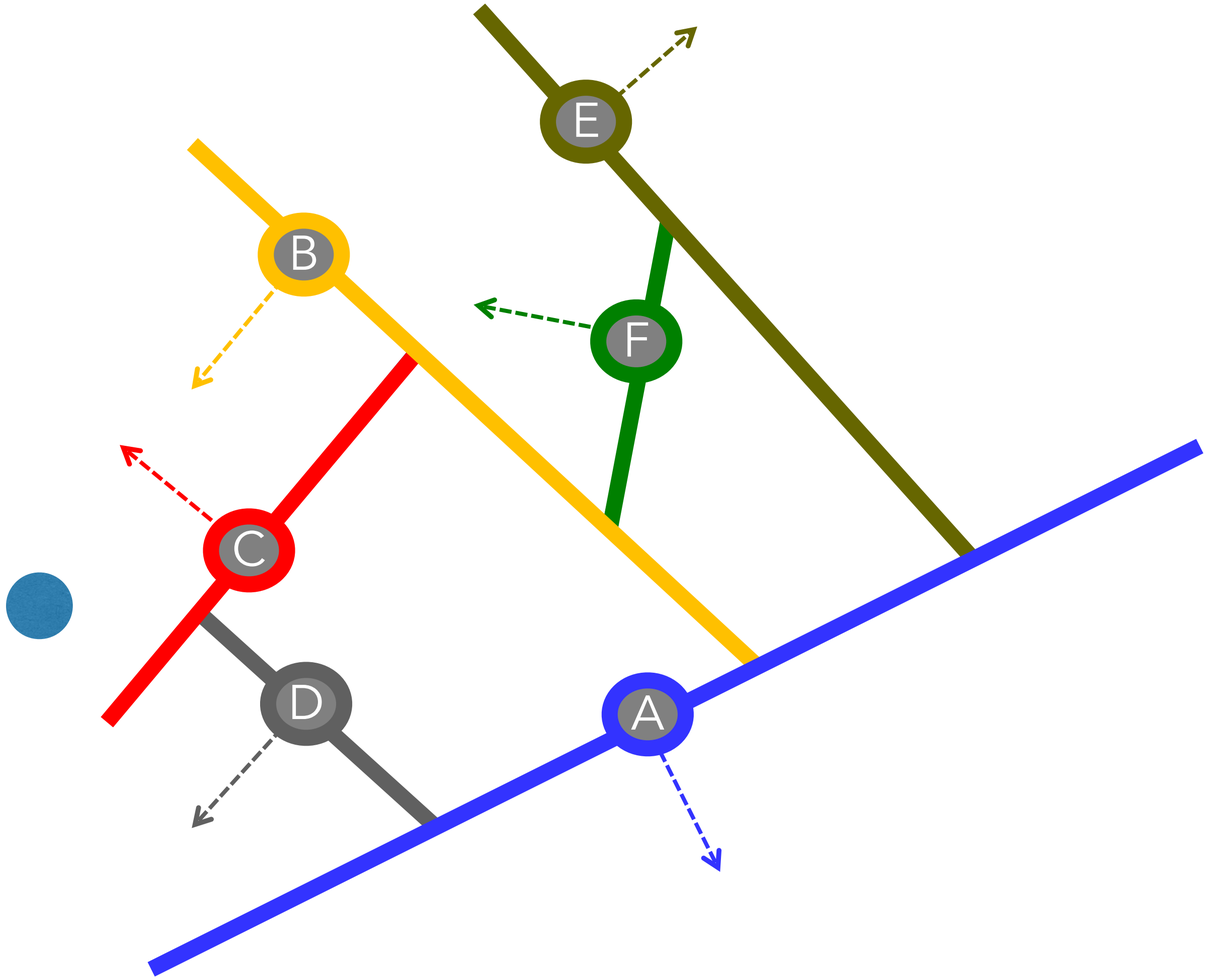


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



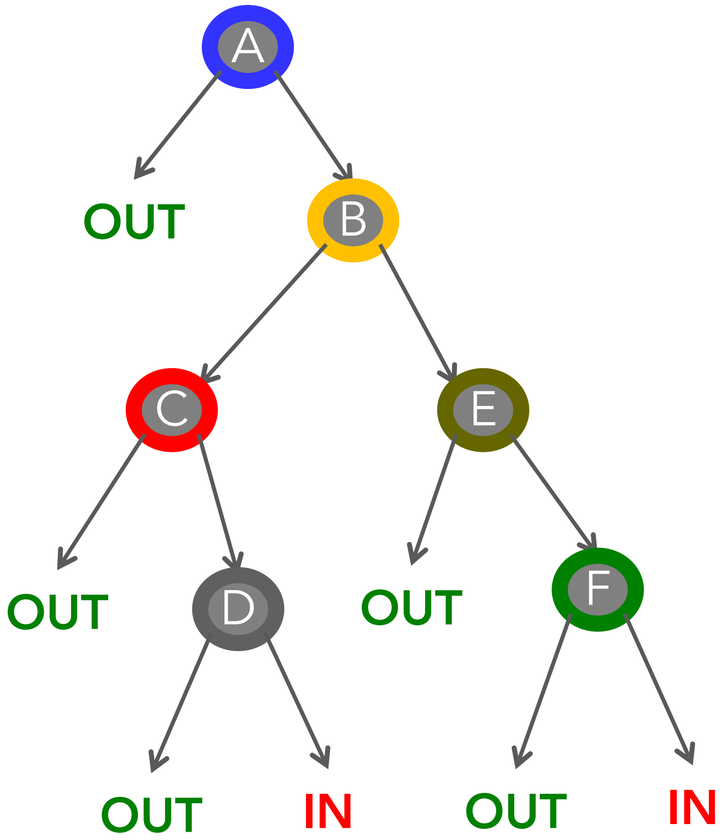
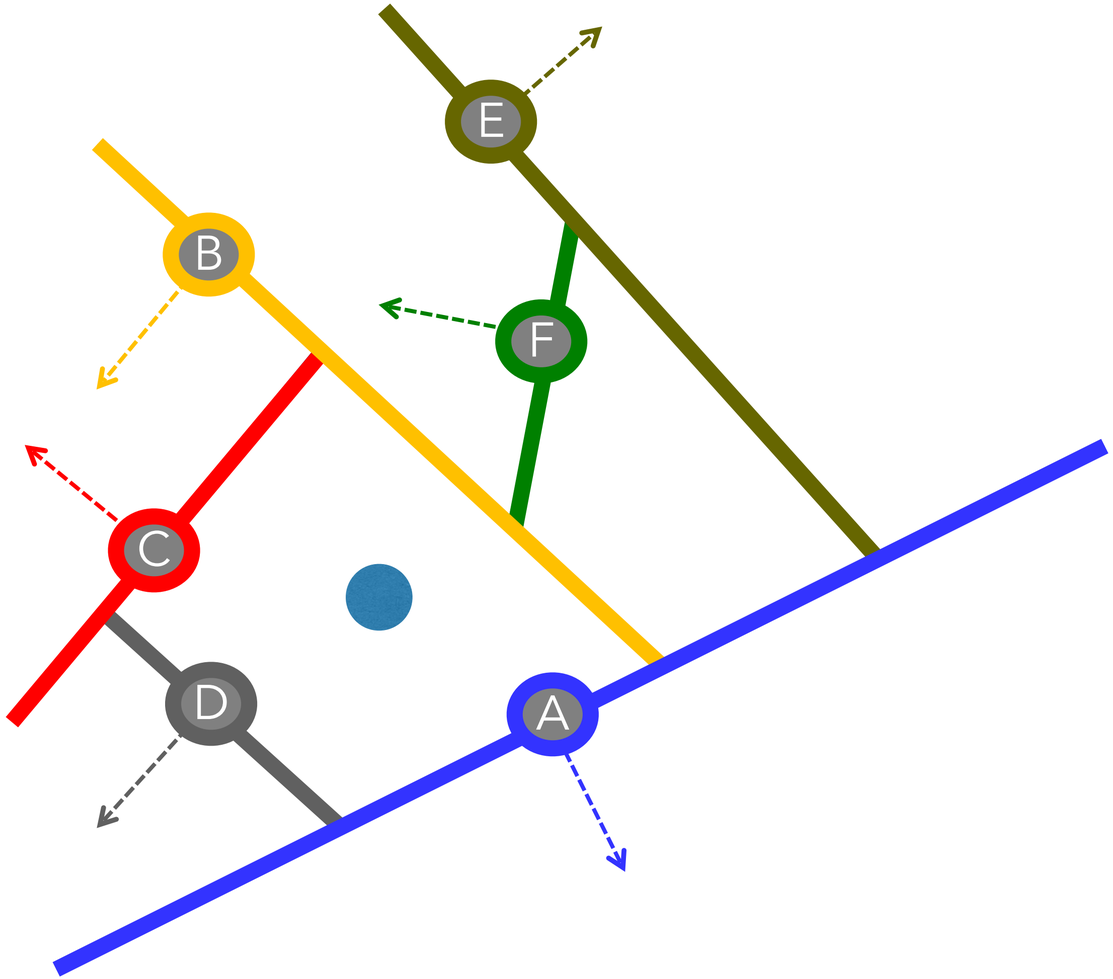


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-trees for Inside-Outside Test



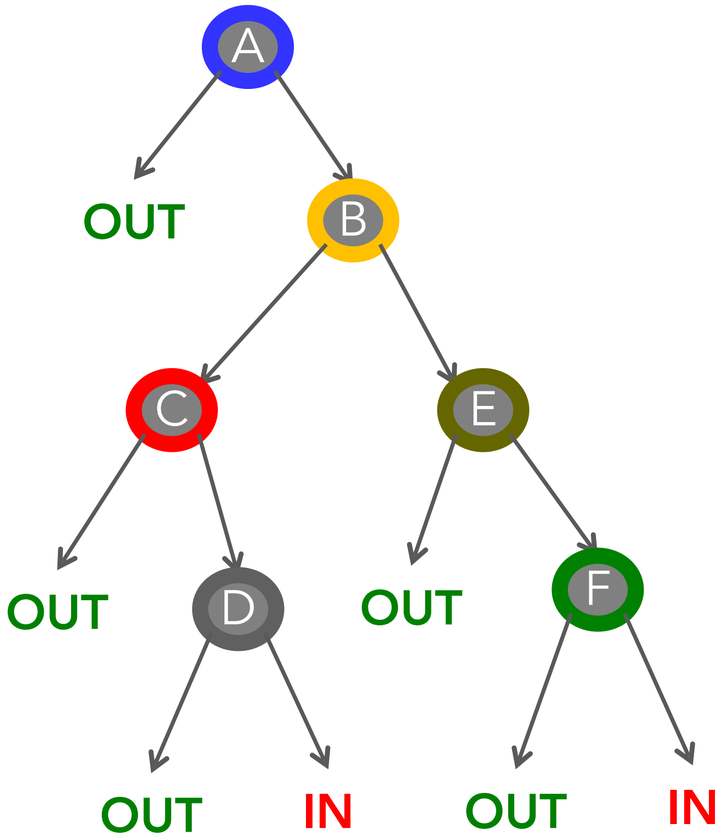


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BSP-tree
Binary Spatial Partitioning tree

- Another variant
  - a binary tree (like the kD-tree)
    - root = all scene (like kD-tree)
  - but, each node is split by an **arbitrary** plane
    - (or a **line**, in 2D)
    - plane is stored at node, as (nx, ny, nz, k)
  - planes can be optimized for a given scene
    - e.g. to go for a 50%-50% object split at each node

University of Victoria | Computer Science

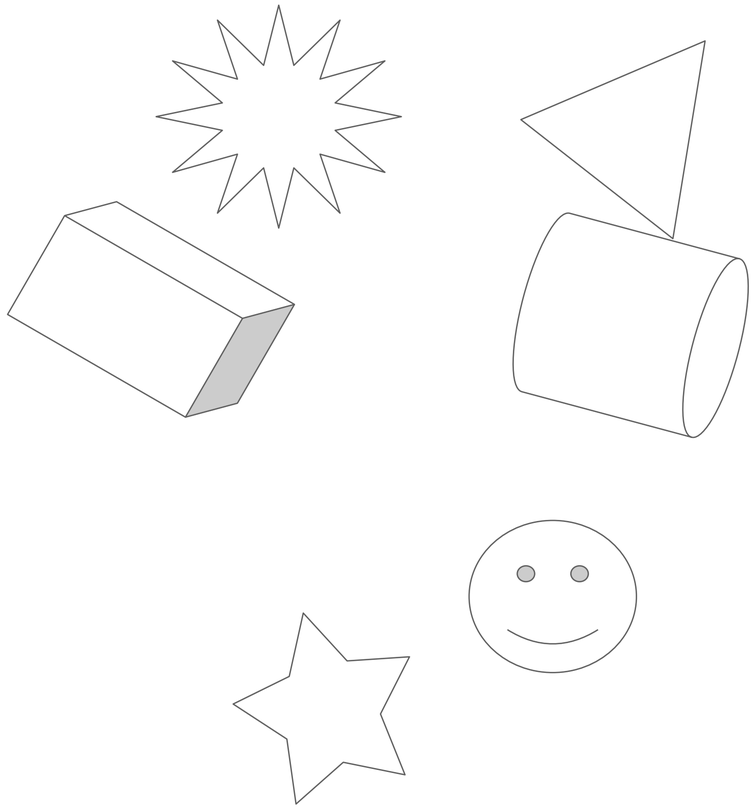
BSP-tree
Binary Spatial Partitioning tree

- Another use: to test (Generic) Polyhedron proxy:
  - note: with planes defined in its **object space**
  - each leaf: inside or outside
    - (no need to store them: left-child = in, right-child = out)
  - tree precomputed for a given object

Primitive Sorting Structures

University of Victoria | Computer Science

Bounding Volume Hierarchies (BVH)



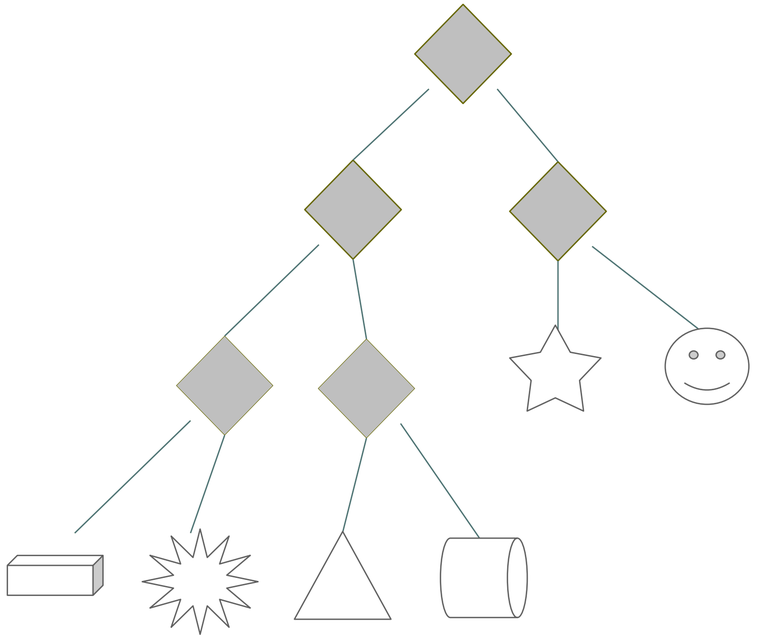


Image Copyright: Marco Tarini

Bounding Volume Hierarchies (BVH)

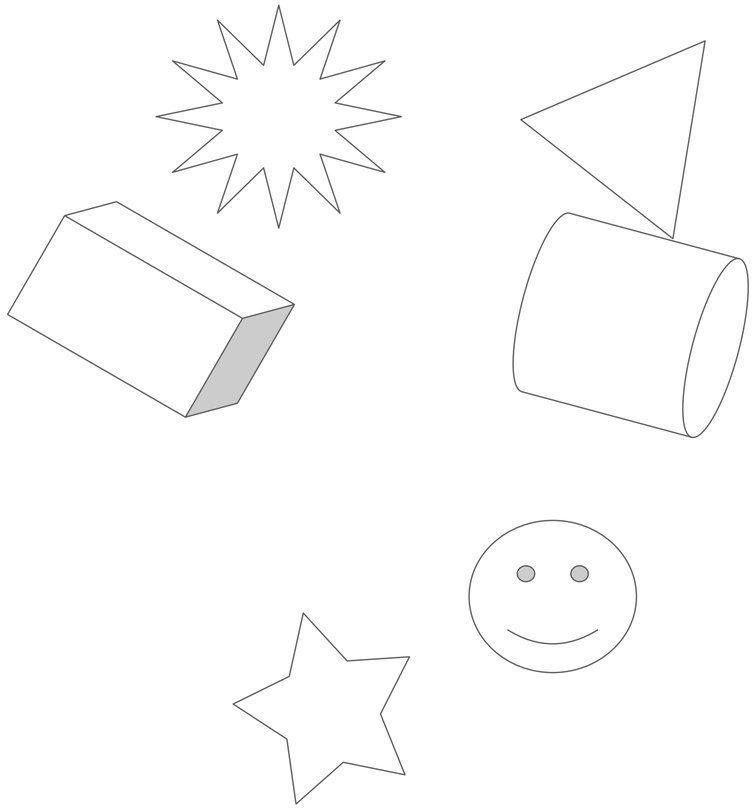
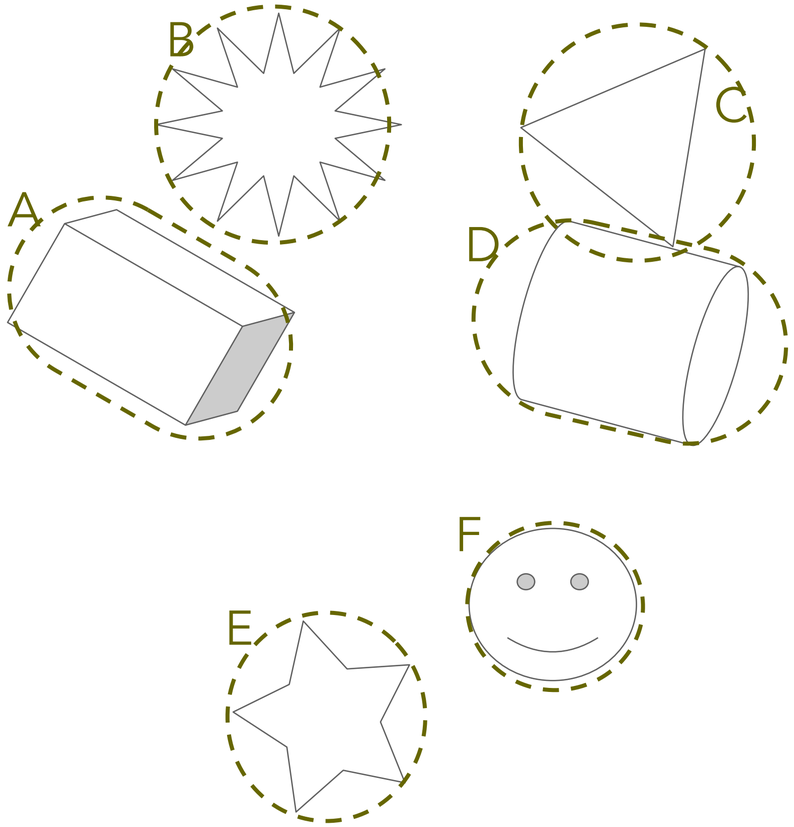
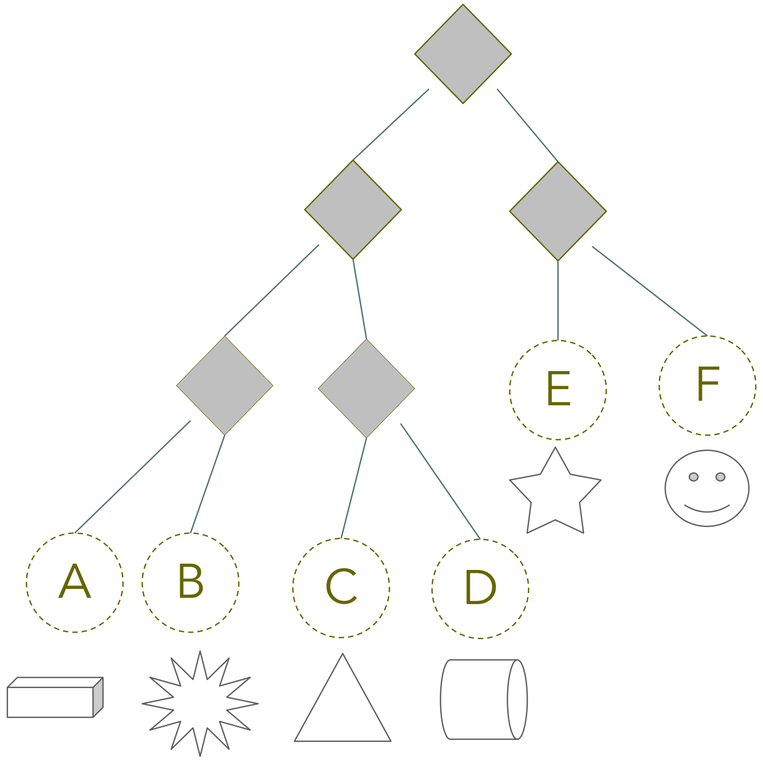




Image Copyright: Marco Tarini

Bounding Volume Hierarchies (BVH)

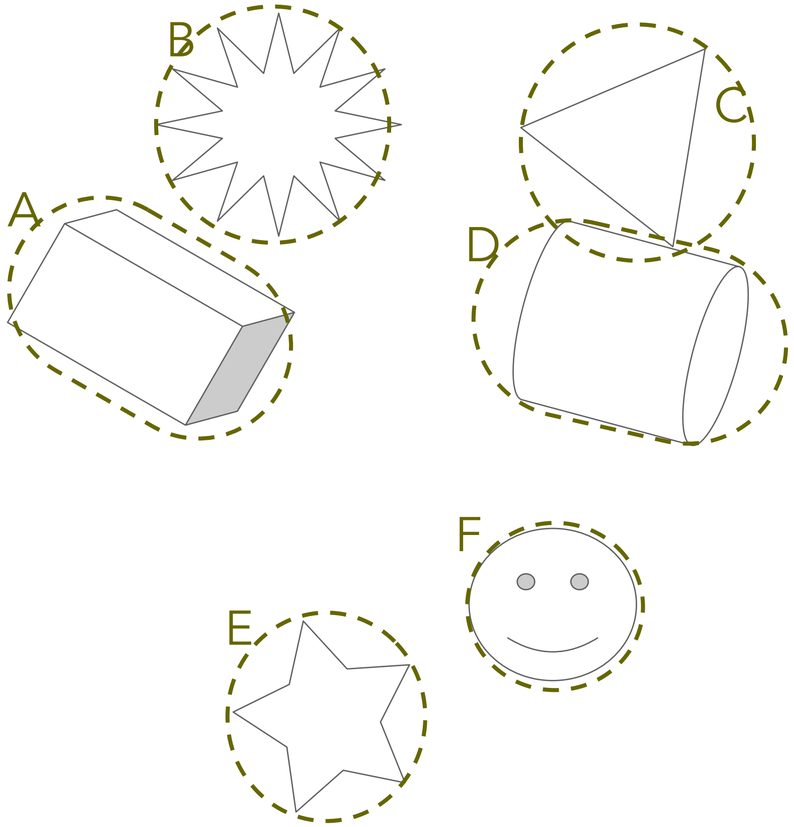




Find the pairs bounded by the smallest volume, merge, repeat.

Image Copyright: Marco Tarini

Bounding Volume Hierarchies (BVH)



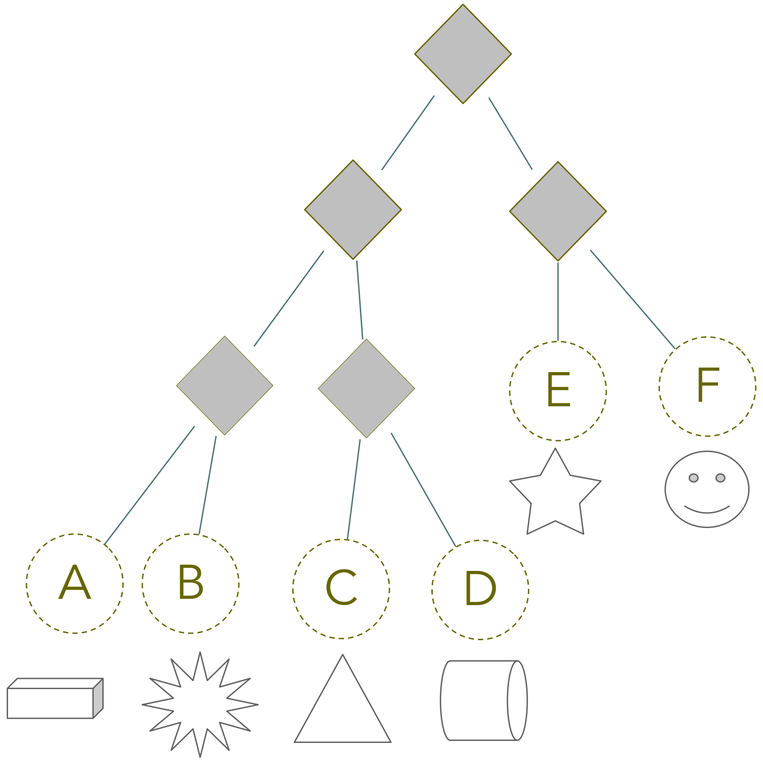
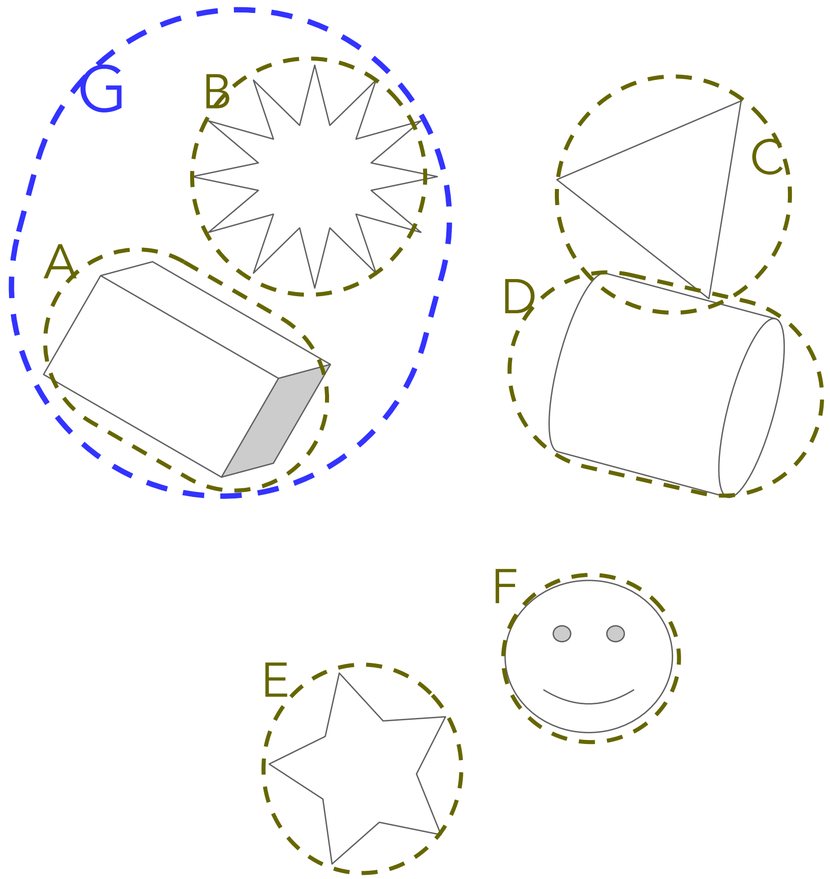


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Bounding Volume Hierarchies (BVH)



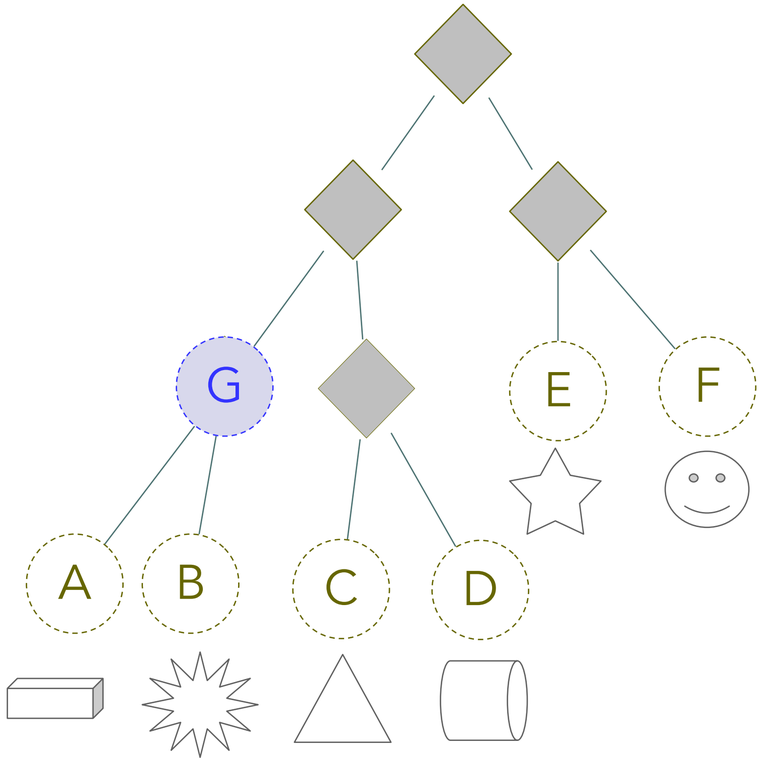
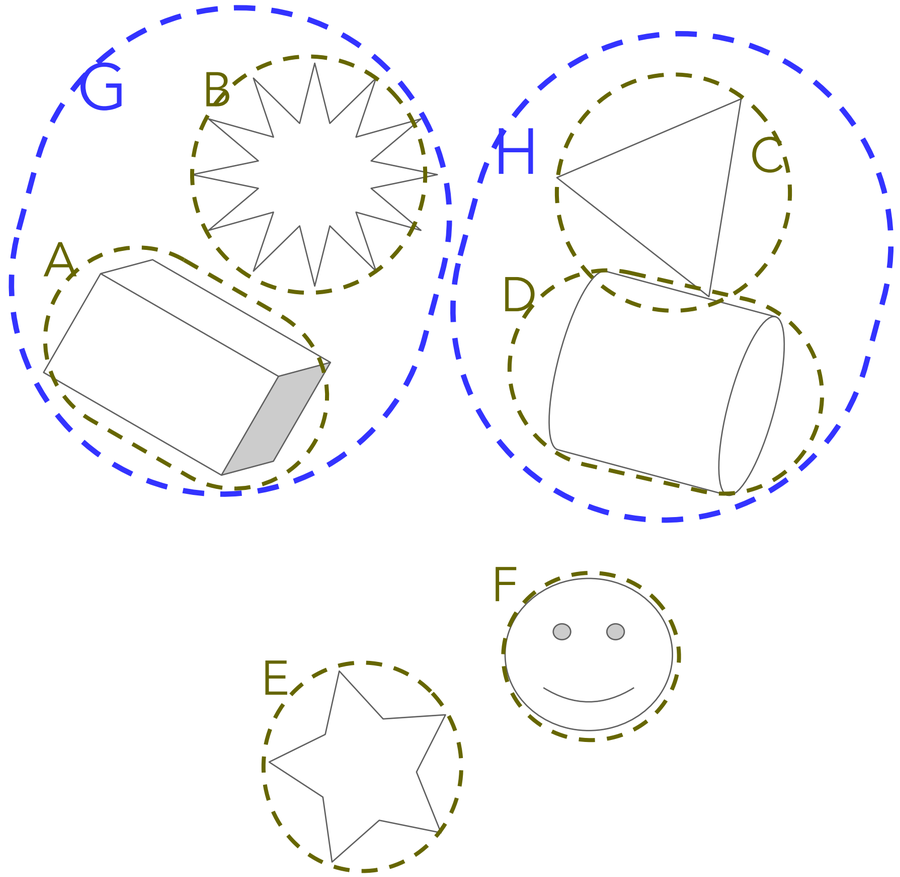


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Bounding Volume Hierarchies (BVH)



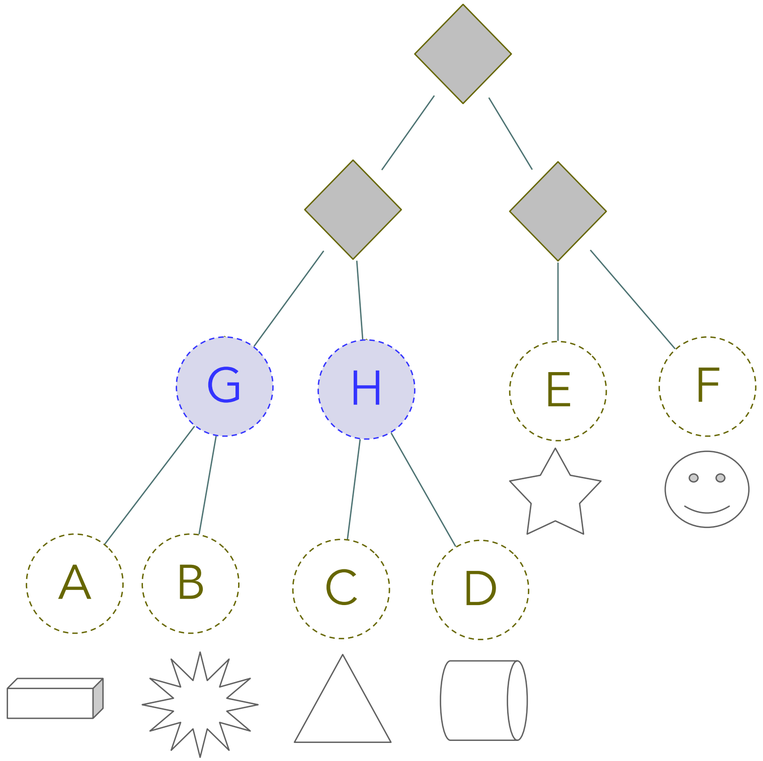
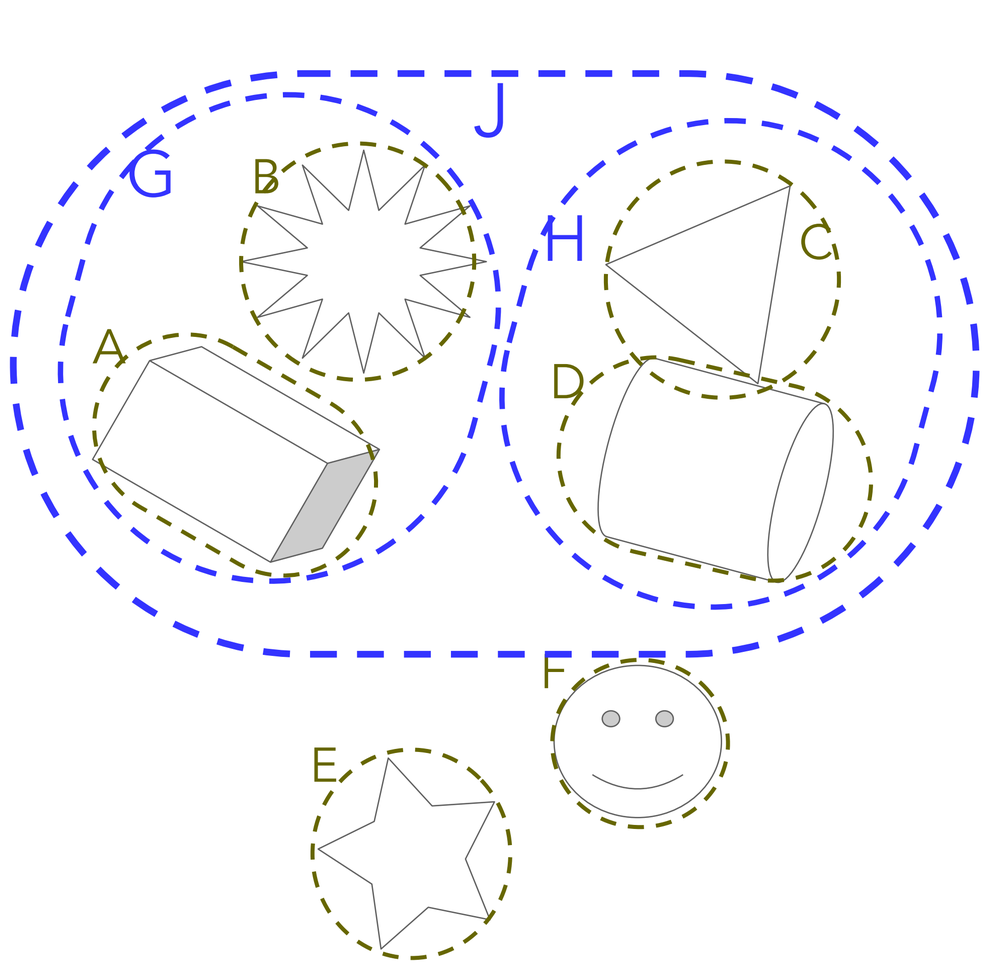


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Bounding Volume Hierarchies (BVH)



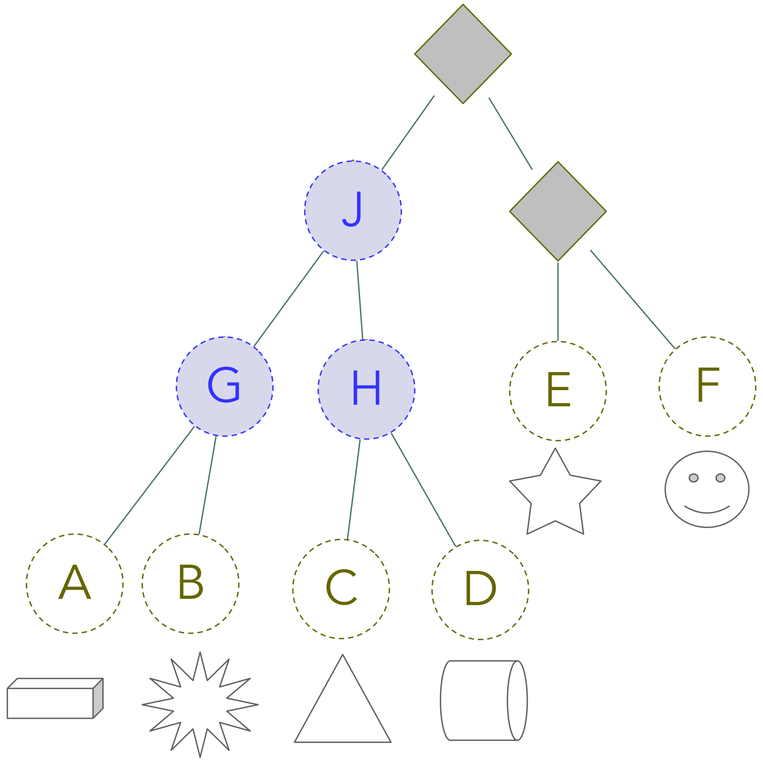
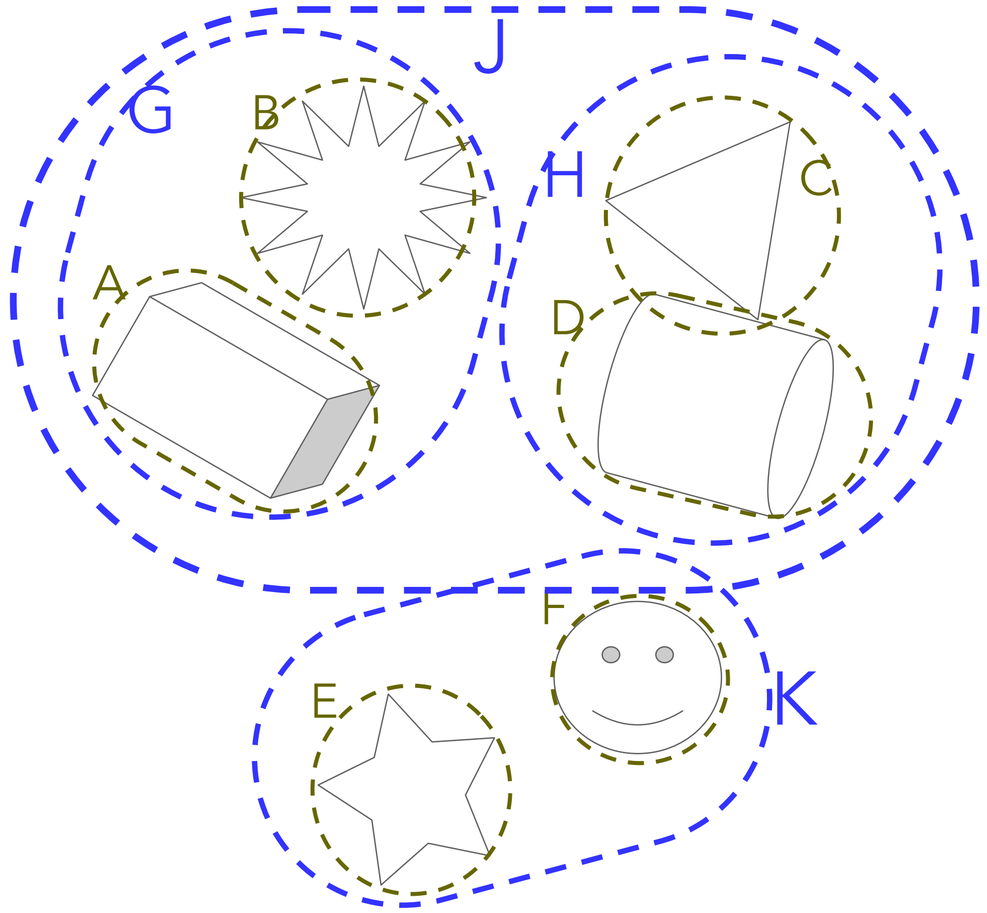


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Bounding Volume Hierarchies (BVH)



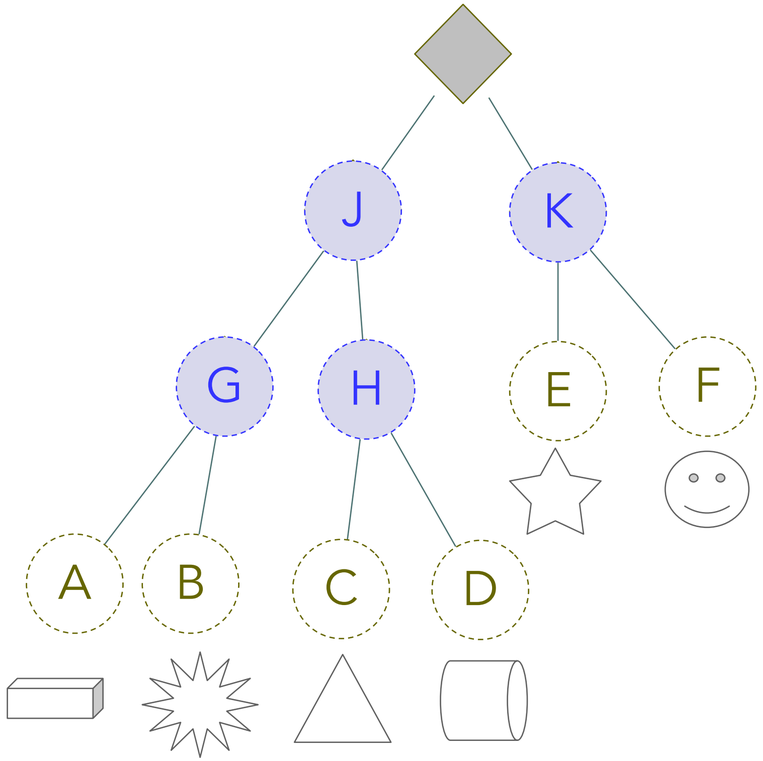
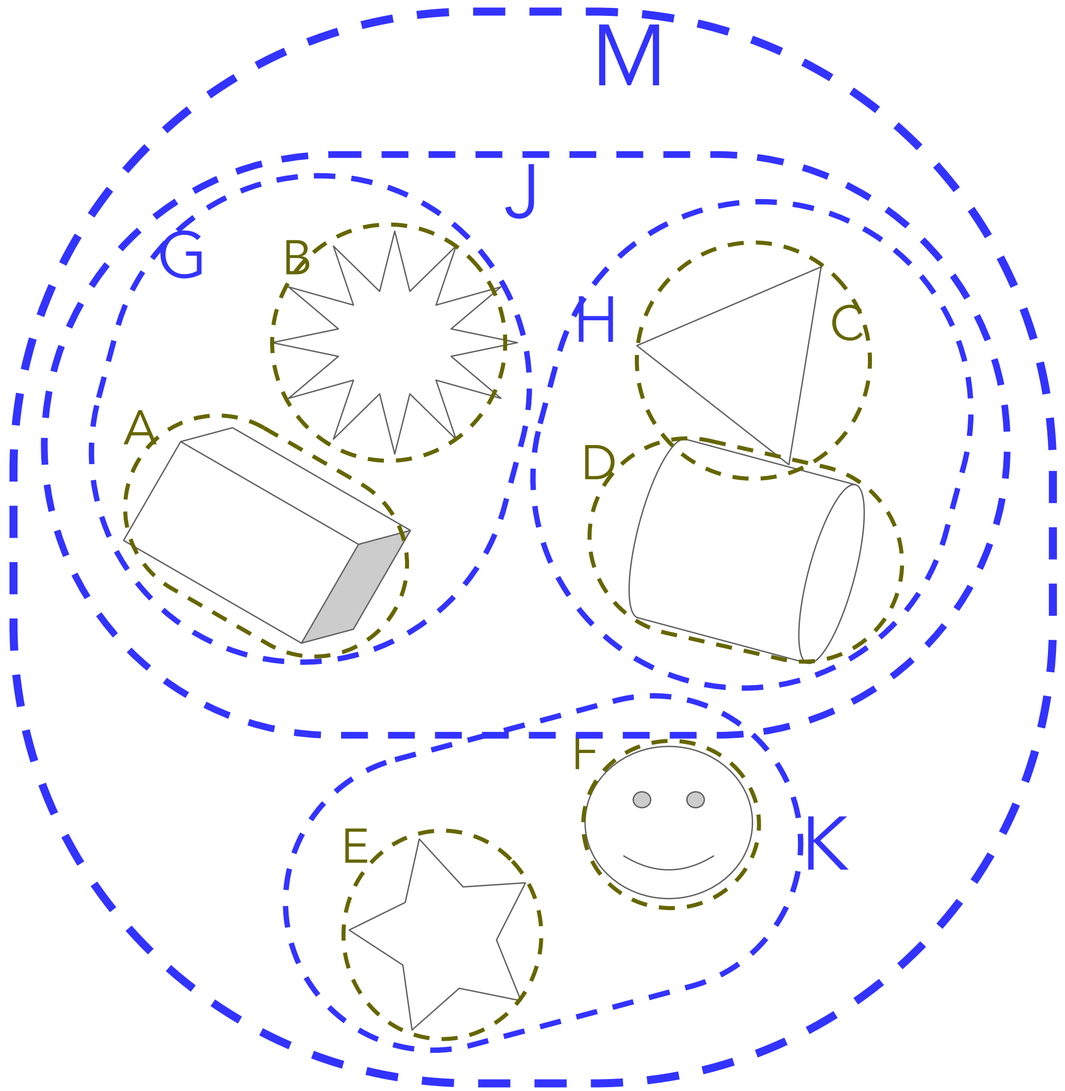


Image Copyright: Marco Tarini

University of Victoria | Computer Science

Bounding Volume Hierarchies (BVH)



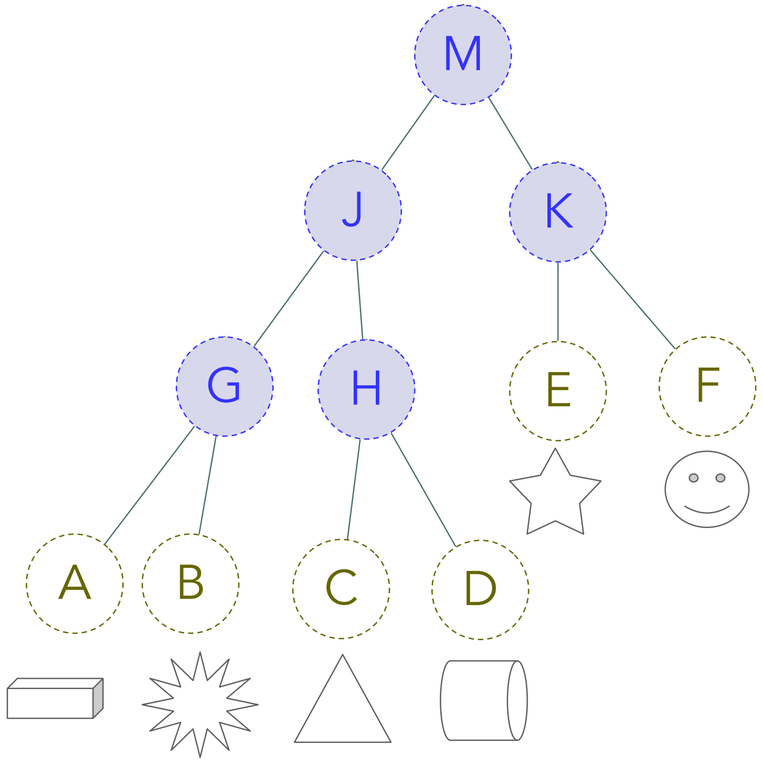


Image Copyright: Marco Tarini

University of Victoria | Computer Science

BVH
Bounding Volume Hierarchy

- Idea: use the scene hierarchy given by the scene graph
  - (instead of a spatial derived one)
- associate a Bounding Volume to each node
  - rule: a BV of a node bounds all objects in the subtree
- construction / update is fast
  - bottom-up: recursive
  - Can also be built top-down
- using it:
  - top-down: visit
  - *note:* ***not*** a single root to leaf path
    - may need to follow *multiple* children of a node (in a BSP-tree: only one)

University of Victoria | Computer Science

Spatial Indexing Structures

- Regular Grid
  - the most parallelizable (to update / construct / use)
  - constant time access (best!)
  - quadratic / cubic space (2D, 3D)
- kD-tree, Oct-tree, Quad-tree
  - compact
  - simple
  - non constant accessing time (still logarithmic on average)
- BSP-tree
  - optimized splits! best performance when accessed
  - optimized splits! more complex construction / update
  - ideal for static parts of the scene
  - (also, used for generic polyhedron inside/outside test)
- BVH
  - simplest construction
  - non necessarily very efficient to access
    - may need to traverse multiple children
    - if you do not have a scene-graph you need to create one
  - ideal for dynamic parts of the scene

Intersection Acceleration Data Structures

University of Victoria | Computer Science

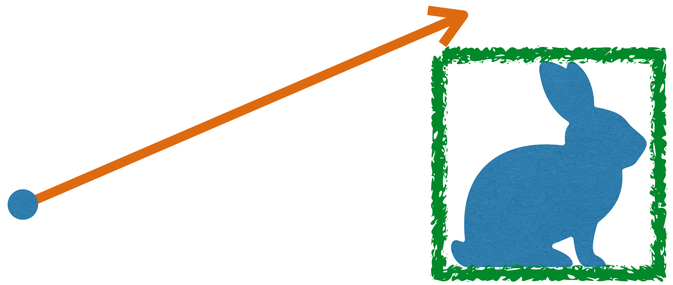
Collision Detection

- It is easy to do, the challenge is to do it efficiently
- An observation:
  - most pair of objects do not intersect each other in a scene, **collisions are rare**
  - optimizing the intersections directly is important but not sufficient, we need to optimize the detection of non intersecting pairs ("**early rejects**")

University of Victoria | Computer Science

Collision Detection

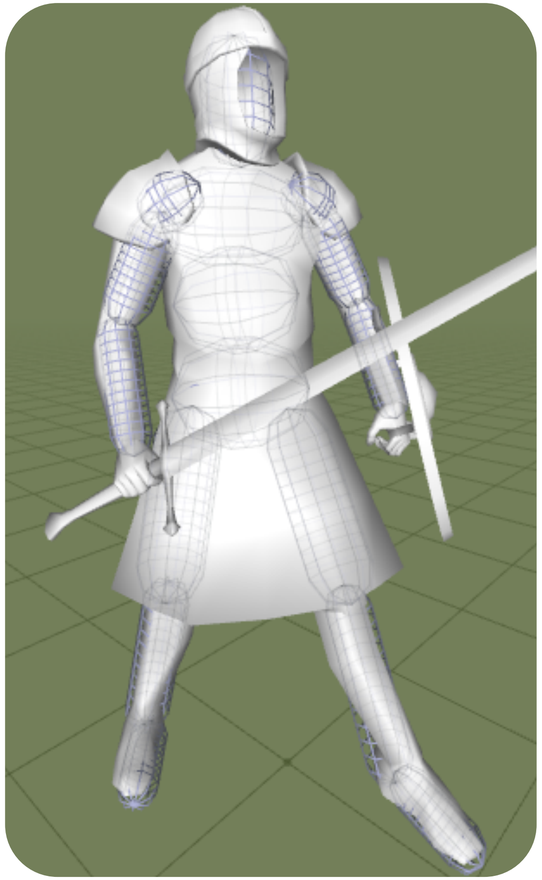
- It is easy to do, the challenge is to do it efficiently
- An observation:
  - most pair of objects do not intersect each other in a scene, **collisions are rare**
  - optimizing the intersections directly is important but not sufficient, we need to optimize the detection of non intersecting pairs ("**early rejects**")

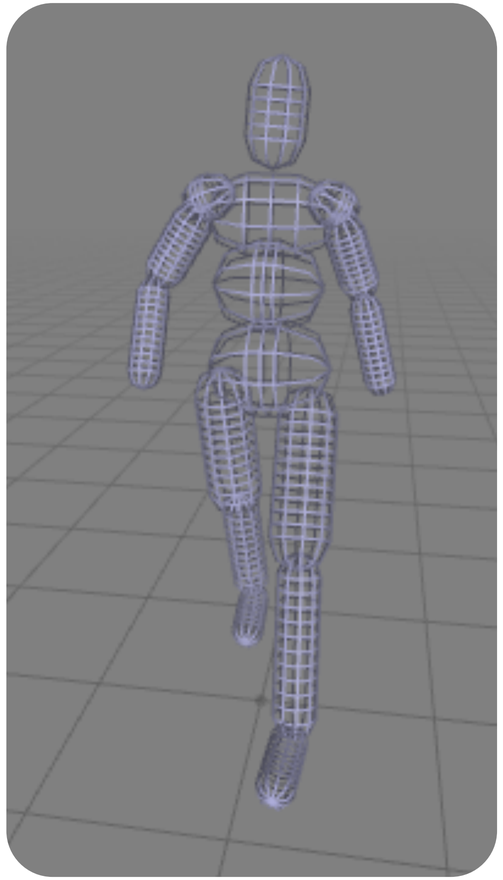


University of Victoria | Computer Science

Geometric Proxies

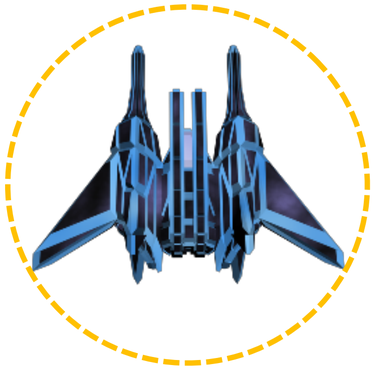
- Idea: use a geometric proxy to approximate the objects in the scene

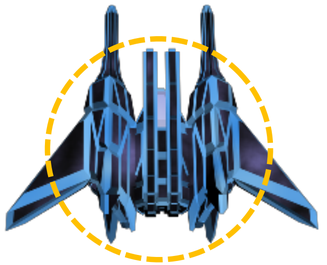




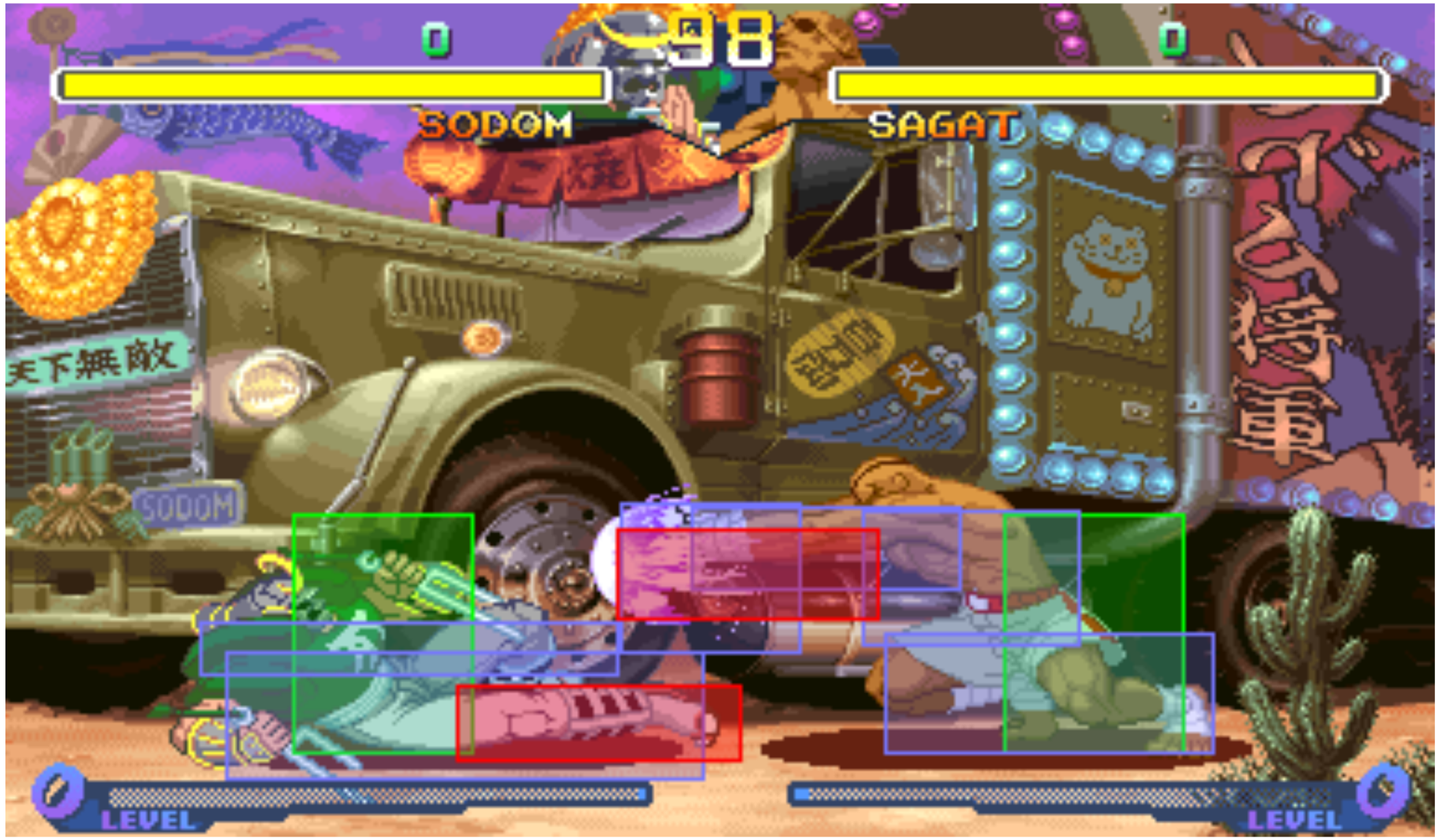
Geometric Proxy

- Extremely coarse approximation
- Used as a:
  - Bounding Volume
    - the entire object must be contained inside
    - exact result, you need to do more work if you detect a collision
  - Collision Object (or “hit-box”)
    - approximation of the object
    - no need to do anything else if an approximation is ok for your use case



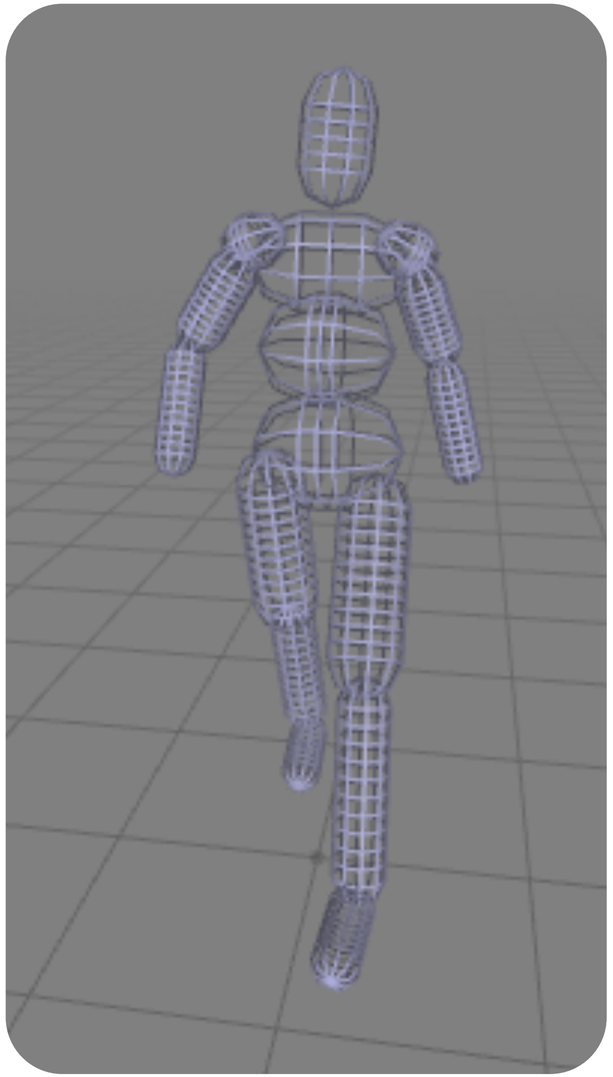


Example: Fighting Games



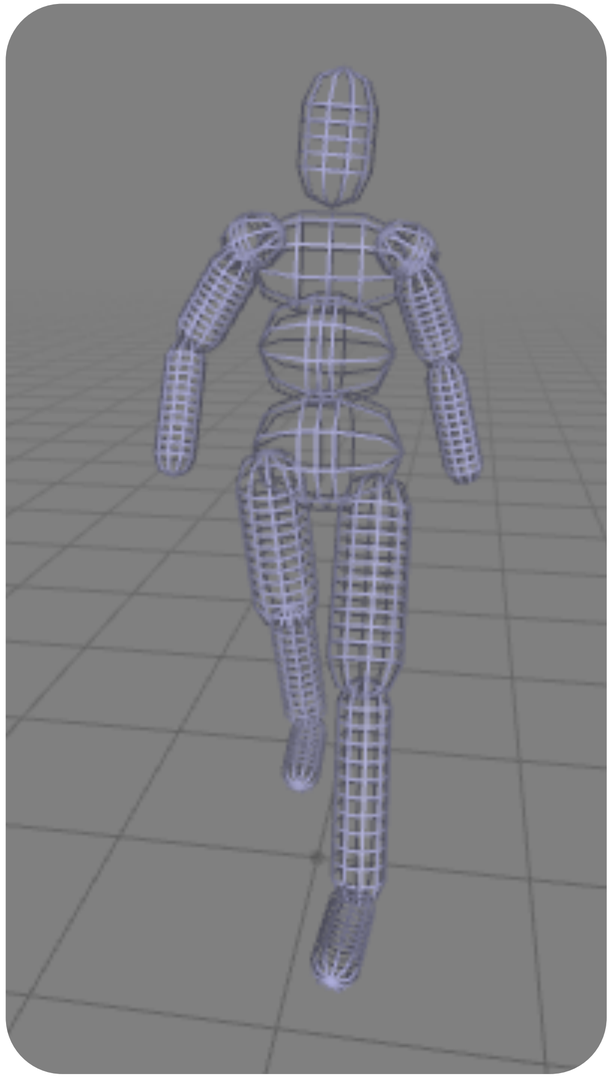
Street Fighter Alpha, CAPCOM 1995

Extremely Common



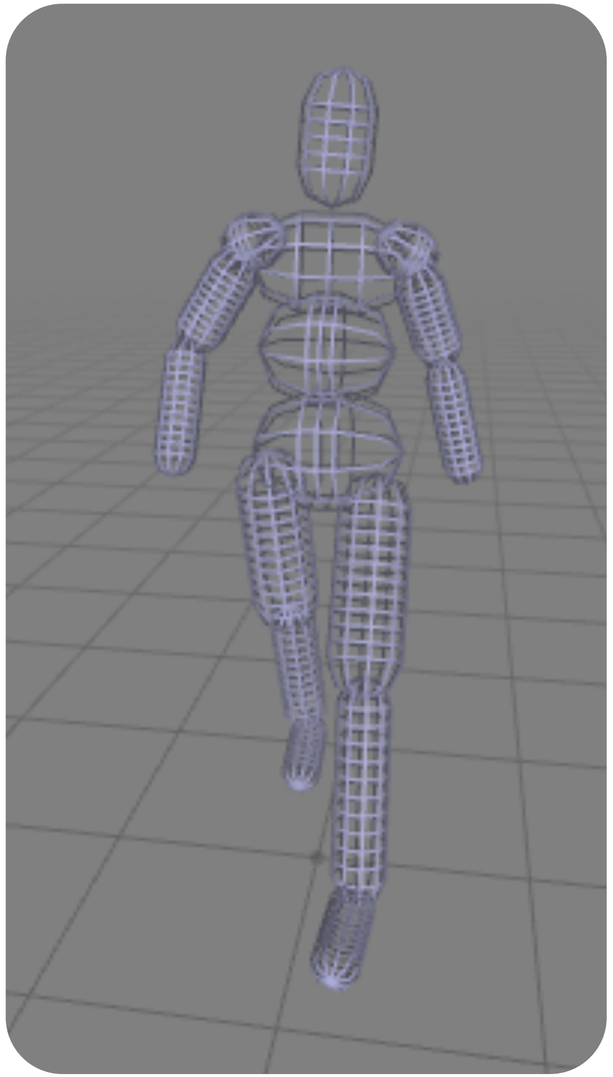
Extremely Common

- Physics engine
  - collision detection
  - collision response



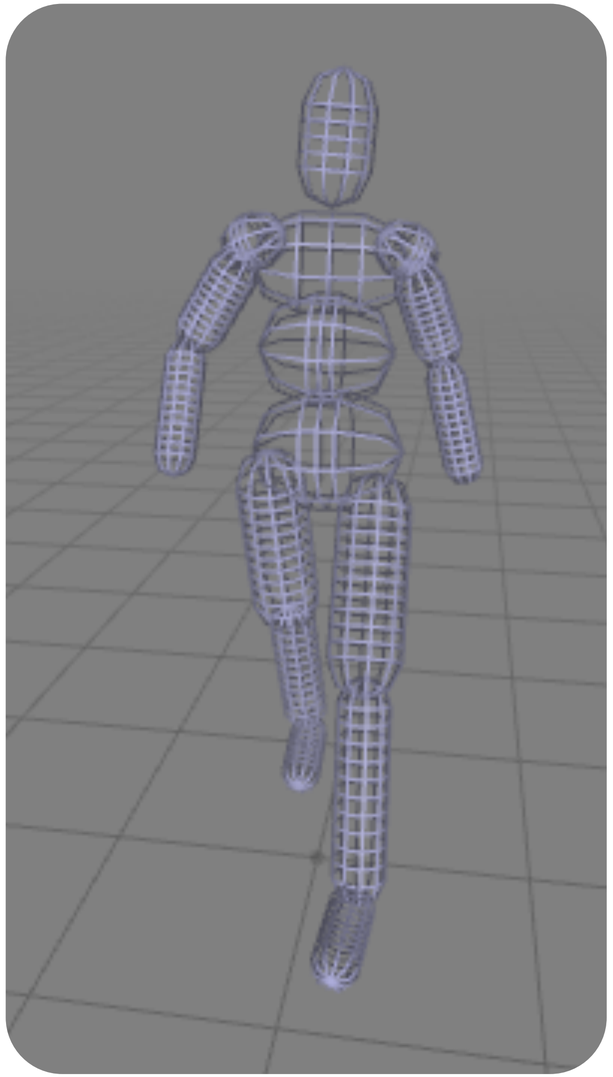
Extremely Common

- Physics engine
  - collision detection
  - collision response
- Rendering
  - view frustum culling
  - occlusion culling



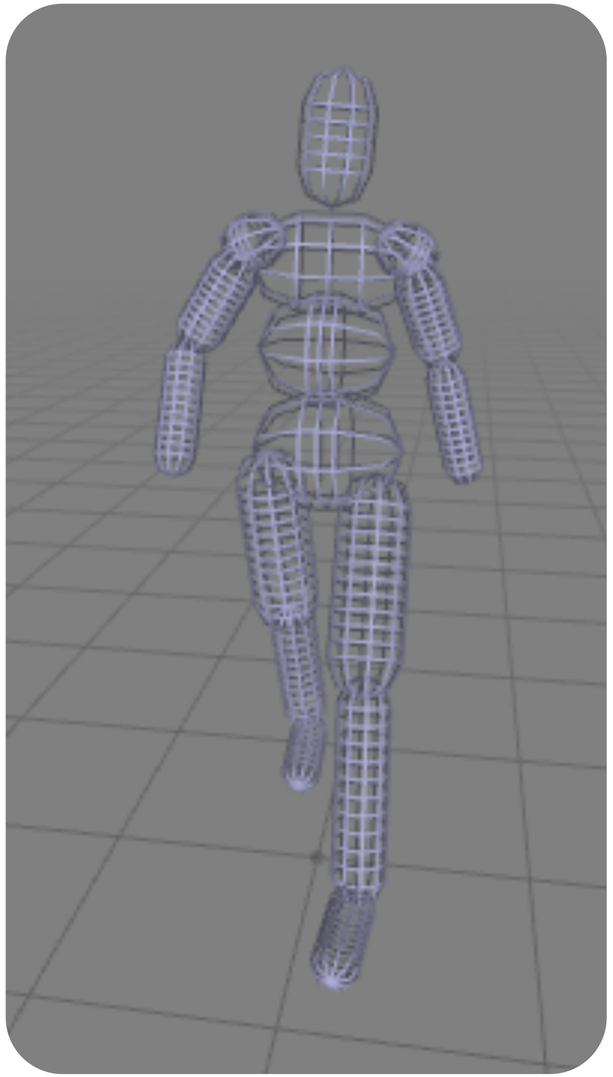
Extremely Common

- Physics engine
  - collision detection
  - collision response
- Rendering
  - view frustum culling
  - occlusion culling
- AI
  - visibility test



Extremely Common

- Physics engine
  - collision detection
  - collision response
- Rendering
  - view frustum culling
  - occlusion culling
- AI
  - visibility test
- GUI
  - picking



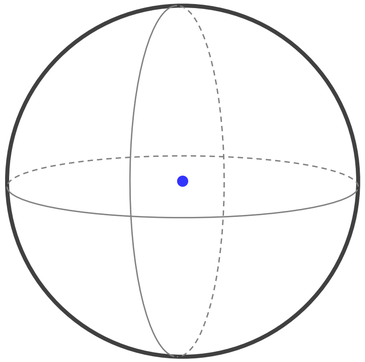
Properties of Geometric Proxies

1. How expensive are they to compute/update?
2. How much space do you need?
3. Are they invariant to the transformations applied on the object?
4. How good is the approximation?
5. How expensive are the collision queries with the other objects in the scene?

University of Victoria | Computer Science

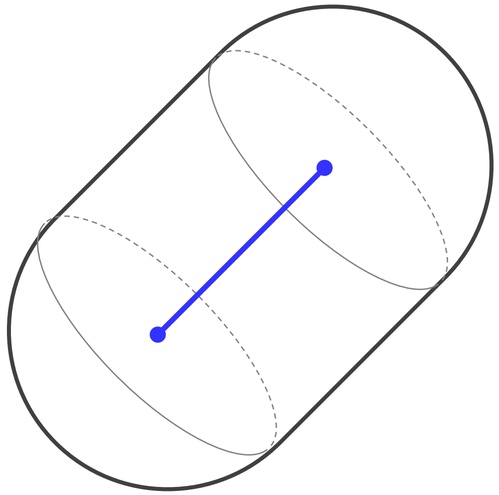
Geometry Proxies: Sphere

- Easy to compute and update
- Compact (center, radius)
- Very efficient collision tests
- Can only be transformed rigidly
- The quality of the approximation is low



Geometry Proxies: Capsule

- Def:
  - Sphere ==
    set of all points with dist from a point < radius
  - Capsule ==
    set of all points with dist from a segment < radius
    - i.e. a cylinder ended with two half-spheres (all same radius)



- Stored with:
  - a segment (two end-points)
  - a radius (a scalar)
- Popular option, compact to store, easy to construct, easy to detect intersections, good approximation

Geometry Proxies: Half Space

- Trivial, but useful
  - e.g. for a flat terrain,
    or a wall



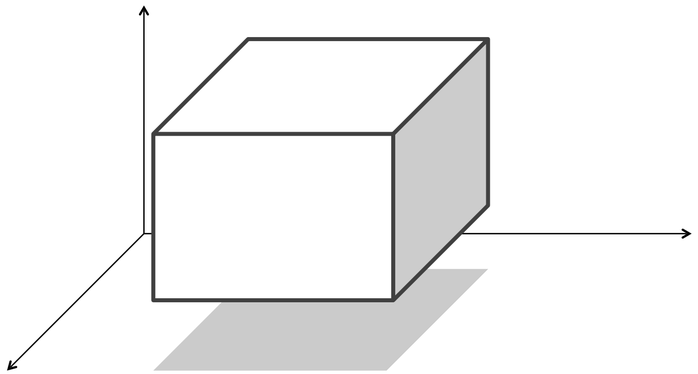
- Storage:
  - (nx, ny, nz, k)
  - a normal, a distance from the origin

- Tests are trivial

University of Victoria | Computer Science

Geometry Proxies:
Axis-Aligned Bounding Box (AABB)

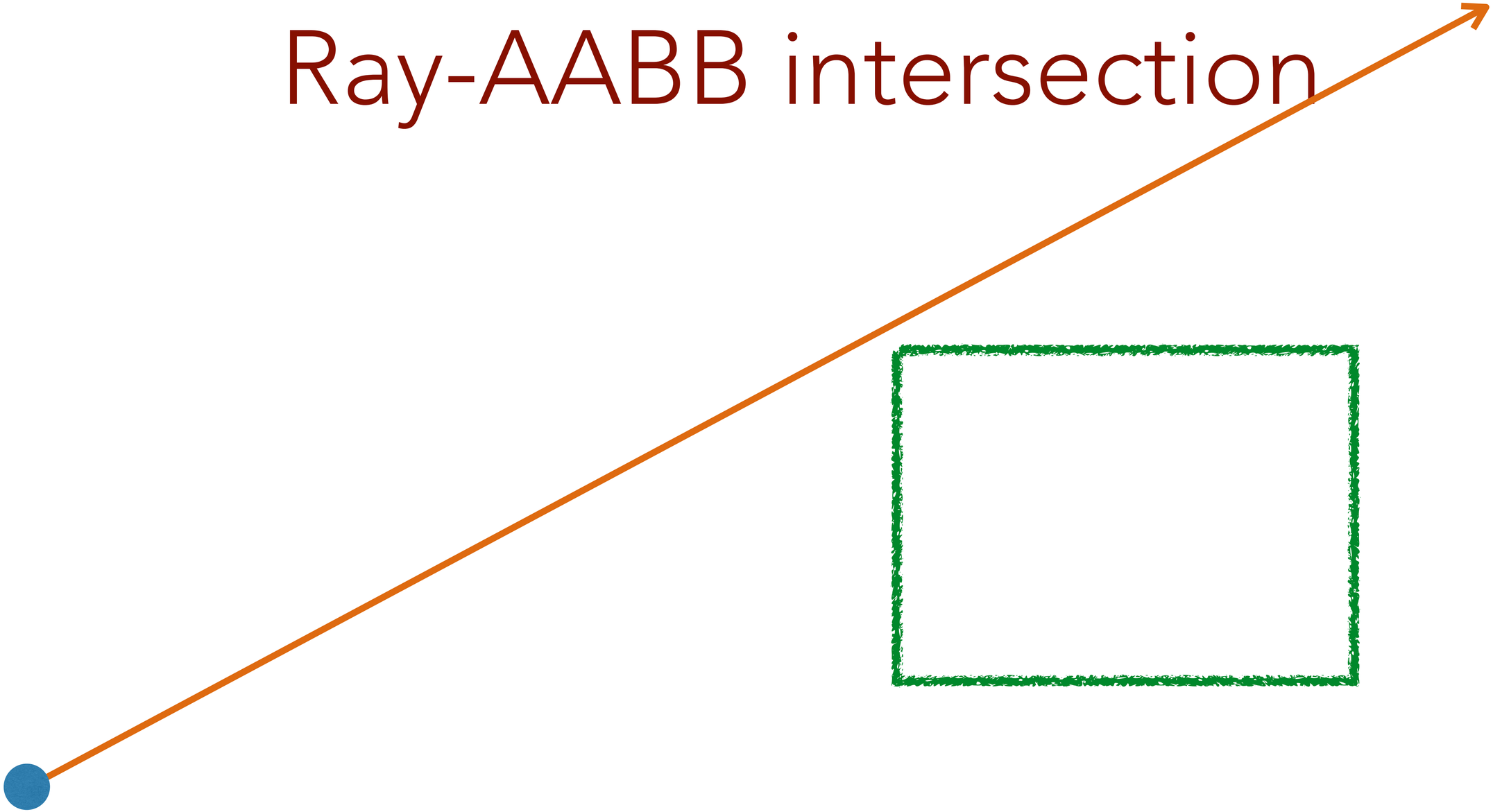
- Easy to update
- Compact (three intervals)
- Trivial to test



- It can only be translated or scaled, rotations are not supported

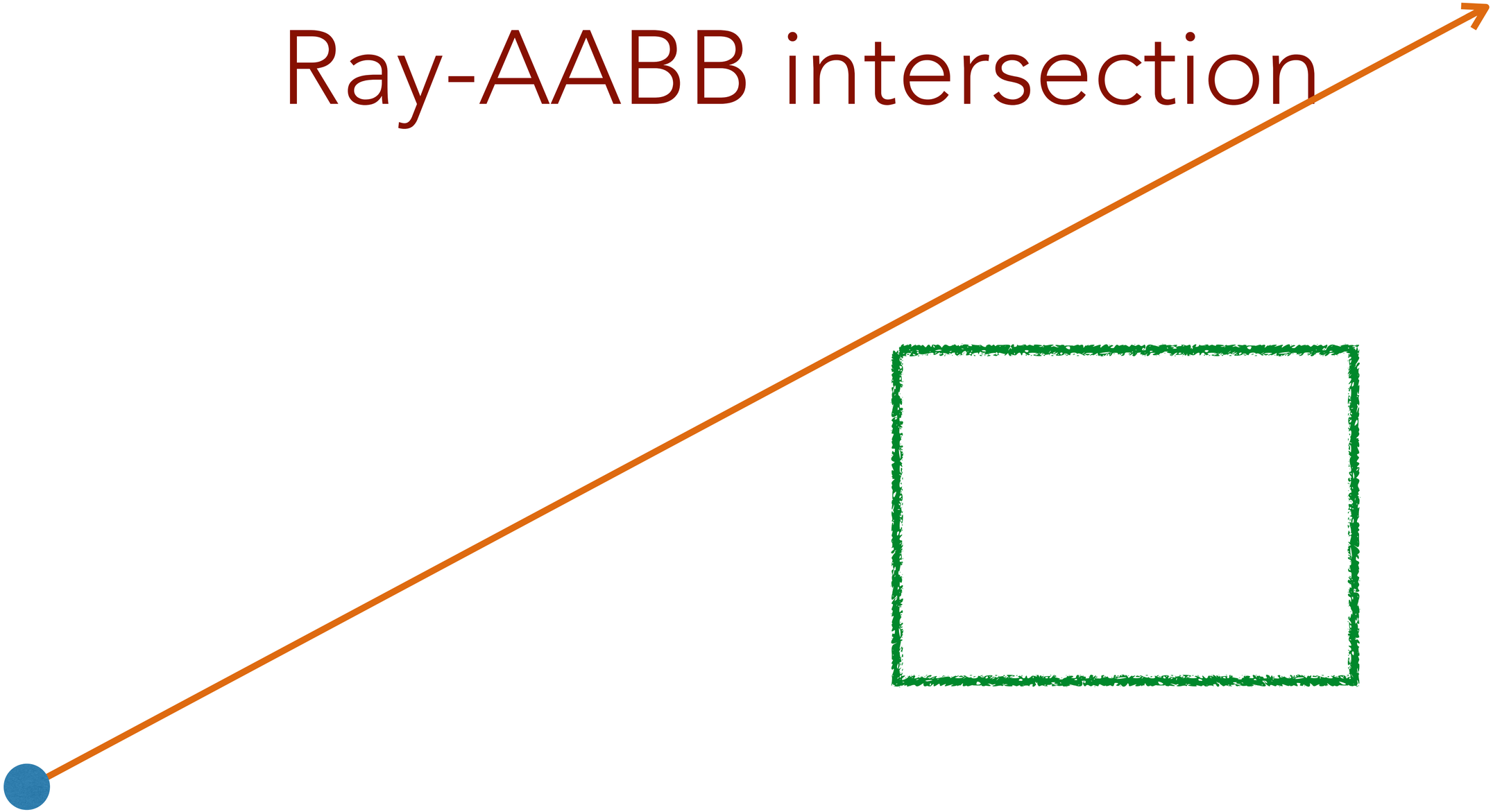
University of Victoria | Computer Science

Ray-AABB intersection



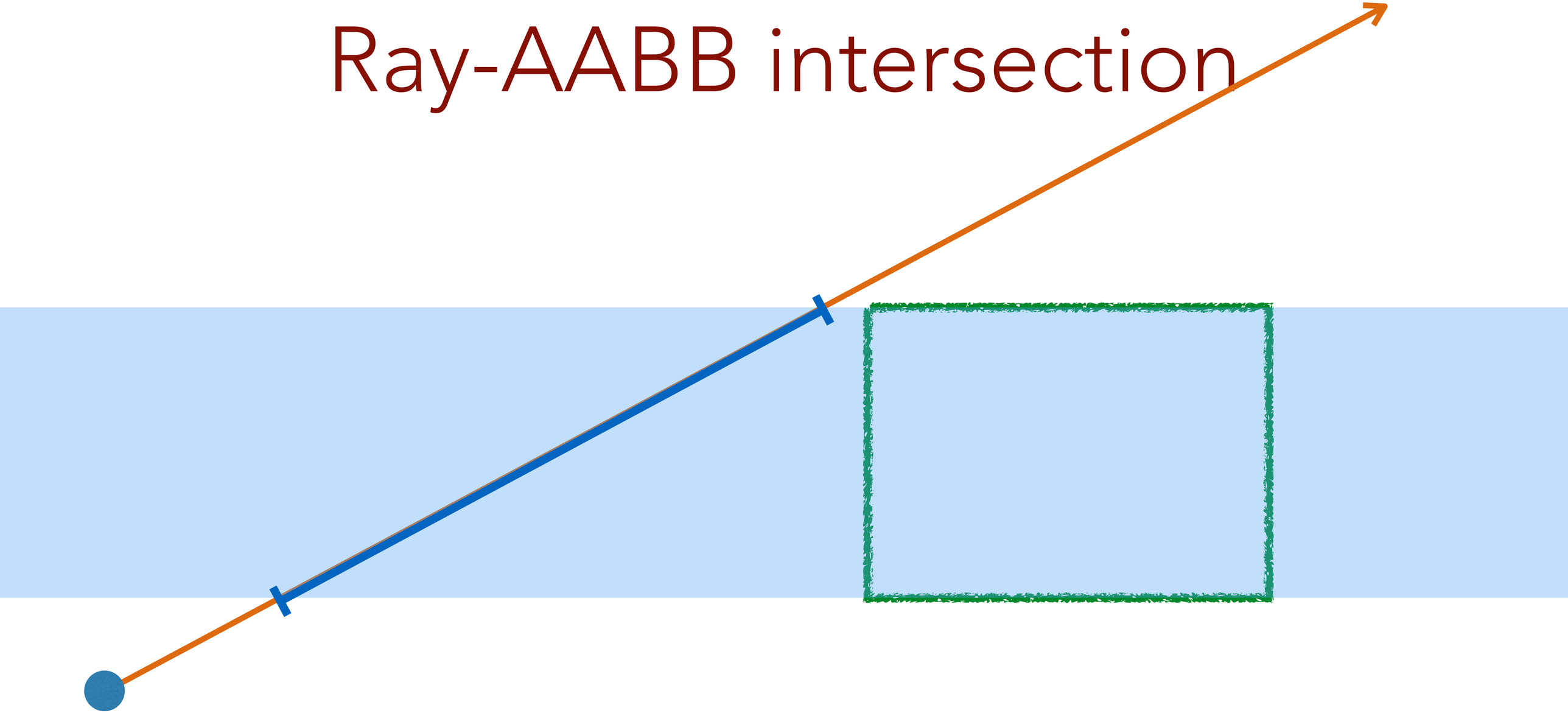
University of Victoria | Computer Science

Ray-AABB intersection



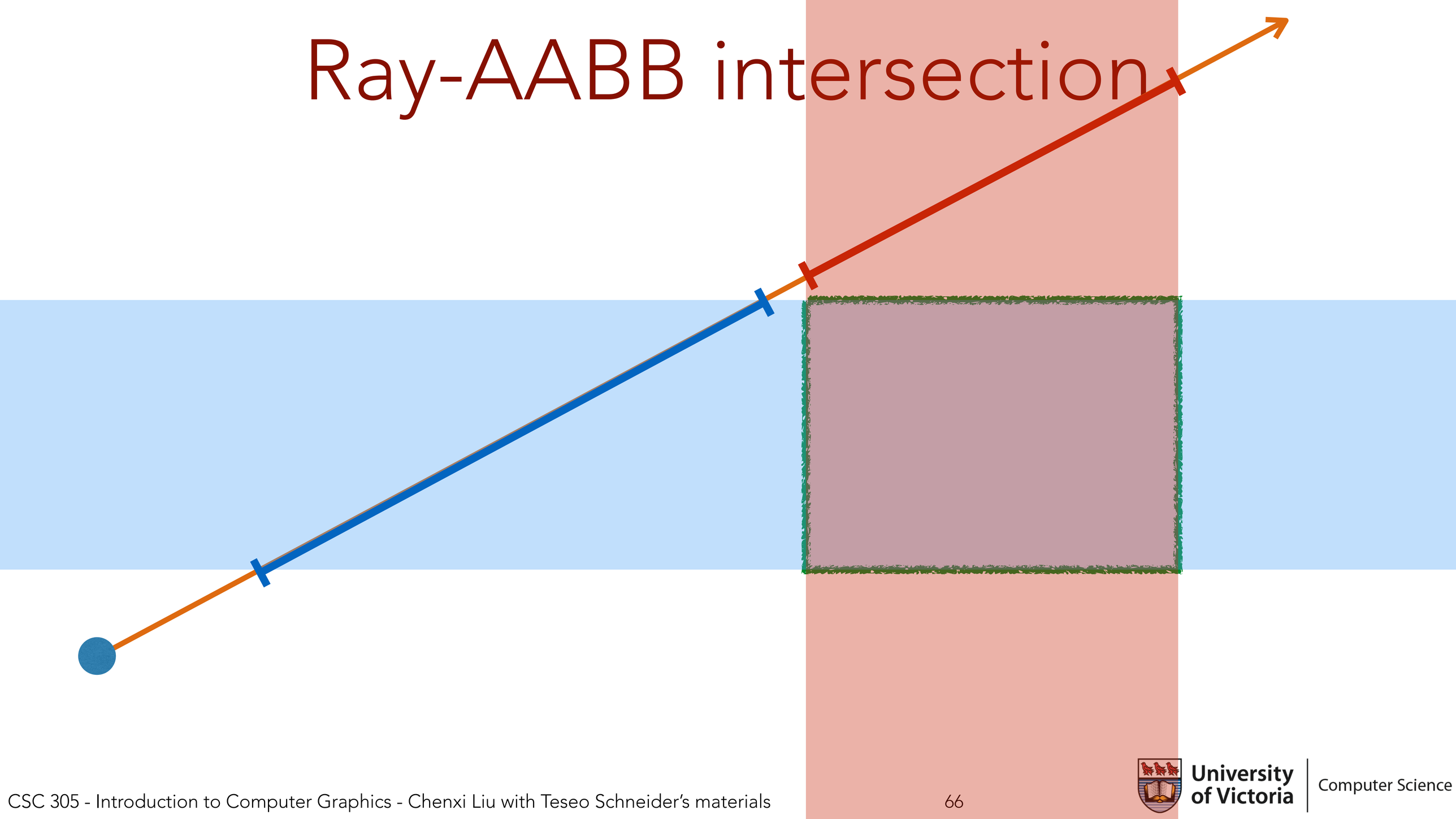
University of Victoria | Computer Science

Ray-AABB intersection

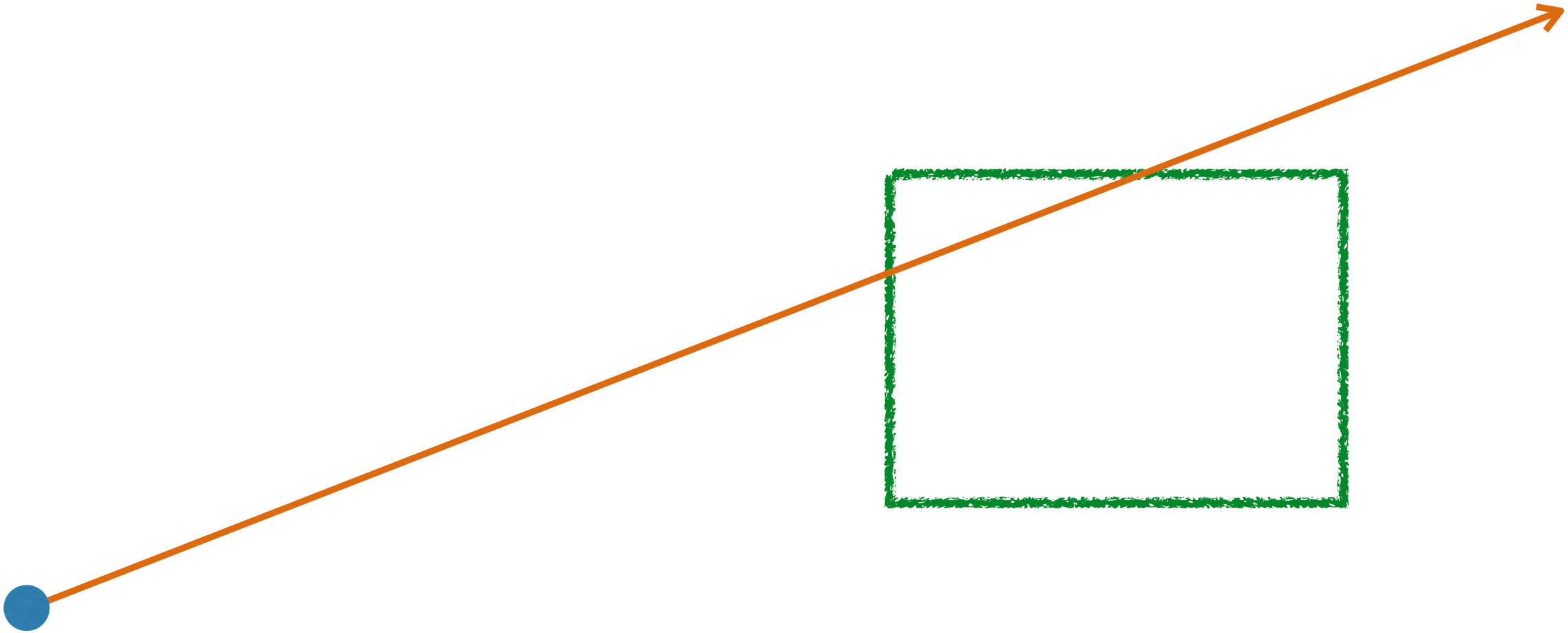


University of Victoria | Computer Science

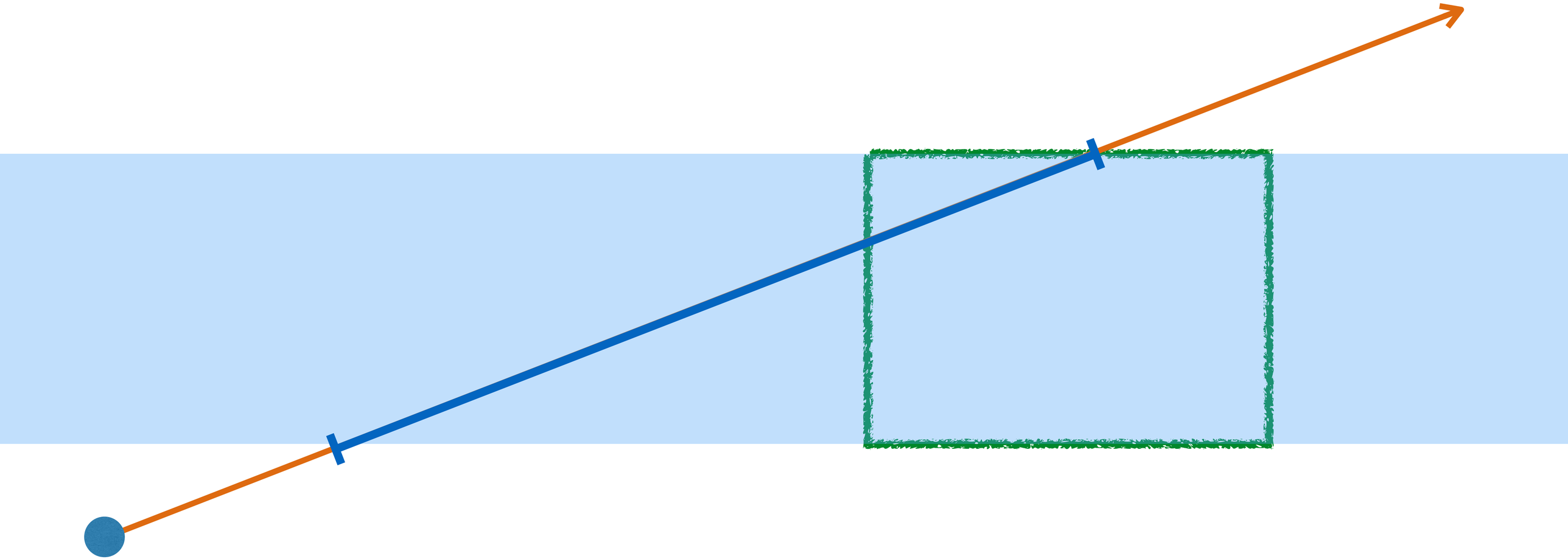
Ray-AABB intersection



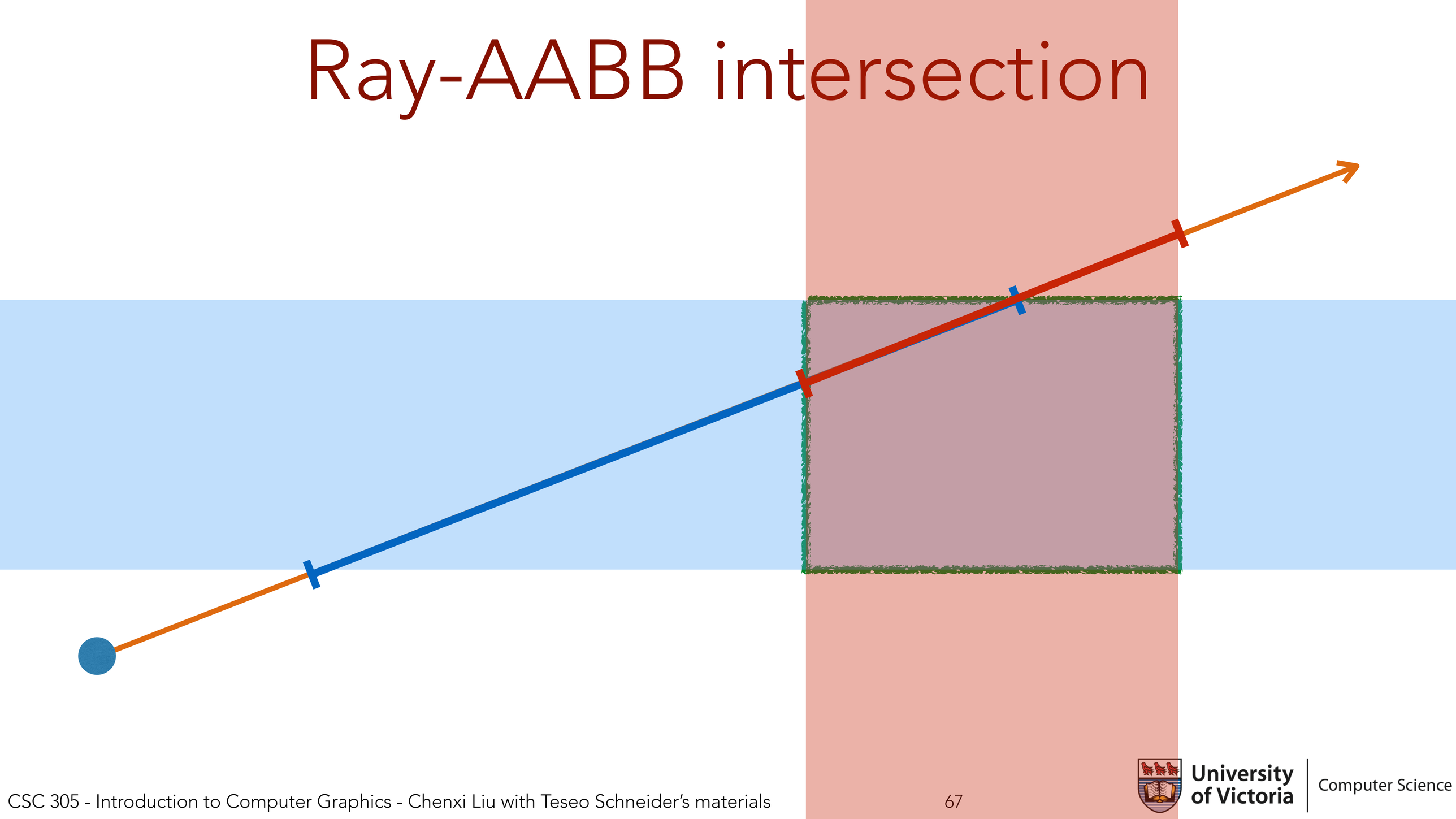
Ray-AABB intersection



Ray-AABB intersection



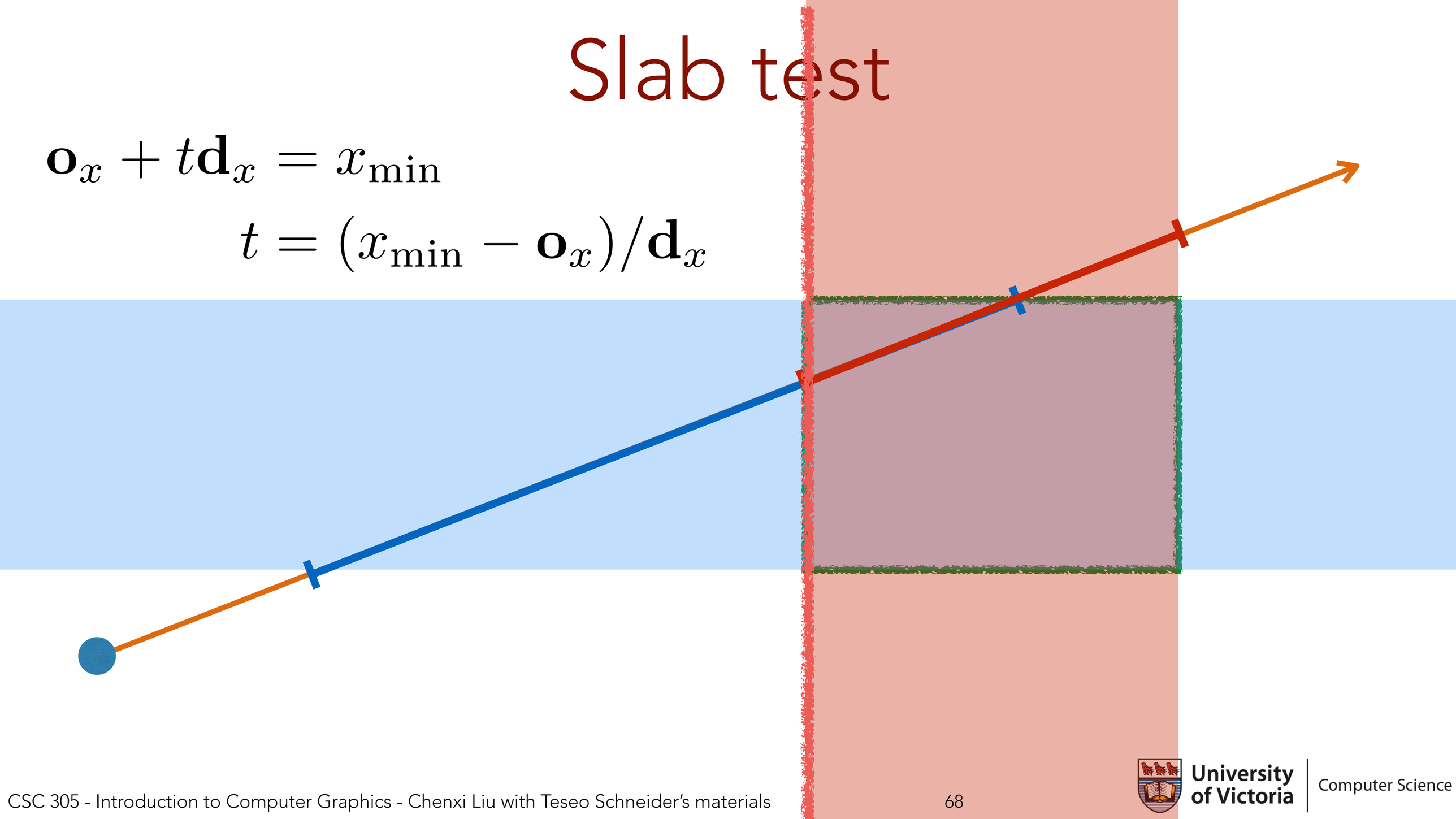
Ray-AABB intersection



Slab test

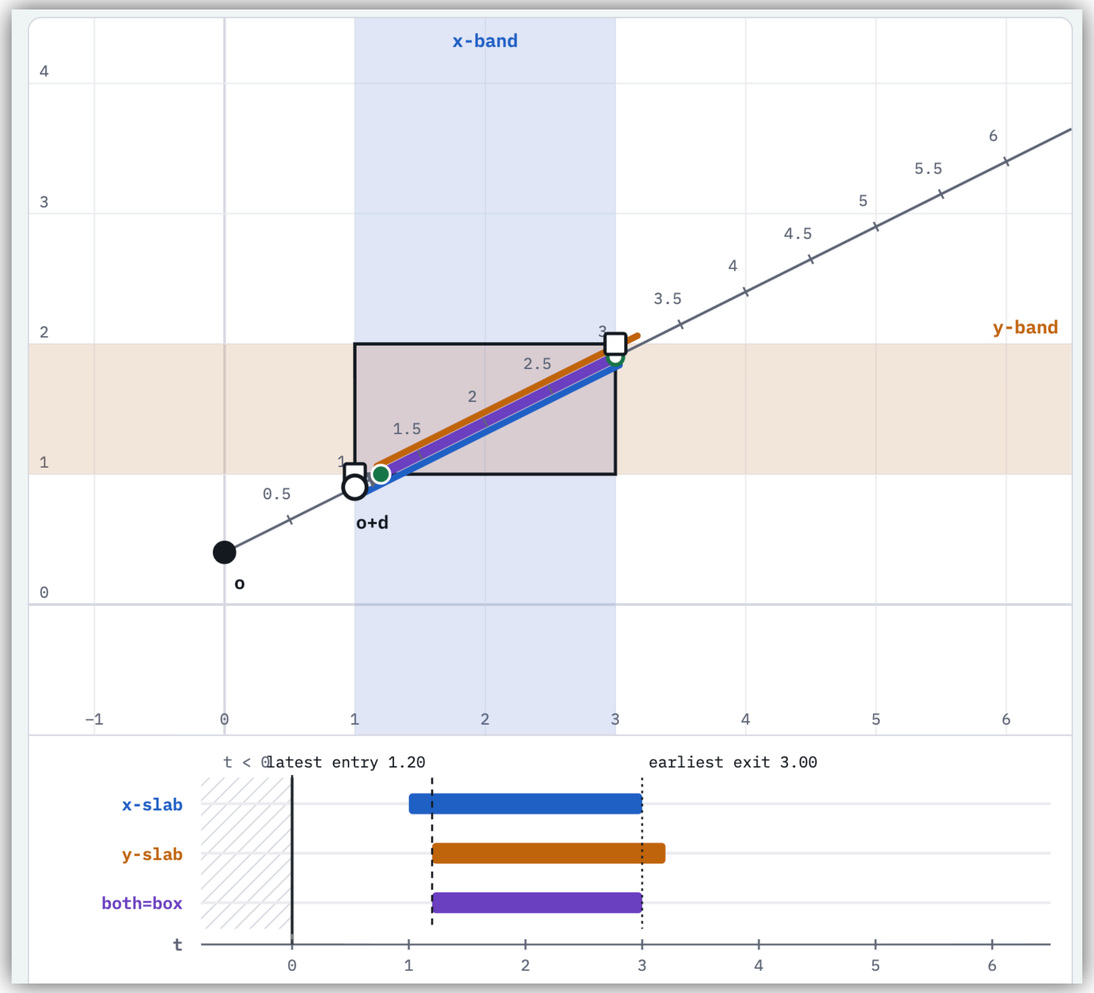
$$\mathbf{o}_x + t\mathbf{d}_x = x_{\min}$$

$$t = (x_{\min} - \mathbf{o}_x)/\mathbf{d}_x$$



University of Victoria | Computer Science

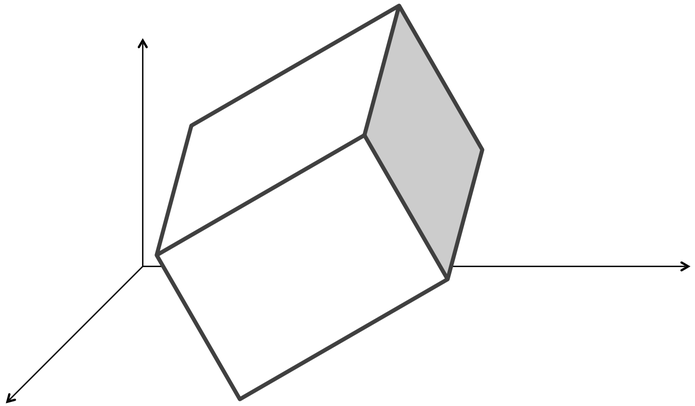
Slab test



University of Victoria | Computer Science

Geometry Proxies: Box

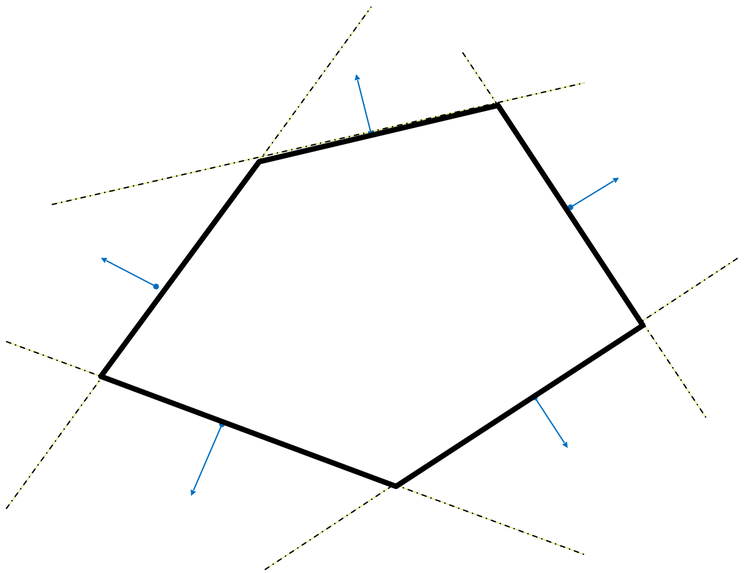
- Similar to AABB, but not axis-aligned
- More expensive to compute and store
  - You need intervals and a rotation
- Still not a great approximation, but it is invariant to rotations and it is fast to compute and use



University of Victoria | Computer Science

Geometry Proxies (in 2D): Convex Polygon

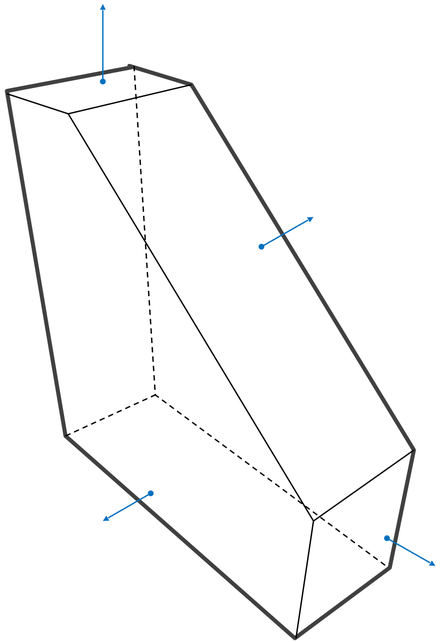
- Intersection of half-planes
  - each delimited by a line
- Stored as:
  - a collection of (oriented) lines
- Test:
  - a point is inside iff
    it is in each half-plane
- Good approximation
- Moderate complexity



University of Victoria | Computer Science

Geometry Proxies (in 3D): Convex Polyhedron

- Intersection of half-spaces
- Similar as previous,
  but in 3D
  - Stored as a collection
    of planes
  - Each plane is a normal + distance from origin
  - Test: inside proxy iff
    inside each half-space



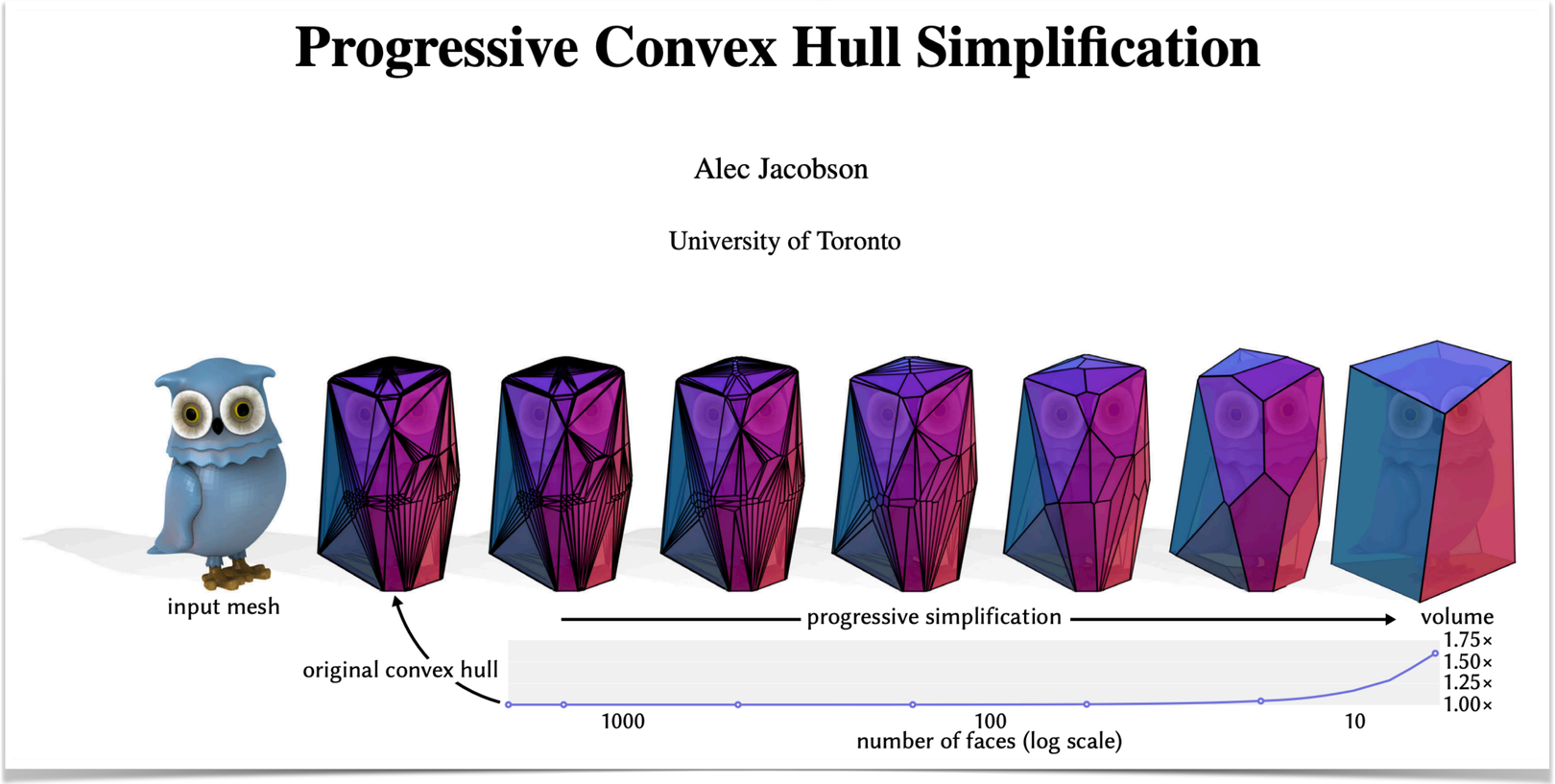
University of Victoria | Computer Science

Geometry Proxies (in 3D): (General) Polyhedron

- Luxury Hit-Boxes :)
  - The most accurate approximations
  - The most expensive tests / storage
- Specific algorithms to test for collisions
  - requiring some preprocessing
  - and data structures (BSP-trees)
- Creation (as meshes):
  - sometimes, with automatic simplification
  - often, hand made (low poly modelling)

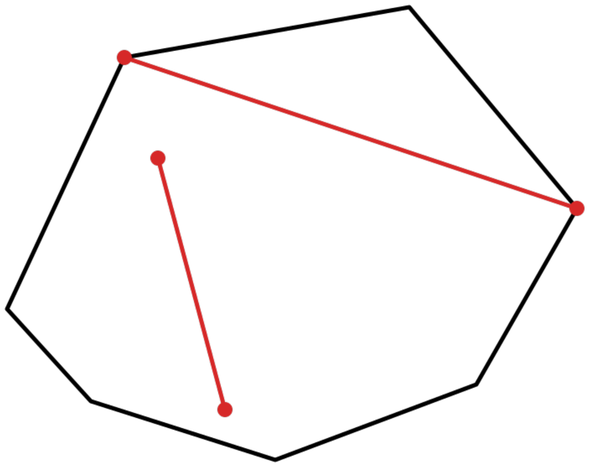
University of Victoria | Computer Science

Geometry Proxies (in 3D): Convex Polyhedron

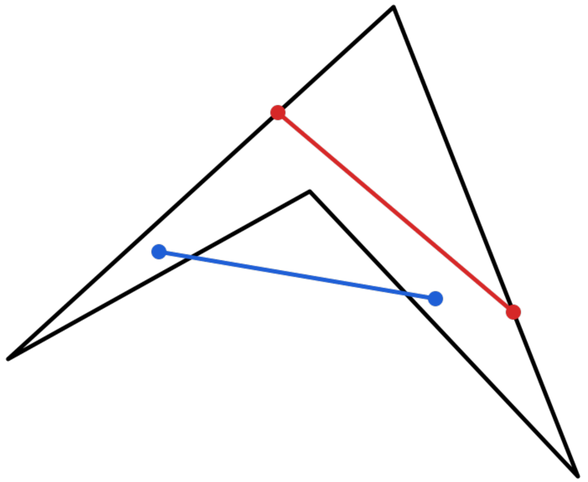


Why convex?

- Definition: a shape is convex if the segment between any two of its points stays inside.



convex



not convex

University of Victoria | Computer Science

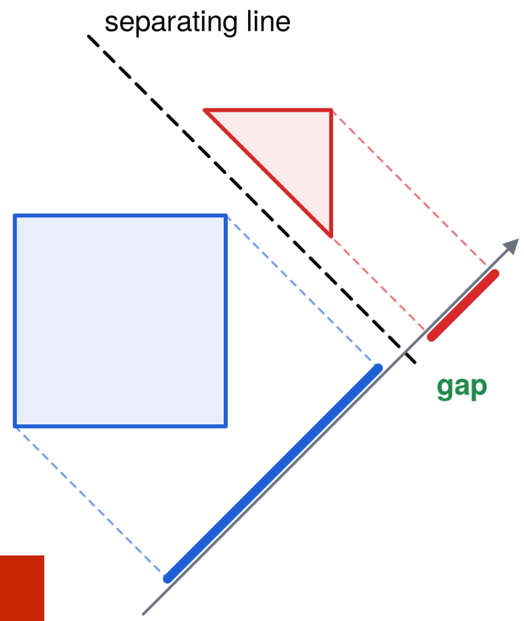
Why convex?

- Consequence 1: a ray enters once and leaves once.
- Consequence 2: point-inside is a simple check.
- Consequence 3: two convex shapes that don't touch always have a separating plane (a line in 2D).

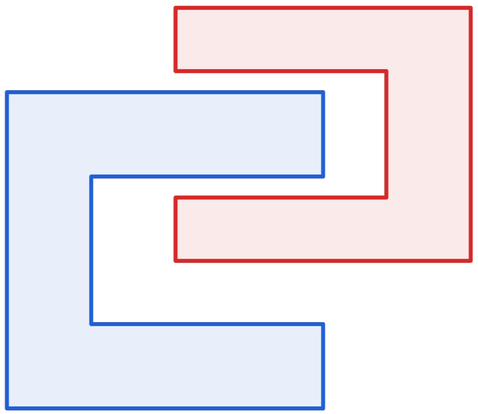
University of Victoria | Computer Science

Separating Axis Theorem (SAT)

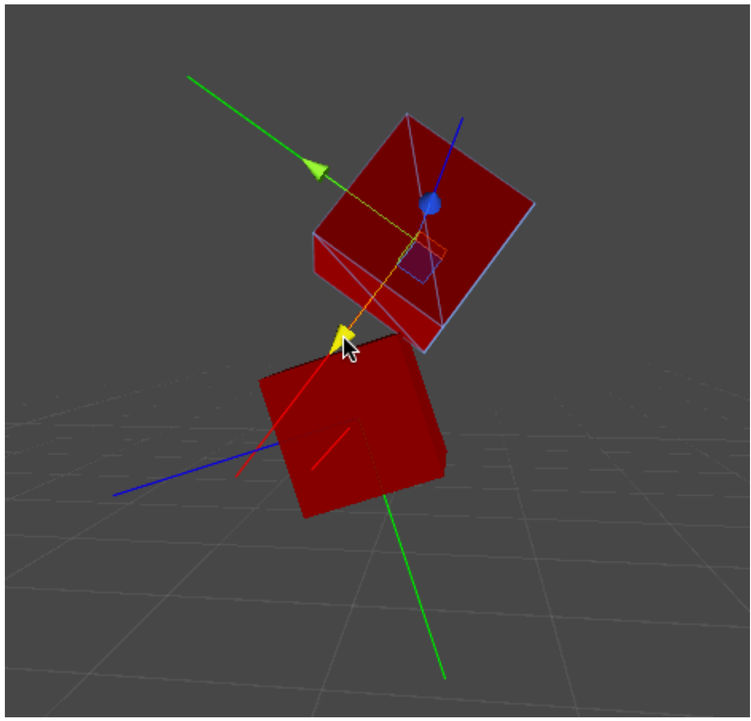
- Two convex shapes do **not** collide **if and only if** there exists at least one line (a separating axis) onto which the projections of both shapes do not overlap



convex: a line fits between



not convex: no line fits

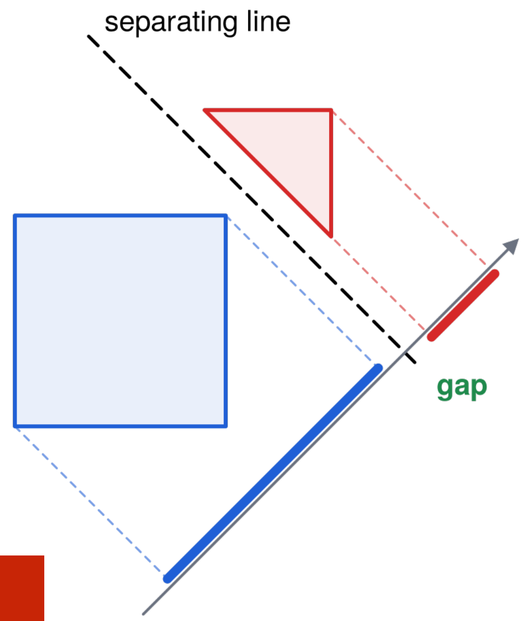


https://github.com/irixapps/Unity-Separating-Axis-SAT

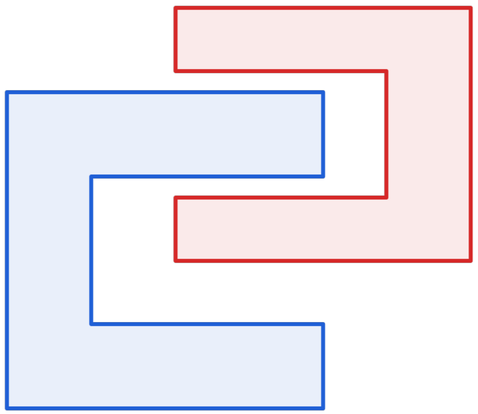
Extra

Separating Axis Theorem (SAT)

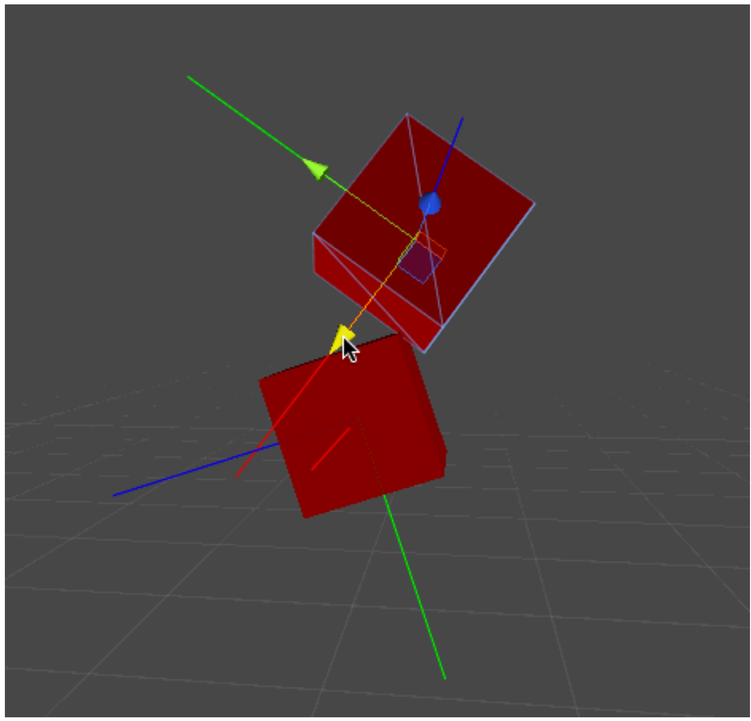
- Two convex shapes do **not** collide **if and only if** there exists at least one line (a separating axis) onto which the projections of both shapes do not overlap



convex: a line fits between



not convex: no line fits

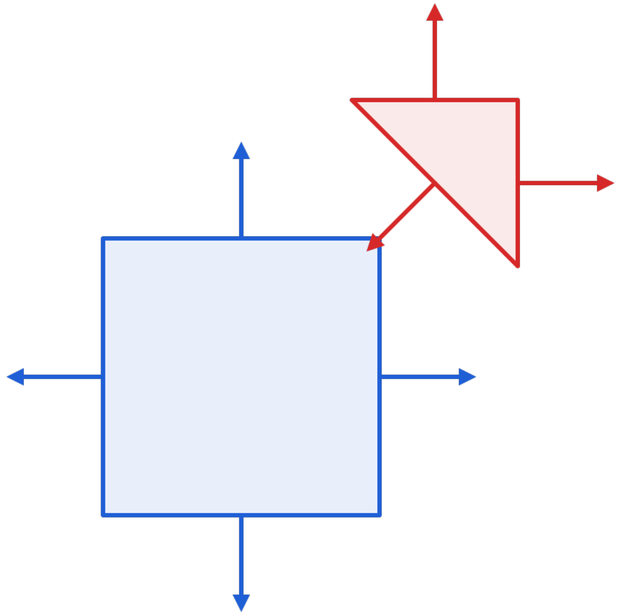


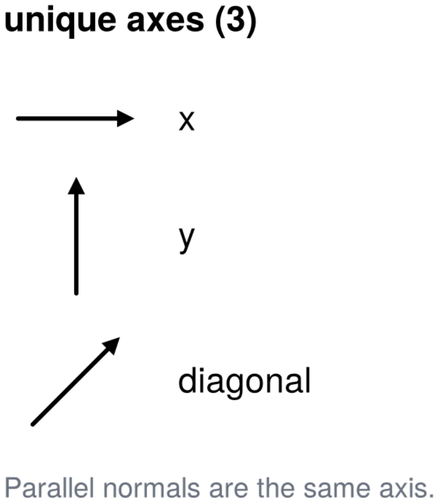
https://github.com/irixapps/Unity-Separating-Axis-SAT

Extra

Separating Axis Theorem (SAT)

- Step 1: Collect candidate axes

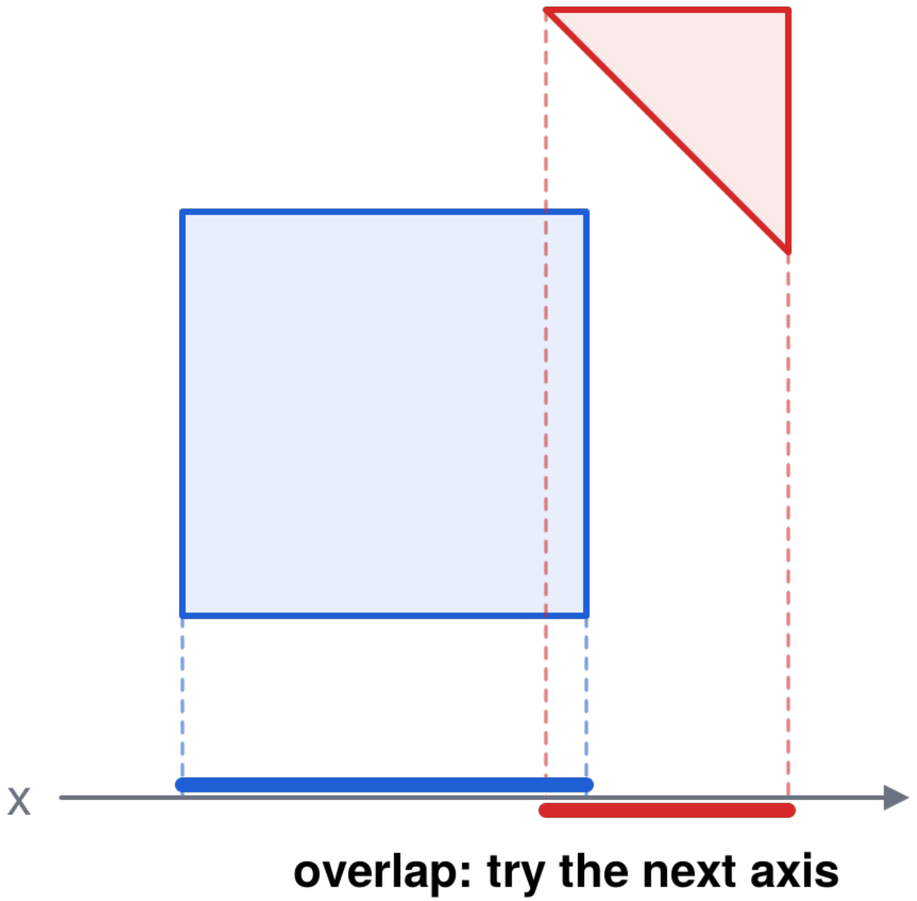




Extra

Separating Axis Theorem (SAT)

- Step 1: Collect candidate axes
- Step 2: Iteratively test if two shapes can be separated along any axis

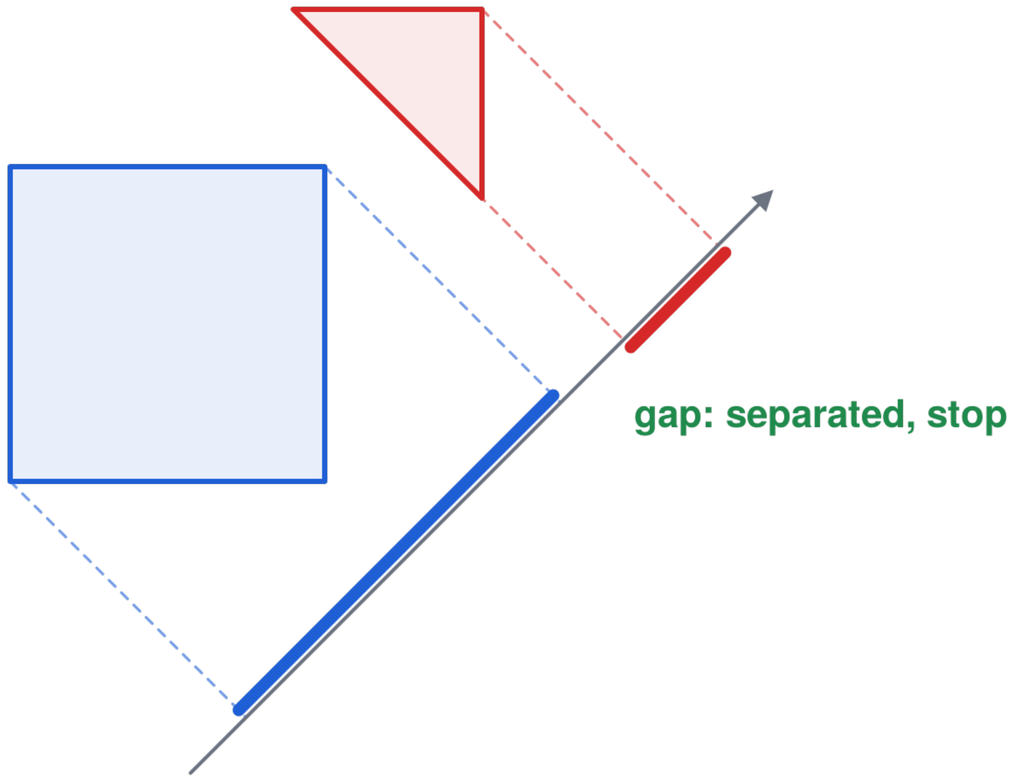


Extra

University of Victoria | Computer Science

Separating Axis Theorem (SAT)

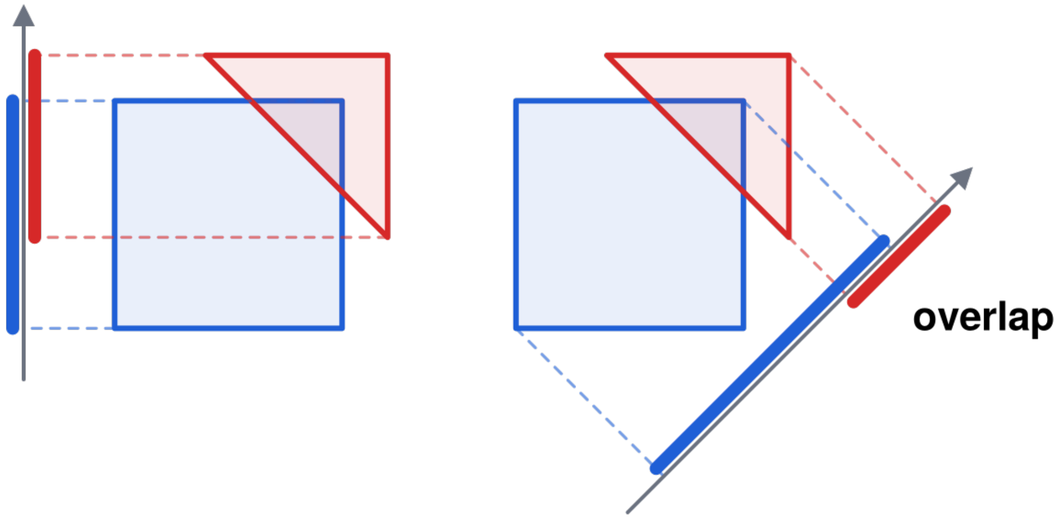
- Step 1: Collect candidate axes
- Step 2: Iteratively test if two shapes can be separated along any axis
  - A gap on any axis means separated



Extra

Separating Axis Theorem (SAT)

- Step 1: Collect candidate axes
- Step 2: Iteratively test if two shapes can be separated along any axis
  - Overlap on every axis means collision



Extra

3D Meshes as Hit-Boxes

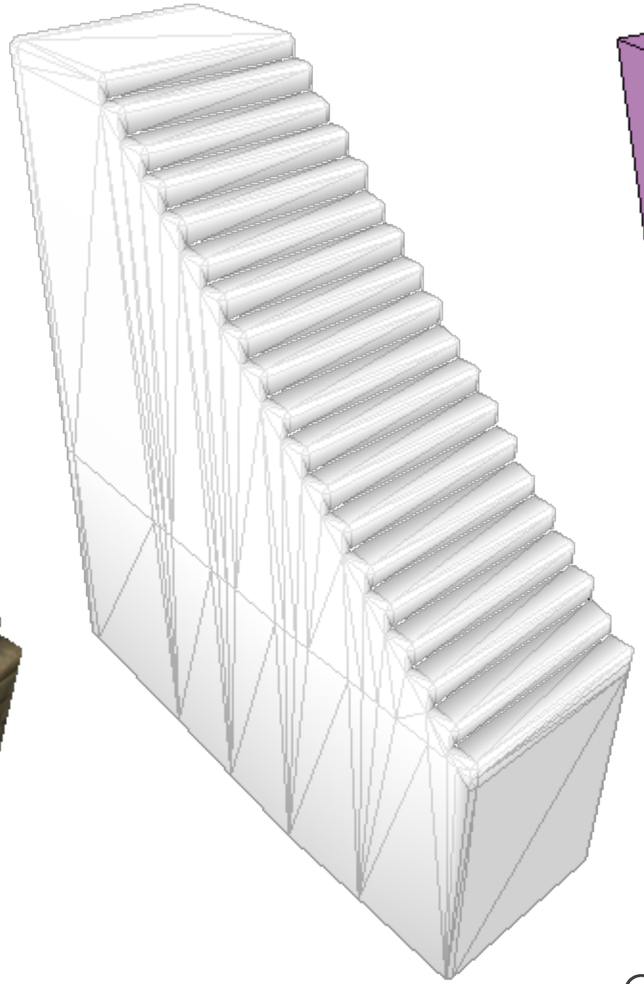
- These are often NOT the meshes that you use for rendering
  - much lower resolution (~ $O(10^2)$)
  - no attributes (no uv-mapping, no col, etc)
  - closed, water-tight (inside != outside)
  - often convex only
  - can be polygonal (as long as the faces are flat)

University of Victoria | Computer Science

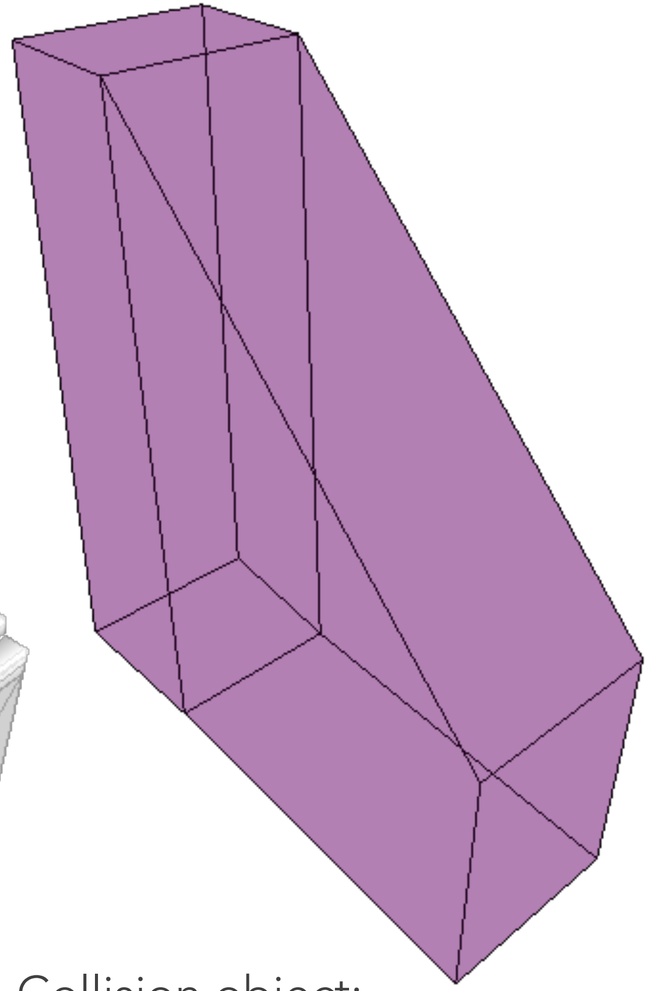
3D Meshes as Hit-Boxes



mesh for rendering
(~600 tri faces)



(in wireframe)

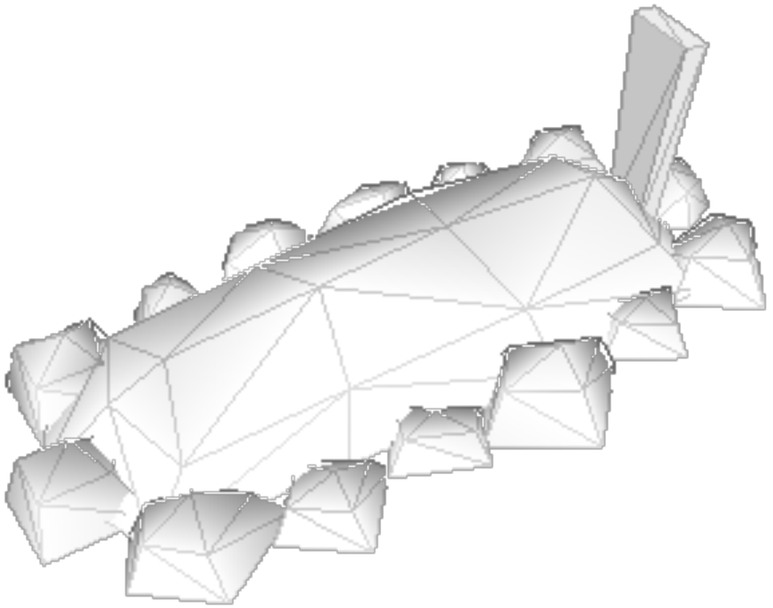


Collision object:
10 (polygonal) faces

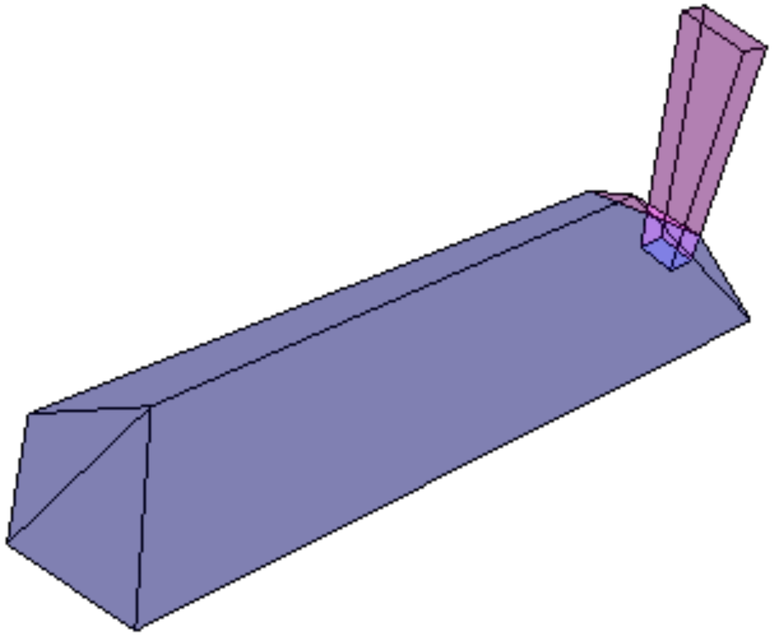
3D Meshes as Hit-Boxes



mesh for rendering
(~300 tri faces)



(in wireframe)



Collision object:
12 (polygonal) faces

Geometry Proxies: Composite Hit-Boxes

- Union of Hit-Boxes
  - inside *iff* inside of *any* sub Hit-Box
- Flexible
  - union of **convex** Hit-Boxes ==> **concave** Hit-Box
  - shape partially defined by a sphere, partially by a box ==> better approximation
- Creation: typically by hand
  - (remember: hit-boxes are usually **assets**)

University of Victoria | Computer Science

How To Choose The Proxy?

- Application dependent
- Note: # of intersection tests to be implemented quadratic wrt # of types supported

VS	Type A	Type B	Type C	Point	Ray
Type A	*algorithm*	*algorithm*	*algorithm*	*algorithm*	*algorithm*
Type B		*algorithm*	*algorithm*	*algorithm*	*algorithm*
Type C			*algorithm*	*algorithm*	*algorithm*

useful,
e.g.
for visibility

University of Victoria | Computer Science

Collision Detection Strategies

Frame 1 Frame 2

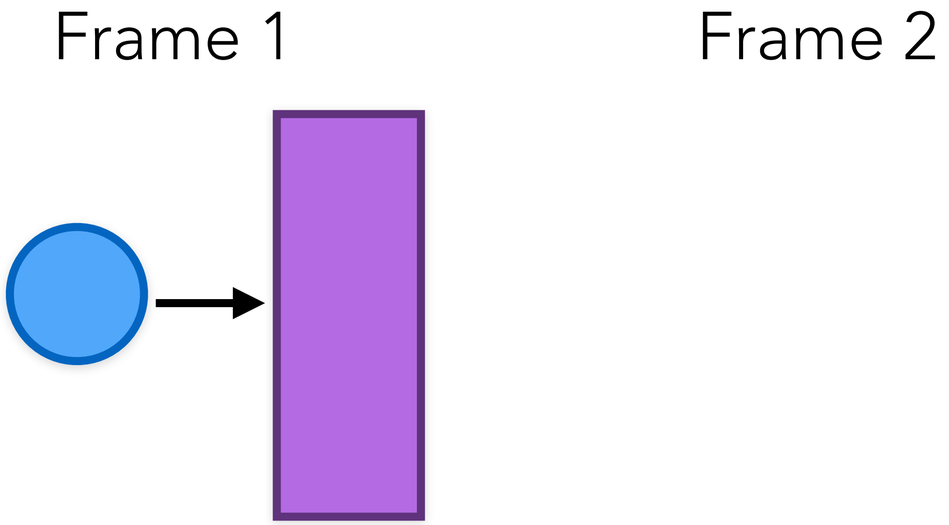
- **Static** Collision detection
  - (“a posteriori”, “discrete”)
  - approximated
  - simple + quick

- **Dynamic** Collision detection
  - (“a priori”, “continuous”)
  - accurate
  - demanding

University of Victoria | Computer Science

Collision Detection Strategies

- **Static** Collision detection
  - (“a posteriori”, “discrete”)
  - approximated
  - simple + quick

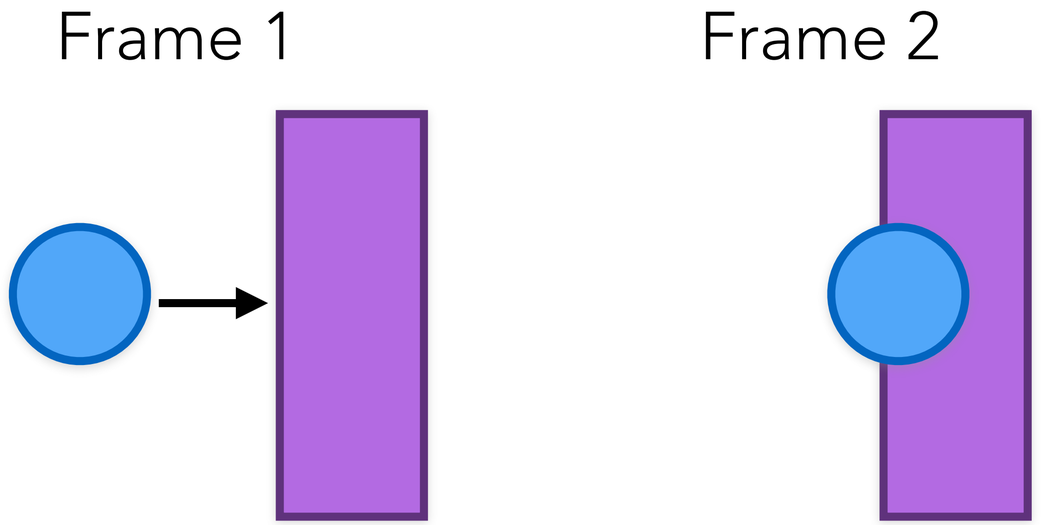


- **Dynamic** Collision detection
  - (“a priori”, “continuous”)
  - accurate
  - demanding

University of Victoria | Computer Science

Collision Detection Strategies

- **Static** Collision detection
  - (“a posteriori”, “discrete”)
  - approximated
  - simple + quick

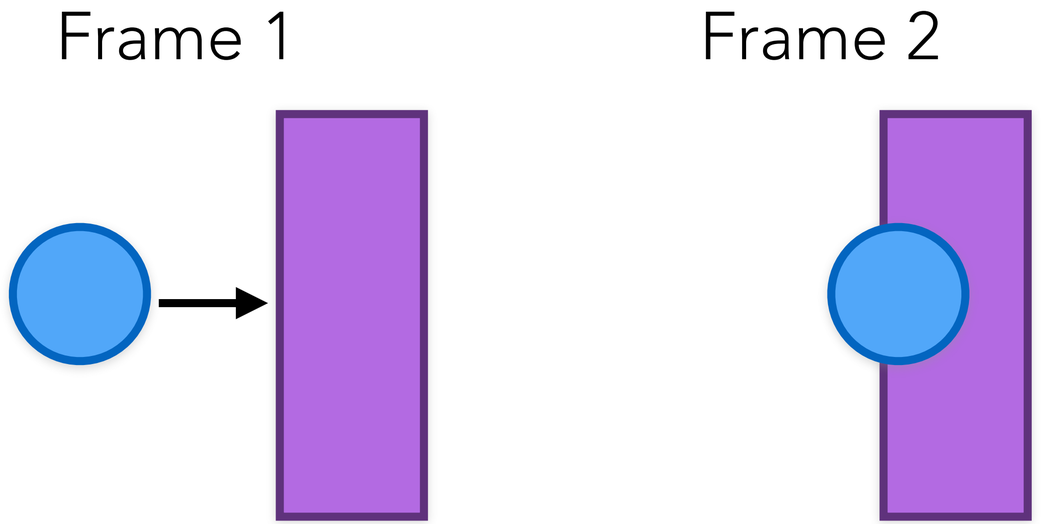


- **Dynamic** Collision detection
  - (“a priori”, “continuous”)
  - accurate
  - demanding

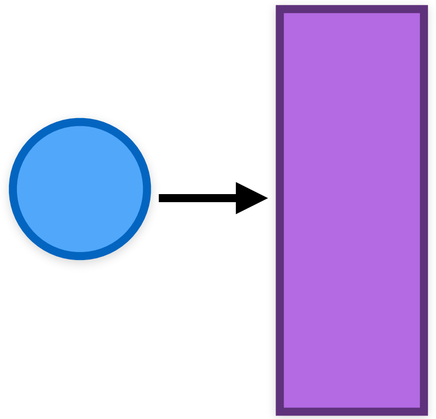
University of Victoria | Computer Science

Collision Detection Strategies

- **Static** Collision detection
  - (“a posteriori”, “discrete”)
  - approximated
  - simple + quick



- **Dynamic** Collision detection
  - (“a priori”, “continuous”)
  - accurate
  - demanding

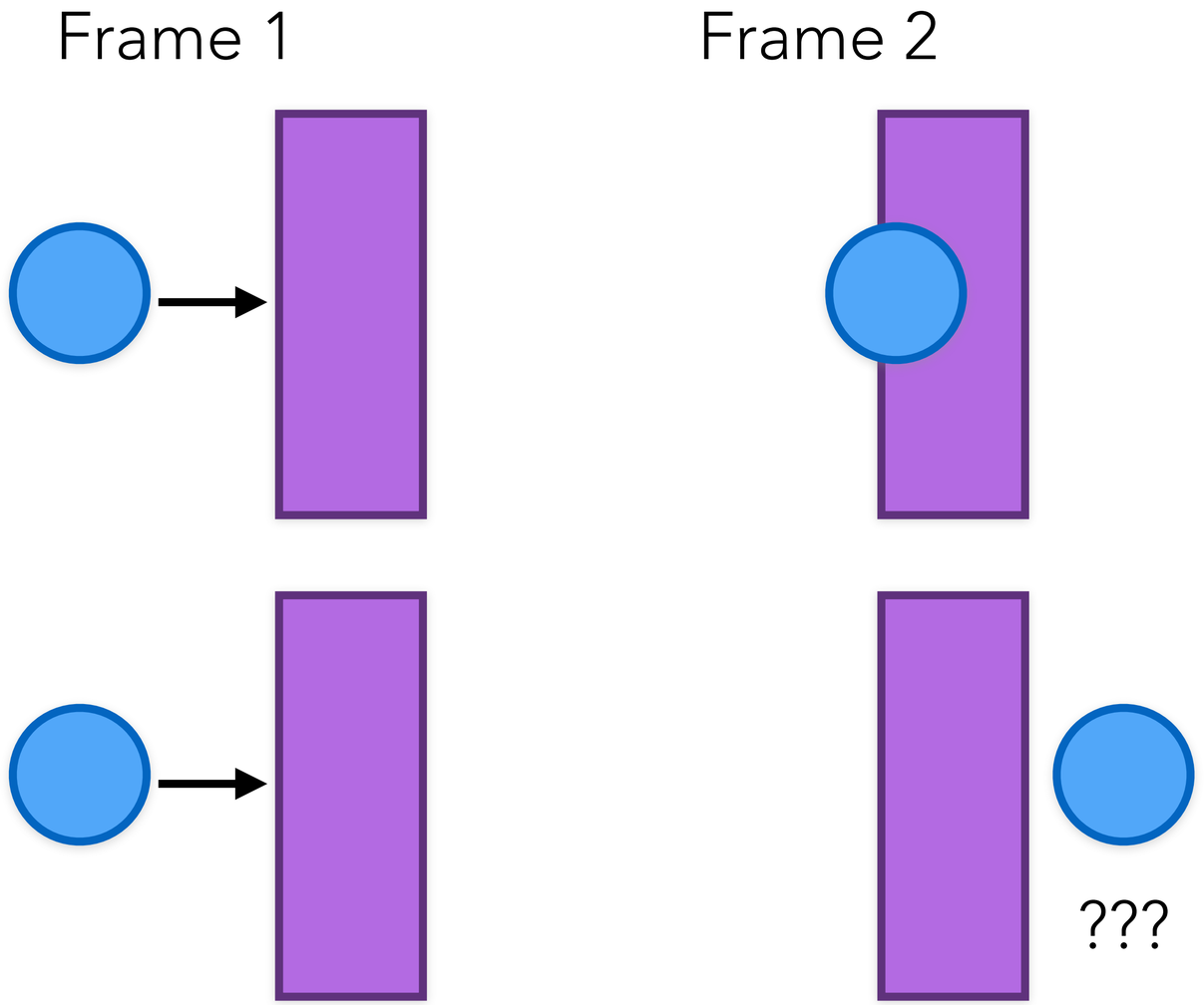


University of Victoria | Computer Science

Collision Detection Strategies

- **Static** Collision detection
  - (“a posteriori”, “discrete”)
  - approximated
  - simple + quick

- **Dynamic** Collision detection
  - (“a priori”, “continuous”)
  - accurate
  - demanding

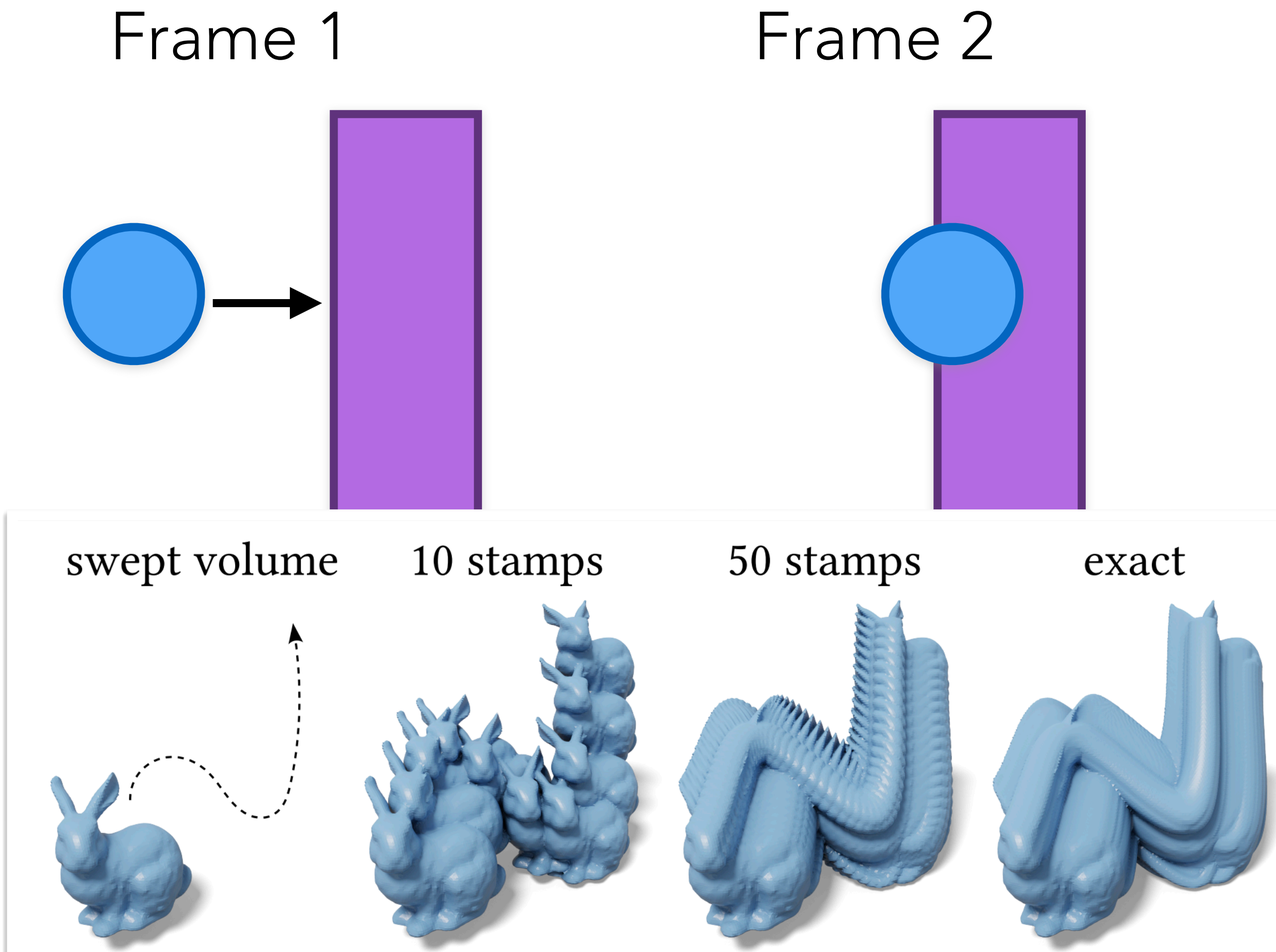


University of Victoria | Computer Science

Collision Detection Strategies

- **Static** Collision detection
  - (“a posteriori”, “discrete”)
  - approximated
  - simple + quick

- **Dynamic** Collision detection
  - (“a priori”, “continuous”)
  - accurate
  - demanding



Marschner, Zoë, et al. "Constructive solid geometry on neural signed distance fields." SIGGRAPH Asia 2023 conference papers. 2023.

Existing Implementations

- Intel Embree - BVH Tree - https://embree.github.io
- Nori - BVH - https://github.com/wjakob/nori
- Approximate knn - https://www.cs.umd.edu/~mount/ANN/
- Intersections - http://www.realtimerendering.com/intersections.html

References

Foundations of Multidimensional and Metric Data Structures
Hanan Samet

http://www.realtimerendering.com/books.html

http://www.realtimerendering.com/intersections.html

Polygon Mesh Processing
Mario Botsch, Leif Kobbelt, Mark Pauly, Pierre Alliez, Bruno Levy

Fundamentals of Computer Graphics, Fourth Edition
4th Edition **by Steve Marschner, Peter Shirley**
Chapter 12